



PL/M-86 USER'S GUIDE

COMPILER V2.3

123461-005

PL/M-86 USER'S GUIDE

Order Number: 121636-003

Additional copies of this manual or other Intel literature may be obtained from:

Literature Department
Intel Corporation
3065 Bowers Avenue
Santa Clara, CA 95051

The information in this document is subject to change without notice.

Intel Corporation makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. Intel Corporation assumes no responsibility for any errors that may appear in this document. Intel Corporation makes no commitment to update nor to keep current the information contained in this document.

Intel Corporation assumes no responsibility for the use of any circuitry other than circuitry embodied in an Intel product. No other circuit patent licenses are implied.

Intel software products are copyrighted by and shall remain the property of Intel Corporation. Use, duplication or disclosure is subject to restrictions stated in Intel's software license, or as defined in ASPR 7-104.9(a)(9).

No part of this document may be copied or reproduced in any form or by any means without the prior written consent of Intel Corporation.

The following are trademarks of Intel Corporation and its affiliates and may be used to identify Intel products:

BCP	Insite	iSBC	MULTICHANNEL
CREDIT	int ₂ l	iSBX	MULTIMODULE
i	Inteleview	iSXM	Plug-A-Bubble
PICE	Inteltec	Library Manager	PROMPT
ICE	Intellink	MCS	RMX/80
iCS	iOSP	Megachassis	RUPI
i _m	iPDS	MICROMAINFRAME	System 2000
iMMX	iRMX	MULTIBUS	UPI

REV.	REVISION HISTORY	DATE
-001	Original issue.	11/80
-002	Added information on DWORD and SELECTOR data types, new built-in functions, segmentation control extensions.	8/81
-003	Added built-ins BLOCK I/O, FLAGS, (flags register), and NIL (pointer value, and extended iAPX 186 instructions.	10/82



HOW TO USE THIS MANUAL

This manual gives instructions for programming in PL/M-86 and for using the PL/M-86 compiler to prepare programs for iAPX 86 microcomputer systems. It is primarily a reference manual for use when you are writing or compiling PL/M-86 programs; however, it also contains some introductory information to help you familiarize yourself with PL/M-86 as you start to use it.

The manual assumes you are familiar with basic programming concepts, including structured programming. The manual, however, does define the language completely, assuming no prior knowledge of PL/M-86.

Following the description of the language, the manual provides instructions for compiling your PL/M-86 programs, linking the compiled code, and executing the final program. It also explains how to interpret compiler output, including error messages.

Finally, the appendixes provide quick reference information plus supplementary instructions for interfacing PL/M-86 modules to modules written in other languages and to your own operating system software.

Manual Organization

This manual contains four kinds of information:

- Introductory and general reference information, including installation instructions
- Language information, for when you are programming in PL/M-86
- Operating instructions for the compiler and descriptions of compiler controls
- Interfacing information you need if you supply some of your own systems software in place of that supplied by Intel (e.g., a non-Intel operating system), or if you are interfacing PL/M-86 modules to modules written in other languages such as ASM86.

If you are a manager evaluating PL/M-86 to determine whether it fits your needs, you will find most of the information you need in Chapter 1, which is an overview of the product.

To get started with PL/M-86, first read this Preface (How to Use this Manual) and Chapter 1. (If you are familiar with assembly languages but not with high-level languages, see section 1.2 for a discussion of the advantages of a high-level language such as PL/M.) Then install the compiler (see instructions in the appendix for your specific operating system) and try compiling, linking, and running the sample program at the end of Chapter 1 to verify that the software operates properly.

After that, if PL/M is a new language to you, study and run the sample program in Chapter 12. Finally, skim through the manual from Chapter 2 to the end and try writing and running a few programs of your own. Once you have become familiar with PL/M-86, you will find this manual useful as a complete reference. For a quick reference, see the *PL/M-86 Pocket Reference* (order number 121622).

If you wish to transport existing PL/M-80 programs to your iAPX 86 application system, refer to Appendix D for a list of the differences between PL/M-80 and PL/M-86. This appendix indicates the areas of your programs that may require modification.

Once you have coded your programs, you are ready to compile, link, and run them. Refer to Chapter 11 for information on how to use compiler controls; refer to your specific host-system appendix for compiler operating instructions. Chapter 14 helps you interpret error messages you may receive when compiling or running your programs. For a detailed explanation of the linking process, refer to the *iAPX 86 Utilities User's Guide* (order number 121616).

If you are coding some of your application software in another language such as ASM86, refer to Appendix F for the information you need. If you are interfacing to your own operating system or providing your own file/device drivers, refer to Appendix G for instructions.

Notational Conventions

Section Numbers

All chapters and appendixes are section-numbered for easy cross-referencing. For instance, heading number 5.3 denotes Chapter 5, section 3. When the text of one section refers to another section, the reference is made by number; e.g., "as described in 7.1." Figures and tables are also numbered to aid in cross-referencing; e.g., "in table 3-1," "see figure 14-1."

Notation for Computer Dialogue

Interactive computer dialogue in this manual consists of commands you enter, which are immediately echoed on the console display, and the text displayed by the operating system, the PL/M-86 compiler, and other Intel-supplied programs. The text you enter is shown in reverse type (white on a black background), and the text displayed by Intel programs is shown in normal black type. The notation < cr > stands for the RETURN key on the console keyboard.

Syntax Notation

In the syntax notation for this manual, the following conventions apply:

- Keywords, letter symbols, and punctuation symbols that you use verbatim in your programs—the *terminal symbols* of the language—are represented in monospace type, in which every character has the same width, just as they do in output media such as CRT console displays and printouts. All letters in terminal symbols are shown in uppercase in the notation; however, you may use either uppercase or lowercase for these symbols in your programs. For example:

E	WHILE	PROCEDURE
(	TO	LITERALLY
:=	DO	END

are terminal symbols.

- Terms standing for language elements or constructs that are defined elsewhere in this notation—in other words, *nonterminal symbols*—are represented in italicized lowercase letters in non-monospace type, in which the width of a character varies. For example:

<i>digits</i>	<i>variable</i>
<i>sign</i>	<i>expression</i>
<i>binary-digit</i>	<i>statement</i>

are nonterminal symbols.

- When two adjacent items must be concatenated, they appear with no space between them. A blank space between two items indicates that the two items may be separated by one or more *logical-blanks*. For example:

digits . *digits* [E [*sign*] *digits*]

specifies that the first set of *digits*, the . symbol, and the second set of *digits* must be concatenated, with no blanks between them. Likewise, the E symbol, the *sign* if included, and the third set of *digits* must be concatenated. Blanks are permitted only between the second set of *digits* and the E symbol.

- Optional constructs are enclosed in square brackets. For example, in the construct represented by

digits . *digits* [E [*sign*] *digits*]

the first and second sets of *digits* and the . symbol are required, and the entire part following the second set of *digits* is optional. If this optional part is included, the *sign* may still be omitted.

- Optional constructs that can be repeated a number of times are marked by a three-dot ellipsis following the closing square bracket. For example:

binary-digit [*binary-digit*] ... B

stands for a concatenated sequence of one or more *binary-digits* followed immediately by a B symbol.

- Alternative constructs are represented as vertically adjacent items separated by extra vertical spacing and enclosed between curly braces that are taller than a single line of type. When these braces appear, choose any one of the constructs enclosed between the braces. For example:

$$\left\{ \begin{array}{l} \textit{digits} \\ \textit{binary-digit} [\textit{binary-digit}] \dots \textit{B} \\ \textit{octal-digit} [\textit{octal-digit}] \dots \textit{O} \\ \textit{hex-digit} [\textit{hex-digit}] \dots \textit{H} \end{array} \right\}$$

indicates that the construct described may have any one of the four forms listed between the large braces.

- Text enclosed between the character sequence /* and the sequence */, when these symbols are in light, non-monospace type, is a prose definition of the given construct. Such definitions are used when symbolic definitions would be more cumbersome. For example:

/* any uppercase or lowercase letter of the alphabet */

is used to avoid listing 52 separate characters vertically between braces.

- The start of a new line in the notation does not mean you must start a new line at that point in your program; however, you may do so for readability. For example, when you use the construct:

```
DO variable = expression TO expression;  
    statement list  
END
```

you need not include a carriage return after the second *expression*, but, in many programs, doing so makes the statement more readable.

(See Appendix C for the BNF notation for the PL/M-86 language.)



CHAPTER 1	PAGE		PAGE
INTRODUCTION			
Product Definition	1-1	Based Variables	3-15
The PL/M-86 Language	1-1	Location References and Based Variables	3-17
Using a High-Level Language	1-1	The AT Attribute	3-17
Why PL/M?	1-2		
The Structure of a PL/M-86 Program	1-2	CHAPTER 4	
PL/M-86 Statements	1-3	EXPRESSIONS AND ASSIGNMENTS	
Declaration Statements	1-3	Operands	4-1
Executable Statements	1-4	Constants	4-1
Built-In Procedures and Variables	1-4	Variable and Location References	4-2
Expressions	1-4	Subexpressions	4-2
Input and Output	1-5	Compound Operands	4-2
The Program Development Process	1-5	Arithmetic Operators	4-3
An Introductory Sample Program	1-6	The +, -, *, and / Operators	4-3
		The MOD Operator	4-4
		Relational Operators	4-4
		Logical Operators	4-5
		Expression Evaluation	4-6
		Precedence of Operators: Analyzing an	
		Expression	4-6
		Compound Operands Have Types	4-8
		Relational Operators Are Restricted	4-9
		Order of Evaluation of Operands	4-9
		Choice of Arithmetic: Summary of Rules	4-10
		Special Case: Constant Expressions	4-11
		Assignment Statements	4-12
		Implicit Type Conversions	4-13
		Constant Expression	4-13
		Multiple Assignment	4-14
		Embedded Assignments	4-14
CHAPTER 2		CHAPTER 5	
LANGUAGE ELEMENTS		ARRAYS AND STRUCTURES	
PL/M-86 Character Set	2-1	Arrays	5-1
Tokens, Separators, and the Use of Blanks	2-2	Subscripted Variables	5-1
Identifiers and Reserved Words	2-3	Structures	5-2
Constants	2-3	Arrays of Structures	5-3
Whole-Number Constants	2-3	Arrays Within Structures	5-3
Floating-Point Constants	2-4	Arrays of Structures with Arrays	
Character Strings	2-5	Inside the Structures	5-3
Comments	2-5	References to Arrays and Structures	5-4
		Fully Qualified Variable References	5-4
		Unqualified and Partially Qualified	
		Variable References	5-4
CHAPTER 3		CHAPTER 6	
DATA DECLARATIONS, TYPES, AND		FLOW CONTROL STATEMENTS	
BASED VARIABLES		DO and END Statements: DO Blocks	6-1
Variable Declaration Statements	3-1	Simple DO Blocks	6-2
Examples	3-2	DO CASE Blocks	6-3
Results	3-3	DO WHILE Blocks	6-4
Combining DECLARE Statements	3-3	Iterative DO Blocks	6-5
Initializations	3-4	The IF Statement	6-7
The Implicit Dimension Specifier	3-6	Nested IF Statements	6-8
Names for Execution Constants: The Use of			
DATA	3-6		
Types of Declaration Statements	3-7		
Compilation Constants (Text Substitution):			
The Use of LITERALLY	3-7		
Declarations of Names for Labels	3-8		
Results	3-9		
Declarations for Procedures	3-9		
Data Types	3-10		
BYTE, WORD, and DWORD Variables:			
Unsigned Arithmetic	3-10		
INTEGER Variables: Signed Arithmetic	3-11		
REAL Variables: Floating-Point Arithmetic	3-11		
Examples of Binary Scientific Notation	3-12		
POINTER Variables and Location References	3-13		
The @ Operator	3-13		
The "Dot" Operator	3-15		
SELECTOR Variables	3-15		

Sequential IF Statements	6-10
GOTO Statements	6-11
The CALL and RETURN Statements	6-12

CHAPTER 7 BLOCK STRUCTURE AND SCOPE

Names Recognized Within Blocks	7-1
Restrictions on Multiple Declarations	7-2
Extended Scope: The PUBLIC and EXTERNAL Attributes	7-4
Scope of Labels and Restriction on GOTOs	7-6

CHAPTER 8 PROCEDURES

Procedure Declarations	8-1
Parameters	8-2
Typed Versus Untyped Procedures	8-3
Activating a Procedure—Function References and CALL Statements	8-4
Indirect Procedure Activation	8-5
Exit from a Procedure: The RETURN Statement ..	8-5
The Procedure Body	8-6
The Attributes: PUBLIC and EXTERNAL, INTERRUPT, REENTRANT	8-7
Interrupts and the INTERRUPT Attribute	8-8
Activating an Interrupt Procedure with a CALL Statement	8-9
Reentrancy and the REENTRANT Attribute	8-10

CHAPTER 9 BUILT-IN PROCEDURES, FUNCTIONS, AND VARIABLES

Obtaining Information About Variables	9-1
The LENGTH Function	9-1
The LAST Function	9-2
The SIZE Function	9-2
Explicit Type and Value Conversions	9-3
The LOW, HIGH, and DOUBLE Functions	9-4
The FLOAT Function	9-5
The FIX Function	9-5
The INT Function	9-5
The SIGNED Function	9-6
The UNSIGN Function	9-6
The ABS and IABS Functions	9-7
Shift and Rotate Functions	9-7
Rotation Functions ROL and ROR	9-7
Logical-Shift Functions SHL and SHR	9-8
Algebraic-Shift Functions SAL and SAR	9-8
String Manipulation Procedures	9-9
The MOVB and MOVW Procedures	9-10
The MOVRB and MOVW Procedures	9-10
The CMPB and CMPW Functions	9-10
The FINDB/FINDW and FINDRB/FINDRW	

Functions	9-11
The SKIPB/SKIPW and SKIPRB/SKIPRW Functions	9-11
The XLAT Procedure	9-12
The SETB and SETW Procedures	9-12
Miscellaneous Built-Ins	9-13
The MOVE Procedure	9-13
The TIME Procedure	9-13
The MEMORY Array	9-14
The LOCKSET Function	9-14
Interrupt-Related Procedures	9-15
The SET\$INTERRUPT Procedure	9-15
The INTERRUPT\$PTR Function	9-15
POINTER and SELECTOR-Related Functions	9-16
The BUILD\$PTR Function	9-16
The SELECTOR\$OF Function	9-16
The OFFSET\$OF Function	9-16
The NIL Function	9-16

CHAPTER 10 PL/M-86 FEATURES INVOLVING iAPX 86 HARDWARE

iAPX 86 Hardware-Dependent Statements	10-1
The ENABLE and DISABLE Statements	10-1
The CAUSE\$INTERRUPT Statement	10-1
The HALT Statement	10-1
The iAPX 86 Hardware Flags	10-1
Optimization and the Hardware Flags	10-1
The CARRY, SIGN, ZERO, and PARITY Functions	10-2
The PLUS and MINUS Operators	10-2
Carry-Rotation Functions	10-3
The DEC Function	10-3
The iAPX 86 Hardware Registers	10-3
The FLAGS Variable	10-3
The STACKPTR and STACKBASE Variables ..	10-4
iAPX 86 Hardware I/O	10-4
The INPUT and INWORD Functions	10-4
The OUTPUT and OUTWORD Arrays	10-5
The BLOCKINPUT and BLOCKINWORD Procedures	10-5
The BLOCKOUTPUT and BLOCKOUTWORD Procedures	10-5
The REAL Math Facility	10-6
Built-ins Supporting the REAL Math Unit	10-8
The SET\$REAL\$MODE Procedure	10-8
The GET\$REAL\$ERROR Function	10-8
Saving and Restoring the REAL Status	10-9
The SAVE\$REAL\$STATUS Procedure	10-9
The RESTORE\$REAL\$STATUS Procedure	10-10



CONTENTS (Cont'd.)

CHAPTER 11

COMPILER CONTROLS

Introduction to Compiler Controls	11-1
The MOD86/MOD186 Control	11-2
The LEFTMARGIN Control	11-2
Object File Controls	11-3
INTVECTOR/NOINTVECTOR	11-4
OVERFLOW/NOOVERFLOW	11-4
OPTIMIZE	11-4
OPTIMIZE(0)	11-5
OPTIMIZE(1)	11-5
OPTIMIZE(2)	11-6
OPTIMIZE(3)	11-8
OBJECT/NOOBJECT	11-16
Object Module Sections	11-16
DEBUG/NODEBUG	11-17
TYPE/NOTYPE	11-18
Program Size Controls	11-18
The SMALL Case	11-18
The COMPACT Case	11-20
The MEDIUM Case	11-21
The LARGE Case	11-22
RAM/ROM Control	11-23
Listing Selection and Content Controls	11-24
PRINT/NOPRINT	11-24
LIST/NOLIST	11-24
CODE/NOCODE	11-25
XREF/NOXREF	11-25
SYMBOLS/NOSYMBOLS	11-25
Listing Format Controls	11-26
PAGING/NOPAGING	11-26
PAGELENGTH	11-26
PAGEWIDTH	11-26
TITLE	11-27
SUBTITLE	11-27
EJECT	11-27
Sample Program Listing	11-28
Symbol and Cross-Reference Listing	11-28
Compilation Summary	11-31
Source Inclusion Controls	11-32
INCLUDE	11-32
SAVE/RESTORE	11-32
Conditional Compilation Controls	11-33
IF/ELSE/ELSEIF/ENDIF	11-33
SET/RESET	11-35
COND/NOCOND	11-36

CHAPTER 12

SAMPLE PROGRAM

CHAPTER 13

PL/M-86 EXTENDED

SEGMENTATION MODELS

Introduction	13-1
Extended Segmentation	13-1
Subsystems	13-2
Open and Closed Subsystems	13-3
The EXPORTS List	13-4
Placement of Controls	13-6

CHAPTER 14

ERROR MESSAGES

Source PL/M-86 Errors	14-1
Fatal Command Tail and Control Errors	14-20
Fatal Input/Output Errors	14-20
Fatal Insufficient Memory Errors	14-20
Fatal Compiler Failure Errors	14-20

APPENDIX A

PL/M-86 RESERVED WORDS AND PREDECLARED IDENTIFIERS

APPENDIX B

PROGRAM CONSTRAINTS

APPENDIX C

GRAMMAR OF THE PL/M-86 LANGUAGE

APPENDIX D

SUMMARY OF DIFFERENCES BETWEEN PL/M-80 AND PL/M-86

APPENDIX E

ASCII CODES

APPENDIX F

LINKING TO MODULES WRITTEN IN OTHER LANGUAGES

APPENDIX G

RUN-TIME INTERRUPT PROCESSING

APPENDIX H

COMPILER INVOCATION AND ADDITIONAL INFORMATION FOR SERIES III USERS

APPENDIX I

COMPILER INVOCATION AND ADDITIONAL INFORMATION FOR iRMX 86 USERS



TABLES

TABLE	TITLE	PAGE	TABLE	TITLE	PAGE
2-1	PL/M-86 Special Characters	2-1	F-1	Summary of iAPX 86 Register Usage	F-5
3-1	Declaration Elements	3-1	F-2	Registers Used to Hold Simple Data Types	F-5
4-1	Operators' Precedence	4-6	F-3	Summary of PL/M-86 Segment Names	F-6
4-2	Summary of Expression Rules	4-10	G-1	Exception and Response Summary	G-5
9-1	Explicit Type and Value Conversions	9-3			
11-1	Compiler Controls	11-2			
11-2	Controls by Categories	11-3			



ILLUSTRATIONS

FIGURE	TITLE	PAGE	FIGURE	TITLE	PAGE
1-1a	Sample Program: Module Sort (module 1 of 2)	1-7	11-4	Sample Program Showing the OPTIMIZE(3) Control	11-14
1-1b	Sample Program: Module Sort (module 2 of 2)	1-8	11-5	Program Listing	11-29
3-1	Successive Byte References of a Structure	3-18	11-6	Cross-Reference Listing	11-30
7-1	Inclusive Extent of Blocks	7-2	11-7	Compilation Summary	11-31
7-2	Outer Level of Block SORT	7-3	11-8	Sample Program Showing the SET(DEBUG=) Control	11-34
7-3	Sample Program Modules Illustrating Valid GOTO Usage	7-7	11-9	Sample Program Showing the NOCOND Control	11-36
7-4	Valid GOTO Transfers	7-9	12-1	Sample Program: Record Sort	12-2
10-1	The iAPX 86 Hardware Flags Register	10-4	F-1	Stack Layout at Point Where a Non-Interrupt Procedure is Activated	F-2
10-2	The REAL Error Byte	10-7	F-2	Stack Layout During Execution of a Non-Interrupt Procedure Body	F-3
10-3	The REAL Mode Word	10-7	G-1	Stack Layout at Point Where an Interrupt Procedure Gains Control	G-2
11-1	Sample Program Showing the OPTIMIZE(0) Control	11-9	G-2	Stack Layout During Execution of Interrupt Procedure Body	G-3
11-2	Sample Program Showing the OPTIMIZE(1) Control	11-11	G-3	Memory Layout of Real Save Area	G-10
11-3	Sample Program Showing the OPTIMIZE(2) Control	11-13			



This chapter introduces the PL/M-86 language and explains the process of developing software for your iAPX 86 application system using PL/M-86.

1.1 Product Definition

PL/M is a high-level language for programming various families of microprocessors. It was designed by Intel Corporation to meet the software requirements of computers in a wide variety of systems and applications work.

The PL/M-86 compiler is a software tool that translates your PL/M-86 source programs into relocatable iAPX 86 object modules. You can then link these to other modules coded in PL/M, assembly, or other high-level languages. The compiler provides a listing output, error messages, and a number of compiler controls to aid in program development and debugging. The compiler runs on an Intel microcomputer development system.

To perform the steps following compilation, use the iAPX 86-based software development utilities – LINK86, LIB86, CREF86, LOC 86, and OH 86. Debug your programs using an applications debugger and the ICE-86 or ICE-88 (In-Circuit Emulator). For firmware systems, you then use the Universal Prom Programmer (UPP) with its Universal Prom Mapper (UPM) software to transfer your programs to PROM.

1.2 The PL/M-86 Language

Using a High-Level Language

High-level languages more closely model the human thought process than lower-level languages such as assembly language. They therefore are easier and faster to write, since one fewer translation step is required from concept to code. High-level language programs are also more likely to be correct, since there is less occasion to introduce error.

Programs in a high-level language are easier to read and understand, and thus easier to modify. As a result, you can develop high-level language programs in a much shorter period of time—programs that are easier to maintain throughout the life of the product. Thus, high-level languages result in lower costs for both developing and maintaining programs.

In addition, programs in a high-level language are easily transferred from one processor to another. Programs that can be transferred between processors without modification are said to be portable.

If PL/M-86 is your first high-level language, you probably want to know how programming in a high-level language differs from assembly language programming. When you use a high-level language:

- You need not know the instruction set of the processor you are using.
- You need not be concerned with the details of the target processor, such as register allocation or assigning the proper number of bytes for each data item—the compiler takes care of these things automatically.

- You use keywords and phrases that are closer to natural English.
- You can combine many operations (including arithmetic and Boolean operations) into expressions; thus, you can perform a whole sequence of operations with one statement.
- You can use data types and data structures that are closer to your actual problem. For instance, in PL/M-86 you can program in terms of Boolean expressions, characters, and data structures, in addition to bytes and words.

The introductory example at the end of this chapter (section 1.6) illustrates these points. Compare this PL/M program with an assembly-language program you might write to solve the same problem.

Coding programs in a high-level language involves thinking differently from coding in assembly language. This level is actually closer to the level of thinking you use when you are planning your overall system design.

Why PL/M?

Many high-level programming languages are available today. Some of them have been around far longer than PL/M. So, once you have decided to use a high-level language, you might ask: How does PL/M differ from other high-level languages? What advantages does it have? When is it the right language to use?

Here are some characteristics of PL/M:

- It has a block structure and control constructs that aid—in fact, encourage and enforce—structured programming.
- It has facilities for such data structures as structured arrays and pointer-based dynamic variables.
- It is a typed language—that is, the compiler does data type compatibility and range checking to help you detect logic errors in your programs at compile time.
- Its data structuring facilities and control statements are designed in a logically consistent way. Thus, PL/M is a good language for expressing algorithms for systems programming.
- Its control constructs make program correctness relatively easy to verify.
- It is a standard language used on Intel microcomputers, so PL/M programs are portable across Intel's processors.

PL/M was designed for programmers (generally systems programmers) who need access to the microprocessor's features—features such as indirect addressing and direct I/O for optimum use of all system resources.

What about the differences between PL/M and older, more established languages like FORTRAN, BASIC, and COBOL? PL/M has many more features than BASIC and is a more general-purpose language than either FORTRAN (best suited for scientific applications) or COBOL (tailored for business data processing). Additionally, PL/M differs from these other languages in its typing and block structure.

1.3 The Structure of a PL/M-86 Program

PL/M-86 is a block-structured language: every statement in a program is part of at least one block—i.e., a well-defined group of statements that begins with a DO statement or a procedure declaration and ends with an END statement.

A module is a labeled simple DO-block; that is, a module must begin with a labeled DO statement and end with an END statement. Between those end points (i.e., within that DO-block), other statements provide the definitions of data and processes that make up the program. These statements are said to be part of the block, contained within the block, or nested within the block. A module can contain other blocks but is never itself contained within another block.

(The reason for saying “simple” DO-block is that DO-blocks are of three other varieties, which are explained in Chapter 6.)

NOTE

The word EOF in PL/M-86 is a reserved word indicating the end of the file. It should not be used as an identifier because the remainder of the module it is used in will be ignored at compile-time.

Every PL/M-86 program consists of one or more modules, separately compiled, each consisting of one or more blocks. There are two kinds of blocks: DO-blocks and procedure definition blocks.

A procedure definition block is a set of statements beginning with a procedure declaration and ending with an END statement. Other declarations and executable statements can go between these endpoints, and are used later when the procedure is actually invoked or called into execution. The definition block is really a further declaration of everything the procedure will use and do. Since it is only executed later, the definition block is considered simply another form of declaration rather than being viewed as immediately executable.

1.4 PL/M-86 Statements

There are two types of statements in PL/M-86: declarations and executable statements.

Declaration Statements

A simple example of a declare statement is:

```
DECLARE WIDTH BYTE;
```

This introduces the identifier WIDTH and associates it with the contents of one byte (8 bits) of memory. The programmer need not know the location of the byte—i.e., its actual address in memory—but will simply refer to the content of this byte using the name WIDTH.

A group of statements intended to perform a function (i.e., a subprogram or subroutine) can be given a name by declaring them to be a procedure:

```
ADDER_UPPER: PROCEDURE (BETA);
```

The statements that define the procedure then follow. This block of PL/M-86 statements is invoked from other points in the program, and may involve passing parameters to it and returning a value. When a procedure has finished executing, control is returned immediately to the main program. This capability is the major feature permitting modular program construction.

Executable Statements

An example of an executable statement is:

```
CLEARANCE = WIDTH + 2 ;
```

Here, we have two identifiers, CLEARANCE and WIDTH. Both must be declared prior to this executable statement, which produces machine code to retrieve the WIDTH value from memory. Once you obtain the WIDTH value, add 2 to it and store the sum in the memory location for CLEARANCE.

For most purposes, the PL/M-86 programmer need not think in terms of memory locations. CLEARANCE and WIDTH are variables, and the assignment statement assigns the value of the expression WIDTH + 2 to the variable CLEARANCE. The compiler automatically generates all the machine code necessary to retrieve data from memory, to evaluate the expression retrieved, and to store the result in the proper location.

The following is a list of all PL/M-86 executable statements and the chapters in which they are fully discussed:

Assignment Statement	Chapter 4
Simple DO Statement	Chapter 6
DO CASE Statement	Chapter 6
DO WHILE Statement	Chapter 6
Iterative DO Statement	Chapter 6
END Statement	Chapter 6
IF Statement	Chapter 6
GOTO Statement	Chapter 6
Null Statement	Chapter 6
CALL Statement	Chapter 8
RETURN Statement	Chapter 8
ENABLE Statement	Chapter 10
DISABLE Statement	Chapter 10
CAUSE\$INTERRUPT Statement	Chapter 10
HALT Statement	Chapter 10

Built-In Procedures and Variables

PL/M-86 provides a large repertoire of built-in procedures and variables. These procedures provide such functions as shifts and rotations, data type conversions, and string manipulation. The built-in procedures and variables are described in Chapter 9.

Expressions

We have already seen simple expressions. A PL/M-86 expression, made up of operands and operators, resembles a conventional algebraic expression.

Operands include numeric constants (such as 3.78 or 105) and variables (as well as other types discussed in Chapters 3 and 4). The operators include + and - for addition and subtraction, * and / for multiplication and division, and MOD for modulo arithmetic.

As in an algebraic expression, elements of a PL/M-86 expression may be grouped with parentheses.

An expression is evaluated using unsigned integer arithmetic, signed integer arithmetic, and/or floating-point arithmetic depending on the types of operands in the expression (see Chapters 3 and 4).

Input and Output

PL/M-86 does not provide formatted I/O capabilities like those of FORTRAN, BASIC, or COBOL. PL/M-86 does provide built-in functions for direct I/O which do not require operating system run-time support. For single-byte or single-word I/O, these functions are INPUT, INWORD, OUTPUT, and OUTWORD. For block I/O, they are BLOCKINPUT, BLOCKINWORD, BLOCKOUTPUT, and BLOCKOUTWORD.

INPUT causes the program to read the 8-bit quantity found in one of the 64K input ports of the iAPX 86. A reference to OUTPUT causes the program to place an 8-bit quantity into one of the 64K output ports of the iAPX 86.

INWORD and OUTWORD have the same effects as INPUT and OUTPUT except that they handle 16-bit (WORD) quantities instead of 8-bit (BYTE) quantities.

BLOCKINPUT causes the program to read a string of 8-bit quantities found in one of the iAPX 86 input ports. BLOCKOUTPUT places a string of 8-bit quantities into one of the iAPX 86 output ports.

BLOCKINWORD and BLOCKOUTWORD have the same effects as BLOCKINPUT and BLOCKOUTPUT except that they handle strings of 16-bit quantities instead of strings of 8-bit quantities.

For more information on these I/O functions, see Chapter 9.

1.5 The Program Development Process

The PL/M-86 compiler is part of an integrated set of tools that make up the total iAPX 86 development solution for your microcomputer system.

The steps in the software development process are as follows:

1. Define the problem completely.
2. Outline the proposed solution in terms of hardware and software. Once this is done, you may begin designing your hardware.
3. Design the software for your system. This important step may consist of several sub-steps, including breaking down the task into modules, choosing the programming language, and selecting the algorithms to be used.
4. Code your programs and prepare them for translation using a text editor.
5. Translate your PL/M program code using the PL/M-86 compiler.
6. If necessary, use the text editor to correct any compile-time errors, then recompile.
7. Using the iAPX 86-based LINK86 (with the BIND option), link the resulting object module to the necessary support libraries and locate your object code. For detailed instructions see the *iAPX 86,88 Family User's Guide*.
8. You can then run your programs and debug them with a debugger such as the ICE-86 debugger.

1.6 An Introductory Sample Program

Figure 1-1 shows the listing of a sample PL/M-86 program divided into two modules. This program contains many undefined words and constructs that will be explained in the upcoming chapters. The programs appear here: 1) to show what a small PL/M-86 program looks like separated into two modules, and 2) to explain, step by step, how to compile, link, and run programs on your development system.

The interactive computer dialogue in this section consists of commands you enter, which are immediately echoed on the console display, and text displayed by the operating system, the PL/M-86 compiler, and other Intel-supplied programs. The text you enter is shown in reverse type (white on a black background), and the text displayed by Intel programs is shown in normal black type. The notation `<cr>` stands for the RETURN key on the console keyboard. Note that the operating system prompt (indicating that it is ready to accept a command) and, for some systems, the name of the loader (e.g., RUN on the Series III) are *not* included. See the appendix for your specific operating system for details.

The main program, to be compiled as a module named "M," does little but define some data and then call the procedure named SORTPROC. This procedure is defined in the other module, which is to be compiled with the name SORTMODULE. To prepare this sample program for execution, first type it in using a text editor.

Then, compile each module separately. For example, if the file PROG1A.SRC contains the module SORTPROC, then you would invoke the compiler with the command:

```
PL/M86 PROG1A.SRC <cr>
```

The compiler responds on the console with a sign-on message:

```
system-id PL / M - 86  COMPILER  Vx.y
```

where

<i>system-id</i>	is the name of your operating system.
<i>x.y</i>	is the version number of the compiler.

This is followed by the console sign-off message:

```
PL / M - 86  COMPILATION COMPLETE.  n WARNINGS,  m ERRORS
```

where

<i>n</i> and <i>m</i>	represent, respectively, the number of warnings and nonfatal errors encountered during compilation.
-----------------------	---

This process would then be repeated with the file PROG1B.SRC, which contains the module M.

Next, link the resulting object modules using the LINK86 command

```
LINK86 PROG1A.OBJ, PROG1B.OBJ TO PROG1.86 BIND <cr>
```

LINK86 displays the sign-on message

```
system-id 8086  LINKER  Vx.y
```

processes your program and returns control to the operating system.

Finally, you can execute your program with the command

```
PROGRAM <pr>
```

```
system-id PL/M-86 V2.1 COMPILATION OF MODULE SORTMODULE
OBJECT MODULE PLACED IN :F1:PROG1A.OBJ
COMPILER INVOKED BY: :F1:PLM86.86 :F1:PROG1A.SRC
```

```

2      1      SORTMODULE: DO; /*Beginning of module*/
3      2      SORTPROC: PROCEDURE(PTR, COUNT, RECSIZE, KEYINDEX) PUBLIC;
              DECLARE PTR POINTER, (COUNT, RECSIZE, KEYINDEX) WORD;

              /* Parameters:
              PTR is pointer to first record
              COUNT is number of records to be sorted.
              RECSIZE is number of bytes in each record - maximum
              is 128.
              KEYINDEX is byte position within each record of a
              BYTE scalar
              to be used as sort key.*/

4      2      DECLARE RECORD BASED PTR(1) BYTE,
              CURRENT(128) BYTE,
              (I,J) WORD;
5      2      SORT: DO J = 1 TO COUNT-1;
6      3      CALL MOVB(@RECORD(J*RECSIZE),@CURRENT, RECSIZE);
7      3      I=J;
8      3      FIND: DO WHILE I>0 AND
              RECORD((I-1)*RECSIZE + KEYINDEX)>
              CURRENT(KEYINDEX);
9      4      CALL MOVB(@RECORD((I-1)*RECSIZE),
              @RECORD(I*RECSIZE), RECSIZE);

10     4      I=I+1;
11     4      END FIND;
12     3      CALL MOVB(@CURRENT, @RECORD(I*RECSIZE), RECSIZE);
13     3      END SORT;
14     2      END SORTPROC;
15     1      END SORTMODULE; /*End of Module*/
```

MODULE INFORMATION:

```

CODE AREA SIZE      = 00A5H      165D
CONSTANT AREA SIZE = 0000H        0D
VARIABLE AREA SIZE = 0084H     132D
MAXIMUM STACK SIZE = 000CH      12D
30 LINES READ
0 PROGRAM WARNINGS
0 PROGRAM ERRORS
```

DICTIONARY SUMMARY:

```

27KB MEMORY AVAILABLE
3KB MEMORY USED      (11%)
0KB DISK SPACE USED
```

END OF PL/M-86 COMPILATION

Figure 1-1a. Sample Program: Module Sort (module 1 of 2)

```

system-id PL/M-86 V2.1 COMPILATION OF MODULE M
OBJECT MODULE PLACED IN :F1:PROG1B.OBJ
COMPILER INVOKED BY: :F1:PLM86.86 :F1:PROG1B.SRC

```

```

M: DO; /*Beginning of module*/

                /* Program to sort two sets of records, using SORTPROC */
2   1   SORTPROC: PROCEDURE (PTR, COUNT, RECSIZE, KEYINDEX) EXTERNAL;
3   2       DECLARE PTR POINTER, (COUNT, RECSIZE, KEYINDEX) WORD;
4   2   END SORTPROC; /* End of usage declaration */

5   1   DECLARE SET1(50) STRUCTURE(ALPHA WORD,
                                BETA(12) BYTE,
                                GAMMA INTEGER,
                                DELTA REAL,
                                EPSILON BYTE);

                /* Key of Nth record in SET1 is SET1(N).BETA(0), the 3rd byte
                in the record. */

6   1   DECLARE SET2(500) STRUCTURE(ITEMS(21) INTEGER,
                                KEY BYTE);

                /* Key of Nth record in SET2 is SET2(N).KEY, the 43rd byte in
                the record. */

                /* Data is read to initialize the records. */

7   1   CALL SORTPROC(@SET1, LENGTH(SET1), SIZE(SET1(1)), 2);
8   1   CALL SORTPROC(@SET2, LENGTH(SET2), SIZE(SET2(1)), 42);

                /* Data is written out from the records. */

9   1   END M; /* End of module */

```

MODULE INFORMATION:

```

CODE AREA SIZE      = 0029H      41D
CONSTANT AREA SIZE = 0000H       0D
VARIABLE AREA SIZE = 5816H    22550D
MAXIMUM STACK SIZE = 000AH      10D
32 LINES READ
0 PROGRAM WARNINGS
0 PROGRAM ERRORS

```

DICTIONARY SUMMARY:

```

27KB MEMORY AVAILABLE
3KB MEMORY USED    (11%)
0KB DISK SPACE USED

```

Figure 1-1b. Sample Program: Module Sort (module 2 of 2)



PL/M-86 programs are written free-form, meaning there is no significance to where a statement is placed on an input line, and blanks can be freely inserted between the elements of the program.

2.1 PL/M-86 Character Set

The character set used in PL/M-86 is a subset of the ASCII character set, as follows:

ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
0123456789

along with the special characters

= . / () + — ' * , < > ; : @ \$ _

and the blank or space; plus the tab, carriage-return, and line-feed characters.

The rules in this section apply to everything in a PL/M-86 program except character string constants (section 2.4), and comments (section 2.5).

If a PL/M-86 program contains any character that is not in the set above, the compiler treats it as an error (or, in some cases, a warning only; see Chapter 14).

Uppercase and lowercase letters are not distinguished from each other except in string constants. For example, xyz and XYZ are interchangeable. In this manual, all PL/M-86 code is in uppercase letters to help distinguish it from explanatory text.

Blanks are not distinguished from each other except in string constants. The compiler treats any unbroken sequence of blanks as a single blank.

Special characters and combinations of them have particular meanings in a PL/M-86 program, as described in the remainder of this manual.

Here is a concise glossary of special characters and combinations:

Table 2-1. PL/M-86 Special Characters

Symbol	Name	Use
=	equal sign	Two distinct uses: (1) assignment operator (2) relational test operator
:=	assign	embedded assignment operator
@	at-sign	location reference operator
.	dot	Three distinct uses: (1) decimal point (2) structure member qualification (3) address operator
/	slash	division operator
/*		beginning-of-comment delimiter
*/		end-of-comment delimiter

Table 2-1. PL/M-86 Special Characters (Cont'd.)

Symbol	Name	Use
(	left paren	left delimiter of lists, subscripts, and some expressions
)	right paren	right delimiter of lists, subscripts, and some expressions
+	plus	addition operator or unary plus operator
—	minus	subtraction or unary minus operator
'	apostrophe	string delimiter
*	asterisk	multiplication operator, implicit dimension specifier
<	less than	relational test operator
>	greater than	relational test operator
<=	less or equal	relational test operator
>=	greater or equal	relational test operator
=	equal	relational test operator
<>	not equal	relational test operator
:	colon	label delimiter
;	semicolon	statement delimiter
,	comma	list element delimiter
_	underscore	significant character in identifier
\$	dollar sign	non-significant character in numbers or identifiers

2.2 Tokens, Separators, and the Use of Blanks

Just as an English sentence is made up of words—the smallest meaningful units of English—so a PL/M-86 statement is made up of *tokens*. Every token belongs to one of the following classes:

- Identifiers
- Reserved words
- Simple delimiters (all of the special characters, except the dollar sign, are simple delimiters.)
- Compound delimiters—these combinations of two special characters:
 $<>$ $<=$ $>=$ $:=$ $/*$ $*/$
- Numeric constants
- Character string constants

For the most part, it is obvious where one token ends and the next one begins. For example, in the assignment statement:

```
EXACT=APPROX*(OFFSET-3)/SCALE;
```

EXACT, APPROX, OFFSET, and SCALE are identifiers, 3 is a numeric constant, and all the other characters are simple delimiters.

Sometimes a simple or compound delimiter does not occur between two identifiers, reserved words, or numeric constants—e.g., DECLAREABYTE. In these cases, a blank must be placed between them as a separator. (Instead of a single blank, any unbroken sequence of blank characters may be used.)

Also, a comment (see section 2.5) may be used as a separator.

Blanks may be inserted freely around any token without changing the meaning of the PL/M-286 statement. Thus, the assignment statement:

```
EXACT = APPROX * ( OFFSET - 3 ) / SCALE ;
```

is equivalent to:

```
EXACT=APPROX*(OFFSET-3)/SCALE ;
```

2.3 Identifiers and Reserved Words

Identifiers are used to name variables, procedures, symbolic constants, and statement labels. Identifiers may be up to 31 characters in length. The first character must be alphabetic, and the remainder may be either alphabetic, numeric, or the underscore (_).

Embedded dollar signs are totally ignored by the compiler, and may be used freely to improve the readability of an identifier or constant (although the \$ may not be the first character). An identifier or constant containing a dollar sign is exactly equivalent to the same identifier with the dollar sign deleted.

Examples of valid identifiers are:

```
INPUT_COUNT
X
GAMM
LONGIDENTIFIERWITHNUMBER3
LONG$$$IDENTIFIER$$$NUMBER$$$3
INPUT$COUNT
INPUTCOUNT
```

The long identifiers are identical (to the compiler). The last two examples are interchangeable, but are different from the first.

Certain *reserved words* must not be used as identifiers because they are actually part of the PL/M-86 language. These are listed in Appendix A.

There is also a set of predeclared identifiers naming built-in procedures and variables. You are permitted to declare these names for your own purposes, but when you do so the built-in value or procedure becomes inaccessible. Appendix A lists these identifiers.

2.4 Constants

A constant is a value that does not change during your program's execution. There are three types of constants: whole-number constants, floating-point constants, and character strings.

Whole-Number Constants

Whole-number constants can be binary, octal, decimal, or hexadecimal. The compiler recognizes these by a suffix of B, O (or Q), D, or H. Numbers without a suffix are considered decimal. If a constant contains characters invalid in the designated number base, it will be flagged as an error.

For example, the maximum whole-number WORD constant is:

$$2^{16}-1 = 1111\$1111\$1111\$1111\text{B} = 177777\text{Q} = 65535\text{D} = 0\text{FFFFH}$$

The first character of a hexadecimal number must be a numeric digit to avoid looking like an identifier. For example, the hexadecimal representation for 163 must be written 0A3H rather than A3H, which would be taken as an identifier.

Examples of valid whole-number constants:

12AH 2 33Q 1010B 55D 0BF3H 65535 777O 3EACH 0F76C05H

Examples of invalid whole-number constants:

12AF	Hexadecimal digits used without an H suffix, hence invalid in the default decimal interpretation.
12AD	Here the final D could be a suffix but the A is not a decimal digit. If hexadecimal is intended, a final H is needed.
11A2B	'A' and '2' are not valid binary digits. If hexadecimal is intended, a final H is necessary.
2ADGH	'G' is not a valid hexadecimal digit.

A whole-number constant can be a BYTE, WORD, DWORD, or INTEGER value depending on its size and context, as explained in Chapter 3. INTEGER context means a signed value from -32768 to $+32767$. POINTER context means the three restricted cases that allow an actual numeric address; the maximum value for these is 1,048,575 (see section 3.4).

Note that a minus sign in front of a constant is not part of the constant. An INTEGER value may be negative, but the range of whole-number constants is non-negative.

Floating-Point Constants

The presence of a decimal point in a decimal constant creates a floating-point constant, i.e., a number of type REAL. Only decimal REALs are allowed.

At least one decimal digit (e.g., 0) must precede the decimal point. A fractional part is optional after the decimal point, as is the base-ten exponent, which is indicated by the letter E. This exponent must have at least one digit. Note that no fractional exponents are possible. The largest REAL constant value is $3.37 \times 10^{+38}$, and the smallest REAL is 1.17×10^{-38} .

Examples of valid REAL constants:

5.3	176.0	1.88	3.14159	0.15	16.	222.2
53.0E-1	1.760E2	0.188E1	314159E-5	1.5E-1	1.6E+1	2.222E+2

The exponents in the third and sixth examples are the same; plus signs don't change the meaning.

Examples of invalid REAL constants:

6	No decimal point
1.3AH	Hexadecimal not allowed in REALs
10.011B	Binary not allowed in REALs
7.52Q	Octal not allowed in REALs
4.8E1AH/2	Only decimal constants in exponents—no hexadecimal, no expressions, no fractions

Character Strings

Character strings are denoted by printable ASCII characters enclosed within apostrophes. To include an apostrophe in a string, write it as two apostrophes; e.g., the string "'Q'" comprises 2 characters, an apostrophe followed by a Q. Spaces are allowed but line-feeds are not. The compiler represents character strings in memory as ASCII codes, one 7-bit character code to each 8-bit byte, with a high-order zero bit. Strings of length 1 translate to single-byte values. Strings of length 2 translate to double-byte values, and those of length 3 or 4 translate to double-word values. For example:

'A' is equivalent to 41H
'AG' is equivalent to 4147H
'AGR' is equivalent to 414752H
'AGRX' is equivalent to 41475258H

(See ASCII code table in Appendix F.)

Therefore, character strings can be used only as BYTE, WORD, or DWORD values since strings longer than 4 characters would exceed the 32-bit capacity of a DWORD value. As constants, however, longer character strings are stored as a sequence of bytes and can be used in a PL/M-86 program (see Chapter 3).

The maximum length of a string constant is 255 characters. It can be used only as an initialization for an array or as part of a location reference pointing to where that string constant is stored.

2.5 Comments

Explanatory *comments* may be interleaved with PL/M-86 program text to improve readability and provide program documentation. A PL/M-86 comment is a sequence of characters delimited on the left by the character pair `/*` and on the right by the character pair `*/`. These delimiters instruct the compiler to ignore any text between them and to consider such text not a part of the program proper.

A comment may contain any printable ASCII character and may also include space, carriage-return, line-feed, and tab characters.

A comment may not be embedded inside a character string constant, i.e., it will become part of the string and the compiler won't recognize it. Apart from this, it may appear anywhere that a blank character may appear—that is, anywhere except embedded within a token. Thus, comments may be freely distributed throughout a PL/M-86 program.

Here is a sample PL/M-86 comment:

```
/*This procedure copies one structure to another.*/
```

In this manual, comments are presented in uppercase and lowercase to help distinguish them visually from program code, which is always presented in uppercase.



CHAPTER 3

DATA DECLARATIONS, TYPES, AND BASED VARIABLES

Four types of objects can be declared to have symbolic names: variables, constants, labels, and procedures. One declaration must be available for each name used in a block—no more, no less. This declaration may appear at the beginning of the block, or in an enclosing outer block. Multiple declarations of the same name in the same block are invalid.

Only after being declared and defined can names for the four elements above be used in executable statements. For variables, constants, and labels, such usage is, in essence, an operational definition. For a procedure, the set of statements between its declaration and its end statement constitutes its definition.

In addition to the item's name, a declaration tells its type, attributes, and/or location. These terms will be clarified in the course of this chapter.

Table 3-1 shows the general appearance of declarations, e.g., required or optional elements.

3.1 Variable Declaration Statements

A DECLARE statement is a non-executable statement that introduces some object or collection of objects, associates names (and sometimes values) with them and allocates storage if necessary. The most important use of DECLARE is for declaring variables.

A variable may be a *scalar*—that is, a single quantity—an *array*, or a *structure*.

A scalar variable is a single object whose value is not necessarily known at compile time and may change during the execution of the program. You therefore refer to it by declaring a name to be used in the program. This name is known as an identifier.

Table 3-1. Declaration Elements

Declaration Statements For	Must Use	Can Use
Variable Names	type: BYTE, WORD, DWORD, INTEGER, POINTER, SELECTOR, REAL, or STRUCTURE	linkage attributes: PUBLIC or EXTERNAL location attributes: AT (location reference) variable initialization attribute: INITIAL (value-list)
Execution Constant Names	type, as above, and constant initialization attribute: DATA (value-list)	linkage attributes as above
Label Names	LABEL	linkage attributes as above
Compilation Constant Names	LITERALLY 'string'	

An array is a list of scalars all named by the same identifier, differentiated from each other by the use of *subscripts*, e.g., A(0), a(1), A(123), and so on.

A structure is a list of scalars and/or arrays that all use the same main identifier and that can be differentiated from each other by their own *member-identifiers* (field names). For example, EMPLOYEES.NAME could refer to the NAME field within the structure EMPLOYEES.

Such variables (arrays and structures) are discussed in greater detail in Chapter 5.

Types are described in section 3.4.

Examples

The following statements declare scalars:

```
DECLARE APPROX REAL;  
DECLARE (OLD, NEW) BYTE;  
DECLARE POINT WORD, VAL12 BYTE;
```

The first example declares a single scalar variable of type REAL, with the identifier (name) APPROX. The second example declares two scalars, OLD and NEW, both of type BYTE. This kind of statement is called a "factored declaration." It is equivalent to the sequence:

```
DECLARE OLD BYTE;  
DECLARE NEW BYTE;
```

except that the factored declaration guarantees the bytes will be contiguous; separated declaration statements do not.

The third example declares two scalars of different types: POINT is of type WORD, and VAL12 is of type BYTE.

The following statements declare arrays:

```
DECLARE DOMAIN (128) BYTE;  
DECLARE GAMMA (19) DWORD;
```

The first statement declares the array DOMAIN, with 128 scalar elements, each of type BYTE. These elements are distinguishable by subscripting the name DOMAIN, using the range 0 to 127 for the subscripts. For example, the third element of DOMAIN can be referred to as DOMAIN(2). The first element of every array has subscript 0.

The second statement declares the array GAMMA, with 19 scalar elements of type DWORD. The subscripts for this array can range from 0 to 18.

The next statement declares a structure with two scalar members:

```
DECLARE RECORD STRUCTURE (KEY BYTE, INFO WORD);
```

The two members are a BYTE scalar that can be referred to as RECORD.KEY and a WORD scalar that can be referred to as RECORD.INFO. The word named by RECORD.INFO is the second and third bytes of this structure.

Further discussion of structures appears in sections 3.4 and 3.6, and in Chapter 5.

Results

The two results of a valid variable declaration are:

1. The name is given a unique address.
2. The variable is considered to have the attributes declared.

This means that all subsequent uses of the variable in the block where it is declared will refer to the same address (except for based variables, discussed in section 3.5).

A valid variable declaration also requires all references to the variable to conform to the rules for the current attributes, i.e., those having priority in the current block. Thus, the compiler can flag a large variety of errors that are caused by incompatible declarations within the current block.

Combining DECLARE Statements

A separate DECLARE statement is not required for each and every declaration. Instead of writing the two DECLARE statements:

```
DECLARE CHR BYTE INITIAL ('A');
DECLARE COUNT INTEGER;
```

we may write both declarations in a single DECLARE statement, as follows:

```
DECLARE CHR BYTE INITIAL ('A'), COUNT INTEGER;
```

This declare statement contains two “declaration elements,” separated by the comma. Every DECLARE statement contains at least one declaration element. If it contains more than one, they are separated by commas.

Previously, most examples have shown only one declaration element in each DECLARE statement. A declaration element is the text for declaring one identifier (or one factored list of identifiers). In the example above, the text `CHR BYTE INITIAL ('A')` is one declaration element, and the text `COUNT INTEGER` is another.

Another way of combining declaration elements is called a factored declaration. For example:

```
DECLARE A BYTE, B BYTE;
DECLARE C WORD, D WORD;
DECLARE E DWORD, F DWORD;
```

can be combined as

```
DECLARE (A,B) BYTE, (C,D) WORD, (E,F) DWORD;
```

In each factored declaration, the allocated locations will be contiguous; elements declared in a non-factored declaration statement may not be.

Variables declared in a “factored” declaration that are not based, and not used as parameters—that is, variables within a parenthesized list—are stored contiguously in the order specified. (If a based variable occurs in a parenthesized list, it is ignored when storage is allocated.)

The declaration elements appearing in a single DECLARE statement are completely independent of each other, as if they were declared in separate DECLARE statements.

3.2 Initializations

Initialization is necessary for every constant and variable name that is used (read) before it is filled (written) during execution since there are no default values. Either kind of name can, of course, be initialized by an assignment statement such as:

```
PI = 3.1415927; /* PI must first be declared REAL. */
VAR13 = 10; /* VAR13 must be declared earlier, not real. */
```

However, the PL/M-86 language provides a means for having the compiler set up these values during the compilation rather than using both instruction space and execution time in your program to do so.

There are two kinds of compile-time initializations: **INITIAL**, used with variables, and **DATA**, used for constants. (**DATA** is explained in greater detail later in this section.) In each case, the initialization attribute is placed after the type in the declaration. For example:

```
DECLARE FAMILY WORD INITIAL (2);
```

INITIAL causes initialization to occur during program loading for variables that have storage allocated for them. Such variables can subsequently be changed during execution, like any other variable. (They will *not* be reinitialized on a program restart.)

The following rules apply to *both* **INITIAL** and **DATA**:

- **INITIAL** and **DATA** may not be used together in the same declaration.
- **INITIAL** may only appear in declarations at the outer level of a module. **DATA**, however, may appear in declarations at any level.
- No initializations are permitted with based variables (discussed in section 3.5), formal parameters (discussed in section 8.1) or with the **EXTERNAL** attribute (discussed in section 7.2).
- Either initialization may follow use of the **AT** attribute (discussed in section 3.6), but if this causes multiple initializations, the result cannot be predicted.

The general form of the **INITIAL** attribute is:

```
INITIAL (value-list)
```

where

value-list is a sequence of values separated by commas.

Values are taken one at a time from the value list and used to initialize the individual scalars being declared. The initialization is performed in the same manner as an assignment. Initial values for members of an array or structure must be specified explicitly. (See also section 9.4 for built-in procedures you can use to initialize **BYTE** and **WORD** strings at run-time.)

Each value may be a string of up to four characters (e.g., 'A', 'NO'), or a *restricted expression*, as explained below. (Byte arrays can accommodate longer strings since each element can represent one character.)

A restricted expression is one of the following three possibilities:

- For **REAL** variables only: a single floating-point constant, with no operator of any kind, to be used *only* to initialize a **REAL** scalar.
- For **POINTER** variables only: a location reference formed with the **@** operator, which must refer to a variable that has already been declared. (Location references are discussed in section 3.4.)

- For all other types: a constant expression containing no operators except + or −. A constant expression has only whole-number constants as operands, e.g., 2048−256+5, as explained in Chapter 4. The expression is evaluated as if it were being assigned to the scalar being initialized, using the rules of that chapter.

NOTE

For compatibility with programs written in PL/M-80, PL/M-86 allows the restricted expression to be an expression containing a location reference formed with the “dot” operator.

The declaration:

```
DECLARE THRESHOLD BYTE INITIAL (48);
```

declares the BYTE scalar THRESHOLD in the usual way, and also initializes it to a value of 48.

The declaration:

```
DECLARE EVEN (5) BYTE INITIAL (2, 4, 6, 8, 10);
```

declares the BYTE array EVEN and initializes its five scalar elements to 2, 4, 6, 8, and 10, respectively.

The declaration:

```
DECLARE COORD STRUCTURE (HIGH$BOUND WORD,
    VALUE (3) BYTE,
    LOW$BOUND BYTE) INITIAL (302, 3, 6, 12, 0);
```

declares the structure COORD and initializes it as follows:

```
COORD.HIGH$BOUND to 302
COORD.VALUE(0) to 3
COORD.VALUE(1) to 6
COORD.VALUE(2) to 12
COORD.LOW$BOUND to 0
```

If a string appears in the value list, it is taken apart from left to right and the pieces are stored in the scalars being initialized. One character is stored in each BYTE scalar, two in each WORD scalar, and four in each DWORD scalar. For example:

```
DECLARE GREETING (5) BYTE AT (0H) INITIAL ('HELLO');
```

causes GREETING(0) to be initialized with the ASCII code for H, GREETING(1) with the ASCII code for E, and so on.

So far, all the examples have shown value lists that match up one-for-one with the scalars being declared. It is permissible for the value list to have *fewer* elements than are being declared. Thus:

```
DECLARE DATUM (100) BYTE INITIAL (3, 5, 7, 8);
```

is permissible. The first four elements of the array DATUM are initialized with the four elements in the value list, and the remainder of the array is left uninitialized. However, the value list may not have *more* elements than are being declared.

The Implicit Dimension Specifier

Often, when you initialize an array, you want the array to have the same number of elements as the value list. This can be done conveniently by using the *implicit dimension specifier* in place of an ordinary dimension specifier (a parenthesized constant). The implicit dimension specifier has the form:

(*)

This may also be used to define an external array whose precise number of elements is either unknown or insignificant. Thus the declaration:

```
DECLARE FAREWELL ( * ) BYTE PUBLIC INITIAL ( 'GOODBYE, NOW' );
```

declares a public BYTE array, FAREWELL, with enough elements to contain the string 'GOODBYE, NOW' (namely 12), and initializes the array elements with the characters of the string. To reference this array in another program module, you can declare it as follows:

```
DECLARE FAREWELL ( * ) BYTE EXTERNAL ;
```

(See Chapter 7 for more information about PUBLIC and EXTERNAL attributes.) Note that the INITIAL and DATA value-lists must not be present when the implicit dimension specifier is used with an external array; otherwise, INITIAL and DATA value-lists are required. Also, the LENGTH, LAST, and SIZE built-ins (see section 9.1) may not be used on an external array that was declared with the implicit dimension specifier.

The implicit dimension specifier may not be used in the following cases:

- After the parenthesized list of identifiers in a factored declaration.
- To specify an array whose elements are structures.
- To specify an array that is a member of a structure.

The implicit dimension specifier may be used with any value list—it is not restricted to strings.

Names for Execution Constants: The Use of DATA

As discussed above, a *variable* is the name of a single data item intended to be used and altered by your program. If it isn't altered during execution, it's a *constant*.

For example, the formula for the circumference of a circle as the product of its radius and two pi could be written in PL/M-86 as:

```
C = R * 2.0 * 3.14159 ;
```

in which C and R would be variables. Their declarations would of course have to precede the above executable statement, and could appear as:

```
DECLARE ( C, R ) REAL ;
```

If pi were used often enough, you might wish to simplify the writing of statements using it to declare a symbolic name with that value:

```
DECLARE PI REAL DATA ( 3.1415927 ) ;
```

An array of constants would require a list of values, for example:

```
DECLARE FIBONACCI(9) BYTE DATA (0,1,1,2,3,5,8,13,21);
```

The form and use of the DATA initialization is *identical* to that of INITIAL except for these four differences:

- DATA causes storage to be allocated in the program's constant data segment. The content and meaning of the name cannot be changed during execution. The name should never appear on the left-hand side of an assignment statement. (This is not the case with INITIAL.)
- Data initializations can be used in declarations at any block level in the program. (INITIAL can only appear at the module level, that is, inside the DO-block that is the module itself, but outside any sub-blocks that the module may contain.)
- If the keyword DATA is used in a PUBLIC declaration when compiling with the ROM option (see section 11.4), DATA must also be used in the EXTERNAL declaration of program modules that reference it. However, no value list is then permitted since the data is defined elsewhere. (INITIAL cannot be combined with EXTERNAL.)
- Use of the AT attribute, as explained in section 3.6. This attribute forces a name to be associated with a specific memory location, which can defeat the purpose of the DATA initialization. (This will not happen with INITIAL unless you explicitly redefine your own variables and location using multiple AT's.)

3.3 Types of Declaration Statements

Compilation Constants (Text Substitution): The Use of LITERALLY

If your program were large enough to have many declarations, you might choose to declare a compilation constant to save time at the keyboard:

```
DECLARE DC LITERALLY 'DECLARE';
```

Thereafter, during compilation, every time DC appears alone (not as part of a word), the full string DECLARE will be substituted by the compiler. Subsequent declarations can thus be written:

```
DC AREA REAL;
DC SIZE WORD;
```

A declaration using the reserved word LITERALLY defines a parameterless "macro" for expansion at compile-time. You declare an identifier to represent a character string, which will then be substituted for each occurrence of the identifier in subsequent text. This expansion will not take place in strings or constants. The form of the declaration is:

```
DECLARE identifier LITERALLY 'string' ;
```

where

<i>identifier</i>	is any valid PL/M-86 identifier.
<i>string</i>	is a sequence of arbitrary characters from the PL/M-86 set, not exceeding 255 in length.

The following example illustrates another use of this facility:

```

DECLARE TRUE LITERALLY 'OFFH', FALSE LITERALLY '0';

DECLARE ROUGH BYTE;
DECLARE (X, Y, DELTA, FINAL) REAL;
...
ROUGH = TRUE;
DO WHILE ROUGH;
    X = SMOOTH (X, Y, DELTA);
    /* SMOOTH is a procedure declared elsewhere. */
    IF (X-FINAL) < DELTA THEN ROUGH = FALSE;

END;
...

```

This example of a **LITERALLY** declaration defines the boolean values **TRUE** and **FALSE** in a manner consistent with the way PL/M-86 handles relational operators (see section 4.3). This kind of literal substitution for fixed values often makes a program more readable.

Another use of the **LITERALLY** declaration is the declaration of quantities that are fixed for one compilation, but are subject to change from one compilation to the next. Consider the example below:

```

DECLARE BUFFER$SIZE LITERALLY '32';
DECLARE PRINT$BUFFER(BUFFER$SIZE) WORD;
...
PRINT$BUFFER(BUFFER$SIZE - 10) = 'G';
...

```

A future change to **BUFFER\$SIZE** can be made in one place, at the first declaration, and the compiler will propagate it throughout the program during compilation. Thus, the programmer is saved the tedious and error-prone process of searching the program for the occurrences of "32" that are buffersize references and not 32's that are used in other contexts.

Declarations for Names of Labels

A *label* marks the location of an instruction as opposed to a data item. Labels are permitted only on executable statements, not on declarations.

There are two ways to declare a name as a label. *Explicit* label declarations are used mainly to allow for module-to-module references, which are discussed in detail in Chapter 9. The three possible forms for explicit label declarations look like this:

```

DECLARE PART3 LABEL;
DECLARE START1 LABEL PUBLIC; /* for intermodule reference */
DECLARE PHASE2 LABEL EXTERNAL; /* for intermodule reference */

```

The rules for the latter two are discussed in Chapter 9.

The more common label declaration, *implicit*, is even simpler: the name is placed at the very beginning of the executable statement it is supposed to point to. For example:

```

START2: ALPHA = 127;

```

This statement *defines* the label START2 as pointing to the location of the PL/M-86 instruction shown. If this block has no explicit declaration of START2, i.e., no statement like:

```
DECLARE START2 LABEL ;
```

then the compiler takes the definition of START2 as an implicit declaration as well as a definition. It is as if the declaration had occurred at the start of the last simple DO or procedure statement. (If there *is* an explicit declaration, then the actual placement of the label remains simply a definition.)

Labels are used to indicate significant instructions or the starting point of instruction sequences. Labels can be useful reference points for understanding the parts of a program, or targets for the transfer of control during execution (as discussed under GOTO and CALL in Chapters 7 and 8).

Results

The results of a valid label declaration are:

1. The declared name can be used to point to an executable instruction.
2. The use of the declared name as a variable in its block is disallowed.
3. If the label is also *defined* in its block by appearing in an executable statement, then the address of that statement is assigned as the value of the label.

Declaration for Procedures

As illustrated earlier, the declaration of a procedure begins by giving its name, with a statement of the form:

name: PROCEDURE

followed optionally by parameters, type and/or attributes. The definition of the procedure then follows, i.e., the set of statements declaring items used in the procedure (including any parameters) and the executable statements of the procedure itself. The definition ends with an END statement, optionally including the procedure name from the declaration.

The complete declaration of a procedure includes all the statements from the PROCEDURE statements through the END statement. This whole definition/declaration must appear before the procedure name is used in an executable statement, just as variable and constant names must be declared before their use.

The only exceptions arise when the full definition appears in another module where it is declared PUBLIC, or when a procedure has been declared REENTRANT. In the first case, if a separate module intends to make use of that public definition, it is required to:

1. Declare the procedure as having the EXTERNAL attribute (so the Linker will search for it).
2. Declare each formal parameter the procedure uses so the compiler can verify correct usage when this module invokes the procedure.
3. End this local declaration with an END statement.

For example:

```
SUMMER: PROCEDURE (A, B) EXTERNAL;  
    DECLARE A WORD, B BYTE;  
END SUMMER;
```

The full details for intermodule references appear in Chapter 7, and the discussion of procedure definition and usage appears in Chapter 8.

3.4 Data Types

The concept of data types applies not only to variables, but to every value processed by a PL/M-86 program. This includes values returned by procedures and values calculated by processing expressions.

PL/M-86 performs calculations using three different kinds of arithmetic: unsigned, signed, or floating-point, depending on the data types involved.

Arithmetic and other expressions using the different types are discussed in detail in Chapter 4.

BYTE, WORD, and DWORD Variables: Unsigned Arithmetic

The value of a BYTE variable is an 8-bit binary number ranging from 0 to 255 and occupying one byte of iAPX 86 memory (two bytes when passed as a parameter on the stack, see Appendix F). The value of a WORD variable is a 16-bit binary number ranging from 0 to 65535 and occupying two contiguous bytes of iAPX 86 memory (the least significant 8 bits are stored in the lower address). The value of a DWORD variable is a 32-bit binary number ranging from 0 to 4,294,967,295 and occupying two contiguous words of iAPX 86 memory (the least significant 16 bits are stored in the lower address). Values of DWORD, WORD, and BYTE variables are treated as *unsigned binary integers*.

NOTE

Support for *, /, and MOD operations on DWORD is located in the PLM86.LIB support library. You must link to this library before executing any program that performs these operations on DWORD types. (The DWORD routines in this library form only one segment: a code segment called LQ_PLM86_LIB_CODE. Since this is read-only code, it may be burned into ROM.)

Unsigned integer arithmetic is used to perform any arithmetic operation on DWORD, WORD, and BYTE variables. All of the PL/M-86 operators may be used with them (see Chapter 4). Arithmetic and logical operations on such variables yield a result of type BYTE, WORD, or DWORD, depending on the operation and the operands. Relational operations always yield a “true” or “false” result of type BYTE.

With unsigned arithmetic, if a large value is subtracted from a smaller one, the result is the two's complement of the absolute difference between the two values. For example, if a BYTE value of 1 (00000001 binary) is subtracted from a BYTE value of 0 (00000000 binary), the result is a BYTE value of 255 (11111111 binary).

Also, the result of a division operation is always truncated (rounded down) to a whole number. For example, if a WORD value of 7 (0000000000000111 binary) is divided by a BYTE value of 2 (00000010 binary), the result is a WORD value of 3 (0000000000000011 binary).

When declaring a variable that may be used to hold or produce a negative result, it is advisable to make it either INTEGER or REAL. If it is supposed to hold or produce a non-integer, it *must* be REAL. Making use of the appropriate data types can help avoid unexpected incorrect results from arithmetic operations (see Chapter 4).

INTEGER Variables: Signed Arithmetic

INTEGER variables represent 16-bit signed integers ranging from -32768 to $+32767$ and occupying two contiguous bytes of iAPX 86 memory (the least significant 8 bits are stored in the lower address). Internally, an INTEGER value is represented in two's complement notation with the following format:



The sign bit is 0 if the INTEGER value is positive or zero, and 1 if the value is negative. The magnitude is given in two's complement notation.

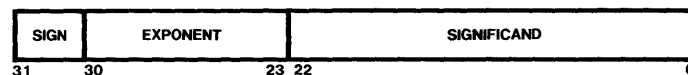
Arithmetic operations on INTEGER variables use signed integer arithmetic to hold an INTEGER result. Thus, addition and subtraction always produce mathematically correct results if overflow does not occur. (See also the OVERFLOW control in section 11.3.) Relational operations are signed arithmetic comparisons that yield a "true" or "false" result of type BYTE.

However, as with BYTE, WORD, and DWORD operands, division produces only an INTEGER result. The result is rounded toward zero, i.e., down if it is positive, up if it is negative.

Only the arithmetic and relational operators may be used with INTEGER operands. Logical operators are not allowed (see Chapter 4).

REAL Variables: Floating-Point Arithmetic

The value of a REAL variable is a signed floating-point number that occupies four contiguous bytes of iAPX 86 memory, which may be viewed as 32 contiguous bits in the short-real format. The bits are divided into fields as follows:



The byte with the *lowest* address contains the least significant 8 bits of the significand, and the byte with the *highest* address contains the sign bit and the most significant 7 bits of the exponent field.

The sign bit is 0 if the REAL value is positive or zero, and 1 if the REAL value is negative.

The exponent field contains a value "offset" by 127—in other words, the actual exponent can be obtained from the exponent field value by subtracting 127. This field is all 0's if the REAL value is zero.

The significand contains the binary digits of the fractional part of the REAL value when this part is represented in "binary scientific" notation (see below). This field is all 0's if the REAL value is zero.

Operations on REAL operands use signed floating-point arithmetic to yield a result of type REAL. The implementation guarantees that the result of each operation is the closest possible floating-point number to the exact mathematical real-number result (if overflow or underflow does not occur). The relational operators and the arithmetic operators $+$, $-$, $*$, and $/$ may be used with REAL operands; the MOD operator and the logical operators are not allowed. Arithmetic operations yield a result of type REAL, and relational operations yield a "true" or "false" result of type BYTE.

The utility of the REAL data type is extended by the PL/M-86 compiler's practice of holding intermediate results in the 8087's temporary-real format. This format preserves 64 bits of precision and the full range of representable numbers. The exponent in this format is 15 bits instead of 8 in the short-real format.

This greater range of exponent greatly reduces the likelihood of underflow and overflow, and eliminates roundoff as a source of error until the final assignment of the result is performed. These advantages arise because underflow, overflow, and roundoff errors are more probable for intermediate computations than for the final result. For example, an intermediate underflow result might later be multiplied by a very large factor, providing a final result of acceptable magnitude.

Examples of Binary Scientific Notation

1. Consider the following binary number (which is equivalent to the decimal value 10.25):

1010.01B

The "." in this number is a *binary point*. The same number can be represented as:

$1.01001B * 2^3$

This is "binary scientific" notation, with the binary point immediately to the right of the most significant digit. The digits 01001 are the fractional part, and 3 is the exponent. This value would be represented as follows:

- The sign bit would be 0, since the value is positive.
- The exponent field would contain the binary equivalent of $127 + 3 = 130$.
- The leftmost digits of the fraction field would be 01001, and the remainder of this field would be all 0's.

The complete 32-bit representation would be

0 10000010 010010000000000000000000

and the contents of the four contiguous memory bytes would be as follows:

highest address:	01000001
	00100100
	00000000
lowest address:	00000000

Note that the most significant digit is not actually represented, since by definition it is a "1" unless the REAL value is zero. If the REAL value is zero, the entire 32-bit representation is all 0's.

2. Consider the fraction $1/16$, or 0.0625. In binary, it is

$1.0000B * 2^{-4}$

The actual exponent, -4 , would be represented as 123 ($127 - 4$), and the fraction field would contain all 0's.

The largest possible value for a valid exponent field is 254, which corresponds to an actual exponent of 127. The largest possible absolute value for a positive or negative REAL value is therefore

1.1111111111111111111111111B * 2¹²⁷

or approximately $3.37 \cdot 10^{38}$.

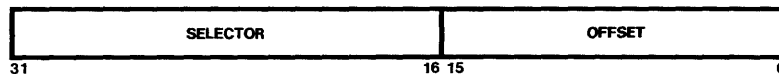
The lowest permissible exponent field value for a non-zero REAL value is 1, which corresponds to an actual exponent of -126 . The smallest possible absolute value for a positive or negative REAL value is therefore

1.0B * 2⁻¹²⁶

or approximately $1.17 \cdot 10^{-38}$.

POINTER Variables and Location References

The value of a **POINTER** variable is the address of an iAPX 86 storage location and is made up of a base selector portion and an offset portion. The bits are divided as follows:



Among other uses, **POINTER** variables are important as bases for based variables (see section 3.5).

Only the relational operators may be used with **POINTER** operands, yielding a “true” or “false” result of type **BYTE**. No arithmetic or logical operations are allowed (see Chapter 4).

There are only a few ways to create or change the value of a **POINTER** variable, that is, the address that the variable points to:

1. The variable can be initialized when declared, using **INITIAL** or **DATA** with an address created with a whole-number constant as described in section 3.2.
2. The variable can be assigned a whole-number constant as described in Chapter 4.
3. The variable can be assigned an address created via the **@** operator (described below). This is the safest and most commonly used method.
4. The variable can be assigned the value of a **POINTER** variable or function.
5. The variable can be assigned a value generated by the **BUILD\$PTR** or **NIL** functions (described in section 9.6).

The @ Operator

A “location reference” is formed by using the @ operator. A location reference has a value of type **POINTER**—that is, a location address. An important use of location references is to supply values for **POINTER** variables.

The basic form of a location reference is:

- **variable-ref**

where

variable-ref is the name of some variable.

The value of this location reference is the actual run-time location of the variable.

The *variable-ref* may also refer to an unqualified array or structure name. The pointer value is the location of the first element or member of the array or structure.

For example, suppose that we have the following declarations:

```
DECLARE RESULT REAL;
DECLARE XNUM(100) BYTE;
DECLARE RECORD STRUCTURE (KEY BYTE,
    INFO(25) BYTE,
    HEAD POINTER);
DECLARE LIST(128) STRUCTURE (KEY BYTE,
    INFO(25) BYTE,
    HEAD POINTER);
```

The @RESULT is the location of the REAL scalar RESULT, while @XNUM(5) is the location of the 6th element of the array XNUM. @XNUM is the location of the beginning of the array, that is, the location of the first element (element 0).

The RECORD structure declares a byte called KEY followed by 25 bytes called INFO(0), INFO(1), and so on, followed by the POINTER variable named HEAD. Since KEY, INFO, and HEAD are all declared part of the RECORD structure, their contents must be referred to as RECORD.KEY, RECORD.INFO(0), ..., RECORD.INFO(24), and RECORD.HEAD.

The addresses of these elements of the RECORD structure can be referred to using the @ operator. @RECORD.HEAD is the location of the POINTER scalar RECORD.HEAD, while @RECORD is the location of the structure, which is the same as that of the BYTE scalar RECORD.KEY. @RECORD.INFO is the location of the first element of the 25-byte array RECORD.INFO, whereas @RECORD.INFO(7) is the location of the 8th element of the same array.

LIST is an array of structures. The location reference @LIST(5).KEY is the location of the scalar LIST(5).KEY. Note that @LIST.KEY is illegal since it does not identify a unique location, i.e., the KEY of *which* LIST.

The location reference @LIST(0).INFO(6) is the location of the scalar LIST(0).INFO(6). Also, @LIST(0).INFO is the location of the first element of the same array, i.e., the location of the array itself.

A special case exists when the identifier used as "*variable-ref*" is the name of a procedure. The procedure must be declared at the outer level of the program module. No actual parameters may be given (even if the procedure declaration includes formal parameters). The value of the location reference in this case is the location of the entry point of the procedure. (Further discussion of procedures appears in Chapter 8 and Appendixes F and G.)

Storing Strings and Constants via Location References. Another form of location reference is:

@ (constant list)

where

constant list is a sequence of one or more constants separated by commas and enclosed by parentheses.

When this type of location reference is made, space is allocated for the constants, the constants are stored in this space (contiguously, in the order given by the list), and the value of the location reference is the location of the first constant.

Strings may be included in the list. For example, if the operand:

```
@('NEXT VALUE')
```

appears in an expression, it causes the string 'NEXT VALUE' to be stored in memory (one character per byte, thus occupying 10 contiguous bytes of storage). The value of the operand is the location of the first of these bytes—in other words, a pointer to the string.

The “DOT” Operator

For compatibility with PL/M-80 programs, a “dot” operator is provided. The dot operator (.) is similar to the @ operator, but produces an address of type WORD that represents an offset to the current data segment (for variables) or to the current code segment (for procedures). This address should be used with caution, since it will not always produce correct results in a PL/M-86 program that contains more than one data segment or more than one code segment. See section 3.5 for indirect variable references and section 8.2 for indirect procedure calling.

SELECTOR Variables

The value of a SELECTOR variable is equivalent to the base portion of a POINTER, and may also be used as the base of a based variable (see section 3.5). The bits are divided as follows:



Only the relational operators may be used with SELECTOR operands, yielding a “true” or “false” result of type BYTE. No arithmetic or logical operations are allowed (see Chapter 4).

There are only three ways to create or change the value of a SELECTOR variable:

1. The variable can be initialized when declared, using INITIAL or DATA with a whole-number constant as described in section 3.2.
2. It can be assigned a whole-number constant as described in Chapter 4.
3. It can be assigned a SELECTOR variable or function, or the build-in function SELECTOR\$OF (see section 9.7).

The results of the @ and dot operators may not be assigned to SELECTOR variables directly. They must first be converted to SELECTOR type with the built-in function SELECTOR\$OF (see section 9.7).

3.5 Based Variables

Sometimes a direct reference to a PL/M-86 data element is either impossible or inconvenient. This happens, for example, when the location of a data element must remain unknown until it is computed at run time. In such cases, it may be necessary to write PL/M-86 code to manipulate the locations of data elements themselves.

To permit this type of manipulation, PL/M-86 uses “based variables.” A based variable is one that is pointed to by another variable called its “base.” This means the base contains the address of the desired (based) variable.

A based variable is not allocated storage by the compiler. At different times during the program run it may actually refer to different places in memory, since its base may be changed by the program.

A based variable is declared by first declaring its base, which must be of type **POINTER**, **SELECTOR**, or **WORD**, and then declaring the based variable itself:

```
DECLARE I BYTE;
DECLARE ITEM$PTR POINTER;
DECLARE ITEM BASED ITEM$PTR BYTE;
```

Given these declarations, a reference to **ITEM** is, in effect, a reference to whatever **BYTE** value is pointed to by the current value of **ITEM\$PTR**. This means that the sequence:

```
ITEM$PTR = @I;
ITEM = 77H;
```

will load the **BYTE** value of 77 (hex) into the variable **I**.

A variable is made **BASED** by inserting in its declaration the word **BASED** and the identifier of the base (which must already have been declared).

The following restrictions apply to bases:

- The base must be of type **POINTER**, **SELECTOR**, or **WORD**. However, use a base of type **WORD** with caution since it does not contain a full iAPX 86 address. **WORD**-based variables are addressed relative to the current **DS** register (see section 11.3).
- The base may not be subscripted—that is, it may not be an array element.
- The base may not itself be a based variable.

The word **BASED** must *immediately* follow the name of the based variable in its declaration, as in the following examples:

```
DECLARE (AGE$PTR, INCOME$PTR, RATING$PTR, CATEGORY$PTR)
    POINTER;
DECLARE AGE BASED AGE$PTR BYTE;
DECLARE (INCOME BASED INCOME$PTR, RATING BASED RATING$PTR)
    WORD;
DECLARE (CATEGORY BASED CATEGORY$PTR)(100) WORD;
```

In the first **DECLARE** statement, the **POINTER** variables **AGE\$PTR**, **INCOME\$PTR**, **RATING\$PTR**, and **CATEGORY\$PTR** are declared. They are used as bases in the next three **DECLARE** statements.

In the second **DECLARE** statement, a **BYTE** variable called **AGE** is declared. The declaration implies that whenever **AGE** is referenced by the running program, its value will be found at the location given by the value of the **POINTER** variable **AGE\$PTR** at the same time.

The third **DECLARE** statement declares two based variables, both of type **WORD**.

The fourth **DECLARE** statement defines a 100-element **WORD** array called **CATEGORY**, based at **CATEGORY\$PTR**. This means that when any element of **CATEGORY** is referenced at run time, the value of **CATEGORY\$PTR** at that same time is the location of the array **CATEGORY**, i.e., its first element.

The other elements follow contiguously. The parentheses around the tokens CATEGORY BASED CATEGORY\$PTR are optional. They help to make the statement more readable, but may be omitted.

Location References and Based Variables

An important use of location references is to supply values for bases. Thus, the @ operator, together with the based variable concept, gives PL/M-86 a very powerful facility for manipulating pointers.

For example, suppose that we have three different REAL variables: NORTH\$ERROR, EAST\$ERROR, and HEIGHT\$ERROR. We want to be able to refer to them at different times by means of the single identifier ERROR. This can be done as follows:

```
DECLARE (NORTH$ERROR, EAST$ERROR, HEIGHT$ERROR) REAL;
DECLARE ERROR$PTR POINTER;
DECLARE ERROR BASED ERROR$PTR REAL;
. . .
ERROR$PTR = @NORTH$ERROR;
```

At this point, the value of ERROR\$PTR is the location of NORTH\$ERROR. A reference to ERROR will, in effect, be a reference to NORTH\$ERROR. Later in the program, we can write:

```
ERROR$PTR = @HEIGHT$ERROR;
```

Now a reference to ERROR will, in effect, be a reference to HEIGHT\$ERROR. In the same way, we can cause the value of the pointer to be the location of EAST\$ERROR, and a reference to ERROR will be a reference to EAST\$ERROR.

This technique is useful for manipulating complicated data structures and for passing locations to procedures as parameters. Examples are given in Chapter 8.

3.6 The AT Attribute

The AT attribute has the form:

AT (*location*)

where

location must be a location reference formed with the @ operator, or a single whole-number constant in the range 0 to 1,048,575.

It must refer to a non-based variable that has already been declared. If there is a subscript expression, it must be a constant expression containing no operators except + and -.

The following are examples of valid AT attributes:

```
AT (4096)
AT (@BUFFER)
AT (@BUFFER(128))
AT (@NAMES(INDEX + 1))
```


In the last example, INDEX represents a whole-number constant that has been previously declared with a "LITERALLY" declaration. The compiler replaces this name with the declared whole-number constant, thus satisfying the restrictions given above.

NOTE

For compatibility with programs written in PL/M-80, PL/M-86 allows the location in an AT attribute to be an expression containing a location reference formed with the "dot" operator. (See Appendix D.)

The affect of an AT attribute is to cause the address of a variable to be the location specified by location. The first non-based variable in a factored declaration will have the address specified by location. Other variables in the same declaration will, in sequence, refer to successive locations thereafter.

For example, the declaration:

```
DECLARE (CHAR$A, CHAR$B, CHAR$C) BYTE AT (@BUFFER);
```

causes the BYTE variable CHAR\$A to refer to the location of BUFFER. The variables CHAR\$B and CHAR\$C are located in the next two bytes after CHAR\$A.

The declaration:

```
DECLARE T (10) STRUCTURE (X(3) BYTE,  
                          Y(3) BYTE,  
                          Z(3) BYTE) AT (@DATA$BUFFER);
```

sets up structure references to 90 bytes. They are organized such that each of the ten members of T refers to 9 bytes—the first three use the name X, the second three Y, and the last three Z. Figure 3-1 may help you visualize this structure.

The declaration above, using the AT attribute, causes the beginning of the structure T—namely the scalar T(0).X(0)—to be located at the same location as a previously declared variable called DATA\$BUFFER. The other scalars making up the structure will follow this location in logical order: T(0).X(1), T(0).X(2), and so on up to T(9).Z(2), the last scalar, which is located in the 89th byte after the location of DATA\$BUFFER.

However, *no memory locations* for these 90 scalars *are allocated* by this declaration. It is up to the programmer to know what else, if anything, will be stored in the memory space starting at @DATA\$BUFFER.

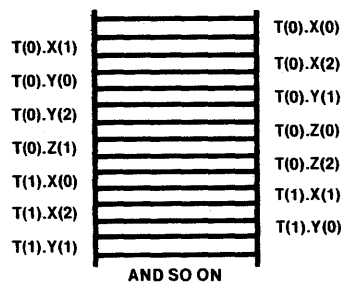


Figure 3-1. Successive Byte References of a Structure

121636-29

The following rules apply to the AT attribute:

- It cannot be used with variables that are based, EXTERNAL, or parameters.
- It can be used with the PUBLIC attribute, in which case it must immediately follow the word PUBLIC. However, the location in this case may *not* be a location reference to a variable that is EXTERNAL.

The AT attribute can be used to make variables “equivalent,” providing more than one way of referring to the same information. For example:

```
DECLARE DATUM WORD;  
DECLARE ITEM BYTE AT (@DATUM);
```

causes ITEM to be declared a BYTE variable at the same location that has just been allocated for the WORD variable DATUM. The result is that any reference to ITEM is, in effect, a reference to the *low-order* byte of DATUM (because WORD values are stored with the low-order 8 bits preceding the high-order 8 bits).

The following is another example using the AT attribute:

```
DECLARE VECTOR (6) BYTE;  
DECLARE SHORT$VECTOR STRUCTURE (FIRST (3) BYTE,  
                                SECOND (3) BYTE)  
                                AT (@VECTOR);
```

Here we first declare a six-element BYTE array, VECTOR. Then we declare a structure of two three-BYTE arrays, SHORT\$VECTOR.FIRST and SHORT\$VECTOR.SECOND.

The first scalar of this structure—SHORT\$VECTOR.FIRST(0)—is located at the same location as the first element of the array VECTOR.

Thus, we have two different ways of referring to the same six bytes. For example, the fifth byte in the group can be referenced as either VECTOR(4) or SHORT\$VECTOR.SECOND(1).

When a variable is declared with the AT attribute, the PL/M-86 Compiler does not optimize the machine code generated to access that variable except under OPTIMIZE(3).



CHAPTER 4

EXPRESSIONS AND ASSIGNMENTS

A PL/M-86 expression consists of scalar operands (values) combined by means of the various arithmetic, logical, and relational operators. Examples are:

$$\begin{aligned} &A + B \\ &A + B - C \\ &A * B + C / D \\ &A * (B + C) - (D - E) / F \end{aligned}$$

where $+$, $-$, $*$, and $/$ are arithmetic operators for addition, subtraction, multiplication, and division, and A, B, C, D, E, and F represent operands. The parentheses serve to group operands and operators, as in ordinary algebra.

This chapter presents a complete discussion of the rules governing PL/M-86 expressions. Although these rules may appear complex, bear in mind that most of the expressions used in actual programs are simple and easy to understand. In particular, when the operands of arithmetic and relational operators are all of the *same type*, the resulting expression is easy to understand.

4.1 Operands

Operands are the building blocks of expressions. An operand is something with a value at run time that can be operated upon by an operator. Thus, in the examples above, A, B, C, etc., might be the identifiers of scalar variables which have values at run time.

Numeric constants and fully qualified variable references may appear as operands in expressions. The following sections describe all of the types of operands that are permitted.

Constants

Any numeric constant may be used as an operand in an expression. Its type must be appropriate, as discussed below.

A numeric constant that contains a decimal point is of type REAL. A numeric constant that does *not* contain a decimal point is called a whole-number constant.

A whole-number constant may be found in either *signed context* or *unsigned context* (see section 4.6). In signed context a whole-number constant is treated as an INTEGER value.

In unsigned context, a whole-number constant is treated as a BYTE value if it is equal to or less than 255, as a WORD value if it is greater than 255 and equal to or less than 65,535, and as a DWORD value if it is greater than 65,535 and equal to or less than 4,294,967,295. A single whole-number constant may also be treated as a POINTER or SELECTOR value (see Chapter 3).

A string constant containing not more than four characters may also be used as an operand. If it has only one character, it is treated as a BYTE constant whose value is the eight-bit ASCII code for the character. If it is a two-character string, it is treated

as a **WORD** constant whose value is formed by stringing together the ASCII codes for the two characters, with the code for the first character forming the most significant eight bits of the sixteen-bit number.

If it is a three- or four-character string, it is treated as a **DWORD** constant whose value is formed by stringing together the ASCII codes for all of the characters. In a three-character string, the most significant 16 bits of the 32-bit number are formed of 8 high-order zeroes, then the code for the first character. In a four-character string, the code for the first two characters forms the most significant 16 bits of the number.

Strings of more than four characters are called string constants. They are illegal as operands in expressions, and may appear in only two contexts: as initialization values for an array or as part of a location reference that points to where the string constant is stored (see Chapter 3).

Variable and Location References

As we have seen, a fully qualified variable reference refers unambiguously to a single scalar value. (Partially qualified references, discussed in Chapter 5, have very restricted uses). Any fully qualified variable reference may be used as an operand in an expression. When the expression is evaluated, the reference is replaced by the value of the scalar.

In addition to the kinds of variable reference described previously, there is another kind called a "function reference."

A function reference is the name of a "typed procedure" that has previously been declared, along with any parameters required by the procedure declaration. The value of a function reference is the value returned by the procedure.

For example, in the statement:

```
I = J + ABS(L);
```

the absolute value of L will be returned by the function ABS and then added to the value of J before being stored in I. If L is -27, the result is exactly as if you had written:

```
I = J + 27;
```

For a complete discussion of procedures and function references, see Chapter 8.

Location references are described in Chapter 3.

Subexpressions

A subexpression is simply an expression enclosed in parentheses. A subexpression may be used as an operand in an expression. This is the same as saying that parentheses may be used to group portions of an expression together, just as in ordinary algebraic notation.

Compound Operands

All the operand types described above are *primary* operands. An operand may also be a value calculated by evaluating some portion of the total expression. For example, in the expression

$$A + B * C$$

(where A, B, and C are variable references), the operands of the * operator are B and C. The operands of the + operator are A and the *compound operand* B * C—or more precisely, the value obtained by evaluating B * C. Notice that this expression is evaluated as if it had been written

$$A + (B * C)$$

This analysis of an expression to determine which operands belong to which operators, and which groups of operators and operands form compound operands, is discussed in section 4.5.

4.2 Arithmetic Operators

There are five principal arithmetic operators in PL/M-86 (two others are described in Chapter 10). The five principal operators are

$$+ - * / \text{MOD}$$

All of these operators are used as in ordinary algebra to combine two operands. Each operand may have a BYTE, WORD, DWORD, INTEGER, or REAL value (except that REAL operands are not allowed with the MOD operator). Arithmetic operations on POINTER and SELECTOR variables are not allowed.

The +, −, *, and / Operators

The operators +, −, *, and / perform addition, subtraction, multiplication, and division on operands of any type except POINTER and SELECTOR. The following rules govern these operations (see also table 4-2).

1. If both operands are of the same type, the result is of the same type as the operands, with only one exception: if both operands are of type BYTE, the * and / operations produce results of type WORD. The type of arithmetic depends on the type of operands, as discussed in Chapter 3.
2. Only three combinations of mixed operand types are allowed. A BYTE operand can be combined with a WORD operand, a BYTE operand can be combined with a DWORD operand, and a WORD operand can be combined with a DWORD operand. In the first two cases, the BYTE operand is extended by 8 high-order zero bits to produce a WORD value, or by 24 high-order zero bits to produce a DWORD value. Similarly, a WORD operand is extended by 16 high-order zero bits to produce a DWORD value. Then the operation is performed as though both operands are of the same type.
3. If one operand is a whole-number constant, and the other is a DWORD, WORD, or BYTE operand, the whole-number constant is treated as a BYTE value if it is equal to or less than 255, a WORD value if it is greater than 255 and equal to or less than 65,535, and a DWORD value if it is greater than 65,535. Then the operation is performed under rule 1 or rule 2 above. If the whole-number constant exceeds 4,294,967,295, the operation is invalid.
4. If one operand is a whole-number constant and the other is an INTEGER operand, the whole-number constant is treated as a positive integer value. Then the operation is performed as if both operands were INTEGER operands. If the whole-number constant exceeds 32,767, the operation is invalid.
5. If one operand is a whole-number constant and the other is of type REAL, POINTER, or SELECTOR, the operation is invalid.

6. If both operands are whole-number constants, the operation depends on the context in which it occurs, as explained in section 4.6.

The result of division by 0 is undefined, except for REAL values—see Appendix G.

A *unary* “−” operator is also defined in PL/M-86. It takes a single operand, to which it is prefixed. In other words, a minus sign that has no operand to the left of it is taken to be a unary minus.

Its effect is that $(-A)$ is equivalent to $(0-A)$, where A is any operand. The 0 is a BYTE value if A is of type BYTE, WORD, or DWORD, an INTEGER value if A is of type INTEGER, or a REAL value if A is of type REAL. If A is a whole-number constant, its type and the unary “−” operation depend on the context as explained in section 4.6.

Finally, a unary “+” operator is defined for the sake of completeness. As in ordinary algebra, a unary “+” has no effect, and $(+A)$ is exactly equivalent to (A) .

The MOD Operator

MOD performs exactly the same as $/$, except as follows:

- REAL operands are not allowed—only BYTE, WORD, DWORD, and INTEGER operands can be used.
- The result is not the quotient, but the *remainder* left after integer division. The result has the same sign as the operand on the left side of the MOD operator.

For example, if A and B are INTEGER variables with values of 35 and 16, respectively, then $A \text{ MOD } B$ yields an INTEGER result of 3, and $-A \text{ MOD } B$ yields -3 .

Unlike the $/$ operator, the MOD operator must be separated from surrounding letters and digits by blanks or other separators.

4.3 Relational Operators

Relational operators are used to compare operands of the same type. They work with all types. They are

<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to
<>	not equal to
=	equal to

Relational operators are always binary operators, taking two operands, to yield a BYTE result, as follows:

If both operands are of the same type, unsigned arithmetic is used to compare two BYTE values, two WORD values, or two DWORD values; signed arithmetic is used to compare two INTEGER values or two REAL values; and POINTER or SELECTOR values are compared for equality or inequality according to the ordering of iAPX 86 locations.

NOTE

If segments are overlapped in your program and you compare **POINTER** or **SELECTOR** values for inequality, the accuracy of your results is not guaranteed.

As with the arithmetic operators, the only legal mixed combinations of operand types are **BYTE/WORD**, **BYTE/DWORD**, and **WORD/DWORD**. Whole-number operands are treated as **BYTE**, **WORD**, **DWORD**, or **INTEGER** values as explained in the rules of section 4.2.

If the specified relation between the operands is “true,” a **BYTE** value of 0FFH (or 1111\$1111B) results. Otherwise, the result is a **BYTE** value of 00H (or 0000\$0000B). Thus, in all cases, the result is of type **BYTE**, with all 8 bits set to 1 for a “true” condition, or to 0 for a “false” condition. For example:

(6>5) result is 0FFH (“true”)
(6<=4) result is 00H (“false”)

Values of “true” and “false” resulting from relational operations are useful in conjunction with **DO WHILE** statements and **IF** statements, as will be seen in Chapter 6. (In the context of a **DO WHILE** statement or **IF** statement, only the least significant bit of a “true” or “false” value is used.)

4.4 Logical Operators

There are 4 logical (boolean) operators in PL/M-86:

NOT AND OR XOR

These operators are used with **BYTE**, **WORD**, **DWORD**, or whole-number constant operands to perform logical operations on 8, 16, or 32 bits.

NOT is a unary operator, taking one operand only. It produces a result of the same type as its operand: each bit of the result is the ones complement of the corresponding bit of the original value.

The remaining operators each take 2 operands, and perform bitwise “and,” “or,” and “exclusive or,” respectively. The bits of an **AND** result are 1 only where the corresponding bit in each operand is 1. The bits of an **OR** result are 1 where the corresponding bit of either operand was a 1, and 0 only where both operands have a 0. The bits of an **XOR** result are 0 only where the corresponding bits of the operand are the same, i.e., both 1 or both 0; the result has a 1 wherever one operand has a 1 and the corresponding bit of the other operand is 0.

If both operands are of the same type, the result is the same type as the operands.

As with the arithmetic and relational operators, the only legal mixed combinations of operand types are **BYTE/WORD**, **BYTE/DWORD**, and **WORD/DWORD** (see rule 2 under section 4.2). Whole-number operands are treated as **BYTE**, **WORD**, or **DWORD** values as explained in rule 3 of section 4.2.

Examples are:

NOT 11001100B	result is 00110011B
10101010B AND 11001100B	result is 10001000B
10101010B OR 11001100B	result is 11101110B
10101010B XOR 11001100B	result is 01100110B

Also, notice that “true” and “false” values resulting from relational operations can be combined meaningfully by means of logical operators.

NOT(6>5)	result is 00H (“false”)
(6>5) AND (1> 2)	result is 00H (“false”)
(6>5) OR (1>2)	result is 0FFH (“true”)
(LIM = Y)XOR(Z<2)	result is 0FFH (“true”) if LIM = Y or if Z <2, but result is 00H (“false”) if both relations are “true” or both “false.”

Note that in the statement:

A = (NOT B)

parentheses *must* be used as indicated. Failure to do so will result in a syntax error.

4.5 Expression Evaluation

Precedence of Operators: Analyzing an Expression

Operators in PL/M-86 have an implied order (stated below) which determines how operands and operators are grouped and analyzed during compilation.

The PL/M operators are listed in table 4-1 from highest to lowest precedence, meaning those which take effect first are listed first. Operators in the same line are of equal precedence and are evaluated as encountered in a left to right reading of an expression.

The order of evaluation in an expression is controlled first by parentheses, then by operator precedence, and finally by left to right order.

The compiler first evaluates operands and operators enclosed in paired parentheses as subexpressions, working from innermost to outermost pairs of parentheses. The value of the subexpression is then used as an operand in the remainder of the expression.

Table 4-1. Operators' Precedence

Operator Class	Operator	Interpretation
Precedence	()	Controls order of evaluation: expressions within parentheses are evaluated before the action of any outside operator on the parenthesized items
Unary	+, −	Single positive operator, single negative operator
Arithmetic	*,/,MOD +, −	Multiplication, division, modulo (remainder) division, addition, subtraction
Relational	<, <=, <>, =, >=, >	Less than, less than or equal to, not equal to, equals, greater than or equal to, greater than
Logical	NOT AND OR, XOR	Logical negation Logical conjunction Logical inclusion disjunction, Logical exclusive disjunction

(Parentheses are also used around both subscripts and the parameters of function or procedure references. These are not subexpressions, but they too must be evaluated before the remainder of the expressions or references can be evaluated at a higher level.)

When you have more than one operator in an expression, you can evaluate the results by beginning with the one having the highest precedence. If the operators are of equal precedence, evaluate them left to right.

EXAMPLE	REASON
$(A + B) * C$ is not the same as $A + B * C$	Parentheses form subexpressions
$A + B * C$ means the same as $A + (B * C)$	Operator precedence
$A / B * C$ means the same as $(A / B) * C$	Left to right, equal precedence

The application of the precedence ranking can also be seen in the following examples:

$A + B * C$	is equivalent to $A + (B * C)$
$A + B - C * D$	is equivalent to $(A + B) - (C * D)$
$A + B + C + D$	is equivalent to $((A + B) + C) + D$
$A / B * C / D$	is equivalent to $((A / B) * C) / D$
$A > B \text{ AND NOT } B < C - 1$	is equivalent to $(A > B) \text{ AND } (\text{NOT}(B < (C - 1)))$

In the last four examples above, we see the application of the "left-to-right" rule for operators with the same precedence. In the second, third, and fifth examples, the left-to-right rule for operators of equal precedence makes no difference in the value of the expression. But in the fourth example, the left-to-right rule is critical.

A further example will show the action of the rules of precedence on a longer expression

$$(-B + \text{SQRT}(B * B - 4.0 * A * C)) / (2.0 * A)$$

We will assume A, B, and C are variables of type REAL, and SQRT is a procedure of type REAL which returns the square root of the value passed to it as a parameter.

In this case, the parameter is the expression $B * B - 4.0 * A * C$. Floating point constants (4.0, 2.0) are used rather than whole-number constants (4, 2) because it is invalid to combine whole-number constants with REAL variables.

As the full expression is analyzed below, association of operands with operators is indicated by brackets drawn over each operator and its operand(s).

The compiler first analyzes the portions of the expressions within the innermost parentheses, then the procedure parameter above and the subexpression $2.0 * A$, also called a compound operand since its result is used in evaluating the whole expression.

$$(-B + \text{SQRT}(\overbrace{B * B} - \overbrace{4.0 * A * C})) / (\overbrace{2.0 * A})$$

In a left-to-right scan, the two operands of the first * operator are both equal to the value of B. The operands of the second * operator are 4.0 and the value of A. The operands of the third * operator are the results of the second evaluation (i.e., the compound operand $4.0 * A$) and the value of C. The operands of the fourth * operator are 2.0 and the value of A.

The subexpression $2.0 * A$ is now completely analyzed, but the parameter expression still contains a minus ($-$) operator that has not been analyzed. The operands of this operator are the result of evaluating $B * B$ and the result of evaluating $4.0 * A * C$. Once this is done, the parameter expression is analyzed and its value can be calculated.

This value does not become an operand in the overall expression. It is passed to the procedure `SQRT`, which returns the square root of the parameter. This returned value then becomes an operand in the remainder of the full original expression:

$$(-B + \text{"returned value"}) / (2.0 * A)$$

Now that the innermost subexpressions have been analyzed and evaluated, what remains is a division whose left operand must be evaluated further. This outer subexpression is $-B +$ the returned square root: there are two operators. The first is a unary minus ($-$) and its operand is the value of B . The second is the binary plus ($+$) operator, with two operands: the value of $-B$ and the value of `SQRT( $B * B - 4.0 * A * C$ )`. $-B$ has the same meaning as $0 - B$, which is to be added to the now-known value of the square root indicated. The final operator is division ($/$), whose two operands are fully known: the value of $(-B + \text{SQRT}(B * B - 4.0 * A * C))$ and the value of $(2.0 * A)$.

Three important points must be emphasized about expression evaluation, as discussed in the next three sections.

Compound Operands Have Types

The first point is that compound operands (as shown between brackets above) have types just as primary operands do. All of the primary operands used in the example above were of type `REAL`, causing the resulting compound operands to be of type `REAL` also. It is always valid for all the operands in an arithmetic expression to be of the same type, and the result is of that type. (The only exception is that combining `BYTE` values can validly create a `WORD` value.)

But, in an expression containing mixed data types, most combinations are not allowed. Again, the only exception involved `BYTE`, `WORD`, and `DWORD` values; these may be mixed as operands in expressions, whether constants or variables.

Mixing any other types in arithmetic, logical, or relational expressions is invalid. For example, if F and G are `INTEGER` variables and H and K are `REAL` variables, then the expressions $F > K$, $H + G$ are invalid.

Due to operator precedence, some combinations can validly occur in the same expression without being directly combined. In the following logical expression

$$F > G \text{ AND } H < K$$

the subexpression $F > G$ yields a byte value, as does the subexpression $H < K$. Then the byte values are `ANDed` together. This expression is legal despite an apparent mixing of types, as follows:

G and H could not be the operands for two reasons:

1. The relational operators are of higher precedence than the `AND` operator.
2. Only `BYTE`, `WORD`, or `DWORD` operands are legal with logical operators.

Relational Operators Are Restricted

The second point is: in the absence of parentheses denoting a subexpression, the result of a relational operation (comparison) is not allowed to become an operand in another relational operation.

The algebraic meaning of $A \leq X \leq B$ is well-defined on paper, but in PL/M-86 the expression

$$A \leq X \leq B$$

is invalid because the second $\leq$ operator would have to use the result of the first $\leq$ operator as one of its operands.

The valid PL/M-86 way to achieve the desired meaning is

$$A \leq X \text{ AND } X \leq B$$

Parentheses could have created a valid expression; for example:

$$(A \leq X) \leq B$$

but the result does not have the desired meaning: $A \leq X$ becomes a byte of value 0 if A is greater than X, 0FFH if not. Thus, if A is 0, X is 1, and B is 2:

$(0 \leq 1) \leq 2$
 becomes $(0FFH) \leq 2$
 yields FALSE, contrary to the original intention

Order of Evaluation of Operands

The third point to be made from the analytical example is that the binding of operators and operands is not the same thing as the order in which operands are evaluated.

As we have just seen, the rules of analysis completely and unambiguously specify which operands are bound to each operator. In the expression

$$A + B * C$$

we know that B and C are the operands of the $*$ operator, while A and the value of $B * C$ are the operands of the $+$ operator. Obviously B and C must be evaluated before the $*$ operation can be carried out. Also, the compound operand $B * C$ must be evaluated before the $+$ is carried out.

But it is not obvious whether B will be evaluated before C or vice versa. Indeed, A could be evaluated before either B or C, and its value stored until the $+$ operation is performed.

The rules of PL/M-86 do not specify the order in which subexpressions or operands are evaluated in each statement. This flexibility allows the compiler to optimize the object code it produces, as discussed in Chapter 11. In most cases, the order of evaluation makes no difference.

However, certain embedded assignments (section 4.7) or function references (section 8.2) have the side effect of changing the value of an operand in the same expression. Due to the variability of evaluation order, this side effect can lead to undesired results. You should avoid such usage.

4.6 Choice of Arithmetic: Summary of Rules

As discussed in Chapter 3, PL/M-86 uses three distinct kinds of arithmetic: unsigned, signed, and floating-point. Whenever an arithmetic or relational operation is carried out, PL/M-86 uses one of these types of arithmetic, depending on the types of the operands.

Table 4-2 is a summary of the rules for which type of arithmetic is used in each case. The table also shows the type of the result in each case (for arithmetic operations). The notes following the table give additional information.

Table 4-2. Summary of Expression Rules

Variable Type	Kind of Arithmetic	Operand Type	Arithmetic Operation	Result	Notes
DWORD, WORD, and BYTE	Unsigned	BYTE w/BYTE	+ or – * or / or MOD	BYTE WORD	range: 0-255 range: 0-65535
		BYTE w/WORD becomes WORD w/WORD	any	WORD	A byte operand is first extended with 8 high-order zeroes to a word value.
		BYTE w/DWORD becomes DWORD w/DWORD	any	DWORD	A byte operand is first extended with 24 high-order zeroes to a dword value.
		WORD w/DWORD becomes DWORD w/DWORD	any	DWORD	A word operand is first extended with 16 high-order zeroes to a dword value.
		BYTE w/ whole-number constant < 256	+ or – * or / or MOD	BYTE WORD	Constant treated as byte. Constant treated as word.
		BYTE or WORD w/ whole-number constant < 65,536	any	WORD	Constant treated as word.
		BYTE or WORD w/ whole-number constant > 65,535	any	DWORD	Constant treated as dword.
		DWORD w/ whole-number constant < 4,294,967,295	any	DWORD	Constant treated as dword.
INTEGER	Signed	INTEGER w/INTEGER	any	INTEGER	range: –32768 to +32767
		INTEGER w/whole number constant becomes INTEGER w/ positive INTEGER	any	INTEGER	Constant treated as a positive INTEGER value. Note: unary minus may be applied to this positive INTEGER value.
REAL	Floating Point	REAL w/REAL	+ or – or * or /	REAL	—

POINTER and SELECTOR variables can only appear in relational expressions, e.g., PTR11 = PTR22, which results in a BYTE value of 0 for FALSE or 0FFH for TRUE. POINTER values are compared as full iAPX 86 addresses. SELECTOR values are compared as 16-bit unsigned numbers.

NOTE: The above are the only permitted combinations of operations and operands. All other combinations are invalid; for example, INTEGER with BYTE, REAL with WORD, and so forth. However, explicit conversions may be coded in-line using the built-in PL/M-86 functions described in Chapter 9.

Special Case: Constant Expressions

The rules already given explain expressions like

$$A + 3 * B$$

where we have a single whole-number constant. However, if we have an expression like

$$3 - 5 + A$$

we must consider which kind of arithmetic will be used to evaluate $3 - 5$, since *both* operands are whole-number constants.

The answer, in this case, depends on the type of operand A. If A is of type BYTE, WORD, or DWORD, we say that $3 - 5$ is in "unsigned context." Unsigned arithmetic is used to evaluate $3 - 5$, giving a BYTE result of 254. Unsigned arithmetic is then used to add this to A.

If A is of type INTEGER, we say that $3 - 5$ is in "signed context." Signed arithmetic is used to evaluate $3 - 5$, giving an INTEGER result of -2 . Signed arithmetic is then used to add this to A.

If A is of type REAL, POINTER, or SELECTOR, the expression is illegal.

Any compound operand, subexpression, or expression that contains only whole-number constants as primary operands is called a *constant expression*. Note that this applies only to whole-number constants less than 65,536 (all DWORD arithmetic is performed at run-time). Floating-point constants are of type REAL and are treated exactly like the values of REAL variables.

In this expression:

$$3 - 5 + 500 + A$$

$3 - 5$ is a constant expression that forms part of the larger constant expression $3 - 5 + 500$.

If the constant expression is not the entire expression, then its value is an operand in the expression. The context is created by the other operand of the same operator.

If the other operand is of type BYTE, WORD, or DWORD, then each whole-number constant is treated as a BYTE value if it is equal to or less than 255, as a WORD value if it is greater than 255 and equal to or less than 65,535, and as a DWORD value if it is greater than 65,535. If the constant exceeds 4,294,967,295 it is illegal. Unsigned arithmetic is used. In the example above, suppose the operand A has a BYTE value. Then the constant expression $3 - 5 + 500$ is in unsigned context. The constants 3 and 5 are treated as BYTE values, and 500 is treated as a WORD value. The operation $3 - 5$ gives a BYTE result of 254, and this is extended to a WORD value of 254 before adding 500, resulting in a WORD value of 754. It is exactly as if the expression had been written as:

$$754 + A$$

Now suppose that A has an INTEGER value. In this case, the constant $3 - 5 + 500$ is in *signed context*, and all three constants are treated as INTEGER values. This time, signed arithmetic is used for the operation $3 - 5$, which results in an INTEGER value of -2 . Then 500 is added, and the INTEGER result is 498. It is as if the expression had been written as:

$498 + A$

To summarize, if the context is created by a BYTE, WORD, or DWORD operand, the constant expression is in unsigned context. If the context is created by an INTEGER operand, the constant expression is in signed context. Note that if the context is created by a REAL, POINTER, or SELECTOR operand, the constant expression is *illegal*.

If the constant expression is the entire expression, then it is one of the following:

- Constant expression as right-hand part of an assignment statement: context is created by the variable to which the expression is being assigned. Rules are given below in section 4.7.
- Constant expression as subscript of an array variable: evaluated as if being assigned to a WORD variable (see section 4.7).
- Constant expression in the IF part of an IF statement: evaluated as if being assigned to a BYTE variable (see sections 6.2 and 4.7).
- Constant expression in a DO WHILE statement: evaluated as if being assigned to a BYTE variable (see sections 6.1 and 4.7).
- Constant expression as “start,” “step,” or “limit” expression in an iterative DO statement: evaluated as if being assigned to a variable of the same type as the index variable in the same iterative DO statement (see sections 6.1 and 4.7).
- Constant expression in a DO CASE statement: evaluated as if being assigned to a WORD variable (see sections 6.1 and 4.7).
- Constant expression as an actual parameter in a CALL statement or function reference: evaluated as if being assigned to the corresponding formal parameter in the procedure declaration (see sections 8.2 and 4.7).
- Constant expression in a RETURN statement: evaluated as if being assigned to a variable of the same type as the (typed) procedure that contains the RETURN statement (see section 4.7).

4.7 Assignment Statements

Results of computations can be stored as values of scalar variables. At any given moment, a scalar variable has only one value—but this value may change with program execution. The PL/M-86 *assignment statement* changes the value of a variable. Its simplest form is:

variable = *expression*;

where *expression* is any PL/M-86 expression, as described in the preceding sections. This expression is evaluated, and the resulting value is assigned to (that is, stored in) *variable*. This variable may be any fully qualified variable reference except a function reference. The old value of the variable is lost.

For example, following execution of the statement:

```
RESULT = A + B;
```

the variable RESULT will have a new value, calculated by evaluating the expression $A + B$.

Implicit Type Conversions

In an assignment statement, if the type of the value of the right-hand expression is not the same as the type of the variable on the left side of the equal sign, then either the assignment is illegal or an *implicit* type conversion occurs. Except for constant expressions, only byte, word, or dword values are converted automatically. Chapter 9 presents eight built-in functions you can invoke to perform *explicit* conversions for use in expressions or assignments. The following paragraphs spell out the rules for the implicit conversions:

Expression with a BYTE value. *WORD variable on the left*: the BYTE value is extended by 8 high-order zero bits to convert it to a WORD value. *DWORD variable on the left*: the BYTE value is extended by 24 high-order zero bits to convert it to a DWORD value.

If the variable on the left is of any type except BYTE, WORD, or DWORD, the assignment is illegal.

Expression with a WORD value. *BYTE variable on the left*: The 8 high-order bits of the WORD value are dropped to convert it to a BYTE value. *DWORD variable on the left*: the WORD value is extended by 16 high-order zero bits to convert it to a DWORD value.

If the variable on the left is of any type except BYTE, WORD, or DWORD, the assignment is illegal.

Expression with a DWORD value. *BYTE variable on the left*: the 24 high-order bits of the DWORD value are dropped to convert it to a BYTE value. *WORD variable on the left*: the 16 high-order bits of the DWORD value are dropped to convert it to a WORD value.

If the variable on the left is of any type except BYTE, WORD, or DWORD, the assignment is illegal.

Expression with an INTEGER value. No implicit conversions are performed. If the variable on the left is of any type except INTEGER, the assignment is illegal.

Expression with a REAL value. No implicit conversions are performed. If the variable on the left is of any type except REAL, the assignment is illegal.

Expression with a POINTER value. No implicit conversions are performed. If the variable on the left is of any type except POINTER, the assignment is illegal.

Expression with a SELECTOR value. No implicit conversions are performed. If the variable on the left is of any type except SELECTOR, the assignment is illegal.

Constant Expression

BYTE variable on the left: The constant expression is evaluated in unsigned context. If the resulting value is less than or equal to 255, it is treated as a BYTE value and no conversion is necessary. If the resulting value is greater than 255, it is converted to type BYTE by dropping all except its 8 low-order bits.

WORD variable on the left: The constant expression is evaluated in unsigned context. If the resulting value is equal to or less than 65,535, it is treated as a WORD value, and no conversion is necessary. If the resulting value is greater than 65,535, it is converted to type WORD by dropping all except its 16 low-order bits.

DWORD variable on the left: The constant expression is evaluated in unsigned context. No conversion is necessary.

INTEGER variable on the left: The constant expression is evaluated in signed context to yield an INTEGER value. No conversion is necessary.

POINTER variable on the left: If the constant expression consists of nothing but a *single* whole-number constant, the constant is treated as a POINTER value. The whole-number constant must not be greater than 1,048,575. If the constant expression consists of anything more than a single whole-number constant, the assignment is illegal. This is one of the three cases in which a whole-number constant can be treated as a POINTER value. The other two cases are described in sections 3.1 and 3.6.

SELECTOR variable on the left: If the constant expression consists of nothing but a single whole-number constant, it is treated as a SELECTOR value. The whole-number constant must not exceed 65,535. If the constant expression consists of anything more than a single whole-number, the assignment is illegal. This is one of two cases where a whole-number constant can be treated as a SELECTOR value. The other case is described in section 3.1.

REAL variable on the left: The assignment is illegal unless all values on the right are REAL. However, the FLOAT function described in section 9.2 can be used to convert the constant expression to a REAL value which can be assigned to that variable.

Multiple Assignment

It is often convenient to assign the same value to several variables at the same time. This is accomplished in PL/M-86 by listing all the variables to the left of the equal sign, separated by commas. The variables LEFT, CENTER, and RIGHT can all be set to the value of the expression INIT + CORR with the single assignment statement:

```
LEFT, CENTER, RIGHT = INIT + CORR;
```

The variables on the left-hand side of a multiple assignment must be all of the same type, with one exception: variables of types BYTE, WORD, and DWORD may be mixed. When this is done, the conversion rules given above are applied separately to each assignment.



The order in which the assignments are performed is not predictable. Therefore, if a variable on the left side of a multiple assignment also appears in the expression on the right side, the results are undefined.

Embedded Assignments

A special form of assignment can be used within PL/M-86 expressions. The form of this "embedded assignment" is:

variable: = expression

and may appear anywhere an expression is allowed. The *expression* (everything to the right of the `:=` assignment symbol) is evaluated and stored into the *variable* on the left. Parentheses are strongly recommended, therefore, to specify the limits of an embedded assignment within an assignment statement. The value of the embedded assignment is the same as that of its right half. For example, the expression:

```
ALT + (CORR := TCORR + PCORR) - (ELEV := HT/SCALE)
```

results in exactly the same value as:

```
ALT + (TCORR + PCORR) - (HT/SCALE)
```

The only difference is the side-effect of storing the intermediate results `TCORR + PCORR` and `HT/SCALE` into `CORR` and `ELEV`, respectively. These names for intermediate results can then be used at a later point in the program without calculating their values. The names must have been declared earlier.

CAUTION

The rules of PL/M-86 do not specify the order in which subexpressions or operands are evaluated. When an embedded assignment changes the value of a variable that also appears elsewhere in the same expression, the results cannot be predicted. The results depend on too many factors, e.g., the optimization level you specify to the compiler (as discussed in Chapter 11).

As an example of this ambiguity, if you write:

```
A = (X := X + 4) + Y * Y + X;
```

you could mean either of the following interpretations:

```
A1 = (X + 4) + Y * Y + (X + 4);
```

```
A2 = (X + 4) + Y * Y + X;
```

You should avoid this ambiguity by removing the embedded assignment from the expression and using a separate assignment statement to achieve the desired effect. For example, each of the above is unambiguously achieved by the following:

```
(1)  X = X + 4;
      A1 = X + Y * Y + X;
```

```
(2)  X = X + 4;
      A2 = X + Y * Y + X - 4;
```

```
(3)  A3 = X + 4 + Y * Y + X;
      X = X + 4;
```

NOTE

Before using floating-point (REAL) arithmetic or assignments, refer to Chapters 3 and 10 and Appendixes F and G, which explain many features and restrictions that affect the result of such operations.



5.1 Arrays

As mentioned briefly in Chapter 3, it is often desirable to use a single identifier to refer to a whole group of scalars, and to distinguish the individual scalars by means of a “subscript,” i.e., a value enclosed in parentheses. The scalars are all the same type. Such a list is called an array.

It is declared by using a “dimension specifier.” The dimension specifier is a nonzero whole-number constant enclosed in parentheses. The value of the constant specifies the number of array elements (individual scalar variables) making up the array. For example:

```
DECLARE ITEMS (100) BYTE;
```

causes the identifier ITEMS to be associated with 100 array elements, each of type BYTE. One byte of storage is allocated for each of these scalars.

The elements of an array are stored contiguously, with the 0th element in the lowest location and the last element in the highest location. (No storage is allocated for a based array, but the elements are considered to be contiguous in memory.)

The declaration:

```
DECLARE (WIDTH, LENGTH, HEIGHT) (100) REAL;
```

is equivalent to the following sequence:

```
DECLARE WIDTH (100) REAL;  
DECLARE LENGTH (100) REAL;  
DECLARE HEIGHT (100) REAL;
```

(except that contiguous storage is guaranteed for variables declared in a single parenthesized list, while variables declared in consecutive declarations are not necessarily stored contiguously).

This causes each of the three identifiers, WIDTH, LENGTH, and HEIGHT, to be associated with 100 array elements of type REAL, so that 300 elements of type REAL have been declared in all. For each of these scalars, four contiguous bytes of storage are allocated.

Subscripted Variables

To refer to a single element of a previously declared array, you use the array name followed by a subscript enclosed in parentheses. This construct is called a “subscripted variable.”

For example, given the DECLARE statement:

```
DECLARE ITEMS (100) BYTE;
```

you can refer to each byte as an individual item using ITEMS(0), ITEMS(1), ITEMS(2), and so on up to ITEMS(99).

Notice that the first element of an array has subscript 0—not 1. Thus, the subscript of the last element is 1 less than the dimension specifier.

If we want to add the third element of the array ITEMS to the fourth, and store the result in the fifth, we can write the PL/M-86 assignment statement:

```
ITEMS(4) = ITEMS(2) + ITEMS(3);
```

Much of the power of a subscripted variable lies in the fact that the subscript need not be a whole-number constant, but can be another variable, or in fact any PL/M-86 expression that yields a BYTE, WORD, or INTEGER value. Thus, the construction:

```
VECTOR(ITEMS(3) + 2)
```

refers to some element of the array VECTOR. Which element it is depends on the expression ITEMS(3) + 2. This value in turn depends on the value stored in ITEMS(3), the fourth element of array ITEMS, at the time when the reference is processed by the running program.

If ITEMS(3) contains the value 5, then ITEMS(3) + 2 is equal to 7 and the reference is to VECTOR(7), the eighth element of the array VECTOR.

The following sequence of statements will sum the elements of the 10-element array NUMBERS by using an “index variable” named I, which takes on values from 0 to 9:

```
DECLARE SUM BYTE;
DECLARE NUMBERS(10) BYTE;
DECLARE I BYTE;

SUM = 0;
DO I = 0 TO 9;
    SUM = SUM + NUMBERS(I);
END;
```

Subscripted array variables are permitted anywhere PL/M-86 permits a variable. They may also appear on the left side of an assignment statement if the array elements are of a scalar type.

5.2 Structures

Just as an array allows one identifier to refer to a collection of elements of the same type, a *structure* allows one identifier to refer to a collection of *structure members* that may have different types. Each member of a structure has a *member identifier*.

The following is an example of a structure declaration:

```
DECLARE AIRPLANE STRUCTURE (SPEED REAL, ALTITUDE REAL);
```

This declares two REAL scalars, both associated with the identifier AIRPLANE. Once this declaration has been made, the first scalar can be referred to as AIRPLANE.SPEED and the second as AIRPLANE.ALTITUDE. These names are also called the “members” of this structure.

A structure may have many members; see Appendix B for the correct limit. The members of a structure are stored contiguously in the order in which they are specified. (No storage is allocated for a based structure, but the members are considered to be contiguous in memory.)

Individual structure members may not be based and may not have any attributes, as discussed in Chapter 3.

Arrays of Structures

We have already seen arrays of scalars. PL/M-86 also allows arrays of structures. The following DECLARE statement creates an array of structures which can be used to store SPEED and ALTITUDE (as in the previous example) for twenty AIRPLANES instead of one:

```
DECLARE AIRPLANE (20) STRUCTURE (SPEED REAL, ALTITUDE REAL);
```

This declares twenty structures associated with the array identifier AIRPLANE, each distinguished by subscripts from 0 to 19. Each of these structures consists of two REAL scalar members. Thus, storage is allocated for 40 REAL scalars.

To refer to the ALTITUDE of AIRPLANE number 17, one would write AIRPLANE(17).ALTITUDE.

Arrays Within Structures

An array may be used as a member of a structure, as in the following DECLARE statement:

```
DECLARE PAYCHECK STRUCTURE (LAST$NAME(15)BYTE,  
    FIRST$NAME(15)BYTE,  
    MI BYTE,  
    AMOUNT REAL);
```

This structure consists of the following members: two 15-element BYTE arrays, PAYCHECK.LAST\$NAME and PAYCHECK.FIRST\$NAME; the BYTE scalar PAYCHECK.MI; and the REAL scalar PAYCHECK.AMOUNT.

To refer to the fourth element of the array PAYCHECK.LASTNAME, we would write PAYCHECK.LASTNAME(3).

Arrays of Structures with Arrays Inside the Structures

Given that an array can be made up of structures, and a structure can have arrays as members, we can combine the two constructions to write:

```
DECLARE FLOOR (30) STRUCTURE (OFFICE (55) BYTE);
```

The identifier FLOOR refers to an array of 30 structures, each of which contains one array of 55 BYTE scalars. This could be thought of as a 30-by-55 matrix of BYTE scalars. To reference a particular scalar value—say element 46 of structure 25—we would write FLOOR(25).OFFICE(46). Note that the scalar elements of each OFFICE array are stored contiguously, and the OFFICE arrays themselves are elements of the FLOOR array and are stored contiguously.

We can alter the PAYCHECK structure declaration above to make it an array of structures, as follows:

```
DECLARE PAYROLL (100) STRUCTURE (LAST$NAME(15) BYTE,
    FIRST$NAME(15) BYTE,
    MI BYTE,
    AMOUNT REAL);
```

Now we have an array of 100 structures, each of which can be used during program execution to store the last name, first name, middle initial, and amount for one employee. LAST\$NAME and FIRST\$NAME in each structure are 15-BYTE arrays for storing the names as character strings. To refer to the Kth character of the first name of the Nth employee, we would write:

```
PAYROLL(N-1).FIRST$NAME(K-1)
```

where N and K are previously declared variables to which we have assigned appropriate values. This might be convenient in a routine for printing out payroll information.

5.3 References to Arrays and Structures

In the preceding sections, we saw numerous examples of variable references. A variable reference is simply the use, in program text, of the identifier of a variable that has been declared.

A variable reference may be “fully qualified,” “partially qualified,” or “unqualified.”

Fully Qualified Variable References

A fully qualified variable reference is one that uniquely specifies a single scalar. For example, if we have the declarations:

```
DECLARE AVERAGE REAL;
DECLARE ITEMS (100) BYTE;
DECLARE RECORD STRUCTURE (KEY BYTE, INFO WORD);
DECLARE NODE (25) STRUCTURE (SUBLIST (100) BYTE, RANK BYTE);
```

then AVERAGE, ITEMS(5), RECORD.INFO, AND NODE(21).SUBLIST(32) are all fully qualified variable references: each refers unambiguously to a single scalar.

It should be noted that qualification may only be applied to variables that have been appropriately declared. A subscript may only be applied to an identifier that has been declared with a dimension specifier. A member-identifier may only be applied to an identifier declared as a structure identifier. The compiler flags violations of these rules as errors.

Unqualified and Partially Qualified Variable References

Unqualified and partially qualified variable references are allowed only in location references, as discussed in Chapter 3, and in the built-in procedures LENGTH, LAST, and SIZE, as discussed in Chapter 9.

An unqualified variable reference is the identifier of a structure or array, without any member-identifier or subscript. For example, with the above declarations, ITEMS and RECORD are unqualified variable references. An unqualified variable reference is a reference to the *entire* array or structure. @ITEMS is the location of the entire array ITEMS—that is, the location of its first byte. Similarly, @RECORD is the location of the first byte of the structure RECORD.

A partially qualified variable reference fails to refer uniquely to a single scalar even using a subscript and/or member-identifier with an identifier. For example, NODE(15) and NODE(12).SUBLIST are partially qualified variable references, given the above declarations.

When used with the @operator, such references are taken to mean the first byte that could fit the description. Thus @NODE(15) is the location of the first byte of the structure NODE(15), which is itself an element of the array NODE. Similarly, @NODE(12).SUBLIST is the location of the first byte of the array NODE(12).SUBLIST, which is itself a member of the structure NODE(12), which in turn is an element of the array NODE.

Note that @NODE.SUBLIST is not permitted because it is completely ambiguous: in a location reference referring to an array made up of structures, a subscript must be given before a member-identifier can be added to the reference. The rule is different for partially qualified variable references in connection with the built-in procedures LENGTH, LAST, and SIZE, as explained in Chapter 9.



CHAPTER 6

FLOW CONTROL STATEMENTS

This chapter describes statements that alter the sequence of execution of PL/M-86 statements and the group statements into blocks.

6.1 DO and END Statements: DO Blocks

Procedures and DO blocks are the basic building units of modular programming in PL/M-86. (Procedures are discussed in Chapter 8.)

The present chapter discusses all four kinds of DO-blocks. Each begins with a DO statement and includes all subsequent statements through the closing END statement. The four kinds are

- The *simple DO block*; for example:

```
DO; /* all statements executed, each in order */
statement-0;
statement-1;
statement-2;
.
.
END;
```

- The *DO CASE block*; for example:

```
DO CASE select_expression;
case-0-statement; /* exactly one statement executed- */
case-1-statement; /* selected by the expression's value */
.
.
END;
```

- The *DO WHILE block*; for example:

```
DO WHILE expression_true;
statement-0; /* all executed as long as expression is */
statement-1; /* true, none when expression is false. */
.
.
END;
```

- The *iterative DO block*; for example:

```
DO counter = start-expr TO limit-expr BY step-expr;
statement-0; /* all statements executed a number */
statement-1; /* of times depending on comparison */
/* of counter with limit_expr. */
.
.
.
END;
```

The last two blocks are also referred to as DO-loops because the executable statements within them may be executed repeatedly (in sequence) depending on the expressions in the DO statement.

As discussed with earlier charts, any DO statement may have multiple labels on it, and the last (only) of these may appear between the word END and the next semicolon. For example:

```
A: B: C: D: EM: DO;
      .
      .
      .
      END EM ; /* indicates end of block EM; */
               /* A, B, C, D also end here. */
```

As mentioned in Chapter 3, the placement of declarations is restricted. Except for use in procedures, declarations are permitted only at the top of a simple DO block, before any executable statements of the block. (This DO can, of course, be nested within other DOs or procedures. Chapter 7 discusses the scope of declared names.)

Each DO block can contain any sequence of executable statements, including other DO blocks. Each block is considered by the compiler as a unit, as if it were a single executable statement. This fact is particularly useful in the DO CASE block and the IF statement, both discussed in this chapter.

The discussions that follow describe the normal flow of control within each kind of DO block. The normal exit from the block passes through the END statement to the statement immediately following. These discussions assume that none of the statements in the block causes control to bypass that process. A GOTO statement with the target outside the block would be one such bypass. (GOTOs are discussed later in this chapter.)

Simple DO Blocks

A simple DO block merely groups, as a unit, a set of statements that will be executed sequentially (except for the effect of GOTOs or CALLs):

```
DO ;
    statement-0;
    statement-1;
    . . .
    statement-n
END ;
```

For example:

```
DO ;
    NEW$VALUE = OLD$VALUE + TEMP;
    COUNT = COUNT + 1
END ;
```

This simple DO block adds the value of TEMP to the value of OLD\$VALUE and stores it in NEW\$VALUE. It then increments the value of COUNT by one.

DO blocks may be nested within each other as shown in the following example:

```
able:  DO;
        statement-0;
        statement-1;
baker:  DO;
        statement-a;
        statement-b;
        statement-c;
        END baker;
        statement-2;
        statement-3;
      END able;
```

The first DO statement and the second END statement bracket one simple DO block. The second DO statement and the first END statement bracket a different DO block inside the first one. Notice how indentation (using tabs or spaces) can be used to make the sequence readable, so that it can be seen at a glance that one DO block is nested inside another. It is recommended that this practice be followed in writing PL/M-86 programs. See Appendix B for the number of DO blocks that can be nested.

A simple DO block can delimit the scope of variables, as discussed in Chapter 7.

DO CASE Blocks

A DO CASE block begins with a DO CASE statement, and selectively executes *one* of the statements in the block. The statement is selected by the value of an expression. The maximum number of cases is given in Appendix B. The form of the DO CASE block is:

```
DO CASE select_expression;
        statement-0;
        statement-1;
        . . .
        statement-n;
END;
```

In the DO CASE statement, *expression* must yield a BYTE, WORD, or INTEGER value. If it is a constant expression, it is evaluated as if it were being assigned to a WORD variable. The value of *expression* must lie between 0 and *n* (call this value *K*). *K* is used to select *one* of the statements in the DO CASE block, which is then executed. The first case (statement-0) corresponds to *K* = 0, the second (statement-1) corresponds to *K* = 1, and so forth. *Only one* statement from the block is selected. This statement is then executed *only once*. Control then passes to the statement following the END statement of the DO CASE block.



If the run-time value of the expression in the DO CASE statement is less than 0 or greater than *n* (where *n*+1 is the number of statements in the DO CASE block), then the effect of the DO CASE statement is *undefined*. This may have disastrous effects on program execution. Therefore if there is any possibility that this out-of-range condition may occur, the DO CASE block should be contained within an IF statement that tests the expression to make sure that it has a value that will produce meaningful results.

An example of a DO CASE block is:

```
DO CASE SCORE;
;                                /* case 0 */
CONVERSIONS=CONVERSIONS+1;    /* case 1 */
SAFETIES = SAFETIES + 1;      /* case 2 */
FIELDGOALS = FIELDGOALS + 1; /* case 3 */
;                                /* case 4 */
;                                /* case 5 */
TOUCHDOWNS=TOUCHDOWNS+1;     /* case 6 */
END;
```

When execution of this CASE statement begins, the variable SCORE must be in the range 0-6. If SCORE is 0, 4, or 5 then a null statement (consisting of only a semicolon, and having no effect) is executed; otherwise the appropriate statement is executed, causing the corresponding variable to be incremented.

A more complex DO CASE block is the following:

```
SELECT = COUNT-5;
IF SELECT <=2 AND SELECT >=0 THEN
DO CASE SELECT;

    X = X + 1;                    /* Case 0 */

DO;                                /* Begin Case 1 */
    X = Y + 10;
    Y = Y + 1;
END;                                /* End Case 1 */

DO I = LAST$HI+1 TO TOP-6 ; /* Begin Case 2 */
    Z(I) = X*Y+1 ;
    W(I) = Z(I)*Z(I) ;
    V(I) = W(I)-Z(I) ;
END;                                /* End Case 2 */

END;                                /* End DO CASE block */
ELSE CALL ERROR;
```

If we assume SELECT and COUNT are INTEGER variables, negative values could occur. The DO CASE block is placed within an IF statement to guarantee that if the value of SELECT is less than 0 or greater than 2, execution of the DO CASE block will not be attempted. Instead, a procedure called ERROR (declared previously) will be activated. IF statements are discussed in section 6.2.

This example illustrates the use of a simple DO block as a single PL/M-86 statement. The DO CASE statement can select Case 1 or Case 2 and cause multiple statements to be executed. This is only possible because they are grouped as a simple DO block, which acts as a single statement.

DO WHILE Blocks

DO WHILE and IF statements examine only the least significant bit of the value of the expression. If the expression is relational, e.g., $A < B$, the result will have a value of 00H or 0FFH, but this is incidental; it may have any BYTE or WORD value. If the value is an odd number (least significant bit = 1), it will be considered "true." If it is even (least significant bit = 0), it will be considered "false."

A DO WHILE block begins with a DO WHILE statement, and has the form:

```
DO WHILE expression; /* expression must yield */
                        /* BYTE or WORD value */
    statement-0;
    statement-1;
    .
    .
    statement-n;
END;
```

The effect of this statement is as follows:

1. First the BYTE or WORD *expression* following the reserved word WHILE is evaluated. If the rightmost bit of result is 1, then the sequence of statements up to the END is executed.
2. When the END is reached, *expression* is evaluated again, and again the sequence of statements is executed only if the value of the expression has a rightmost bit of 1.
3. The block is executed over and over until *expression* has a value whose rightmost bit is 0. Execution then skips the statements in the block and passes to the statements following the END statement.

Consider the following example:

```
AMOUNT = 1;
DO WHILE AMOUNT <= 3;
    AMOUNT = AMOUNT + 1;
END;
```

The statement AMOUNT = AMOUNT + 1 is executed exactly 3 times. The value of AMOUNT when program control passes out of the block is 4.

Iterative DO Blocks

An iterative DO block begins with an iteration statement and executes each statement in order in the block, repeating the entire sequence as described in this section. The form of the iterative DO block is:

```
DO counter = start-expr TO limit-expr BY step-expr ;
    statement-0 ;
    statement-1 ;
    .
    .
    .
END ;
```

The BY *step-expr* phrase is optional; if omitted, a step of 1 is used.

The *counter* must be a non-subscripted variable of type BYTE, WORD, or INTEGER. The *start-expr*, *limit-expr*, and *step-expr* may be any valid PL/M-86 expressions also of these types. However, if *counter* or any of these expressions has type INTEGER, then all must be INTEGER, as explained in Chapter 4.

An example of an iterative DO block is:

```
DO I = 1 TO 10;
    CALL BELL;
END;
```

where BELL is the name of a procedure that causes a bell to be rung. The bell is rung ten times.

Another example shows how the index-variable can be used within the block:

```
AMOUNT = 0;
DO I = 1 TO 10;
    AMOUNT = AMOUNT + I;
END;
```

The assignment statement is executed 10 times, each time with a new value for I. The result is to sum the numbers from 1 to 10 (inclusive) and leave the sum (namely, 55) as the value of AMOUNT.

The next example uses *step-expr*:

```
/*Compute the product of the first N odd integers */
PROD = 1;
DO I = 1 TO (2*N-1) BY 2;
    PROD = PROD*I;
END;
```

The type of counter affects two important factors in the execution flow of iterative DOs:

- a. When *step-expr* is evaluated.
- b. What causes execution to exit the DO block.

The following steps constitute the general execution sequence of an iterative DO block, with BYTE, WORD, or INTEGER variables and expressions in the DO itself. Type is mentioned only for steps where it matters, i.e., where the actions or consequences are different for different types. Where the INTEGER case is different, it is described in parentheses. The discussion following this description summarizes all the rules and their results.

1. The *start-expr* is evaluated and assigned to *counter*.
2. The *limit-expr* is evaluated and compared with *counter*. (If these are INTEGER type, then *step-expr* is also newly evaluated at this time.)
 - a. If *counter* is greater than *limit-expr*, execution exits the DO and passes to the statement following the next END (unless *step-expr* is a negative INTEGER; if so, the exit occurs only if *counter* is less than *limit-expr*.)
 - b. Otherwise, the statements within the DO block are executed in order until the END statement is reached.
 - c. At the END, a *step-expr* of type BYTE or WORD is newly evaluated.
3. The *counter* is incremented by the value of *step-expr*. For BYTE or WORD *counters*, if the new value is less than the old value (due to modulo arithmetic as explained below), the loop is exited immediately. Otherwise, control returns to step 2 above.

(An 8-bit byte can represent numbers no larger than 11111111B (255 decimal). The largest number a 16-bit word can represent is 1111111111111111B, which is 65535 decimal. If you add 1 to these values, you get 0. Thus, the new *counter* can be less than the old.)

These rules and their consequences can be summarized in two broad cases:

- A. If you start with a non-negative *step-expr*, then the loop is exited as soon as any one of the following becomes true:
 1. The new *counter* is greater than the new *limit-expr*.
 2. An INTEGER *step-expr* becomes negative AND the new *counter* is still less than the new *limit-expr*.
 3. A BYTE or WORD *step-expr* causes a lower *counter* than the one just used.
- B. If you start with a negative INTEGER *step-expr*, then the loop is exited as soon as either of the following two conditions occurs:
 1. The new *counter* is less than the new *limit-expr*.
 2. The new *step-expr* becomes non-negative AND the new *counter* is greater than the new *limit-expr*.

Upon exit from the iterative DO block:

1. In all cases *step-exp* has been reevaluated.
2. In all but one case *limit-expr* has been re-evaluated. When a non-INTEGER *counter* has just "gone over" and become smaller, *limit-expr* is unchanged from its value during the last loop.
3. In all cases *counter* has been changed, but the step value that was added to it varies. If INTEGER, *counter* has been incremented by the former step value before it was re-evaluated. For BYTE or WORD *counters*, the newer step has been used.

The following distinctions can be important:

- In every case, *start-expr* is evaluated only once and *limit-expr* is evaluated before any execution.
- An INTEGER *step-expr* is evaluated in step 2; other *step-exprs* are evaluated in step 3.
- With a *counter* of BYTE or WORD, there is no such thing as a negative step. For example, if *step-expr* is -5 , 251 is used. Furthermore, stepping down to a *limit-expr* that is less than *start-expr* is not possible because the loop will be exited immediately.

6.2 The IF Statement

The IF statement provides conditional execution of statements. It takes the form:

```
IF expression THEN statement-a;  
   ELSE statement-b; /* optional */
```

The reserved word THEN and the statement following it are required and are called the "THEN part." The reserved word ELSE and the statement following it are optional, and are called the "ELSE part."

The IF statement has the following effect: first *expression* is evaluated as if it were being assigned to a variable of type BYTE. If the result is "true" (rightmost bit is 1) then *statement-a* is executed. If the result is "false" (rightmost bit is 0), then *statement-b* is executed. Following execution of the chosen alternative, control passes to the next statement following the IF statement. Thus, of the two statements (*statement-a* and *statement-b*) one and only one is executed.

Consider the following program fragment:

```
IF NEW > OLD THEN RESULT = NEW;
  ELSE RESULT = OLD;
```

Here, RESULT is assigned the value of NEW or the value of OLD, whichever is greater. This code causes exactly one of the two assignment statements to be executed. RESULT always gets assigned some value, but only one assignment to RESULT is executed.

In the event that *statement-b* is not needed, the ELSE part may be omitted entirely. Such an IF statement takes the form:

```
IF expression THEN statement-a;
```

Here, *statement-a* is executed if the value of *expression* has a rightmost bit of 1. Otherwise, nothing happens, and control immediately passes on to the next statement following the IF statement.

For example, the following sequence of PL/M-86 statements will assign to INDEX either the number 5, or the value of THRESHOLD, whichever is larger. The value of INIT will change during execution of the IF statement only if THRESHOLD is greater than 5. The final value of INIT is copied to INDEX in any case:

```
INIT = 5;
IF THRESHOLD > INIT THEN INIT = THRESHOLD;
INDEX = INIT;
```

The power of the IF statement is enhanced by using DO blocks in the THEN and ELSE parts. Since a DO block is allowed wherever a single statement is allowed, each of the two statements in an IF statement may be a DO block. For example:

```
IF A = B THEN
  DO;
    EQUAL$EVENTS = EQUAL$EVENTS + 1;
    PAIR$VALUE = A;
    BOTTOM = B;
  END;
ELSE
  DO;
    UNEQUAL$EVENTS = UNEQUAL$EVENTS + 1;
    TOP = A;
    BOTTOM = B;
  END;
```

DO blocks nested within an IF statement can contain further nested DO blocks, IF statements, variable and procedure declarations, and so on.

Nested IF Statements

Any IF statement (including the ELSE part, if any) may be considered a single PL/M-86 statement (although it is *not* a block). Thus, the statement to be executed in a THEN or an ELSE clause may in fact be another IF statement.

An IF statement inside a THEN clause is called a "nested" IF. Nesting may be carried to several levels without needing to enclose any of the nested IF statements in DO blocks, as in the following construction:

```
IF expression-1 THEN
    IF expression-2 THEN
        IF expression-3 THEN statement-a;
```

Here we have three levels of nesting. Note that *statement-a* will be executed only if the values of all three expressions are "true." Thus, the above is equivalent to:

```
IF (expression-1) AND (expression-2) AND (expression-3)
THEN statement-a;
```

Notice that the above example of nesting does not have an ELSE part. When using nested IF statements, it is important to understand the following important rule of PL/M-86:

A set of nested IF statements may have only *one* ELSE part, and it belongs to the *innermost* (that is, the last) of the nested IF statements.

This rule could also be restated as:

When an IF statement is nested within the THEN part of an outer IF statement, the outer IF statement may not have an ELSE part.

In other words, the construction:

```
IF expression-1 THEN
    IF expression-2 THEN statement-a
    ELSE statement-b;
```

is legal and means that if the values of both *expression-1* and *expression-2* are "true," then *statement-a* will be executed. If the value of *expression-1* is "true" and the value of *expression-2* is "false," then *statement-b* will be executed. If the value of *expression-1* is "false," neither *statement-a* nor *statement-b* will be executed, regardless of the value of *expression-2*.

The construction above is equivalent to:

```
IF expression-1 THEN
    DO ;
        IF expression-2 THEN statement-a;
        ELSE statement-b;
    END ;
```

This construction is much more readable and offers less opportunity for error.

If the intention is for the ELSE part to belong to the outer IF statement, then the nesting *must* be done by means of a DO block:

```
IF expression-1 THEN
    DO ;
        IF expression-2 THEN
            statement-a;
        END ;
    ELSE statement-b;
```

Note that the meaning of this construction differs completely from the previous one.

Finally, consider the following:

```
IF expression-1 THEN
    IF expression-2 THEN
        IF expression-3 THEN statement-a;
        ELSE statement-b;
    ELSE statement-c;          /* illegal statement */
ELSE statement-d;             /* illegal statement */
```

This construction is *illegal* because only one ELSE part is allowed. If the intention is for the ELSE parts to match the IF parts as indicated by the indenting, the nesting *must* be done with DO blocks, as follows:

```
IF expression-1 THEN
    DO;
        IF expression-2 THEN
            DO;
                IF expression-3 THEN statement-a;
                ELSE statement-b;
            END;
        ELSE statement-c;
    END;
ELSE statement-d;
```

Sequential IF Statements

Consider the following example. An ASCII-coded character is stored in a BYTE variable named CHAR. If the character is an A, we want to execute *statement-a*. If the character is a B, we want to execute *statement-b*. If the character is a C, we want to execute *statement-c*. If the character is not A, B, or C, we want to execute *statement-x*. The code for doing this could be written as follows, using IF statements that are completely independent of one another:

```
IF CHAR = 'A' THEN statement-a;
IF CHAR = 'B' THEN statement-b;
IF CHAR = 'C' THEN statement-c;
IF CHAR <> 'A' AND CHAR <> 'B' and CHAR <> 'C' THEN statement-x;
```

This sequence is inefficient because all four IF statements (six tests in all) will be carried out in every case, which is wasteful when one of the earlier tests succeeds.

We need to test for 'A' in all cases. But we need to test for 'B' only if the test for 'A' fails and we need to test for 'C' only if both previous tests fail. Finally, if the tests for 'A', 'B', and 'C' all fail, no further tests are needed—we must execute *statement-x*. To improve the code, we rewrite it as follows:

```
IF CHAR = 'A' THEN statement-a;
ELSE IF CHAR = 'B' THEN statement-b;
ELSE IF CHAR = 'C' THEN statement-c;
ELSE statement-x;
```

Notice that this sequence is *not* a case of “nested IF statements” as described in the preceding section. IF statements are said to be nested only when one IF statement is inside the THEN part of another. Here, we have IF statements inside the ELSE parts of other IF statements. This construction is called “sequential IF statements.” It is equivalent to the following:

```
IF CHAR = 'A' THEN statement-a;  
ELSE DO;  
    IF CHAR = 'B' THEN statement-b;  
    ELSE DO;  
        IF CHAR = 'C' THEN statement-c;  
        ELSE statement-x;  
    END;  
END;
```

Sequential IF statements are useful whenever a set of tests is to be made, but you want to skip the remaining tests whenever one of the tests succeeds. This construction works in such cases because all the remaining tests are in the ELSE part of the current test.

6.3 GOTO Statements

A GOTO statement alters the sequential order of program execution by transferring control directly to a labeled statement. Sequential execution then resumes, beginning with the “target” statement. The GOTO statement has the following form:

```
GOTO label;
```

For example:

```
GOTO ABORT;
```

The appearance of *label* in a GOTO statement is *not* a “label definition”—it is a label reference.

The reserved word GOTO can also be written GO TO, with an embedded blank.

For reasons discussed in Chapter 7, GOTO statements are restricted. The only possible GOTO transfers are the following:

- From a GOTO statement in the outer level of some block to a labeled statement in the outer level of the *same* block.
- From a GOTO statement in an inner block to a labeled statement in the outer level of an enclosing block (not necessarily the smallest enclosing block). However, if the inner block is a procedure block, the transfer may only be to a statement in the outer level of the main program module.
- From any point in one program module to a labeled statement in the outer level of the main program module. To jump to such a label, you must declare the label to have “extended scope,” i.e., declare it PUBLIC in the main module and EXTERNAL in the module containing the GOTO.

The use of GOTOs is necessary in some situations. However, in most situations where control transfers are desired, the use of an iterative DO, DO WHILE, DO CASE, IF, or a procedure activation (see Chapter 8) is preferable. Indiscriminate use of GOTOs will result in a program that is difficult to understand, correct, and maintain.

6.4 The CALL and RETURN Statements

The CALL and RETURN statements are mentioned here only for completeness, since they do control the flow of a program. However, they are not discussed in detail until Chapter 8.

The CALL statement is used to activate an untyped procedure (one that does not return a value).

The RETURN statement is used within a procedure body to cause a return of control from the procedure to the point from which it was activated.



CHAPTER 7

BLOCK STRUCTURE AND SCOPE

This chapter is intended to clarify the meaning of *outer level* and the concept of *scope*, including the use of the linkage attributes, PUBLIC and EXTERNAL.

The outer level of a block means statements (or labels) contained in the block but not contained in any nested blocks. The term “exclusive extent” also has this meaning. The inner level, or “inclusive extent,” includes this outer level and all nested blocks as well.

A block “at the same level” as another block means both are contained by exactly the same outer blocks.

The scope of an object means those parts of a program where its name, type, and attributes are recognized (i.e., handled according to a given declaration). An object means a variable, label, procedure, or symbolic (named) constant (i.e., a compilation constant or execution constant as discussed in Chapter 3). A program is the complete set of modules that are ultimately executed as a unit.

These definitions are explained further by the text and examples that follow.

7.1 Names Recognized Within Blocks

Throughout this manual we have seen that PL/M-86 is a block-structured language, enabling you to implement your design for solving a problem, processing data, or controlling hardware.

You create blocks of code containing declarations followed by executable statements. You order and nest the blocks in such a way as to simplify and clarify the flow of data and control. (The maximum nest is 18 blocks deep.) A collection of these blocks that performs a single function, or a small set of related functions, is usually compiled as one module, as discussed in Chapter 1.

Beyond the advantages of modularity, simplicity, and clarity, the nesting of blocks serves another very basic purpose: names declared at an outer level are known to all statements of all nested blocks as well.

You can always declare a new meaning for any such name within a nested simple-DO or procedure block, thereby cutting off its earlier meaning for this block. But if you don't choose this option, its meaning is established by a single declaration at an outer level. (The only objects that don't require declarations prior to use are labels and re-entrant procedures.)

In figure 7-1, everything inside the solid line constitutes the inclusive extent of block MMM (in this case, *module* MMM). KK is known throughout this block, including within all nested blocks.

Everything inside the dashed line constitutes the inclusive extent of block SORT. JJ and II are known throughout this block, but not outside it, that is, not before the label SORT or after the END SORT statement.

Everything inside the dotted line constitutes the inclusive extent of block FIND. Since this is not a simple-DO or procedure block, declarations are not allowed. All prior declarations shown are available for use within FIND.

```

MMM: DO; /*Beginning of module*/
      DECLARE RECORD (128) STRUCTURE
            (KEY BYTE,
             INFO WORD);
      DECLARE CURRENT STRUCTURE
            (KEY BYTE,
             INFO WORD);
      DECLARE KK BYTE:
      KK = 127;
      /*Instructions here would read in data.*/

      SORT: DO;
            DECLARE (JJ,II) INTEGER;
            DO JJ = 1 TO 127;
                  CURRENT.KEY = RECORD(JJ).KEY;
                  CURRENT.INFO = RECORD(JJ).INFO;
                  II = JJ;
            FIND: DO WHILE II > 0 AND
                  RECORD(II-1).KEY > CURRENT.KEY;
                  RECORD(II).KEY = RECORD(II-1).KEY;
                  RECORD(II).INFO = RECORD(II-1).INFO;
                  II = II-1;
            END FIND;
            RECORD(II).KEY = CURRENT.KEY;
            RECORD(II).INFO = CURRENT.INFO;
            END;
            END SORT;

      /*Instructions here would write out
        data from the records.*/
END MMM; /*End of module*/

```

Figure 7-1. Inclusive Extent of Blocks

See also figure 7-2.

The shaded area is the exclusive extent (the outer level) of block SORT. The unshaded area *within* SORT is the exclusive (and inclusive) extent of block FIND. To the instructions within the FIND block, SORT's exclusive extent is an outer level. The *outermost* level (or module level) is the area *outside* the solid lines enclosing the SORT block.

Restrictions on Multiple Declarations

In any given block, a known name cannot be redeclared at the same level as its original declaration. A new declaration is permitted inside a nested simple-DO or procedure block, where it automatically identifies a new object despite the existence of the same name at a higher level. The new object will be the only one known by this name within its block, and it will be unknown outside its block, where the prior name maintains its meaning. These observations also apply when a name is redeclared in another block at the same level as the block containing the original declaration.

```

MMM:  DO; /*Beginning of module*/
      DECLARE RECORD (128) STRUCTURE
        (KEY BYTE,
         INFO WORD);
      DECLARE CURRENT STRUCTURE
        (KEY BYTE,
         INFO WORD);
      DECLARE KK BYTE:
      KK = 127;
      /*Instructions here would read in data.*/

      SORT:  DO;
              DECLARE (JJ,II) INTEGER;
              DO JJ = 1 TO 127;
                CURRENT.KEY = RECORD(JJ).KEY;
                CURRENT.INFO = RECORD(JJ).INFO;
                II = JJ;
              FIND:  DO WHILE II > 0 AND
                      RECORD(II-1).KEY > CURRENT.KEY;
                      RECORD(II).KEY = RECORD(II-1).KEY;
                      RECORD(II).INFO = RECORD(II-1).INFO;
                      II=II-1;
                    END FIND;
              RECORD(II).KEY = CURRENT.KEY;
              RECORD(II).INFO = CURRENT.INFO;
              END;
            END SORT;

      /*Instructions here would write out
        data from the records.*/
END MMM;  /*End of module*/

```

Figure 7-2. Outer Level of Block SORT

When a name is declared *only* in a separate block at the same level, there is no way to access it except in that block where it is declared. The definition is not at an outer level to the block in which you are now programming. Any local declaration you supply will establish a new separate object whose values bear no relation to those of the other.

The reason for these rules, as for many in programming, is that there must be no ambiguity about what address/location is meant by each name in the program. The declaration rules above give you freedom to choose whatever names seem appropriate within a given block, without interfering with exterior uses of them. But when you re-declare a name, its outer-level meaning is inaccessible until execution exits the block containing the new declaration. For example:

```

A:  DO;
    DECLARE X, Y, Z BYTE;

L1: X=2;
    Y=X;
    Z=X;

```



```

B:      DO;
        DECLARE X, Y BYTE;
        X=3;
        Y=X;
L2:     Z=X;

END B;

L3:     /*At this point, X=2, Y=2, Z=3, because the
        value of the redeclared X was used to fill
        Z*/

        /*If statement L2 were outside the loop labeled
        B, then Z would be 2 because the outer X
        value would be used.*/

```

7.2 Extended Scope: The PUBLIC and EXTERNAL Attributes

These attributes permit you to extend the scope of names for all objects except modules; a module name may not be declared with either attribute.

By “extend the scope,” we mean make the names available for use in modules other than the one where they are defined. (The names are already available to nested blocks in *this* module.) To be specific, this includes names for variables, labels, procedures, and execution constants.

For example, the statement:

```
DECLARE FLAG BYTE PUBLIC;
```

causes a byte to be allocated, named FLAG, and its address made known to any other module where the following declaration occurs:

```
DECLARE FLAG BYTE EXTERNAL;
```

Similarly, if one module has a procedure declaration block that begins:

```

SUMMER: PROCEDURE (A,B) WORD PUBLIC;
        DECLARE (A,B) BYTE;
        . /*other declarations can go here*/
        . /*executable statements go here,
           defining the procedure*/
        .
END SUMMER;

```

then any other module may invoke SUMMER if it first declares:

```

SUMMER:  PROCEDURE (A,B) WORD EXTERNAL;
        /*A,B can be any names*/
        DECLARE (A,B) BYTE;
        /*but these names must match them,*/
        /*and each type must match its public definition*/
END SUMMER;

```

Since no ambiguity of location or definition is permissible, the use of PUBLIC and EXTERNAL must follow a strict set of rules, as follows:

1. These attributes may only be used in a declaration at the outermost level of a module, i.e., never in a nested block.
2. Only one may appear on any declaration, and only once. Thus:

```
DECLARE ZETA BYTE PUBLIC EXTERNAL; /*error*/
DECLARE RHO WORD PUBLIC PUBLIC; /*error */
```

and similar constructs are all invalid.

3. Names may be declared PUBLIC, at most, once. The PUBLIC declaration is the defining declaration: the address it creates is used in each procedure or module where the same name is declared EXTERNAL. Clearly you must not create more than one PUBLIC address for any name.
4. Names may be declared EXTERNAL only if they are also declared PUBLIC in a different module of the program. The EXTERNAL attribute is essentially a request to use a PUBLIC address. An EXTERNAL without a PUBLIC is a dead letter. Lack of a definition elsewhere will result in a link-time error.
5. Where the name is declared EXTERNAL, it must be given the same type as where it is declared PUBLIC. Any contradiction of type would violate the intention to use the location(s) and content(s) defined elsewhere. If the name is declared PUBLIC and has the DATA attribute, all EXTERNAL declarations must also use DATA.
6. Similarly, names declared EXTERNAL must not be given a location (using the AT clause), or an initialization (using DATA or INITIAL). Such usage would contradict the fact that names are being defined in another module. However, in the module where this name is declared PUBLIC, the use of AT, DATA, or INITIAL is allowed.
7. Neither PUBLIC nor EXTERNAL may be applied to a name that is based. For example:

```
DECLARE PTR1 POINTER;
DECLARE V1 BASED PTR1 PUBLIC;
```

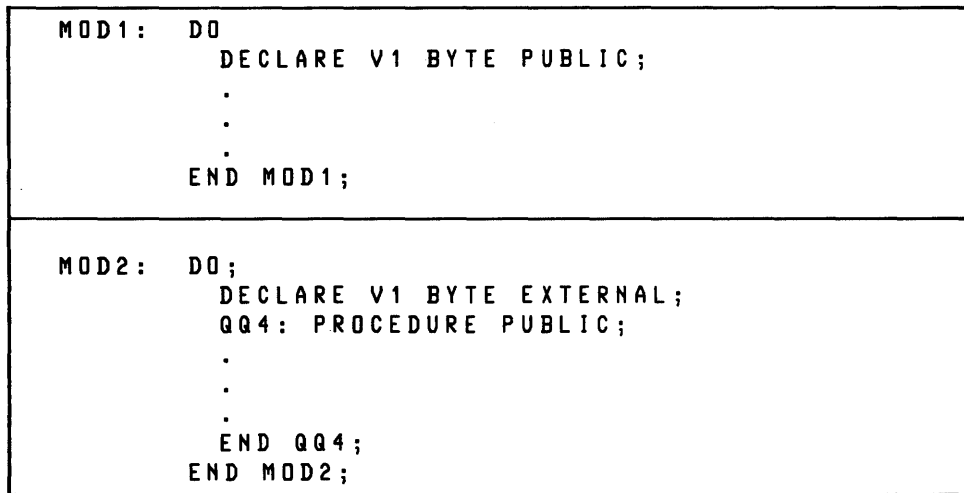
is invalid. The reason: by definition, V1 has no home of its own; its location is always determined by PTR1. Thus, to declare V1 PUBLIC or EXTERNAL does not permit the correct assignment of addresses. PTR1, on the other hand, always contains the current address of V1. Declaring the base, in this case PTR1, to be PUBLIC and EXTERNAL is always permissible since it permits valid results.

NOTE

The PL/M-86 compiler will generate external records only for items that are actually referenced in the program.

(Four additional restrictions on the use of PUBLIC and EXTERNAL procedures appear in Chapter 8.)

Following these rules will permit consistent and reliable execution of programs using names with extended scope. A PUBLIC definition occurring in one module will then be used by all related references to that name in separate modules; that is, references which declare the name EXTERNAL. The diagram below illustrates this:



Both references to V1 will use the same definition (location) for V1, namely, the definition in module MOD1. Similarly, if any module needed to call procedure QQ4, it would first need a declaration like this:

```
QQ4: PROCEDURE EXTERNAL ;
END QQ4 ;
```

so that a subsequent CALL QQ4 would correctly pass control to that procedure in module MOD2.

7.3 Scope of Labels and Restrictions on GOTOs

Labels are subject to exactly the same rules of scope discussed above.

One consequence is that a label is unknown outside the block where it is declared. As discussed in Chapter 1, a label is either declared explicitly at the beginning of a simple-DO or procedure block, or the compiler *considers* it to be declared there as soon as it is defined by use anywhere in the block. Therefore, the discussion of what names are known in which blocks applies directly to labels as well as to other names.

The label on a block is not part of the block it names. For example, the name on the DO enclosing the module itself is not part of that DO; it merely names it. For nested blocks, a label is again not part of the block it names, but belongs instead to the outer level as part of that first enclosing block.

If a name used as a label on a block is defined inside that block, it will name a new thing, be it label, variable, or constant. There will be no confusion with the outer label name. This fact leads to important restrictions on the use of the GOTO statement:

1. It is impossible for a GOTO to transfer control from an outer block to a labeled statement inside a nested block.
2. Moreover, a GOTO can transfer control from one block to another in the same module *only* if the target block encloses the one containing the GOTO (and only if the name of that target label is not declared in the nested block.)

Furthermore, a label with the PUBLIC attribute is permitted only in the main module. (This has the interesting consequence of forcing all other transfers of control—that is, those not involving a return to the main module—to use procedure calls. This favors the development of orderly, modularized, traceable programs.)

In fact, only four GOTO transfers are possible, as follows:

1. From one point in a block to another statement also in the same level of the same block
2. From an inner, nested DO-block (not a nested procedure) to a statement in the outer level of *any* enclosing block
3. From a procedure to a statement in the outer level of the main program in the same module
4. To a main-program label that is declared PUBLIC, from *any* point in any module that declares that label EXTERNAL

(Recall that *only* labels at the outer level of a main program may be declared PUBLIC.)

Given the program structure and declarations shown in figure 7-3, the only valid GOTO transfers are shown in figure 7-4. A single-headed arrow means the transfer is valid only in the direction shown. A double-headed arrow means that a GOTO can be used in either direction.

```

MAIN:   DO;
        DECLARE (LAB33, LAB77) LABEL PUBLIC;
        DECLARE IT BYTE;
        .
        .
        .
LAB33:  . . . ;
        DO;
        .
        .
        .
        END;
        .
        .
        .
LAB77:  . . . ;
        DO WHILE IT > 0;
        .
        .
        .
        END;
        .
        .
        .
END MAIN;

```

Figure 7-3. Sample Program Modules Illustrating Valid GOTO Usage

```

MOD1: DO;
    P1: PROCEDURE;
        DECLARE (LAB33,LAB77) LABEL EXTERNAL;
        L1: . . . ;
        DO;
            DECLARE KO BYTE;
            P2: PROCEDURE
                .
                .
                .
                L2: . . . ;
                .
                .
                .
            END P2;
        L3: . . . ;
        .
        .
        .
        END P1;
    END MOD1;

```

```

MOD2: DO;
    .
    .
    .
    P4: PROCEDURE;
        DECLARE (LAB33,LAB77) LABEL EXTERNAL;
        .
        .
        .
        L4: . . . ;
        .
        .
        .
        L5: . . . ;
        DO;
            .
            .
            .
            L6: . . . ;
            .
            .
            .
        END;
        .
        .
        .
        L7: . . . ;
        .
        .
        .
        END P4;
    L8: . . . ;
    END MOD2;

```

Figure 7-3. Sample Program Modules Illustrating Valid GOTO Usage (Cont'd.)

Figure 7-4 illustrates legal GOTO transfers that are the only transfers permitted among the given labels in figure 7-3.

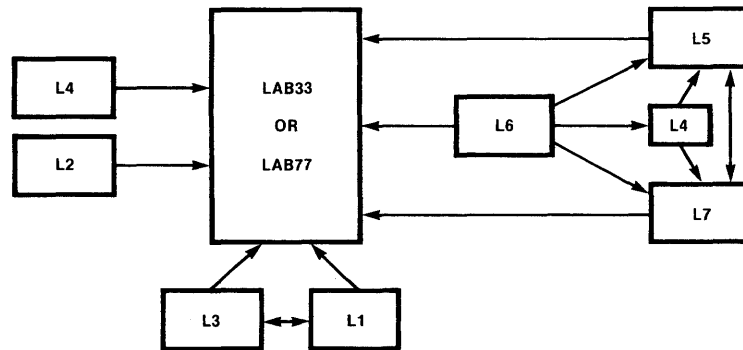


Figure 7-4. Valid GOTO Transfers

121636-1



A procedure is a section of PL/M-86 code that is *declared* without being executed, and then *activated* from other parts of the program. A *function reference* or *CALL statement* activates the procedure, causing the procedure code to be executed (even if it is physically located elsewhere). Program control is then transferred from the point of activation to the beginning of the procedure code, and the code is executed. Upon exit from the procedure code, program control is passed back to the statement immediately after the point of activation.

The use of procedures forms the basis of modular programming. It facilitates making and using program libraries, eases programming and documentation, and reduces the amount of object code generated by a program. The following sections review how to declare and activate procedures.

8.1 Procedure Declarations

You must declare procedures, just as you must declare variables. Thereafter, any reference to a procedure must occur within the scope defined by the procedure declaration. Also, a procedure may not be used (called, or invoked in an expression) until *after* the END statement of the procedure declaration (unless re-entrant—see section 8.5).

A procedure declaration consists of three parts: a PROCEDURE statement, a sequence of statements forming the “procedure body,” and an END statement.

The following is a simple example:

```
DOOR$CHECK:PROCEDURE;  
    IF FRONT$DOOR$LOCKED AND SIDE$DOOR$LOCKED  
    THEN CALL POWER$ON;  
    ELSE CALL DOOR$ALARM;  
END DOOR$CHECK;
```

where POWER\$ON and DOOR\$ALARM are procedures declared elsewhere in the same program.

NOTE

The name in a PROCEDURE statement has the same appearance as a label definition—but it is *not* considered a label definition, and a procedure name is not a label. PROCEDURE statements may not be labeled.

The name is a PL/M-86 identifier, which is associated with this procedure. The scope of a procedure is governed by the placement of its declaration in the program text, just as the scope of a variable is governed by the placement of its DECLARE statement (see Chapter 7 for a detailed description). Within this scope, the procedure can be activated by the name used in the PROCEDURE statement.

A procedure declaration, like a DO block, controls the scope of variables as described in Chapter 7. Also, like a simple DO block, a procedure declaration may contain DECLARE statements, which must precede the first executable statement in the procedure body.

As in a DO block, the identifier in the END statement has no effect on the program, but helps legibility and debugging. If used, it must be the same as the procedure name.

The parameter list and the type are discussed in the following two sections.

Parameters

Formal parameters are non-based scalar variables declared within a procedure declaration; their identifiers appear in the parameter list in the PROCEDURE statement. The identifiers in the list are separated by commas and the list is enclosed in parentheses. No subscripts or member-identifiers are allowed in the parameter list.

If the procedure has no formal parameters, the parameter list (including the parentheses) is omitted from the PROCEDURE statement.

Each formal parameter must be declared as a non-based scalar variable in a DECLARE statement preceding the first executable statement in the procedure body. However, procedure parameters are not stored according to the same rules as other declared variables. In particular, do not assume that a parameter is stored contiguously with other variables declared in the same factored variable declaration.

When a procedure that has formal parameters is activated, the CALL statement or function reference contains a list of *actual parameters*. Each actual parameter is an expression whose value is assigned to the corresponding formal parameter in the procedure before the procedure begins to execute.

For example, the following procedure takes four parameters, called PTR, N, LOWER, and UPPER. It examines N contiguously stored BYTE variables. The parameter PTR is the location of the first of these variables. If any of these variables is less than the parameter LOWER or greater than the parameter UPPER, the ERRORSET procedure (declared elsewhere in the program) is activated:

```
RANGE$CHECK: PROCEDURE(PTR,N,LOWER,UPPER);
              DECLARE PTR POINTER;
              DECLARE (N,LOWER,UPPER,1)BYTE;
              DECLARE ITEM BASED PTR(1)BYTE;

              DO I = 0 TO N-1;
                IF (ITEM(I) < LOWER) OR (ITEM(I) > UPPER)
                  THEN CALL ERRORSET;

              /*ERRORSET is a procedure declared elsewhere*/

              END;
END RANGE$CHECK;
```

Notice that the array ITEM is declared to have only one element. Since it is a based array, a reference to any element of ITEM is really a reference to some location relative to the location represented by PTR. In writing the procedure RANGE\$CHECK, we must supply a dimension specifier above zero for ITEM so that references to ITEM can be subscripted. But it does not matter what the dimension specifier is. We arbitrarily use 1 here.

Having made this declaration, suppose that we have 25 variables stored contiguously in an array called QUANTS. We want to check that all of these variables have values within the range defined by the values of two other BYTE variables, SMALL and LARGE.

We write:

```
CALL RANGE$CHECK (@QUANTS, 25, SMALL, LARGE);
```

When this call statement is processed, the following sequence occurs:

- The four actual parameters in the CALL statement—@QUANTS, 25, SMALL, and LARGE—are assigned to the formal parameters PTR, N, LOWER, and UPPER, which were declared within the procedure RANGE\$CHECK. Since ITEM is based on PTR and the value of PTR is @QUANTS, every reference to an element of ITEM becomes a reference to the corresponding element of QUANTS.
- The executable statements of the procedure RANGE\$CHECK are executed, and if any of the values are less than the value of SMALL or greater than the value of LARGE, the procedure ERRORSET is activated.
- Finally, control returns to the statement following the CALL statement.

Notice how the use of a based variable, with the base passed as a parameter, allows the procedure to have its own unchanging name (ITEM) for a set of variables which may be a different set each time the procedure is activated.



When a procedure has more than one parameter, PL/M-86 does not guarantee the order in which actual parameters will be evaluated when the procedure is activated. If one actual parameter changes another actual parameter, the results are *undefined*. This can occur if an expression used as an actual parameter contains an embedded assignment or function reference which changes another actual parameter for the same procedure. See the next caution also.

Typed Versus Untyped Procedures

The RANGE\$CHECK procedure (shown above) is an *untyped* procedure. No type is given in the PROCEDURE statement, and it does not return a value. An untyped procedure is activated by using its name in a CALL statement, as explained in section 8.2.

A *typed procedure*, also called a *function*, has a type in its PROCEDURE statement: BYTE, WORD, DWORD, INTEGER, REAL, POINTER, or SELECTOR. Such a procedure returns a value of this type, which is used in an expression or stored as the value of a variable. The procedure is activated by using its name as an operand in an expression; this special type of variable reference is called a “function reference.”

When the expression is processed at run time, the function reference causes the procedure to be executed. The function reference itself is then replaced by the value returned by the procedure. The expression containing the function reference is then evaluated, and program execution continues in normal sequence.

Like an untyped procedure, a typed procedure may have parameters. They are handled as described in the previous section.

The body of a typed procedure must always contain a RETURN statement with an expression, as explained later in this chapter.



The body of a typed procedure may contain code (such as an assignment statement) that changes the value of some variable declared outside the procedure. This is called a “side effect.”

Recall that PL/M-86 does not guarantee the order in which operands in an expression are evaluated. Therefore, if a function used in an expression changes the value of another variable in the same expression, the value of the expression depends on whether the function reference or the variable is evaluated first.

If the analysis of the expression does not force one of these operands to be evaluated before the other, then the value of the expression is *undefined*.

This situation can be avoided by using parentheses to segregate any typed procedure that has a side effect, or by using this procedure in an assignment statement first to create an unambiguous sequence.

8.2 Activating a Procedure—Function References and CALL Statements

The two forms of procedure activation depend on whether the procedure is typed or untyped. An untyped procedure is activated by means of a **CALL** statement, which has the form:

CALL *name*

or:

CALL *name (parameter list);*

An example is the following:

CALL REORDER (@RANK\$TABLE, 3);

(An alternate form of the **CALL** statement is discussed later.)

A typed procedure is activated by means of a function reference, which is an operand in an expression. A function reference has the form:

name

or

name (parameter list)

This occurs as an *operand* in an expression, as in the following example:

TOTAL = SUBTOTAL + SUM\$ARRAY (@ITEMS, COUNT);

where **SUM\$ARRAY** is a previously declared typed procedure. The value added to **SUBTOTAL** will be the value returned by **SUM\$ARRAY** using the actual parameters (**@ITEMS, COUNT**). See the cautionary note in section 8.1.

In both forms of procedure activation, the elements of the parameter list are called “actual parameters” to distinguish them from the “formal parameters” of the procedure declaration. At the time of activation, each actual parameter is evaluated and the result is assigned to the corresponding formal parameter in the procedure declaration. Then, the procedure body is executed. Any PL/M-86 expression may be an actual parameter if its type is the same as that of the corresponding formal parameter.

The actual parameter list in a procedure activation must also match the formal parameter list in the procedure declaration—that is, it must contain the same number of parameters of the same type in the same order. If the procedure is declared without a formal parameter list, then no actual parameter list can be used in the activation.

As in expression evaluation and assignment statements (see Chapter 4), a few type conversions are performed automatically when necessary in activating and returning from a procedure. The built-in explicit type conversion procedures of Chapter 9 can also be used to force the value of an expression to a desired type.

Indirect Procedure Activation

The CALL statement, in the form shown above, activates an untyped procedure by its name. It is also possible to activate an untyped procedure by its location. This is done by means of a CALL statement with the form:

```
CALL identifier [.member-identifier][(parameter list)];
```

The identifier may not be subscripted, though it may be a structure reference. It must be a fully qualified POINTER or WORD type variable reference, and its value is assumed to be the location of the entry-point of the procedure being activated.

NOTE

Calls through 32-bit POINTERS will be translated into “long” calls while those through 16-bit WORDs or POINTERS (in the SMALL case) will be translated into “short” calls (relative to the current code segment). The compiler will issue a warning if the wrong addressing type is used to gain a procedure address for later indirect calls.

A normal CALL uses the *name* of the procedure; the compiler checks to make sure that the correct number of parameters is supplied and performs automatic type conversion on the actual parameters.

When the CALL statement uses a *location*, the compiler does not check the number of parameters or perform type conversion. If the number of parameters is wrong or if an actual parameter is not of the same type as the corresponding formal parameter, the results are unpredictable.

8.3 Exit from a Procedure: The RETURN Statement

The execution of a procedure is terminated in one of three ways:

- By execution of a RETURN statement within the procedure body. A typed procedure *must* terminate with a RETURN statement that has an expression.
- By executing a GOTO to a statement outside the procedure body. The target of the GOTO must be at the outer level of the main program (see Chapter 7).
- By reaching the END statement that terminates the procedure declaration.

The RETURN statement takes one of two forms:

```
RETURN ;
```

or

```
RETURN expression;
```

The first form is used in an untyped procedure. The second form is used in a typed procedure. The value of *expression* becomes the value returned by the procedure. It is evaluated as if it were being assigned to a variable of the same type as used on the PROCEDURE statement.

8.4 The Procedure Body

The statements within the procedure body may be any valid PL/M-86 statements, including CALL statements and nested procedure declarations.

Examples

1. The following is a typed procedure declaration:

```
AVG: PROCEDURE (X,Y) REAL;
      DECLARE (X,Y) REAL;
      RETURN (X + Y)/2.0;
END AVG;
```

This procedure could be used as follows:

```
LOW  = 3.0;
HIGH = 4.0;
MEAN = AVG (SMALL, LARGE);
```

The affect would be to assign the value 3.5 to MEAN.

2. The following is an untyped procedure:

```
ADUT: PROCEDURE (ITEM);
      DECLARE ITEM WORD;
      IF ITEM = 0FFH THEN COUNTER = COUNTER + 1;
      RETURN;
END ADUT;
```

Here COUNTER is some variable declared outside the procedure, i.e., it is a "global" variable. This procedure could be activated as follows:

```
CALL ADUT (UNKNOWN);
```

If the value of the variable UNKNOWN is greater than or equal to 0FFH, the value of COUNTER will be incremented.

3. This example demonstrates an important use of based variables:

```
SUM$ARRAY: PROCEDURE (PTR,N) BYTE;
      DECLARE PTR POINTER,
      ARRAY BASED PTR(1) BYTE,
      (N,SUM,I) BYTE;
```

```

        SUM = 0;
        DO I = 0 TO N;
            SUM = SUM + ARRAY(I);
        END;
        RETURN SUM;
    END SUM$ARRAY;

```

This procedure returns the sum of the first $N + 1$ elements (from the 0th to the Nth) of a BYTE array pointed to by PTR. Notice that ARRAY is declared to have 1 element. Since it is a based variable, no space is allocated for it. It must be declared as an array (with a non-zero dimension) so that it can be subscripted in the iterative DO block. The choice of 1 as the constant in the dimension specifier is arbitrary and does not restrict the value of N that may be supplied when the procedure is activated.

The procedure could be used as follows to sum the elements of a 100-element BYTE array named PRICE, and to assign the sum to the variable TOTAL:

```
TOTAL = SUM$ARRAY(@PRICE,99);
```

8.5 The Attributes: PUBLIC and EXTERNAL, INTERRUPT, REENTRANT

The PUBLIC and EXTERNAL attributes can be included in PROCEDURE statements to give procedures extended scope. Extended scope is discussed in Chapter 7.

A procedure declaration with the PUBLIC attribute is called a “defining declaration.” A procedure declaration with the EXTERNAL attribute is called a “usage declaration.” Most of the rules for PUBLIC and EXTERNAL appear in Chapter 7. The following additional rules apply to the use of the EXTERNAL attribute in a procedure declaration:

1. The EXTERNAL attribute may not be used in the same PROCEDURE statement as a PUBLIC or REENTRANT attribute (see below). Note, however, that the defining declaration of a procedure may have the REENTRANT attribute.
2. A usage (EXTERNAL) declaration of a procedure should have the same number of parameters as the defining (PUBLIC) declaration. Variable types and dimension specifiers should match up in the same sequence in both declarations. The names of the parameters need not be the same. Note that a discrepancy between the parameter lists in the defining declaration and in a usage declaration will not be automatically detected. (See Chapter 11 for a description of the TYPE control to detect such an error at module linkage time.)
3. The procedure body of a usage declaration may not contain anything except the declarations of the formal parameters. The formal parameters must be declared with the same types as in the defining declaration.
4. No labels may appear in a usage declaration.

NOTE

The PL/M-86 compiler will only generate external records for items that are actually referenced in the program.

For example, we can alter the procedure AVG (from example 1 above) by giving it the PUBLIC attribute:

```

AVG: PROCEDURE (X,Y) REAL PUBLIC;
    DECLARE (X,Y) REAL;
    RETURN (X + Y)/2.0;
END AVG;

```

In another module, we can have a usage declaration:

```
AVG: PROCEDURE (X,Y) REAL EXTERNAL;
      DECLARE (X,Y) REAL;
END AVG;
```

Now, in the module with the usage declaration, we can reference AVG in an executable statement:

```
MIDDLE = AVG (FIRST, LATEST);
```

thereby activating the procedure AVG as declared in the first module.

Interrupts and the INTERRUPT Attribute

The INTERRUPT attribute allows you to define a procedure to handle some condition signaled by an iAPX 86 interrupt, e.g., from a peripheral device. A procedure with this attribute is activated when the corresponding interrupt signal is received in the iAPX 86-based system. The PL/M-86 statement CAUSE\$INTERRUPT (*constant*) can also be used to initiate an interrupt signal (see Chapter 10).

Note that the following discussion applies only to interrupt procedures; interrupt tasks are discussed in Appendix G.

The INTERRUPT attribute can be used only in declaring an untyped procedure with no parameters at the outermost level of a program module. It must be declared PUBLIC or EXTERNAL (and optionally REENTRANT). The form is

```
INTERRUPT n
```

where *n* is any decimal number from 0 to 255. Each number can only be used once in a program. Each such procedure is then referred to as an interrupt procedure. Such a procedure is necessary if you wish to provide non-default handling of exception conditions arising in floating-point arithmetic (see appendixes F and G).

The following discussion of the iAPX 86 interrupt mechanism clarifies how interrupt procedures work. Additional information can be found in Appendix G.

The iAPX 86 interrupt mechanism has two states: *enabled* or *disabled*. The ENABLE statement allows interrupts to take affect and the DISABLE statement prevents them from having any affect. The HALT statement also enables interrupts. (The state of the iAPX 86 interrupt mechanism upon initialization is determined by the operating system.)

When some peripheral device sends an interrupt to the iAPX 86 CPU, it is ignored if the interrupt mechanism is disabled. If interrupts are enabled, the interrupt is processed as follows:

1. The CPU completes any instruction currently in execution.
2. The CPU sends an "acknowledge interrupt" signal, then the interrupting device sends its interrupt number.
3. The interrupt mechanism is disabled. This prevents any other device from interfering.
4. Control passes to the interrupt procedure whose number matches the number sent by the peripheral device. If no such procedure has been established, the results are undefined, since the vector that transfers control may be uninitialized.

5. When the procedure is through (by executing a RETURN or reaching the END of the procedure), the interrupt mechanism is enabled so other devices may be serviced, and control returns to the point where the interrupt occurred.

It is possible (as with other untyped procedures) for the procedure to terminate by executing a GOTO with a target outside the procedure in the outer level of the main program module. In this case, control will never be returned to the point where the program was interrupted, and interrupts will not be enabled automatically.

The following is an example of an interrupt procedure for a hypothetical system where a peripheral device initiates an interrupt whenever the temperature of a device exceeds a certain threshold. The interrupt procedure turns on the annunciator light, updates a status word, and returns control to the program:

```
HITEMP; PROCEDURE INTERRUPT PUBLIC:
    CALL ANNUNCIATOR(1);
    /* This will result in an output
       from the iAPX 8086 to turn on
       annunciator light number 1, the
       high-temperature warning. */

    ALERT = ALERT OR 00000010B;
    /* This puts a 1 in one of the
       bit positions of ALERT, which
       contains a bit pattern represent-
       ing current alerts. */

END HITEMP;
```

Activating an Interrupt Procedure with a CALL Statement

A procedure with the INTERRUPT attribute may also be activated by means of a CALL statement, like any other untyped procedure. However, when this is done, the programmer must bear in mind that interrupts are *not* automatically disabled upon activation of the procedure. If interrupts are enabled when the CALL is executed, then unless the procedure has a DISABLE as its first executable statement, it will run with interrupts enabled and should have the REENTRANT attribute (see next section).

WARNING

If using an interrupt procedure that will call another interrupt procedure, you must manually extend the length of the stack storage, as the compiler allocates only enough space to save one procedure. See Appendix G and use the LINK and LOC sequence (the *Family Utilities User's Guide*) for more information.

In every other respect, an interrupt procedure activated by a CALL statement is like any other procedure so activated.

NOTE

Unlike PL/M-80, PL/M-86 interrupt routines activated with a CALL statement do not alter the interrupt enable status. This means that termination of the procedure by means of a RETURN statement or the END statement will not automatically enable interrupts.

Section 6 of Chapter 9 discusses the built-in function `INTERRUPT$PTR`, which returns the interrupt entry point, given an interrupt procedure name, and also the built-in procedure `SET$INTERRUPT`, which sets an interrupt vector given the interrupt procedure name and number.

The `CAUSE$INTERRUPT` statement causes a software interrupt to the vector specified in the statement:

```
CAUSE$INTERRUPT(constant);
```

where *constant* is in the range 0 to 255.

Reentrancy and the REENTRANT Attribute

The `REENTRANT` attribute allows a procedure to suspend execution temporarily, restart with new parameters, and then later complete the original execution successfully as if there had been no interruption.

This ability is desirable in two circumstances: (1) if your procedure (`PROC1`) activates itself (called *direct recursion*), or (2) if your procedure activates another procedure (`PROC2`) that will reactivate `PROC1` before `PROC1` has finished its original processing (called *indirect recursion*).

Without the `REENTRANT` attribute, storage for procedure variables is allocated statically, in fixed locations within the Data Segment of the object module. Re-entering such a procedure would write over the earlier contents of such locations making it impossible to complete the original, suspended execution.

When you use the attribute `REENTRANT` in declaring a procedure, its variables are not stored with other variables in the data section, but are stored on the stack. Thus preserved, each set can be used independently by each invocation of the procedure.

Multiple sets of variables may therefore need to be stored on the stack during recursive use of such procedures. You must be sure to specify (at relocation and link time) a stack size large enough for all such storage needed by all multiple invocations that may be active at one time.

A procedure with the `REENTRANT` attribute may be activated before it is declared. This permits *direct recursion*, where the procedure activates itself, and *indirect recursion*, where the procedure activates a second procedure and the second procedure activates the first—or activates a third procedure, which activates a fourth, etc., with the result that the first procedure is activated before it terminates.

The following rules summarize the use of the `REENTRANT` attribute:

- Any procedure that may be interrupted and is also activated from within an interrupt procedure should have the `REENTRANT` attribute.
Note that this may apply to an interrupt procedure that runs with interrupts enabled because it contains an `ENABLE` statement. If there is any possibility that it will be interrupted by its own interrupt, it should have the `REENTRANT` attribute. This situation is equivalent to recursion.
- Any procedure that is directly recursive (activates itself) should have the `REENTRANT` attribute.
- Any procedure that is indirectly recursive (activates another procedure and is activated itself as a result) should have the `REENTRANT` attribute.

- Any procedure that is activated by a reentrant procedure should also have the REENTRANT attribute.

In other words: if there is any possibility that a procedure can be activated while it is already running, it should be reentrant.

- The REENTRANT attribute cannot be used in the same declaration as the EXTERNAL attribute. (It may be used with the PUBLIC attribute.)
- The REENTRANT attribute may only be used in a PROCEDURE statement at the outer level of a module (discussed in Chapter 7).
- A procedure declaration with the REENTRANT attribute may not have another procedure declaration nested inside it.



CHAPTER 9

BUILT-IN PROCEDURES, FUNCTIONS, AND VARIABLES

Built-in procedures, functions, and variables act as if they were declared in an all encompassing global block invisible to the programmer.

These identifiers are subject to the rules of scope. This means the name of a built-in procedure or variable can be declared to have a local meaning within the program. Within the scope of such a declaration, the built-in is unavailable. This distinguishes these identifiers from reserved words, listed in Appendix A, which cannot be used as identifiers in declarations.

No built-in procedure may be used within a location reference. No built-in variable may be used within a location reference, except as specifically noted in the following sections.

9.1 Obtaining Information About Variables

PL/M-86 has three built-in procedures that take variable names as actual parameters and return information based on the declarations of the variables: LENGTH, LAST, and SIZE.

The LENGTH Function

LENGTH is a WORD function that returns the number of elements in an array. It is activated by a function reference with the form:

`LENGTH (variable-ref)`

where

variable-ref must be a *non-subscripted* reference to an array.

The array may be a member of a structure; it may not be an EXTERNAL array using the implicit dimension specifier (see section 3.2).

The WORD value returned is the number of elements in the array—that is, it is equal to the dimension specifier in the array declaration.

If the array is not a structure member, then the reference must be an unqualified variable reference. If the array is a structure member, then the reference is a partially qualified variable reference (see section 5.3). For example, given the declaration:

```
DECLARE RECORD STRUCTURE (KEY BYTE,  
                           INFO(3) WORD);
```

then LENGTH(RECORD.INFO) is a valid function reference and returns a WORD value of 3.

If the array is a member of a structure, and the structure is an element of an array, a special case arises. Given the declaration:

```
DECLARE LIST (4) STRUCTURE (KEY BYTE,  
                           INFO (3) WORD);
```

then all of the following function references are correct and return the value 3.

```
LENGTH(LIST(0).INFO)
LENGTH(LIST(1).INFO)
LENGTH(LIST(2).INFO)
LENGTH(LIST(3).INFO)
```

In other words, the subscript for the array LIST is irrelevant when a member-identifier is supplied, since the arrays within the structures are all the same length. PL/M-86 allows a "shorthand" form of partially qualified variable reference in the LENGTH, LAST, and SIZE function references. For example:

```
LENGTH(LIST.INFO)
```

is a valid function reference and returns the value 3.

The LAST Function

LAST is a WORD function that returns the subscript of the last element in an array. It is activated by a function reference with the form:

```
LAST ( variable-ref )
```

where

variable-ref must be a *non-subscripted* reference to an array.

The array may be a member of a structure; it may not be an EXTERNAL array using the implicit dimension specifier (see section 3.2).

The WORD value returned is the subscript of the last element of the array. Note that for a given array, LAST will always be one less than LENGTH.

As in the LENGTH function, a "shorthand" form of partially qualified variable reference is allowed in the case where the array is a member of a structure and the structure is an array element.

The SIZE Function

SIZE is a WORD function that returns the number of bytes occupied by an array or structure. It is activated by a function reference with the form:

```
SIZE ( variable-ref )
```

where

variable-ref is a fully qualified, partially qualified, or unqualified reference to any scalar, array, or structure. The *variable-ref* may not be an EXTERNAL declaration that uses the implicit dimension specifier (see section 3.2).

The WORD value returned is the number of bytes required by the object referenced.

If the reference is fully qualified, it refers to a scalar, and the value is the number of bytes required for the scalar. If the reference is unqualified, it refers to an entire structure or array, and the value is the total number of bytes required for the structure or array.

If the reference is partially qualified, it refers either to a structure member that is an array, or to an array element that is a structure. The value is the number of bytes required for the array or structure.

As in the LENGTH function, a “shorthand” form of partially qualified variable reference is allowed in the case where the array or scalar is a member of a structure and the structure is an array element.

9.2 Explicit Type and Value Conversions

The ten functions in this section provide explicit conversion from one type to another and from signed values to or from absolute magnitudes.

Explicit type-conversion functions are invoked as

function-name (*expression*)

where

function-name has a type and *expression* has a value.

In table 9-1, each function name is followed by its type, the expression type expected, the purpose of the function, and the nature of the value it returns to the expression that invoked it.

Table 9-1. Explicit Type and Value Conversions

Procedure Name	Type	Parameter Expression Type	Function	Result Returned
LOW	BYTE	BYTE WORD	Converts WORD value to BYTE value	BYTE value unchanged Low-order byte of WORD
	WORD	DWORD	Converts DWORD value to WORD value	Low-order word of DWORD
HIGH	BYTE	BYTE WORD	Converts WORD value to BYTE value	0 (zero) High-order byte of WORD
	WORD	DWORD	Converts DWORD value to WORD value	High-order word of DWORD
DOUBLE	WORD	BYTE	Converts BYTE value to WORD value	WORD value, by appending 8 high-order zero bits
	DWORD	WORD DWORD	Converts WORD value to DWORD value	DWORD value, by appending 16 high-order zero bits DWORD value unchanged
FLOAT	REAL	INTEGER	Converts INTEGER value to REAL value	Same value of type REAL
FIX	INTEGER	REAL	Converts REAL value to INTEGER value (rounds toward zero)	INTEGER value modulo 32768, i.e., within range ± 32767
INT	INTEGER	BYTE WORD	Converts BYTE or WORD to INTEGER; interprets parameter as positive	Corresponding INTEGER value within range 0 to 32767

Table 9-1. Explicit Type and Value Conversion (Cont'd.)

Procedure Name	Type	Parameter Expression Type	Function	Result Returned
SIGNED	INTEGER	BYTE WORD	Converts a WORD value to an INTEGER value	BYTE value is extended with 8 high-order zeros; WORD value unchanged
UNSIGN	WORD	INTEGER	Converts an INTEGER value to a WORD value	The bit pattern is unchanged but can now be used in WORD expressions
ABS	REAL	REAL	Converts negative value to positive	Absolute value of expression supplied. If positive, returned unchanged; if negative, $-(expression)$ is returned
IABS	INTEGER	INTEGER	Converts negative value to positive	Absolute value of expression supplied. If positive, returned unchanged; if negative, $-(expression)$ is returned

The LOW, HIGH, and DOUBLE Functions

LOW and HIGH are BYTE functions that convert WORD values to BYTE values, or WORD functions that convert DWORD values to WORD values. They are activated by function references with the forms:

LOW (*expression*)
HIGH (*expression*)

where

expression has a DWORD, WORD, or BYTE value.

If *expression* has a DWORD value, LOW returns the value of the low-order (least significant) word of the expression value whereas HIGH returns the value of the high-order (most significant) word of the expression value.

If *expression* has a WORD value, LOW returns the value of the low-order (least significant) byte of the expression value whereas HIGH returns the value of the high-order (most significant) byte of the expression value.

If *expression* has a BYTE value, then LOW will return this value unchanged. However, HIGH will return 0.

DOUBLE is a WORD function that converts a BYTE value to a WORD value, or a DWORD function that converts a word value to a DWORD value. It is activated by a function reference with the form:

DOUBLE *expression*

where

expression has a BYTE, WORD, or DWORD value.

If *expression* has a BYTE value, the function appends 8 high-order zero bits to convert it to a WORD value and returns this WORD value. If *expression* has a WORD value, the function appends 16 high-order zero bits to convert it to a DWORD value and returns this value. If *expression* has a DWORD value, the function returns it unchanged.

The FLOAT Function

FLOAT is a REAL function that converts an INTEGER value to a REAL value. It is activated by a function reference, with the form:

F L O A T (*expression*)

where

expression has an INTEGER value.

FLOAT converts the INTEGER value to the corresponding REAL value and returns this REAL value.

The FIX Function

FIX is an INTEGER function that converts a REAL value to an INTEGER value. It is activated by a function reference with the form:

F I X (*expression*)

where

expression has a REAL value.

FIX rounds the REAL value to the nearest INTEGER. If both INTEGERS are equally near, FIX rounds to the even one. The resulting INTEGER value is then returned. Thus, FIX(1.4) would result in the INTEGER value 1, FIX(−1.8) in −2, FIX(3.5) in 4, and FIX(6.5) in 6.

If the result calculated by FIX is not within the implemented range of INTEGER values, the result is *undefined*.

NOTE

FIX is affected by your choice of rounding mode—see section 10.5. The above examples assume the default mode, which is “round to nearest or even.”

The INT Function

T is an INTEGER function that converts a BYTE or WORD value to an INTEGER value. It is activated by a function reference with the form:

I N T (*expression*)

where

expression has a BYTE or WORD value.

INT interprets the BYTE or WORD value as a positive number and returns the corresponding INTEGER value.

If the result calculated by INT is not within the implemented range of INTEGER values, the result is *undefined*.

The SIGNED Function

SIGNED is an INTEGER function that converts a BYTE or WORD value to an INTEGER value. It is activated by a function reference with the form:

SIGNED (*expression*)

where

expression has a WORD or BYTE value.

If *expression* has a BYTE value, it will be extended by 8 high-order zero bits to produce a WORD value.

SIGNED interprets the WORD value as a 16-bit two's-complement number and returns the corresponding integer value.

This means that if the highest-order (most significant) bit of the WORD valued is a 0, SIGNED interprets the WORD value as a positive number and returns the corresponding INTEGER value. For example:

SIGNED (0000\$0000\$0000\$0100B)

returns an INTEGER value of 4.

But if the highest-order bit of the WORD value is a 1, SIGNED returns a negative INTEGER value whose absolute magnitude is the two's complement of the WORD value. For example:

SIGNED(1111\$1111\$1111\$1100B)

returns an INTEGER value of -4.

The UNSIGN Function

UNSIGN is a WORD function that converts an INTEGER value to a WORD value. It is activated by a function reference with the form:

UNSIGN (*expression*)

where

expression has an INTEGER value.

UNSIGN converts the INTEGER value to a WORD value.

If the INTEGER value is positive, then the WORD value will be numerically the same as the INTEGER value. But if the INTEGER value is negative, then the WORD value will be the two's complement of the absolute magnitude of the INTEGER value. For example:

UNSIGN(-4)

returns a WORD value of

1111\$1111\$1111\$1100B

The ABS and IABS Functions

ABS is a REAL function that returns the absolute value of a REAL value. It is activated by a function reference with the form:

ABS (*expression*)

where

expression has a REAL value.

If the value of *expression* is positive, ABS returns it unchanged. If the value of *expression* is negative, ABS returns $-(\textit{expression})$.

IABS is an INTEGER function that returns the absolute value of an INTEGER value. It is activated by a function reference with the form:

IABS (*expression*)

where

expression has an INTEGER value.

If the value of *expression* is positive, IABS returns it unchanged. If the value of *expression* is negative, IABS returns $-(\textit{expression})$.

9.3 Shift and Rotate Functions

In shift and rotate operations, a value is handled as a pattern of 8 bits (for a BYTE value), 16 bits (for a WORD or INTEGER value), or 32 bits (for a DWORD value). The pattern is moved to the right or left by a specified number of bits called the "bit count."

In a shift, bits moved off one end of the pattern are lost, and zero bits move into the pattern from the other end (except in the case of SAR—see below). In a rotate, bits moved off one end move onto the other end.

Rotation Functions ROL and ROR

ROL and ROR are functions whose type depends on the type of the expression given as an actual parameter. They are activated by function references with the forms:

ROL (*pattern*, *count*)

ROR (*pattern*, *count*)

where

pattern and *count* are expressions with BYTE, WORD, or DWORD values.

If *count* has a WORD or DWORD value, all but the 8 low-order bits will be dropped to produce a BYTE value. If the value of *count* is 0, no rotation occurs.

The value of *pattern* is handled as an 8-bit, 16-bit, or 32-bit binary quantity that is rotated to the left (by ROL) or to the right (by ROR). The type of *pattern* determines whether a byte, word, or dword rotate is performed. The number of bit positions by which it is rotated is specified by *count*.

The following are examples of the action of these procedures:

```
ROR(10011101B, 1) returns a value of 11001110B.
ROL(10011101B, 2) returns a value of 01110110B.
ROR(1101011010011010B, 9) returns a value of 0100110101101011B.
```

Logical-Shift Functions SHL and SHR

SHL and SHR are functions whose type depends on the type of the expression given as an actual parameter. They are activated by function references with the forms:

```
SHL (pattern, count)
SHR (pattern, count)
```

where

pattern and *count* are expressions with BYTE, WORD, or DWORD values.

If *count* has a WORD or DWORD value, all but the 8 low-order bits will be dropped to produce a BYTE value. If the value of *count* is 0, no shift occurs.

The value of *pattern* may be either a BYTE, WORD, or DWORD value and will not be converted. If it is a BYTE value, the function will return a BYTE value. If *pattern* is a WORD value, the function will return a WORD value; if it is a DWORD value, the function will return a DWORD value.

The value of *pattern* is shifted left (by SHL) or right (by SHR), with the bit count given by *count*.

A shift operation can force a 1-bit out of the pattern. For example:

```
SHL(1000$0001B, 1)
```

becomes 0000\$0010, losing the former high-order bit, and:

```
SHR(1000$0001B, 1)
```

becomes 0100\$0000, losing the former low-order bit.

If the specified pattern and count do not cause such a “lost information” shift, then a shift of one bit position has the effect of multiplication by 2 for a left shift, or division by 2 for a right shift. For example, suppose that VAR is a BYTE variable with a value of 8. This is represented as 0000\$1000. SHL(VAR,1) will return 0001\$0000, which represents 16, while SHR(VAR,1) will return 0000\$0100, which represents 4.

Algebraic-Shift Functions SAL and SAR

SAL and SAR are INTEGER functions. They are activated by function references with the forms:

```
SAL (pattern, count)
SAR (pattern, count)
```

where

pattern is an expression with an INTEGER value.

count is an expression with a BYTE, WORD, or DWORD value.

If *count* has a WORD or DWORD value, all but the 8 low-order bits will be dropped to produce a BYTE value. If the value of *count* is 0, no shift occurs.

SAL and SAR treat the INTEGER value of *pattern* as a bit pattern. This pattern is shifted to the left or to the right.

In a left shift (SAL), 0-bits move into the pattern from the right (as in SHL and SHR).

In a right shift (SAR), either zero bits or one bits move into the pattern from the left. If the original value of *pattern* is positive, the sign bit (leftmost bit) is a 0, and zero bits move in from the left. If the original value is negative, the sign bit is a 1, and one bits move in from the left.

This means, in some instances, that just as with the logical shifts, an algebraic shift of one bit position can have the effect of multiplication by 2 for a left shift or division by 2 for a right shift. For example, suppose that VAL is an INTEGER variable with a value of -8 . This value is represented as 1111\$1111\$1111\$1000B. SAL(VAL,1) will return 1111\$1111\$1111\$0000B, which represents -16 , while SAR(VAL,1) will return 1111\$1111\$1111\$1100B, which represents -4 .

9.4 String Manipulation Procedures

The term “string” is used here in a broader sense than previously. The “character strings” mentioned in sections 2.4, 3.1, and 4.4 are BYTE strings.

Considered more broadly, a string is any contiguously stored set of BYTE values or WORD values. We can regard a string as if it were a BYTE or WORD array, and refer to the BYTE or WORD values as “elements.”

We will use the word “index” to refer to the position of a given element within a string. The index is like the subscript of an array reference. Thus, the index of the first element of a string is 0, the index of the second element is 1, etc.

In the following descriptions, the “location” of a string always means the location of its first element. In each string-manipulation procedure, the location of a string is specified by a parameter called “source” or “destination,” which is an expression with a POINTER value. Thus, the source points to the lowest element. For MOVW and MOVV, this is the first element to be processed. For MOVWB and MOVVW, it is the last element to be processed, as discussed below.

The “length” of a string is the number of elements it contains. In each string-manipulation procedure, the number of elements to be processed is specified by a parameter called “count,” which is an expression with a WORD or BYTE value.

NOTE

If the source or destination string address is in SELECTOR or WORD form, use the @ operator of a variable based on the address. Otherwise, the built-in function BUILD\$PTR can be used to construct the pointer-parameter for the string built-in.

The string-manipulation procedures (with the exception of XLAT) are available in pairs. One of each pair is for BYTE strings and the other is for WORD strings.

The MOVB and MOVW Procedures

MOVB is an untyped procedure that copies a BYTE string from one location to another. It is activated by a CALL statement with the form:

```
CALL MOVB ( source, destination, count ) ;
```

where

source and *destination* are expressions with POINTER values.

count is an expression with a BYTE or WORD value.

The string elements are copied in *ascending* order—that is, element 0 is copied first, then element 1, etc. This is significant if the source string and the destination string overlap. If the value of *destination* is higher than the value of *source*, and the two strings overlap, elements in the overlap area will be overwritten before they are copied. This can be avoided by using MOVRB instead of MOVB.

MOVW is the same as MOVB except that it copies a WORD string instead of a BYTE string.

The MOVRB and MOVW Procedures

MOVRB is the same as MOVB except that the elements in the source string are copied to the destination string in *descending order*. This is significant when the two strings overlap. If the value of “destination” is higher than the value of “source,” and an overlap exists, elements in the overlap area will not be overwritten until *after* they have been copied. However, if the value of “source” is higher than the value of “destination,” then elements in the overlap area will be overwritten before they are copied.

MOVW is the same as MOVRB except that it copies a WORD string instead of a BYTE string.

NOTE

If two strings overlap, a procedure such as the following can be used to make the correct choice between MOVB and MOVRB, so that elements in the overlap area will not be overwritten until after they have been copied.

```
MOVBYTES: PROCEDURE (SRC, DST, CNT);
    DECLARE (SRC, DST) POINTER, CNT WORD;
    IF SRC > DST THEN CALL MOVB (SRC, DST, CNT);
    ELSE CALL MOVRB (SRC, DST, CNT);
END MOVBYTES;
```

This procedure can be activated without the need to consider whether overlap may occur or whether *source* or *destination* is higher.

The CMPB and CMPW Functions

CMPB is a WORD function that compares two BYTE strings. It is activated by a function reference with the form:

```
CMPB ( source1, source2, count )
```

where

source1 and *source2* are expressions with POINTER values.

count is an expression with a WORD value.

CMPB compares two BYTE strings of length *count* whose locations are *source1* and *source2*.

If every element in the string at *source1* is equal to the corresponding element in the string at *source2*, CMPB returns a WORD value of 0FFFFH. Otherwise, it returns the index (position within the strings) of the first pair of elements found to be unequal.

CMPW is the same as CMPB except that it compares two WORD strings instead of two BYTE strings.

The FINDB/FINDW and FINDRB/FINDRW Functions

FINDB is a WORD function that searches a BYTE string to find an element that has a specified value. It is activated by a function reference of the form:

F I N D B (*source*, *target*, *count*)

where

<i>source</i>	is an expression with a POINTER value.
<i>target</i>	is an expression with a BYTE or WORD value. If <i>target</i> has a WORD value, the 8 high-order bits will be dropped to produce a BYTE value.
<i>count</i>	is an expression with a WORD value.

FINDB examines each element of the source string (in ascending order) until it finds an element whose value is equal to the BYTE value of *target*—or until *count* elements have been searched without any of them matching *target*. If the search is successful, FINDB returns the index of the first element of the string that matches *target*. If the search is unsuccessful, FINDB returns a WORD value of 0FFFFH.

FINDW is the same as FINDB except that it searches a WORD string instead of a BYTE string. If the *target* parameter has a BYTE value, it is extended by 8 high-order zero bits to produce a WORD value.

FINDRB is the same as FINDB except that it searches the source string in *descending* order. Thus, if the search is successful, FINDRB returns the index of the *last* (highest subscript) element that matches the BYTE value of *target*.

FINDRW is the same as FINDRB except that it searches a WORD string instead of a BYTE string (in descending order).

The SKIPB/SKIPW and SKIPRB/SKIPRW Functions

SKIPB is the “converse” of FINDB (see above). Instead of searching for the first element in the BYTE source string that matches the BYTE value of *target*, SKIPB searches for the first element that does *not* match.

In every other respect, the operation is exactly the same as FINDB.

SKIPW is the “converse” of FINDW (see above). Instead of searching for the first element in the WORD source string that matches the WORD value of *target*, SKIPW searches for the first element that does *not* match.

In every other respect, the operation is exactly the same as FINDW.

SKIPRB is the “converse” of FINDRB (see above). Instead of searching for the last element in the BYTE source string that matches the BYTE value of *target*, SKIPRB searches for the last element that does *not* match.

In every other respect, the operation is exactly the same as FINDRB.

SKIPRW is the “converse” of FINDRW (see above). Instead of searching for the last element in the WORD source string that matches the WORD value of *target*, SKIPRW searches for the last element that does *not* match.

In every other respect, the operation is exactly the same as FINDRW.

The XLAT Procedure

XLAT is an untyped procedure that uses a translation table to “translate” a BYTE string to produce another BYTE string. It is activated by a CALL statement of the form:

```
CALL XLAT ( source, destination, count, table ) ;
```

where

<i>source</i> , <i>destination</i> , and <i>table</i>	are expressions with POINTER values.
<i>count</i>	is an expression with a WORD value.

XLAT “translates” the BYTE elements in the source string, placing the translated elements in the destination string. The value of *table* is assumed to be the location of a BYTE string of up to 256 elements. This string is used as a *translation table*.

The value of an element in the source string is used as an index for the translation table. The index selects one element from the translation table; this element is then copied into the destination string.

For example, if the fifth element in the source string is 202, then 202 is used as an index for the translation table. The 203rd element of the table is copied into the fifth position in the destination string.

The elements of the source string are translated into the destination string in *ascending* order.

The SETB and SETW Procedures

SETB is an untyped procedure that sets each element of a BYTE string to a single specified value. It is activated by a CALL statement with the form:

```
CALL SETB ( newvalue, destination, count ) ;
```

where

<i>newvalue</i>	is an expression with a BYTE or WORD value. If it has a WORD value, the 8 high-order bits are dropped to produce a BYTE value.
<i>destination</i>	is an expression with a POINTER value.
<i>count</i>	is an expression with a BYTE or WORD value.

SETB assigns the BYTE value of *newvalue* to each element of a BYTE string.

SETW is the same as SETB except that it assigns a single WORD value to each element of a WORD string instead of a BYTE string.

If *newvalue* has a BYTE value, it will be extended by 8 high-order zero bits to produce a WORD value.

9.5 Miscellaneous Built-Ins

The MOVE Procedure

MOVE is an untyped procedure that is provided for compatibility with PL/M-80 programs. It is activated by a CALL statement with the form:

```
CALL MOVE (count, source, destination) ;
```

where

count, *source*, and
destination are expressions with WORD or BYTE values.

If any of these parameters has a BYTE value, it will be extended by 8 high-order zero bits to produce a WORD value.

The values of *source* and *destination* are assumed to be the WORD-type addresses of the source string and the destination string. The operation differs from MOVB as follows:

- All three parameters must have either BYTE or WORD values, and will be converted to WORD values if they are BYTE values. POINTER values for *source* and *destination* are not allowed and therefore the values cannot be supplied by means of the @ operator. Thus, MOVE can only handle strings whose locations can be expressed as WORD addresses.
- Note that the parameters are in a different order than the one used by the other built-in string functions.
- If the source and destination strings overlap, the results are always *undefined*.

The TIME Procedure

The untyped procedure TIME causes a time delay specified by its actual parameter. It is activated by a CALL statement with the form:

```
CALL TIME (expression) ;
```

where the expression is converted, if necessary, to a WORD quantity. The length of time measured by the procedure is a multiple of 100 microseconds. If the actual parameter evaluates to *n*, then the delay caused by the procedure is 100*n* microseconds. For example, the statement:

```
CALL TIME (45) ;
```

causes a delay of 4.5 milliseconds. Since the maximum delay offered by the procedure is about 6.55 seconds, longer delays must be obtained by repeated activations. The following block takes one second to execute:

```
DO I = 1 TO 40 ;
    CALL TIME (250) ;
END ;
```


The TIME procedure is based on iAPX 86 CPU cycle times, and assumes that the system is running at 5 MHz (8 MHz for the iAPX 186) without interruptions.

The MEMORY Array

MEMORY is a BYTE array of *unspecified* length which represents an uninitialized (free) segment of iAPX 86 storage. References to MEMORY may be subscripted. The maximum subscript allowed depends on both the system environment and the program. References to MEMORY, either subscripted or unqualified, may be used in location references. For example, @ MEMORY is the location of the beginning of free memory space, i.e., byte 0 of the memory segment.

A reference to MEMORY may not be used as an actual parameter for the LENGTH, LAST, and SIZE procedures. However, some systems provide service routines you can use to learn the size of free memory.

The LOCKSET Function

The LOCKSET function permits a programmer to implement a simple software synchronization lock. It is a BYTE procedure called by a function reference with the form:

LOCKSET (*lockptr*, *newvalue*)

where

<i>lockptr</i>	is an expression with a POINTER value.
<i>newvalue</i>	is an expression with a BYTE or WORD value. If <i>newvalue</i> has a WORD value, the 8 high-order bits will be dropped to produce a BYTE value.

The action of LOCKSET is as follows: the *lockptr* parameter is used as a pointer to a BYTE variable; the value of *newvalue* is assigned to this variable, and LOCKSET returns the original value of the variable. During this transaction, the iAPX 86 CPU prevents any other process from accessing the same memory location.

To see how this facility can be used, consider a system having more than one iAPX 86 processor using the same memory, and consider a program in one of these processors. Suppose that this program uses memory locations that are also used by other processors in the system.

Within certain "critical" regions of our program, we want to ensure that no other processor will access the shared memory locations. To achieve this, we declare a BYTE variable called LOCK and establish a convention that if LOCK=0, any processor in the system may access the shared memory locations. But if LOCK=1, no processor may access the shared memory locations unless it was the one that set LOCK to 1.

Now, if we write the function reference LOCKSET(@LOCK,1), the value 1 is assigned to LOCK. Furthermore, if the value returned by LOCKSET is 0, then LOCK was not already set and *this* processor is the one that set it. We are now allowed, by convention, to enter the critical region of our program and access the shared memory locations. At the end of the critical region, we must release the lock by writing LOCK=0.

But, if LOCKSET returns a value of 1, then LOCK was already set and this processor was not the one that set it. By convention, we must wait until a LOCKSET(@LOCK,1) function reference returns a value of 0 before assessing the shared memory locations.

Thus, our program could contain the following construction:

```
/*Begin critical region*/

DO WHILE LOCKSET(@LOCK,1);
    /*Do nothing but repeat until LOCKSET returns 0*/
END;

/*Now LOCK has been set to 1 by this processor*/

. . .
/*Critical region of program, where
  shared memory locations are accessed*/
. . .

LOCK=0;
/*End critical region*/
```

In the simplest case just described, only one software lock is used. It is represented by the variable **LOCK**. But, if more than one set of memory locations needed protection at different times, we could establish as many different software locks as necessary, with each lock using a different **BYTE** variable.

Also, note that a software lock can be used for other purposes than protecting memory locations. **LOCKSET** provides a mechanism that can be used to implement various types of synchronization in a multiprocessor system.

9.6 Interrupt-Related Procedures

The two capabilities described in this section permit programs to set interrupt vectors and learn the entry point of an interrupt-handling procedure.

The SET\$INTERRUPT Procedure

This procedure permits a program in execution to set an interrupt vector to point to the interrupt entry point of a separately compiled interrupt handling routine, or to alter such vectors dynamically. See also section 8.3 and Appendix G.

The procedure is invoked by a **CALL** of the form:

```
CALL SET$INTERRUPT (constant,name)
```

where *name* is the interrupt procedure name, and *constant* is an interrupt number, i.e., a whole-number constant between 0 and 255.

The INTERRUPT\$PTR Function

This built-in function returns the interrupt entry point. Its form is:

```
INTERRUPT$PTR (name)
```

It is typically used in an assignment statement, for example:

```
INT$ARRAY(4) = INTERRUPT$PTR (HANDLER_PROG_4)
```

The interrupt entry point is not accessible without using this function, since the @ operator refers to the procedure entry point instead. These differences are discussed in greater detail in Appendix G.

9.7 POINTER and SELECTOR-Related Functions

The following built-in functions permit programs to manipulate POINTER and SELECTOR values that serve as location addresses in iAPX 86 memory. (For more information on POINTER and SELECTOR, see section 3.4.)

The BUILD\$PTR Function

BUILD\$PTR is a POINTER function that takes a SELECTOR value (the base portion) and a WORD value (the offset portion), and returns a POINTER value. It is activated by a function reference with the form:

BUILD\$PTR (*base*, *offset*)

where

<i>base</i>	is an expression with a SELECTOR value.
<i>offset</i>	is an expression with a WORD value.

The SELECTOR\$OF Function

SELECTOR\$OF is a SELECTOR function that returns the base portion of a POINTER. It is activated by a function reference with the form:

SELECTOR\$OF (*pointer*)

where

<i>pointer</i>	is an expression with a POINTER value.
----------------	--

The OFFSET\$OF Function

OFFSET\$OF is a WORD function that returns the offset portion of a POINTER. It is activated by a function reference with the form:

OFFSET\$OF (*pointer*)

where

<i>pointer</i>	is an expression with a POINTER value.
----------------	--

The NIL Function

NIL is a POINTER function that returns a pointer with all bytes set to zero. It is activated by a function reference with the form:

NIL

The pointer value NIL points to no object at all. You may assign the value NIL to a pointer to indicate, for instance, the end of a linked list.

Note that pointer values generated with the NIL function cannot be used to dereference data values. For example, if your program contains the following statements:

```
DECLARE P POINTER;
DECLARE B BASED P BYTE;
P = NIL;
```

then any subsequent references to B are invalid and will cause a trap.

NOTE

Usage of the NIL pointer can lead to undesirable results.

In the SMALL case, a NIL pointer is defined to be DS:0, which is not distinguishable from a valid pointer to the first variable in the data segment (DS). For example:

```
$SMALL
M:DD;
    DECLARE P POINTER INITIAL (@P);
    DECLARE B BYTE;
    B = (P=NIL); /* will assign TRUE (0FFH) to B */
END;
```

In all other cases, the NIL pointer is defined to be 0:0, which is not distinguishable from a valid pointer to a variable located at absolute address 0. For example:

```
$LARGE
M:DD;
    DECLARE Q BYTE AT (0);
    DECLARE B BYTE;
    B = (@Q=NIL); /* will assign TRUE (0FFH) to B */
END;
```




CHAPTER 10

PL/M-86 FEATURES INVOLVING iAPX 86 HARDWARE

The PL/M-86 features described in this chapter make use, directly or indirectly, of the iAPX 86 hardware.

10.1 iAPX 86 Hardware-Dependent Statements

The ENABLE and DISABLE Statements

These two statements enable and disable the iAPX 86 interrupt mechanism. They have the forms

```
ENABLE ;  
DISABLE ;
```

ENABLE generates an STI instruction, causing the iAPX 86 to enable interrupts. DISABLE generates a CLI instruction, causing the iAPX 86 to disable interrupts.

The CAUSE\$INTERRUPT Statement

The CAUSE\$INTERRUPT statement causes a software interrupt to be generated. It takes the form:

```
CAUSE$INTERRUPT (constant) ;
```

where

constant is a whole-number constant in the range 0 to 255.

CAUSE\$INTERRUPT generates an INT instruction with the constant as the interrupt type, causing the iAPX 86 to transfer control to the appropriate interrupt vector (see section 8.5 and Appendix G).

The HALT Statement

The HALT statement has the form:

```
HALT ;
```

It generates an STI instruction followed by an HLT instruction, causing the iAPX 86 to come to a halt with interrupts enabled (see section 8.5).

10.2 The iAPX 86 Hardware Flags

Optimization and the Hardware Flags

To produce an efficient machine-code program from a PL/M-86 source, the PL/M-86 compiler performs extensive optimizations of the machine code. This means that the exact sequence of machine code produced to implement a given sequence of PL/M-86 source statements cannot be predicted.

Consequently, the state of the iAPX 86 hardware flags cannot be predicted for any given point in the program. For example, suppose that a source program contains the following fragment:

...

```
SUM = SUM + 250;
```

...

where

SUM is a BYTE variable.

Now, if the value of SUM before this assignment statement is greater than 5, the addition will cause an overflow and the hardware CARRY flag will be set.

If there were no optimization of the machine code, one could follow this assignment statement with one of the PL/M-86 features described in the following sections and be sure that the feature would operate in a certain fashion depending on whether or not the addition caused the CARRY flag to be set. However, because of the optimization, some machine code instructions may occur immediately after the addition and change the CARRY flag. One cannot safely predict whether this will happen or not.

Accordingly, any PL/M-86 feature that is dependent on the CARRY flag (or any of the other hardware flags) may cause the program to run incorrectly. These features must therefore be used with caution, and any program that uses them must be checked carefully to make sure that it operates correctly.

The CARRY, SIGN, ZERO, and PARITY Functions

These four built-in BYTE functions return the logical values of the iAPX 86 hardware flags. These functions take no parameters, and are activated by function references with the following forms

```
CARRY  
ZERO  
SIGN  
PARITY
```

An occurrence of one of these activations (in an expression) generates a test of the corresponding condition flag. If the flag is set (=1), a value of 0FFH is returned. If the flag is clear (=0), a value of 0 is returned.

The PLUS and MINUS Operators

In addition to the arithmetic operators described in section 4.2, PL/M-86 has two more: PLUS and MINUS.

PLUS and MINUS perform similar to + and -, and have the same precedence. However, they take into account the current setting of the iAPX 86 CPU hardware CARRY flag in performing the operation.

Carry-Rotation Functions

SCL and SCR are rotation functions whose type depends on the type of the expression given as an actual parameter. They are activated by function references with the forms:

```
SCL (pattern, count) ;
SCR (pattern, count) ;
```

where

pattern and *count* are expressions with BYTE or WORD values.

If *count* has a WORD value, all but the 8 low-order bits will be dropped to produce a BYTE value. If the value of count is 0, no shift occurs.

The value of *pattern* is handled as an 8-bit or 16-bit binary quantity that is rotated to the left (by SCL) or to the right (by SCR), similar to the ROL and ROR functions described in Chapter 9. The type of pattern determines whether a byte or word rotate is performed. The number of bit positions by which it is rotated is specified by count.

The bit rotated off one end of pattern is rotated into the CARRY flag, and the old value of CARRY is rotated to the other end of pattern. In effect, SCL and SCR perform 9-bit rotations on 8-bit values and 17-bit rotations on 16-bit values.

For example, if the value of CARRY is 0, then

SCL(11001010B, 2) returns a value of 00101001B and CARRY is set to 1
 SCR(11001010B, 1) returns a value of 01100101B and CARRY remains 0

The DEC Function

DEC is a BYTE function that uses the value of the hardware CARRY flag internally. It is activated by a function reference with the form:

```
DEC (expression) ;
```

where

expression is converted, if necessary, to a BYTE value.

The procedure performs a decimal adjust operation on the actual parameter value and returns the result of this operation.

10.3 The iAPX 86 Hardware Registers

The FLAGS Variable

FLAGS is a WORD variable that provides access to the iAPX 86 hardware flags register (see figure 10-1). The iAPX 86 hardware flags register is a 16-bit register that contains the hardware flags that are altered by the execution of various instructions.

The FLAGS register is assigned to change the setting of the various flags. It can also be read to determine the current flag settings.

Note that the IOPL and IF flags can only be altered when you are executing in protection mode at level 0.

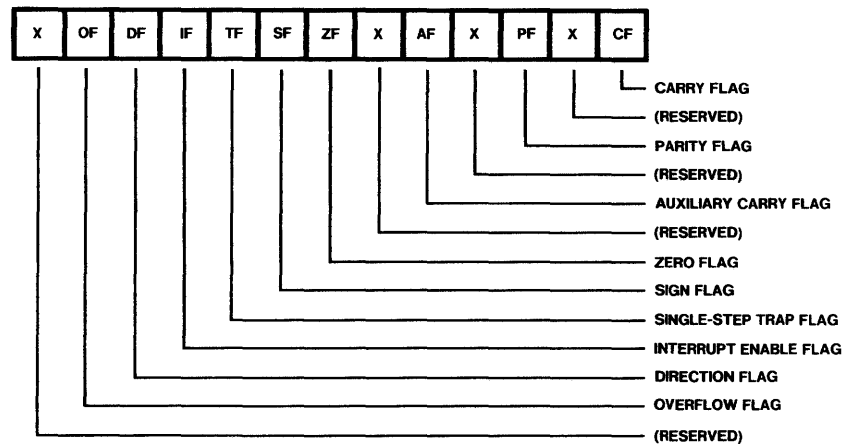


Figure 10-1. The iAPX 86 Hardware Flags Register

945-2

The STACKPTR and STACKBASE Variables

STACKPTR and STACKBASE are WORD variables. They provide access to the iAPX 86 hardware stack pointer and stack base registers, i.e., SP and SS.

When setting these registers (that is, using STACKPTR or STACKBASE on the left side of an assignment), care must be exercised because this action removes control of the stack from the compiler. Thus, the compile-time checks on stack overflow and the compiler's assumptions about the run-time status of the stack may be invalid.

10.4 iAPX 86 Hardware I/O

Single byte or word input is performed as a function invocation in an expression on the right-hand side of an assignment statement. Single byte or word output is achieved by filling the appropriate element of the output array corresponding to the desired output port of the iAPX 86 CPU.

Multiple byte or word input is performed as a procedure invocation, reading in a string from an iAPX 86 CPU port and storing it in a user-specified memory location. Multiple byte or word output is also performed as a procedure invocation, using a CALL statement to send a string from memory into an iAPX 86 CPU port.

The INPUT and INWORD Functions

INPUT is a BYTE function and INWORD is a WORD function. They are activated by function references with the forms:

```
INPUT (expression)
INWORD (expression)
```

where

expression has a BYTE or WORD value.

The value of *expression* specifies one of the input ports of the iAPX 86 CPU. The value returned by INPUT is the BYTE quantity found in the specified input port. The value returned by INWORD is the WORD quantity found in the specified input port.

The OUTPUT and OUTWORD Arrays

OUTPUT is a BYTE array and OUTWORD is a WORD array. Each array has 65536 elements, corresponding to the number of output ports on the iAPX 86 CPU.

A reference to OUTPUT or OUTWORD may only appear as the left part of an assignment statement or embedded assignment; anywhere else it is illegal. The right-hand side of the assignment must have a BYTE or WORD value.

The effect of an assignment to an element of OUTPUT is to place the BYTE value of the expression on the right side of the assignment into the corresponding output port. (Since OUTPUT is a BYTE array, the value of the expression will be converted automatically to a type BYTE if necessary.)

The effect of an assignment to an element of OUTWORD is to place the WORD value of the expression on the right side of the assignment into the corresponding output port.

The BLOCKINPUT and BLOCKINWORD Procedures

BLOCKINPUT and BLOCKINWORD are untyped procedures. They are activated by CALL statements with the forms:

```
CALL BLOCKINPUT (port, destination, count) ;  
CALL BLOCKINWORD (port, destination, count) ;
```

where

port and *count* are expressions with WORD values.
destination is an expression with a POINTER value.

The value of *port* specifies one of the input ports of the iAPX 86 CPU. The value of *count* specifies the length of the BYTE or WORD string found in the specified port, and *destination* is the location (in memory) where this string is to be stored.

The BLOCKOUTPUT and BLOCKOUTWORD Procedures

BLOCKOUTPUT and BLOCKOUTWORD are untyped procedures. They are activated by CALL statements with the forms:

```
CALL BLOCKOUTPUT (port, source, count) ;  
CALL BLOCKOUTWORD (port, source, count) ;
```

where

port and *count* are expressions with WORD values.
source is an expression with a POINTER value.

The value of *port* specifies one of the output ports of the iAPX 86 CPU. The value of *count* specifies the length of the BYTE or WORD string output to the specified port, and *source* is the location (in memory) where this string is currently stored.

10.5 The REAL Math Facility

REAL math support for PL/M-86 is provided by the 8087 Numeric Data Extension. From the program's point of view, The REAL Math Facility consists of the following:

- The REAL stack, used to hold operands and results during REAL operations.
- The REAL Error Byte (see figure 10-2), consisting of 7 exception flags initialized to all 0's. (The reserved bit is set to 1 by the 8087.)

The first six bits in this byte correspond to the possible errors that can arise during REAL operations (see Appendix G). When an error occurs, the facility sets the corresponding bit to 1. A program can invoke a built-in procedure (described in section 10.7) that reads and clears the REAL Error Byte.

The exception/error categories are discussed in Appendix G.

- The REAL mode word (see figure 10-3), consisting of 16 bits initialized to 03FFH.
 1. Bits 0-7 determine whether the corresponding error condition is to be handled by using the default recovery described below or by using the programmer-supplied exception procedure. (See Appendix G for details on writing these.) When the bit is 1, the default is used; when it is 0, the user routine is used. In either case, the facility records the error by setting the corresponding bit of the REAL Error Byte. For most uses, the default recovery is appropriate and less work.

This mode word is often called a mask; i.e., it lets some signals through (for interrupt processing), but not others. If one of the bits 0-5 is a 0, the corresponding error is said to be unmasked. (See section 10.7 on how to set the mode word.)

If the interrupt is enabled ($IEM = 0$), one of the masked bits is 0, and the corresponding error occurs during floating point processing, then the REAL math facility interrupts the host CPU. The 8087 interrupt number is dependent on the internal configuration. The exception condition is thus reported and control is passed to the user-written error handling routine. This situation is called an unmasked error. Sections 8.2 and Appendix G discuss aspects of interrupt procedures.

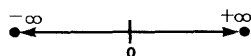
Conversely, a "masked error" means the mode bit corresponding to that error is 1. Masked errors do not cause an interrupt, but are handled as described in Appendix G.

Bits 13, 14, and 15 are reserved and are not for PL/M-86 use.

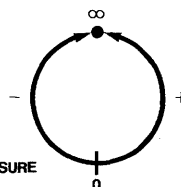
Bits 8-12 provide options for controlling precision, rounding, and infinity representation (see figure 10-3).

2. All intermediate results are held in an internal format of 64-bit precision. The most-significant 24 bits of the final result are returned (plus sign and 7-bit exponent) as the PL/M-86 answer, and rounded, if needed, according to the user-specified control. The default precision setting preserves extended precision and operates slightly faster than the other.
3. Rounding introduces an error of less than one unit in the last place to which the result was rounded. Statistically, the default provides the most accurate and unbiased estimate of the "true result," i.e., the 64-bit result. In all rounding modes except "round down," subtracting a number from itself yields $+0$; round down yields -0 .

4.



AFFINE CLOSURE



PROJECTIVE CLOSURE

121945-4

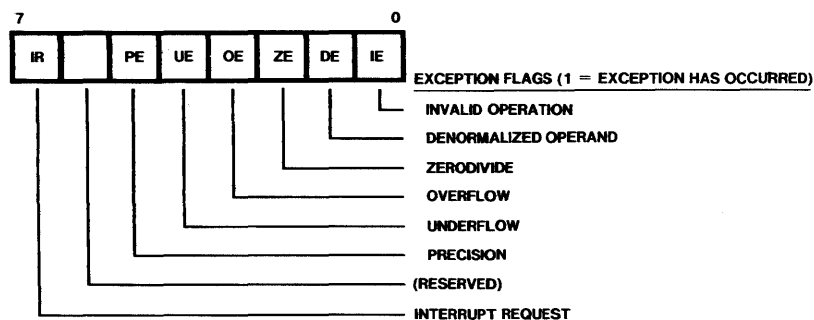
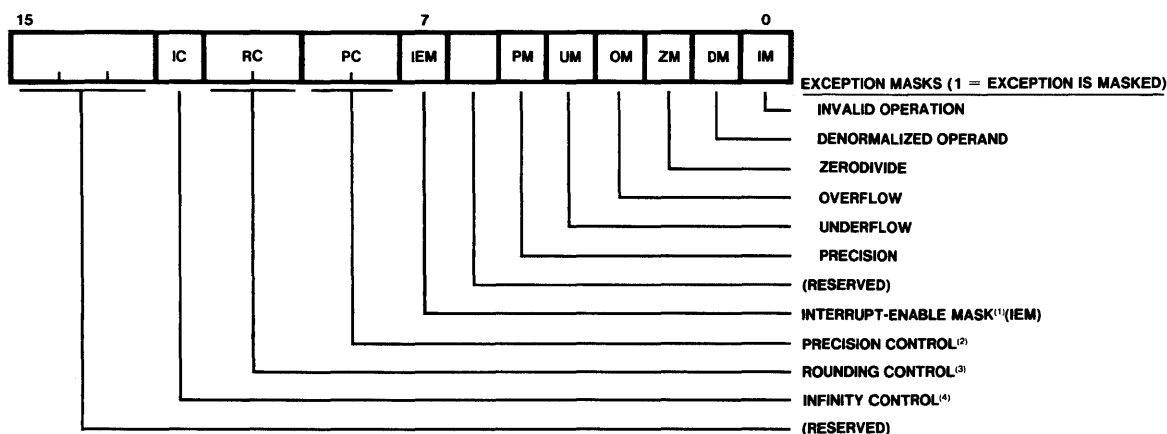


Figure 10-2. The REAL Error Byte

121636-2



⁽¹⁾ Interrupt-Enable Mask:
 0 = Interrupts Enabled
 1 = Interrupts Disabled (Masked)

⁽²⁾ Precision Control:
 00 = 24 bits
 01 = (reserved)
 10 = 53 bits
 11 = 64 bits

⁽³⁾ Rounding Control:
 00 = Round to Nearest or Even
 01 = Round Down (toward $-\infty$)
 10 = Round Up (toward $+\infty$)
 11 = Chop (Truncate Toward Zero)

⁽⁴⁾ Infinity Control:
 0 = Projective
 1 = Affine

Figure 10-3. The REAL Mode Word

121636-3

The full extent of the 8087's numeric and operational capabilities are discussed in the *8086 Family User's Manual Supplement for the 8087 Numeric Data Processor*.

10.6 Built-ins Supporting the REAL Math Unit

The INIT\$REAL\$MATH\$UNIT Procedure

INIT\$REAL\$MATH\$UNIT is a built-in untyped procedure activated by a CALL statement, as follows:

```
CALL INIT$REAL$MATH$UNIT;
```

This call is required as the first access to the REAL math facility (the 8087 Numeric Data Extension).

The effect of this call is to initialize the REAL math unit for subsequent operations. This includes setting a default value into the control word, namely 03FFH or 0000001111111111 in binary. The effect of this setting is to mask all exceptions and interrupts, to set precision to 64 bits, and to cause rounding to even. This means no interrupts will occur from the REAL Math Facility regardless of what errors are detected.

Procedures activated after this call has taken effect do not need to do such initialization.

The SET\$REAL\$MODE Procedure

This procedure should only be invoked if you wish to change the default mode word (for example, to unmask the invalid exception).

SET\$REAL\$MODE is a built-in untyped procedure, activated by a CALL statement with the following form:

```
CALL SET$REAL$MODE (modeword);
```

where

modeword is an expression with a WORD value.

The value of modeword becomes the new contents of the REAL mode word (see figure 10-3). The suggested value for modeword is 033EH, that is, 0000001100111110 in binary. This value provides maximum precision, default rounding, and masked handling of all exception conditions except invalid, which can alert you to errors of initialization or stack usage. See Appendix G for facts and references on writing an interrupt handling procedure.

The GET\$REAL\$ERROR Function

GET\$REAL\$ERROR is a built-in BYTE function activated by a function reference with the following form:

```
GET$REAL$ERROR
```

The BYTE value returned is the current contents of the REAL error byte (see figure 10-1). This function also clears the error byte in the REAL math facility.

Saving and Restoring REAL Status

If an interrupt procedure performs any floating-point operation, it will change the REAL status. If such an interrupt procedure is activated during a floating-point operation, the program will be unable to continue the interrupted operation correctly after returning from the interrupted procedure. Therefore, it is first necessary for any interrupt procedure that performs a floating-point operation to save the REAL status and subsequently restore it before returning. The built-in procedures `SAVE$REAL$STATUS` and `RESTORE$REAL$STATUS` make this possible. `SAVE$REAL$STATUS` initializes the 8087.

These procedures can also be used in a multi-tasking environment where a running task using the 8087 may be preempted by another task that also uses the 8087. The preempting task must call `SAVE$REAL$STATUS` before it executes any statements that affect the 8087, i.e., before calling `SET$REAL$MODE` and before any arithmetic or assignment of REALs (other than `GET$REAL$ERROR`, if needed).

New vectors will be required for the interrupt handlers appropriate to each new task, e.g., to handle unmasked exception conditions. These vectors must be initialized by the operating system.

After its processing is complete and it is ready to terminate, the preempting task must call `RESTORE$REAL$STATUS` to reload the state information that applied at the time the former running task was preempted. This enables that task to resume execution from the point where it relinquished control.

NOTE

REAL functions without REAL parameters should not call `GET$REAL$ERRORS` or `SAVE$REAL$STATUS` before executing at least one floating-point instruction. To do so may result in loss of processor synchronization.

The SAVE\$REAL\$STATUS Procedure

`SAVE$REAL$STATUS` is a built-in untyped procedure activated by a `CALL` statement with the form:

```
CALL SAVE$REAL$STATUS (location);
```

where

location is a pointer to a memory area of 94 bytes where the REAL status information will be saved.

The REAL status is saved at the specified location, and the REAL stack and error byte are reinitialized.

If the state of the REAL math unit is unknown to this procedure when it is called, as in the case mentioned above for preempting tasks, then you do not want to do an initialization because that will destroy existing error flags, masks, and control settings. The action appropriate to these circumstances (except for error-recovery routines, discussed in Appendix G) is to issue:

```
CALL SAVE$REAL$STATUS (@location_1);
```

before any REAL math usage, and

```
CALL RESTORE$REAL$STATUS (@location_1);
```

prior to the procedure's return. The save automatically reinitializes the math unit and the error byte.

This protects the status of preempted tasks or prior procedures and establishes a known initialization state for the current procedure's actions. iAPX 86 interrupts are disabled during the save.



The iAPX 86 processor must be able to acknowledge 8087 interrupts or loss of synchronization will occur.

The **RESTORE\$REAL\$STATUS** Procedure

RESTORE\$REAL\$STATUS is a built-in untyped procedure activated by a **CALL** statement with the form:

```
CALL RESTORE$REAL$STATUS (location) ;
```

where

location is a pointer to a memory area where the **REAL** status information was previously saved by a call to the **SAVE\$REAL\$STATUS** procedure.

This procedure should be called prior to returning from an interrupt procedure where the real math unit's status was saved using **SAVE\$REAL\$STATUS**.



11.1 Introduction to Compiler Controls

The exact operation of the compiler may be controlled by a number of *controls* that specify options such as the type of listing to be produced and the destination of the object file. Controls may be specified as part of the command invoking the compiler, or as *control lines* appearing as part of the source input file.

A *control line* is a source line containing a dollar sign (\$) in the left margin. Normally, the left margin is set at column one, but this may be changed with the LEFTMARGIN control. Control lines are introduced into the source to allow selective control over sections of the program. For example, it may be desirable to suppress the listing for certain sections of the program, or to cause page ejects at certain places.

A line is considered a control line by the compiler if there is a dollar sign in the left margin, even if it appears to be part of a PL/M-86 comment or character string constant.

On a control line, the dollar sign is followed by a sequence of controls. These controls must be separated from each other by one or more blanks.

The following are examples of control lines:

```
$NOCODE XREF  
$ EJECT CODE
```

There are two types of controls: *primary* and *general*. Primary controls must occur either in the invocation command or in a control line which precedes the first noncontrol line of the source file. Primary controls may not be changed within a module. General controls may occur either in the invocation command or on a control line located anywhere in the source input, and may be changed freely within a module.

A large number of controls are available, but you may only need to specify a few of them for most compilations because a set of defaults is built into the compiler. The controls are summarized in alphabetic order in table 11-1.

A *control* consists of a control-name which, depending on the particular control, may be followed by a parenthesized *control parameter*.

The following are examples of controls:

```
LIST  
NOXREF  
OBJECT(PROG2.OJB)
```

Table 11-2 lists the controls in the order in which they are discussed in this chapter, which is approximately in order of importance or usage. Primary controls have an asterisk attached. Sample invocation lines for most compiler controls are provided on a foldout page in your specific operating-system appendix.

Table 11-1. Compiler Controls

Primary Control Names	Abbreviation	Default	Section
DEBUG/NODEBUG	DB	NODEBUG	11.4
INTVECTOR/NOINTVECTOR	IV	INTVECTOR	11.4
MOD86/MOD186	none	MOD86	11.2
OBJECT/NOOBJECT	OJ	OBJECT(<i>source-file.OBJ</i>)	11.4
OPTIMIZE	OT	OPTIMIZE(1)	11.4
PAGING/NOPAGING	PI	PAGING	11.6
PAGELNGTH	PL	PAGELNGTH(60)	11.6
PAGewidth	PW	PAGewidth(120)	11.6
PRINT/NOPRINT	PR	PRINT(<i>source-file.LST</i>)	11.5
RAM/ROM	none	ROM	11.4
SMALL/COMPACT/MEDIUM/LARGE	SM/CP/MD/LA	LARGE	11.4
SYMBOLS/NOSYMBOLS	SB	NOSYMBOLS	11.5
TITLE	TT	<i>module name</i>	11.6
TYPE/NOTYPE	TY	TYPE	11.4
XREF/NOXREF	XR	NOXREF	11.5
General Control Names	Abbreviation	Default	Section
CODE/NOCODE	CO	NOCODE	11.5
COND/NOCOND	none	COND	11.8
EJECT	EJ	—	11.6
IF/ELSEIF/ELSE/ENDIF	none	—	11.8
INCLUDE	IC	—	11.7
LEFTMARGIN	LM	LEFTMARGIN(1)	11.3
LIST/NOLIST	LI	LIST	11.5
OVERFLOW/NOOVERFLOW	OV	NOOVERFLOW	11.4
SAVE/RESTORE	SA/RS	—	11.7
SET/RESET	none	RESET (0)	11.8
SUBTITLE	ST	no subtitle	11.6

11.2 The MOD86/MOD186 Control

MOD86 and MOD186 are primary controls with the form:

```
MOD86
MOD186
Default: MOD86
```

The MOD86 control specifies that the object module will contain instructions for execution on the iAPX-86 processor.

The MOD186 control allows the compiler to generate an extended set of instructions in the object module for use on the iAPX-186 processor.

11.3 The LEFTMARGIN Control

This is the only control for specifying the format of the source input. It is a general control with the form:

```
LEFTMARGIN(column)
Default: LEFTMARGIN(1)
```

where

column is a non-zero, unsigned integer specifying the left margin of the source input.

Table 11-2. Controls by Categories

Section	Category	Controls
11.2	Code Generation	*MOD86/MOD186
11.3	Input Format	LEFTMARGIN
11.4	Object File	*INTVECTOR/NOINTVECTOR OVERFLOW/NOOVERFLOW *OPTIMIZE *OBJECT/NOOBJECT *DEBUG/NODEBUG *TYPE/NOTYPE *SMALL/COMPACT/MEDIUM/LARGE *ROM/RAM
11.5	Listing Content	*PRINT/NOPRINT LIST/NOLIST CODE/NOCODE *XREF/NOXREF *SYMBOLS/NOSYMBOLS
11.6	Listing Format	*PAGING/NOPAGING *PAGELENGTH *PAGEWIDTH *TITLE SUBTITLE EJECT
11.7	Source Inclusion and Control Status	INCLUDE SAVE/RESTORE
11.8	Conditional Compilation	IF/ELSEIF/ELSE/ENDIF SET/RESET COND

* denotes primary control.

All characters to the left of this position on subsequent input lines are not processed by the compiler (but do appear on the listing). The first character on a line is in column 1.

The new setting of the left margin takes effect on the next input line. It remains in effect for all input from the source file and any INCLUDE files until it is reset by another LEFTMARGIN control.

Note that a control line is one that contains a dollar sign in the column specified by the most recent LEFTMARGIN control.

11.4 Object File Controls

These controls determine what type of object file is to be produced and on which device it is to appear. The controls are discussed in the following order:

INTVECTOR/NOINTVECTOR
OVERFLOW/NOOVERFLOW
OPTIMIZE
OBJECT/NOOBJECT
DEBUG/NODEBUG
TYPE/NOTYPE
Program Size Controls
RAM/ROM

INTVECTOR/NOINTVECTOR

These are primary controls. They have the form

```
INTVECTOR  
NOINTVECTOR  
Default: INTVECTOR
```

Under the INTVECTOR control, the compiler creates an interrupt vector consisting of a 4-byte entry for each interrupt procedure in the module. For Interrupt n , the interrupt vector entry is located at absolute location $4*n$. See Chapter 8 and Appendix G for further discussion.

Alternatively, it may be desirable to create the interrupt vector independently, using either PL/M-86 or assembly language. In this case, the NOINTVECTOR control is used and the compiler does not generate any interrupt vector. The implications of this are discussed in Appendix G.

OVERFLOW/NOOVERFLOW

These are general controls. They have the form:

```
OVERFLOW  
NOOVERFLOW  
Default: NOOVERFLOW
```

These controls specify whether overflow is to be detected in performing signed (INTEGER) arithmetic. If the NOOVERFLOW control is specified, no overflow detection is implemented in the compiled module and the results of overflow in signed arithmetic are undefined. If the OVERFLOW control is specified, overflow in signed arithmetic results in a nonmaskable Interrupt 4, and it is the programmer's responsibility to provide an interrupt procedure to handle the interrupt. Failure to provide such a procedure may result in unpredictable program behavior when overflow occurs.

If this control is nested within a program statement, overflow detection will begin when the next complete statement is evaluated.

Note that the use of the OVERFLOW control results in some expansion of the object code.

OPTIMIZE

This is a primary control. It has the form:

```
OPTIMIZE( $n$ )  
Default: OPTIMIZE(1)
```

where

n may be 0, 1, 2, or 3.

This control governs the kinds of optimization to be performed in generating object code.



OPTIMIZE(0) is the only level of optimization that does not optimize code between program lines. Thus, it is the only one that gives guaranteed results when your programs are debugged with a debugger such as the ICE 86 debugger.

OPTIMIZE(0)

OPTIMIZE(0) specifies only "folding" of constant expressions.

Folding means recognizing, during compilation, operations that are superfluous or combinable, and removing or combining them so as to save memory space or execution time. Examples include addition with a zero operand, multiplication by one, and logical expressions with "true" or "false" constants. Also, in the statement:

```
A = 6 + 3 + A;
```

the compiler will add 6 and 3, producing code to add 9 to A.

OPTIMIZE(1)

OPTIMIZE(1) specifies strength reduction, elimination of common subexpressions, short-circuit evaluation of some Boolean expressions, and the above optimizations of level (0).

Strength reduction means substituting quick operations in place of longer operations, e.g., shifting left by 1 instead of multiplying by 2. This instruction requires less space and executes faster. The addition of identical subexpressions may also generate left shift instructions.

Elimination of common subexpressions means that if an expression reappears in the same block, its value is re-used rather than being recomputed. The compiler also recognizes commutative forms of subexpressions, e.g., $A + B$ and $B + A$ are seen to be the same. Intermediate results during expression evaluation are saved in registers and/or on the stack for later use. For example:

```
A = B + C * D / 3;
C = E + D * C / 3;
```

The value of $C * D / 3$ will not be recomputed for the second statement.

Optimizing the evaluation of Boolean expressions uses the fact that in certain cases some of the terms are not needed to determine the value of the expression. For example, in the expression:

```
( A > B AND I > J )
```

if the first term ($A > B$) is false, the entire expression is false, and it is not necessary to evaluate the second term. The use of PL/M-86 built-in procedures does not change this optimization. However, if a user-function or an embedded assignment is part of the expression, this short evaluation is not done. For example:

```
( A > B AND ( UFUN ( A ) > J ) )
```

is evaluated in full.

OPTIMIZE(2)

OPTIMIZE(2) specifies all of the above, plus the following:

- Machine code optimizations (e.g., short jumps, moves)
- Elimination of superfluous branches
- Re-use of duplicate code
- Removal of unreachable code and reversal of branch-condition

Optimizing machine code means saving space by using shorter forms for identical machine instructions. This is possible because the iAPX 86 has multiple forms for some of its instructions. For example:

```
MOV RESULT1, AX; /* move accumulator value to location RESULT1 */
```

can be generated in 3 bytes as A30800, or in 4 bytes as B9060800. The former choice saves a byte of storage for the program. Similarly, jumps that the compiler can recognize as within the same segment or even closer, within 127 bytes, permit the use of fewer byte instructions.

Elimination of superfluous branches means optimizing consecutive or multiple branches into a single branch example. For example:

```

                JZ      LAB1;      /* Jump on zero to LAB1 */
                JMP     LAB2;      /* unconditional jump to LAB2 */
LAB1:           .....
                ...
                ...
LAB2:           .....
```

will be transformed into:

```

                JNZ     LAB2;      /* Jump on non-zero to LAB2 */
LAB1:           .....
                ...
                ...
LAB2:           .....
```

Similarly, multiple branches like the following are eliminated:

```

LAB0:           JMP     LAB1
                ...
                ...
LAB1:           JMP     LAB2
                ...
                ...
LAB2:           .....
```

and transformed into:

```

LAB0:           JMP     LAB2
                ...
                ...
LAB1:           JMP     LAB2
                ...
                ...
LAB2:           .....
```

Reuse of duplicate code can refer to identical code at the end of two converging paths. In such a case, the code is inserted in only one path, and a jump to that path is inserted in the other path. For example:

```
DECLARE A BYTE, SPOT POINTER;
DECLARE S BASED SPOT STRUCTURE (B BYTE, C BYTE);
IF A = 1 THEN
    S.C = INPUT (0F7H) AND 07FH;
ELSE
    S.C = INPUT (0F9H) and 07FH;
```

	Before		After
	CMP A, 1H		CMP A, 1H
	JZ 4+5H		JNZ @1
	JMP @1		
	IN 0F7H		IN 0F7H
	AND AL, 7FH		JMP @2
	MOV BX, SPOT		
	MOV S [BX+1H], AL		
	JMP @2		
@1:	IN 0F9H	@1:	IN 0F9H
	AND AL, 7FH	@2:	AND AL, 7FH
	MOV BX, SPOT		MOV BX, SPOT
	MOV S [BX+1H], AL		MOV S [BX+1H], AL
@2:			

Reuse of duplicate code can also refer to machine instructions immediately preceding a loop—instructions that are identical to those ending the loop. A branch can be generated to re-use the code generated at the beginning of the loop. For example:

	Before		After
	ADD AX, BX	LAB0:	ADD AX, BX
	MOV ANS, AX		MOV ANS, AX
LAB0:	MOV AL, DUM1		MOV AL, DUM1
	CMP AL, DUM2		CMP AL, DUM2
	JNZ LAB1		JNZ LAB1
	...		...
	...		...
	ADD AX, BX		JMP LAB0
	MOV ANS, AX	LAB1:	
	JMP LAB0		
LAB1:			

This is safe so long as LAB0 is not the target of a jump instruction. The compiler normally handles a whole procedure at a time, and is thus aware of such a condition. The optimization cannot be safely applied to labels in the outer level of the main program module. This optimization will not change your program and will save code space.

The optimization that removes unreachable code takes a second look at the generated object code and finds areas which can never be reached due to the control structures created earlier.

For example, if the following code were generated before optimization:

```

        MOV    AX, A
        RCR    AL, 1
        JB     @1
        JMP     @2

@1:     MOV    AX, 0FFFFH
        OUTW   DF6H
        JMP     @2
        MOV    AX, B
        ADD    A, AX
        JMP     @3
@2:     ....
        ....
        ....
@3:     .....
        .....

```

then the removal of unreachable code would produce:

```

        MOV    AX, A
        RCR    AL, 1
        JB     @1
        JMP     @2

@1:     MOV    AX, 0FFFFH
        OUTW   DF6H
        JMP     @2
@2:     ...
        ...
@3:     .....

```

This can be further optimized by reversing the branch condition in the third instruction and removing the unnecessary JMP @2:

```

        MOV    AX, A
        RCR    AL, 1
        JNB     @2

@1:     MOV    AX, 0FFFFH
        OUTW   DF6H
@2:     ...
        ...
@3:     .....

```

OPTIMIZE(3)

OPTIMIZE(3) includes all of the above optimizations. It also optimizes indeterminate storage operations (e.g., those using based variables or variables declared with the AT attribute). The assumption validating this new optimization is that based variables (or AT variables) are not overlaying other user-declared variables.

On this optimization level, all Boolean expressions are short-circuited except those containing embedded assignments. (For a description of how this optimization occurs, see OPTIMIZE(1).)

The benefits of this optimization level include more efficient use of code space because the user guarantees that needed values are not overlaid.

The first guarantee is a consequence of user caution in variable-declaration and usage. For example, the sequence:

```
DECLARE (I,J) WORD;
DECLARE THETA (19) AT (@I);
DECLARE A BASED J (10)
      STRUCTURE (F1 BYTE,F2 WORD);
..
J=.1;
....
..
A(I).F1 = 7;
A(I).F2 = 99;
THETA(I) = 31;
..
..
```

violates this caution and guarantee because it causes the values being used as pointers and subscripts to be overlaid. The compiler normally takes steps to avoid the difficulties implied here. But, in OPTIMIZE(3), these steps are omitted due to the implicit user guarantee that such situations have been avoided.

Figures 11-1 through 11-4 illustrate the levels of optimization described above.

```
system-id PL/M-86 V2.1 COMPILATION OF MODULE EXAMPLES_OF_OPTIMIZATIONS
OBJECT MODULE PLACED IN :F1:EXMPLE.OBJ
COMPILER INVOKED BY:  :F1:PLM86.86 :F1:EXMPLE.SRC NOPAGING COMPACT CODE OPTIMIZE(0)
```

```

      EXAMPLES_OF_OPTIMIZATIONS: DO;
2   1   DECLARE (A,B,C) WORD, D(100) WORD, (PTR_1, PTR_2) POINTER,
      ABASED BASED PTR 1 (10) WORD;
3   1   DO WHILE D(A+B) < D(A+B+1);
4   2   IF PTR_1 = PTR_2 THEN DO;
6   3   A = A * 2;
7   3   ABASED(A) = ABASED(B);
8   3   ABASED(B) = ABASED(C);
9   3   END;
10  2   ELSE A = A + 1;
11  2   END;
12  1   END EXAMPLES_OF_OPTIMIZATIONS;
                                           ; STATEMENT # 3
```

```

0000 8BEC          MOV     BP,SP
0002 FB          STI
      @1:
0003 8B1E0000      MOV     BX,A
0007 031E0200      ADD     BX,B
000B D1E3          SHL     BX,1
000D 8B360000      MOV     SI,A
0011 03360200      ADD     SI,B
0015 D1E6          SHL     SI,1
0017 8B870600      MOV     AX,D[BX]
001B 3B840800      CMP     AX,D[SI+2H]
001F 7203          JB      $+5H
0021 E97700        JMP     @2
```

Figure 11-1. Sample Program Showing the OPTIMIZE(0) Control

```

0024 C406CE00      LES     AX,PTR_1      ; STATEMENT # 4
0028 06           PUSH    ES             ; 1
0029 C416D200      LES     DX,PTR_2
002D 8CC7         MOV     DI,ES
002F 5E           POP     SI             ; 1
0030 B104         MOV     CL,4H
0032 8BD8         MOV     BX,AX
0034 D3EB         SHR     BX,CL
0036 03F3         ADD     SI,BX
0038 8BDA         MOV     BX,DX
003A D3EB         SHR     BX,CL
003C 03FB         ADD     DI,BX
003E 3BF7         CMP     SI,DI
0040 7507         JNZ     $+9H
0042 240F         AND     AL,0FH
0044 80E20F       AND     DL,0FH
0047 3AC2         CMP     AL,DL
0049 7403         JZ      $+5H
004B E94100       JMP     @3

                                ; STATEMENT # 6
004E 8B060000     MOV     AX,A
0052 D1E0         SHL     AX,1
0054 89060000     MOV     A,AX

                                ; STATEMENT # 7
0058 8B360200     MOV     SI,B
005C D1E6         SHL     SI,1
005E 8B3E0000     MOV     DI,A
0062 D1E7         SHL     DI,1
0064 C41ECE00     LES     BX,PTR_1
0066 268B00       MOV     AX,ES:[BX].ABASED[SI]
006B C41ECE00     LES     BX,PTR_1
006F 268901       MOV     ES:[BX].ABASED[DI],AX
                                ; STATEMENT # 8
0072 8B360400     MOV     SI,C
0076 D1E6         SHL     SI,1
0078 8B3E0200     MOV     DI,B
007C D1E7         SHL     DI,1
007E C41ECE00     LES     BX,PTR_1
0082 268B00       MOV     AX,ES:[BX].ABASED[SI]
0085 C41ECE00     LES     BX,PTR_1
0089 268901       MOV     ES:[BX].ABASED[DI],AX
                                ; STATEMENT # 10
008C E90900       JMP     @4
                                @3:
008F 8B060000     MOV     AX,A
0093 40           INC     AX
0094 89060000     MOV     A,AX

                                ; STATEMENT # 11
                                @4:
0098 E968FF       JMP     @1
                                @2:

                                ; STATEMENT # 12

```

Figure 11-1. Sample Program Showing the OPTIMIZE(0) Control (Cont'd.)

MODULE INFORMATION:

```

CODE AREA SIZE      = 009BH      155D
CONSTANT AREA SIZE  = 0000H      0D
VARIABLE AREA SIZE  = 00D6H      214D
MAXIMUM STACK SIZE  = 0002H      2D
12 LINES READ
0 PROGRAM WARNINGS
0 PROGRAM ERRORS

```

DICTIONARY SUMMARY:

```

27KB MEMORY AVAILABLE
3KB MEMORY USED      (11%)
0KB DISK SPACE USED

```

END OF PL/M-86 COMPILATION

Figure 11-1. Sample Program Showing the OPTIMIZE(0) Control (Cont'd.)

```

system-id PL/M-86 V2.1 COMPILATION OF MODULE EXAMPLES_OF_OPTIMIZATIONS
OBJECT MODULE PLACED IN :F1:EXMPLE.OBJ
COMPILER INVOKED BY:   :F1:PLM86.86 :F1:EXMPLE.SRC NOPAGING COMPACT CODE OPTIMIZE(1)

```

```

2  1      EXAMPLES_OF_OPTIMIZATIONS: DO;
      DECLARE (A,B,C) WORD, D(100) WORD, (PTR_1, PTR_2) POINTER,
      ABASED BASED PTR_1 (10) WORD;
3  1      DO WHILE D(A+B) < D(A+B+1);
4  2          IF PTR_1 = PTR_2 THEN DO;
6  3              A = A * 2;
7  3              ABASED(A) = ABASED(B);
8  3              ABASED(B) = ABASED(C);
9  3              END;
10 2          ELSE A = A + 1;
11 2          END;
12 1      END EXAMPLES_OF_OPTIMIZATIONS;

```

; STATEMENT # 3

```

0000 8BEC          MOV     BP,SP
0002 FB           STI
      @1:
0003 8B1E0000      MOV     BX,A
0007 8B060200      MOV     AX,B
000B 03D8          ADD     BX,AX
000D D1E3          SHL     BX,1
000F 8B8F0600      MOV     CX,D[BX]
0013 3B8F0800      CMP     CX,D[BX+2H]
0017 7203          JB      $+5H
0019 E96800        JMP     @2

```

; STATEMENT # 4

```

001C C406CE00      LES     AX,PTR_1
0020 06           PUSH    ES      ; 1
0021 C416D200      LES     DX,PTR_2
0025 8CC7          MOV     DI,ES
0027 5E           POP     SI      ; 1
0028 B104          MOV     CL,4H

```

Figure 11-2. Sample Program Showing the OPTIMIZE(1) Control

```

002A 8BD8      MOV     BX,AX
002C D3EB      SHR     BX,CL
002E 03F3      ADD     SI,BX
0030 8BDA      MOV     BX,DX
0032 D3EB      SHR     BX,CL
0034 03FB      ADD     DI,BX
0036 3BF7      CMP     SI,DI
0038 7507      JNZ     $+9H
003A 240F      AND     AL,0FH
003C 80E20F    AND     DL,0FH
003F 3AC2      CMP     AL,DL
0041 7403      JZ      $+5H
0043 E93700    JMP     @3
                                ; STATEMENT # 6
0046 8B060000  MOV     AX,A
004A D1E0      SHL     AX,1
004C 89060000  MOV     A,AX
                                ; STATEMENT # 7
0050 8B360200  MOV     SI,B
0054 D1E6      SHL     SI,1
0056 D1E0      SHL     AX,1
0058 C41ECE00  LES     BX,PTR_1
005C 268B08    MOV     CX,ES:[BX].ABASED[SI]
005F 8BF0      MOV     SI,AX
0061 268908    MOV     ES:[BX].ABASED[SI],CX
                                ; STATEMENT # 8
0064 8B360400  MOV     SI,C
0068 D1E6      SHL     SI,1
006A 8B3E0200  MOV     DI,B
006E D1E7      SHL     DI,1
0070 C41ECE00  LES     BX,PTR_1
0074 268B00    MOV     AX,ES:[BX].ABASED[SI]
0077 268901    MOV     ES:[BX].ABASED[DI],AX
                                ; STATEMENT # 10
007A E90400    JMP     @4
007D FF060000  INC     A
                                ; STATEMENT # 11
0081 E97FFF    JMP     @1
                                ; STATEMENT # 12

```

MODULE INFORMATION:

```

CODE AREA SIZE      = 0084H    132D
CONSTANT AREA SIZE  = 0000H     0D
VARIABLE AREA SIZE  = 00D6H   214D
MAXIMUM STACK SIZE  = 0002H     2D
12 LINES READ
0 PROGRAM WARNINGS
0 PROGRAM ERRORS

```

DICTIONARY SUMMARY:

```

27KB MEMORY AVAILABLE
3KB MEMORY USED    (11%)
0KB DISK SPACE USED

```

Figure 11-2. Sample Program Showing the OPTIMIZE(1) Control (Cont'd.)

system-id PL/M-86 V2.1 COMPILATION OF MODULE EXAMPLES_OF_OPTIMIZATIONS
 OBJECT MODULE PLACED IN :F1:EXMPLE.OBJ
 COMPILER INVOKED BY: :F1:PLM86.86 :F1:EXMPLE.SRC NOPAGING COMPACT CODE OPTIMIZE(2)

```

      EXAMPLES_OF_OPTIMIZATIONS: DO;
2    1      DECLARE (A,B,C) WORD, D(100) WORD, (PTR_1, PTR_2) POINTER,
      ABASED BASED PTR_1 (10) WORD;
3    1      DO WHILE D(A+B) < D(A+B+1);
4    2      IF PTR_1 = PTR_2 THEN DO;
5    3          A = A * 2;
6    3          ABASED(A) = ABASED(B);
7    3          ABASED(B) = ABASED(C);
8    3          END;
9    3      ELSE A = A + 1;
10   2      END;
11   2      END EXAMPLES_OF_OPTIMIZATIONS;
12   1
; STATEMENT # 3

```

```

0000 8BEC      MOV     BP,SP
0002 FB       STI
      @1:
0003 8B1E0000   MOV     BX,A
0007 A10200     MOV     AX,B
000A 03D8      ADD     BX,AX
000C D1E3      SHL     BX,1
000E 8B8F0600   MOV     CX,D[BX]
0012 3B8F0800   CMP     CX,D[BX+2H]
0016 7361      JAE     @2
; STATEMENT # 4
0018 C406CE00     LES     AX,PTR_1
001C 06         PUSH    ES ; 1
001D C416D200     LES     DX,PTR_2
0021 8CC7      MOV     DI,ES
0023 5E       POP     SI ; 1
0024 B104      MOV     CL,4H
0026 8BD8      MOV     BX,AX
0028 D3EB      SHR     BX,CL
002A 03F3      ADD     SI,BX
002C 8BDA      MOV     BX,DX
002E D3EB      SHR     BX,CL
0030 03FB      ADD     DI,BX
0032 3BF7      CMP     SI,DI
0034 7507      JNZ     $+9H
0036 240F      AND     AL,0FH
0038 80E20F    AND     DL,0FH
003B 3AC2      CMP     AL,DL
003D 7534      JNZ     @3
; STATEMENT # 6
003F A10000     MOV     AX,A
0042 D1E0      SHL     AX,1
0044 A30000     MOV     A,AX
; STATEMENT # 7
0047 8B360200   MOV     SI,B
004B D1E6      SHL     SI,1
004D D1E0      SHL     AX,1
004F C41ECE00     LES     BX,PTR_1
0053 268B08     MOV     CX,ES:[BX].ABASED[SI]
0056 8BF0      MOV     SI,AX
0058 268908     MOV     ES:[BX].ABASED[SI],CX

```

Figure 11-3. Sample Program Showing the OPTIMIZE(2) Control

```

                                ; STATEMENT # 8
005B 8B360400      MOV     SI,C
005F D1E6          SHL     SI,1
0061 8B3E0200      MOV     DI,B
0065 D1E7          SHL     DI,1
0067 C41ECE00      LES     BX,PTR_1
006B 268B00        MOV     AX,ES:[BX].ABASED[SI]
006E 268901        MOV     ES:[BX].ABASED[DI],AX
                                ; STATEMENT # 10
0071 EB90          JMP     @1
                                @3:
0073 FF060000      INC     A
                                ; STATEMENT # 11
0077 EB8A          JMP     @1
                                @2:
                                ; STATEMENT # 12

```

MODULE INFORMATION:

```

CODE AREA SIZE      = 0079H    121D
CONSTANT AREA SIZE  = 0000H     0D
VARIABLE AREA SIZE  = 00D6H   214D
MAXIMUM STACK SIZE  = 0002H     2D
12 LINES READ
0 PROGRAM WARNINGS
0 PROGRAM ERRORS

```

DICTIONARY SUMMARY:

```

27KB MEMORY AVAILABLE
3KB MEMORY USED    (11%)
0KB DISK SPACE USED

```

END OF PL/M-86 COMPILATION

Figure 11-3. Sample Program Showing the OPTIMIZE(2) Control (Cont'd.)

```

system-id PL/M-86 V2.1 COMPILATION OF MODULE EXAMPLES_OF_OPTIMIZATIONS
OBJECT MODULE PLACED IN :F1:EXMPLE.OBJ
COMPILER INVOKED BY:   :F1:PLM86.86 :F1:EXMPLE.SRC NOPAGING COMPACT CODE OPTIMIZE(3)

```

```

EXAMPLES OF OPTIMIZATIONS: DO;
2  1      DECLARE (A,B,C) WORD, D(100) WORD, (PTR_1, PTR_2) POINTER,
          ABASED BASED PTR_1 (10) WORD;
3  1      DO WHILE D(A+B) < D(A+B+1);
4  2          IF PTR_1 = PTR_2 THEN DO;
5  3              A = A * 2;
6  3              ABASED(A) = ABASED(B);
7  3              ABASED(B) = ABASED(C);
8  3              END;
9  3          ELSE A = A + 1;
10 2          END;
11 2      END EXAMPLES_OF_OPTIMIZATIONS;
12 1

```

Figure 11-4. Sample Program Showing the OPTIMIZE(3) Control

```

; STATEMENT # 3
0000 8BEC      MOV    BP,SP
0002 FB       STI
      @1:
0003 8B1E0000  MOV    BX,A
0007 A10200    MOV    AX,B
000A 03D8      ADD    BX,AX
000C D1E3      SHL    BX,1
000E 8B8F0600  MOV    CX,D[BX]
0012 3B8F0800  CMP    CX,D[BX+2H]
0016 733D      JAE    @2
      ; STATEMENT # 4
0018 C41ECE00  LES     BX,PTR_1
001C 8CC6      MOV    SI,ES
001E 3B36D400  CMP    SI,PTR_2+2H
0022 7504      JNZ    $+6H
0024 3B1ED200  CMP    BX,PTR_2
0028 7525      JNZ    @3
      ; STATEMENT # 6
002A A10000      MOV    AX,A
002D D1E0      SHL    AX,1
002F A30000      MOV    A,AX
      ; STATEMENT # 7
0032 8B3E0200  MOV    DI,B
0036 D1E7      SHL    DI,1
0038 D1E0      SHL    AX,1
003A 268B09      MOV    CX,ES:[BX].ABASED[DI]
003D 96          XCHG   AX,SI
003E 268908      MOV    ES:[BX].ABASED[SI],CX
      ; STATEMENT # 8
0041 8B360400  MOV    SI,C
0045 D1E6      SHL    SI,1
0047 268B08      MOV    CX,ES:[BX].ABASED[SI]
004A 268909      MOV    ES:[BX].ABASED[DI],CX
      ; STATEMENT # 10
004D EBB4      JMP     @1
      @3:
004F FF060000  INC     A
      ; STATEMENT # 11
0053 EBAA      JMP     @1
      @2:
      ; STATEMENT # 12

```

MODULE INFORMATION:

```

CODE AREA SIZE      = 0055H      85D
CONSTANT AREA SIZE  = 0000H      0D
VARIABLE AREA SIZE  = 00D6H     214D
MAXIMUM STACK SIZE  = 0000H      0D
12 LINES READ
0 PROGRAM WARNINGS
0 PROGRAM ERRORS

```

DICTIONARY SUMMARY:

```

27KB MEMORY AVAILABLE
3KB MEMORY USED      (11%)
0KB DISK SPACE USED

```

END OF PL/M-86 COMPILATION

Figure 11-4. Sample Program Showing the OPTIMIZE(3) Control (Cont'd.)

OBJECT/NOOBJECT

These are primary controls. They have the form:

```
OBJECT
OBJECT (pathname)
NOOBJECT
Default: OBJECT (source-file.OBJ)
```

The OBJECT control specifies that an object module is to be created during the compilation. The *pathname* is a standard operating system pathname that specifies the file to receive the object module. If the control is absent or if an OBJECT control appears without a pathname, the object module is directed to the same file name used for source input, but with the extension OBJ.

The NOOBJECT control specifies that an object module is not to be produced.

Object Module Sections

The output of the compiler is an object file containing the compiled module. This object module may be linked with other object modules and located using LINK86 and LOC86. A knowledge of the makeup of an object module is not necessary for PL/M-86 programming, but it can aid you in understanding the controls for program size, linkage, and location.

The object module output by the compiler contains five *sections*.

- Code Section
- Constant Section (absent in LARGE case and in ROM—see below)
- Data Section
- Stack Section
- Memory Section

As explained later in this chapter, these sections can be combined in various ways into “memory segments” for execution, depending on the size of the program (SMALL, COMPACT, MEDIUM, or LARGE).

Code Section. This section contains the object code generated by the source program. If either the LARGE control or the ROM control is used, this section also contains the information that would otherwise be in the constant section.

In addition, the code section for the main program module contains a “main program prologue” generated by the compiler. This code precedes the code compiled from the source program, and sets the CPU up for program execution by initializing various registers and enabling interrupts.

Constant Section. This section contains all variables initialized with the DATA attribute, as well as all REAL constants and all constant lists. If the LARGE or ROM controls are used, this information is placed in the code section and no constant section is produced.

Data Section. All variables are allocated space in this section except parameters, based variables, and variables that are located with an AT attribute, initialized with the DATA attribute, or local to a REENTRANT procedure.

Further, if a nested procedure refers to any parameter of an enclosing procedure, then during execution all parameters of the enclosing procedure will be placed in the data segment. The compiler reserves enough space during compilation to prepare for this.

Stack Section. The stack section is used in executing procedures, as explained in Appendixes F and G. It is also used for any temporary storage used by the program but not explicitly declared in the source module (such as temporary variables generated by the compiler).

The exact size of the stack is automatically determined by the compiler except for possible multiple incarnations of reentrant procedures. The user can override this computation of stack size and explicitly state the stack requirement during the relocation process.

NOTE

When using reentrant procedures or interrupt procedures the user must be careful to allocate a stack section large enough to accommodate all possible storage required by multiple incarnations of such procedures. The stack size can be explicitly specified during the relocation and linkage process.

The stack space requirement of each procedure is shown in the listing produced by the SYMBOLS or XREF control. This information can be used to compute the additional stack space required for reentrant or interrupt procedures.

Memory Section. This is the area of memory referenced by the built-in PL/M-86 identifier MEMORY. Its maximum allowable size depends on the size control used in compilation (SMALL, COMPACT, MEDIUM, or LARGE) as explained below.

The compiler generates a memory section of length zero, and it is your responsibility to specify the actual (run-time) space required during the linkage and relocation process. The *iAPX 86,88 Family Utilities User's Guide* can assist you in this.

DEBUG/NODEBUG

These are primary controls. They have the form

```
DEBUG
NODEBUG
Default: NODEBUG
```

The DEBUG control specifies that the object module is to contain the statement number and relative address of each source program statement, information about each local symbol (including based symbols and procedure parameters), and block information for each procedure. This information may be used later by a debugger such as the ICE 86 debugger.



OPTIMIZE(0) is the only level of optimization that does not optimize code between program lines. Thus, it is the only one that gives guaranteed results when your programs are debugged with a debugger such as the ICE 86.

TYPE/NOTYPE

These are primary controls. They have the form

TYPE

NOTYPE

Default: **TYPE**

The **TYPE** control specifies that the object module is to contain information on the variable types output in symbol records. This information may be used later for type checking by the iAPX 86 Linker, or by a debugger such as the ICE 86 debugger.

The **NOTYPE** control specifies that such type definitions are not to be placed in the object module.

Program Size Controls

These controls specify the memory size requirements of the program that is to contain the module being compiled. They affect the operation of the compiler in various ways and impose certain constraints on the source module being compiled.

These controls also influence how locations are referenced in the compiled program, which leads to certain programming restrictions for each size control.

Note that for maximum efficiency of the object code, the smallest possible size should be used for any given program. Also note that all modules of a program must be compiled with the same size control. These are primary controls. They have the form:

SMALL

COMPACT

MEDIUM

LARGE

Default: **SMALL**

Extensions to these controls are discussed in Chapter 13.

NOTE

Usage of the **NIL** pointer can lead to undesirable results (see section 9.7).

The SMALL Case

When modules compiled with the **SMALL** control are linked, the code sections from all modules are combined and are allocated space within one segment. The segment address for this segment is kept in the **CS** register. The constant, data, stack, and memory sections from all modules are allocated space within a second segment. The segment address for this second segment is kept in the **DS** register, with an identical copy in the **SS** register.

Therefore, the **SMALL** control may be used if the total size of all code sections does not exceed 64K, and the total size of all constant, data, stack, and memory sections does not exceed 64K.

Since there is only one segment for code, the segment address for this segment (CS register) is never updated during program execution. Likewise, since there is only one segment for constants, data, stack, and memory sections, the segment address for this segment (DS and SS registers) is never updated (except when an interrupt occurs, as explained in Appendix G). Therefore, when any location is referenced, only a 16-bit offset is calculated and then used in conjunction with the appropriate segment address. POINTER values are therefore 16-bit values in the SMALL case, and this leads to the following restrictions.

1. POINTER variables may not be initialized with, or assigned, whole-number constants, for example:

```
DECLARE RR POINTER INITIAL (2277); /*invalid under SMALL*/
DECLARE SS POINTER;
SS = 100; /*invalid under SMALL*/
```

2. The @ operator must not be used with a variable that was located at an absolute address that was specified by a whole-number constant, for example:

```
DECLARE JD BYTE AT (100), PD POINTER;
PD = @JD; /*invalid under SMALL*/
```

This restriction does not apply if the absolute address in the declaration is *created* by the @ operator with a variable, for example:

```
DECLARE UKE BYTE, NAGE POINTER;
DECLARE SKI BYTE AT (@UKE);
NAGE = @SKI; /*valid*/
```

3. The PUBLIC attribute must not be used with a variable located at an absolute address specified by a whole-number constant. As above, this restriction does not apply when @ is used:

```
DECLARE SHOMEN BYTE PUBLIC AT (100); /*invalid*/

DECLARE IKYO BYTE;
DECLARE SANKYO BYTE PUBLIC AT (@IKYO); /*valid*/
```

This restriction arises because external variables are assumed to be in the DATA segment.

4. Restrictions 1 and 2 apply also to WORD variables when used as offset pointers and to the use of the dot operator.
5. The @ and dot operators may not be used with variables based on SELECTOR, for example:

```
DECLARE SEL SELECTOR;
DECLARE R BASED SEL BYTE;
DECLARE PD POINTER;
PD = @R /* invalid under SMALL */
```

6. If the built-in function SELECTOR\$OF is used, it will always return the value of the DS register. If BUILD\$PTR is used, it will ignore the SELECTOR expression (see section 9.7).

PL/M-80 Compatibility. The SMALL control is the most likely choice when compiling a program written in PL/M-80. The compiler produces error messages to flag violations of any of the restrictions or to flag the use of the new reserved keywords

(DWORD, INTEGER, REAL, POINTER, SELECTOR, and CAUSEINTER-
RUPT) as programmer-defined identifiers. Otherwise, complete upwards
compatibility is provided by PL/M-86.

NOTE

Care must be used with the dot operator under conditions other than SMALL
and RAM.

The COMPACT Case

A program compiled with the COMPACT control has four segments: code, data, stack, and memory. Each of these is the result of combining the same-type sections from all modules, and each has a maximum size of 64K bytes. The constant sections from all modules are grouped with the data segment unless the ROM control is used, which causes all constant sections to be merged into the CODE segment instead. Since the code, data, and stack segments are fully defined by the time the program is loaded, the segment base addresses in the CS and SS registers are never changed. (The DS register may change when an interrupt occurs, as explained in Appendix F.)

All code and a few prologue constants are addressed relative to CS. All data except absolute data (declared with the AT (*constant*) attribute) are addressed relative to DS. The stack is addressed relative to SS. ES is not initialized and can change during execution. References to any location require only a 16-bit offset address against these segment base addresses.

The sole programming restriction in the COMPACT case is that PUBLIC variables may not be declared AT an absolute location, for example:

```
DECLARE ANVIL BYTE PUBLIC AT (100);
```

is not allowed. This restriction does not apply when the "location" within the AT attribute is formed with the @ operator, i.e., DECLARE ANVIL BYTE PUBLIC AT (@HAMR); is valid. However, the phrases "@ MEMORY" and "MEMORY" are not allowed in defining a PUBLIC variable.

Programming Restrictions in the COMPACT Case. The following restrictions must be observed:

1. When an exported procedure is indirectly activated, a POINTER variable must be used in the CALL statement, for example:

```
$COMPACT(SUBSYS HAS MOD1, MOD2, MOD3; EXPORTS PROC)
MOD1: DO
    DECLARE P POINTER, W WORD;
    PROC: PROCEDURE PUBLIC;
    .
    .
    .
    END PROC;
    P=@PROC; CALL P; /* POINTER must be used */
    W=.PROC; CALL W; /* not allowed */

END MOD1;
```

2. When a procedure that is not exported is indirectly activated, a **WORD** variable must be used. Note that **WORD** variables do not range over the entire iAPX 86 address space but are restricted to offsets within the current code segment, for example:

```

DECLARE P POINTER, W WORD;
LPROC: PROCEDURE; /* local */
.
.
.
END LPROC;
P=@LPROC; CALL P; /* not allowed */
W=.LPROC; CALL W; /* WORD must be used */

```

The MEDIUM Case

In a program compiled with the **MEDIUM** control, a separate segment is used for the code section of each compiled module. Therefore, the total space required for code may exceed 64K, although the maximum size of any one code section is still limited to 64K.

The constant, data, stack, and memory sections of all modules are combined and are allocated space within a single segment.

At any moment during program execution, one segment of code is the “current” segment, and its segment address is kept in the CS register.. This segment address is updated whenever a **PUBLIC** or **EXTERNAL** procedure is activated, since this may involve a new code segment.

The segment address for the segment containing constants, data, stack, and memory sections is kept in the DS register (with an identical copy in the SS register) and is never changed (except when an interrupt occurs, as explained in Appendix G).

With the **MEDIUM** option, a **POINTER** value is a four-byte quantity containing a segment address and an offset. Therefore, the first three restrictions of the **SMALL** case do not apply. However, the **MEDIUM** case introduces two minor restrictions on indirect procedure activation.

Programming Restrictions in the MEDIUM Case. The following restrictions must be observed:

1. When a **PUBLIC** or **EXTERNAL** procedure is indirectly activated, a **POINTER** variable must be used in the **CALL** statement, for example:

```

DECLARE P POINTER, W WORD;
PROC: PROCEDURE PUBLIC;
.
.
.
END PROC;

P=@PROC; CALL P; /*recommended where an indirect
                  call must be used*/

W=.PROC; CALL W; /*not allowed*/

```

- When a procedure that is not PUBLIC or EXTERNAL is indirectly activated, a WORD variable must be used. This is consistent with PL/M-80, and is not recommended in the PL/M-86 programs because WORD variables do not range over the entire iAPX 86 address space (but are restricted to offsets within an assumed segment), for example:

```

DECLARE P POINTER, ^W WORD;
LPROC: PROCEDURE;      /*local*/
.
.
.
        END LPROC;
P=@LPROC; CALL P;      /*not allowed*/

W=.LPROC; CALL W;      /*not recommended, but
                        allowed*/

```

- A variable that is absolutely located (by using the AT attribute with a numeric constant) may not have the PUBLIC attribute. For example:

```
DECLARE B BYTE PUBLIC AT(100);
```

is not allowed.

Restrictions 1 and 2 arise from the fact that the code segment address may change during program execution. Restriction 3 is the same as Restriction 3 in the SMALL case, and arises for the same reason.

The LARGE Case

In a program compiled with the LARGE control, a separate segment is used for the code section (with constants) from each compiled module. Thus the total space required for code and constants may exceed 64K, but the total for the code section (with constants) from any one module is limited to 64K.

A separate segment is used for the data section from each compiled module. Thus the total space required for data sections may exceed 64K, although the size of any one data section is limited to 64K.

The stack sections from all modules are combined in one segment, and the memory sections for all modules are combined in another segment. Thus the total space required for stack is limited to 64K, and the total space required for memory is also limited to 64K.

At any moment during program execution, one code segment and one data segment are "current." Code and data segments are paired, so that the current code and data segments are always from the same module. The compiler implements this pairing by placing the segment address for the data segment in a reserved location in the code section. During program execution, the segment addresses for the current code and data segments are kept in the CS and DS registers, respectively, and are updated whenever a PUBLIC or EXTERNAL procedure is activated, as this may involve new code and data segments.

The stack segment address is kept in the SS register.

Programming Restrictions in the LARGE Case. These first two are the same as Restrictions 1 and 2 in the MEDIUM case, and arise for the same reason.

1. When a PUBLIC or EXTERNAL procedure is indirectly activated, a POINTER variable must be used in the CALL statement, for example:

```

DECLARE P POINTER, W WORD;
PROC: PROCEDURE PUBLIC;
.
.
.
END PROC;

P=@PROC; CALL P; /*recommended where an indirect
                  call must be made*/

W=.PROC; CALL W ; /*not allowed*/

```

2. When a procedure that is not PUBLIC or EXTERNAL is indirectly activated, a WORD variable must be used. This is consistent with PL/M-80, and is not recommended in PL/M-86 programs because WORD variables do not range over the entire iAPX 86 address space (but are restricted to offsets within an assumed segment), for example:

```

DECLARE P POINTER, W WORD;
LPROC: PROCEDURE;      /*local*/
.
.
.
END LPROC;

P=@LPROC; CALL P;      /*not allowed*/

W=.LPROC; CALL W;      /*not recommended, but
allowed*/

```

RAM/ROM Control

This primary control directs the object-module placement of all constants, both user-defined and compiler-generated. Its form is:

```

RAM
ROM

```

Default: RAM (except in LARGE model)

The default setting places the CONSTANT section within the DATA segment in all segmentation models (sizes) except LARGE, in which constants are placed in the CODE segment instead.

The ROM setting places constants in the CODE segment. Under this setting, the INITIAL attribute on a variable produces a warning message. The dot operator is not advised for variable references under the ROM option since constants and variables will be relative to different segment registers. If SMALL is also specified, then pointers will be four bytes instead of two. (See also Appendix F.)

If the keyword DATA is used in a PUBLIC declaration when compiling with the ROM control, DATA must also be used in the EXTERNAL declaration of program modules that reference it. However, no value list is then permitted since the data is defined elsewhere.

11.5 Listing Selection and Content Controls

These controls determine what types of listings are to be produced and on which device they are to appear. The controls are discussed in the following order:

PRINT/NOPRINT
LIST/NOLIST
CODE/NOCODE
XREF/NOXREF
SYMBOLS/NOSYMBOLS

A sample listing is discussed at the end of this section.

PRINT/NOPRINT

These are primary controls. They have the form:

PRINT
PRINT (*pathname*)
NOPRINT
Default: PRINT (*source-file.LST*)

The PRINT control specifies that printed output is to be produced. *Pathname* is a standard operating system pathname that specifies the file to receive the printed output. Any output-type device, including a disk file, may also be given. If the control is absent, or if a PRINT control appears without a pathname, printed output is sent to a file that has the same name as the source file but with the extension LST.

The NOPRINT control specifies that no printed output is to be produced, even if implied by other listing controls such as LIST and CODE.

LIST/NOLIST

These are general controls. They have the form:

LIST
NOLIST
Default: LIST

The LIST control specifies that listing of the source program is to resume with the next source line read.

The NOLIST control specifies that listing of the source program is to be suppressed until the next occurrence, if any, of a LIST control.

When LIST is in effect, all input lines (from the source file or from an INCLUDE file), including control lines, are listed. When NOLIST is in effect, only source lines associated with error messages are listed.

Note that the LIST control *cannot* override a NOPRINT control. If NOPRINT is in effect, no listing whatsoever is produced.

CODE/NOCODE

These are general controls. They have the form:

CODE
NOCODE
Default: **NOCODE**

The **CODE** control specifies that listing of the generated object code in standard assembly language format is to begin. This listing is placed at the end of the program listing on the listing file.

The **NOCODE** control specifies that listing of the generated object code is to be suppressed until the next occurrence, if any, of a **CODE** control.

Note that the **CODE** control cannot override a **NOPRINT** control.

XREF/NOXREF

These are primary controls. They have the form:

XREF
NOXREF
Default: **NOXREF**

The **XREF** control specifies that a cross-reference listing of source program identifiers is to be produced on the listing file.

The **NOXREF** control suppresses the cross-reference listing.

Note that the **XREF** control cannot override a **NOPRINT** control.

SYMBOLS/NOSYMBOLS

These are primary controls. They have the form:

SYMBOLS
NOSYMBOLS
Default: **NOSYMBOLS**

The **SYMBOLS** control specifies that a listing of all identifiers in the PL/M-86 source program and their attributes is to be produced on the listing file.

The **NOSYMBOLS** control suppresses such a listing.

Note that the **SYMBOLS** control cannot override a **NOPRINT** control.

11.6 Listing Format Controls

Format controls determine the format of the listing output of the compiler. The controls are discussed in the following order:

PAGING/NOPAGING
PAGELENGTH
PAGEWIDTH
TITLE
SUBTITLE
EJECT

PAGING/NOPAGING

These are primary controls. They have the form:

P A G I N G
N O P A G I N G
Default: P A G I N G

The PAGING control specifies that the listed output is to be formatted onto pages. Each page carries a heading identifying the compiler and a page number, and possibly a user specified title.

The NOPAGING control specifies that page ejecting, page heading, and page numbering are not to be performed. Thus, the listing appears on one long “”page” as would be suitable for a slow serial output device. If NOPAGING is specified, a page eject is not generated if an EJECT control is encountered.

PAGELENGTH

This is a primary control. It has the form:

P A G E L E N G T H (*length*)
Default: P A G E L E N G T H (6 0)

where

<i>length</i>	is a non-zero, unsigned integer specifying the maximum number of lines to be printed per page of listing output. This number is taken to include the page headings appearing on the page.
---------------	---

The minimum value for *length* is 5.

PAGEWIDTH

This is a primary control. It has the form:

P A G E W I D T H (*width*)
Default: P A G E W I D T H (1 2 0)

where

<i>width</i>	is a non-zero, unsigned integer specifying the maximum line width, in characters, to be used for listing output.
--------------	--

The minimum value for *width* is 60; the maximum value is 132.

TITLE

This is a primary control. It has the form:

TITLE (' *title* ')

Default: **TITLE (' *module name* ')**

where

title is a sequence of printable ASCII characters that are enclosed in quotes.

The sequence, truncated on the right to fit, if necessary, is placed in the title line of each page of listed output.

The maximum length allowed for *title* is 60 characters, but a narrow pagewidth may restrict this number further. For example:

TITLE ('TEST PROGRAM 4')

SUBTITLE

This is a general control. It has the form:

SUBTITLE (' *subtitle* ')

Default: no subtitle

where

subtitle is a sequence of printable ASCII characters that are enclosed in single quotes.

The sequence, truncated on the right to fit, if necessary, is placed in the subtitle line of each page of listed output.

The maximum length allowed for *subtitle* is 60 characters, but a narrow pagewidth may restrict this number further. For example:

SUBTITLE ('TEST PROGRAM 4')

When a SUBTITLE control appears before the first noncontrol line in the source file, it causes the specified subtitle to appear on the first page and all subsequent pages until another SUBTITLE control appears.

A subsequent SUBTITLE control causes a page eject, and the new subtitle appears on the next page and all subsequent pages until the next SUBTITLE control.

EJECT

This is a general control. It has the form:

EJECT

It causes printing of the current page to terminate and a new page to be started. The control line containing the EJECT control is the last one printed on the old page.

Sample Program Listing

During the compilation process, a listing of the source input is produced. Each page of the listing carries a numbered page-header that identifies the compiler, prints a time and date as designated by the host operating system, and optionally gives a title and a subtitle, and/or a date (see figure 11-5).

The first part of the listing contains a summary of the compilation, beginning with the compiler identification and the name of the source module being compiled. The next line names the file receiving the object code. Then, the command line used to invoke the compiler is reproduced. The listing of the program itself is shown in figure 11-5.

The listing contains a copy of the source input plus additional information. Two columns of numbers appear to the left of the source image. The first column provides a sequential numbering of PL/M-86 statements. Error messages, if any, refer to these statement numbers. The second column gives the block nesting depth of the current statement.

Lines included with the INCLUDE control are marked with an equal sign (=) just to the left of the source image. If the included file contains another INCLUDE control, lines included by this nested INCLUDE are marked with an "=1". For yet another level of nesting, "=2" is used to mark each line, and so forth up to the compiler's limit of five levels of nesting. These markings make it easy to see where included text begins and ends.

Should a source line be too long to fit on the page in one line, it will be continued on the following line. Such continuation lines are marked with a hyphen (-) just to the left of the source image.

The CODE control may be used to obtain the iAPX 86 assembly code produced in the translation of each PL/M-86 statement. This code listing appears interspersed in the source text in six columns of information in a pseudo-assembly language format:

1. Location counter (hexadecimal notation)
2. Resultant binary code (hexadecimal notation)
3. Label field
4. Opcode mnemonic
5. Symbolic arguments
6. Comment field

Not all six of these columns will appear on any one line of the code listing. Compiler generated labels (e.g., those which mark the beginning and ending of a DO WHILE loop) are preceded by an AT sign (@). The comments appearing on PUSH and POP instructions indicate the stack depth associated with the stack instruction.

Symbol and Cross-Reference Listing

If specified by the XREF or SYMBOLS control, a summary of all identifier usage appears following the program listing.

Depending on whether the SYMBOLS or XREF control was used to request the identifier usage summary, six or seven types of information are provided in the symbol or cross-reference listing. These are as follows:

1. Statement number where identifier was defined
2. Relative address associated with identifier

```

system-id PL/M-86 V2.1 COMPILATION OF MODULE STACK
OBJECT MODULE PLACED IN :F1:STACK.OBJ
COMPILER INVOKED BY: :F1:PLM86.86 :F1:STACK.SRC NOPAGING CODE XREF
                        TITLE(STACK MODULE)

```

```

                STACK: DO;
                /* This module implements a BYTE stack with push and pop */
2      1      DECLARE S(100) BYTE, /*Stack Storage*/
                T BYTE PUBLIC INITIAL(-1); /*Stack Index*/

3      1      PUSH: PROCEDURE (B) PUBLIC; /*Pushes B onto the stack*/
4      2      DECLARE B BYTE;
5      2      S(T:=T+1) = B; /*Increment T and store B*/
6      2      END PUSH;

7      1      POP: PROCEDURE BYTE PUBLIC; /*Returns value popped from stack*/
8      2      RETURN S((T:=T-1)+1); /*Decrement T, return S(T+1)*/
9      2      END POP;
10     1      END STACK; /*Module ends here*/

```

```

; STATEMENT # 3

```

```

                PUSH      PROC NEAR
0000  55      PUSH      BP
0001  8BEC    MOV       BP,SP
; STATEMENT # 5
0003  8A066400 MOV      AL,T
0007  FEC0    INC       AL
0009  B400    MOV       AH,0H
000B  88066400 MOV      T,AL
000F  8BD8    MOV       BX,AX
0011  8A4604  MOV       AL,[BP].B
0014  88870000 MOV      S[BX],AL
; STATEMENT # 6
0018  5D      POP       BP
0019  C20200  RET       2H

```

```

                PUSH      ENDP
; STATEMENT # 7

```

```

                POP       PROC NEAR
001C  55      PUSH      BP
001D  8BEC    MOV       BP,SP
; STATEMENT # 8
001F  8A066400 MOV      AL,T
0023  FEC8    DEC       AL
0025  B400    MOV       AH,0H
0027  88066400 MOV      T,AL
002B  8BD8    MOV       BX,AX
002D  8A870100 MOV      AL,S[BX+1H]
0031  5D      POP       BP
0032  C3      RET
; STATEMENT # 9

```

```

                POP      ENDP
; STATEMENT # 10

```

```

DEFN  ADDR  SIZE  NAME, ATTRIBUTES, AND REFERENCES

```

Figure 11-5. Program Listing

- 3. Size of object identified (in bytes)
- 4. The identifier
- 5. Attributes of the identifier (including expansion for LITERALLYs and scoping information for local variables and parameters)
- 6. Statement numbers where identifier was referenced (XREF control only)
- 7. Statement numbers where identifier was assigned a value (XREF control only)

Notice that a single identifier may be declared more than once in a source module (i.e., an identifier defined twice in different blocks). Each such unique object, even though named by the same identifier, appears as a separate entry in the listing.

The address given for each object is the location of that object relative to the start of its associated section. The object's attributes determine which section is applicable.

Identifiers in the SYMBOLS or XREF listing are given in alphabetical order with the following exception: members of structures are listed, in order of declaration, immediately following the entry for the structure itself. Indentation is used to differentiate between these entries.

The XREF listing differentiates between items 6 and 7 by adding the character to statement numbers where a value is assigned. For example, if statement 17 read:

```
I = I + 1;
```

the list of statement numbers would include 17 and 17*, indicating a reference and an assignment in statement 17.

The AUTOMATIC attribute indicates that the identifier was declared as a parameter or as a local variable in a REENTRANT procedure and therefore is allocated dynamically on the stack.

Figure 11-6 is an example of the cross-reference listing.

P /M-86 COMPILER

STACK MODULE
CROSS-REFERENCE LISTING

DEFN	ADDR	SIZE	NAME, ATTRIBUTES, AND REFERENCES
4	0004H	1	B. BYTE IN PROC (PUSH) PARAMETER AUTOMATIC 4 5
7	001CH	23	POP. PROCEDURE BYTE PUBLIC STACK=0002H
3	0000H	28	PUSH PROCEDURE PUBLIC STACK=0004H
2	0000H	100	S. BYTE ARRAY(100) 5* 8
1	0000H		STACK. PROCEDURE STACK=0000H
2	0064H	1	T. BYTE PUBLIC INITIAL 5 5* 8 8*

Figure 11-6. Cross-Reference Listing

Compilation Summary

Following the listing (or appearing alone if NOLIST is in effect) is a compilation summary. Eight pieces of information are provided:

- *Code area size* gives the size in bytes of the *code section* of the output module.
- *Constant area size* gives the size in bytes of the *constant section* of the output module.
- *Variable area size* gives the size in bytes of the *data section* of the output module.
- *Maximum stack size* gives the size in bytes of the *stack section* allocated for the output module.
- *Lines read* gives the number of source lines processed during compilation.
- *Program warnings* give the number of warning messages issued during compilation.
- *Program errors* give the number of error messages issued during compilation.
- *Dictionary summary* gives the amount of memory available for the dictionary, the amount of memory that it actually used, and the amount of disk space that it occupied.

Figure 11-7 is an example of the compilation summary. Refer to Chapter 13 for an explanation of the various object module sections.

```

MODULE INFORMATION:

CODE AREA SIZE      = 0033H      51D
CONSTANT AREA SIZE  = 0000H      0D
VARIABLE AREA SIZE  = 0065H     101D
MAXIMUM STACK SIZE  = 0004H      4D
14 LINES READ
0 PROGRAM WARNINGS
0 PROGRAM ERRORS

DICTIONARY SUMMARY:

27KB MEMORY AVAILABLE
3KB MEMORY USED    (11%)
0KB DISK SPACE USED

END OF PL/M-86 COMPILATION

```

Figure 11-7. Compilation Summary

11.7 Source Inclusion Controls

These controls allow the input source to be changed to a different file. The controls are:

INCLUDE
SAVE/RESTORE

INCLUDE

INCLUDE is a general control, with the form:

I N C L U D E (*pathname*)

where

pathname is a standard operating system pathname specifying a file.

An INCLUDE control must be the rightmost control in a control line or in the invocation command.

The INCLUDE control causes subsequent source lines to be input from the specified file. Input will continue from this file until an end-of-file is detected. At that time, input will be resumed from the file which was being processed when the INCLUDE control was encountered.

An included file may itself contain INCLUDE controls. Note that such nesting of included files may not exceed a depth of *five*.

SAVE/RESTORE

These are general controls. They have the form:

SAVE
RESTORE

These controls allow the settings of certain general controls to be saved on a stack before an INCLUDE control switches the input source to another file, and then restored after the end of the included file. However, SAVE and RESTORE can be used for other purposes as well. The controls whose settings are saved and restored are:

LIST/NOLIST
CODE/NOCODE
OVERFLOW/NOOVERFLOW
LEFTMARGIN
COND/NOCOND

The SAVE control saves all of these settings on a stack. This stack has a maximum capacity of five sets of control settings, which corresponds to the maximum nesting depth of five for the INCLUDE control.

The RESTORE control restores the most recently saved set of control settings from the stack.

11.8 Conditional Compilation Controls

These controls allow selected portions of the source file to be skipped by the compiler if specified conditions are not met. Figure 11-8 shows an example program using the conditional compilation controls, while figure 11-9 shows the same example with NOCOND being used.

The controls are:

```
IF/ELSEIF/ELSE/ENDIF
SET/RESET
COND/NOCOND
```

IF/ELSE/ELSEIF/ENDIF

These controls provide the actual conditional capability using conditions which are based on the values of switches.

These controls cannot be used in the invocation of the compiler, and each must be the only control on its control line.

An IF control and an ENDIF control are used to delimit an "IF element," which can have several different forms. The simplest form of IF element is:

```
$ IF condition
text
$ END IF
```

where

condition is a limited form of PL/M expression in which the only operators allowed are OR, XOR, NOT, AND, <, <=, =, >=, and >, and the only operands allowed are switches and whole-number constants in the range 0 to 255. If the switch did not appear in a set control, a value of "false" (0) is assumed. Parenthesized subexpressions are not allowed. Within these restrictions, *condition* is evaluated according to the PL/M-86 rules for expression evaluation. Note that *condition* must be followed by a carriage return.

text is text which will be processed normally by the compiler if the least significant bit of the value of *condition* is a 1, or skipped if the bit is a 0. Note that text may contain any mixture of PL/M-86 source and compiler controls. If the text is skipped, any controls within it are not processed.

The second form of IF element contains an ELSE element:

```
$IF condition
text 1
$ ELSE
text 2
$ END IF
```

In this construction, *text 1* will be processed normally if the least significant bit of the value of *condition* is a 1, while *text 2* will be skipped. If the bit is a 0, *text 1* will be skipped and *text 2* will be processed normally.

Note that only one ELSE element is allowed within an IF element.

PL/M-86 COMPILER EXAMPLE

system-id PL/M-86 V2.1 COMPILATION OF MODULE EXAMPLE
 OBJECT MODULE PLACED IN :F1:CEX.OBJ
 COMPILER INVOKED BY: :F1:PLM86.86 :F1:CEX.SRC SET(DEBUG=3)

```

EXAMPLE: DO;

2  1  DECLARE BOOLEAN LITERALLY 'BYTE', TRUE LITERALLY '0FFH',
      FALSE LITERALLY '0';
3  1  PRINT$DIAGNOSTICS: PROCEDURE (SWITCHES, TABLES) EXTERNAL;
4  2  DECLARE (SWITCHES, TABLES) BOOLEAN;
5  2  END PRINT$DIAGNOSTICS;

6  2  DISPLAY$PROMPT: PROCEDURE EXTERNAL; END DISPLAY$PROMPT;

8  2  AWAIT$CR: PROCEDURE EXTERNAL; END AWAIT$CR;

      $IF DEBUG = 1
          CALL PRINT$DIAGNOSTICS (TRUE, FALSE);
      $  RESET (TRAP)
      $ELSEIF DEBUG = 2
          CALL PRINT$DIAGNOSTICS (TRUE, TRUE);
      $  RESET (TRAP)
      $ELSEIF DEBUG = 3
10  1  CALL PRINT$DIAGNOSTICS (TRUE, TRUE);
11  1  CALL PRINT$DIAGNOSTICS (TRUE, TRUE);
      $  SET (TRAP)
      $ENDIF

      $IF TRAP
12  1  CALL DISPLAY$PROMPT;
13  1  CALL AWAIT$CR;
      $ENDIF

14  1  END EXAMPLE;

```

MODULE INFORMATION:

```

CODE AREA SIZE      = 0017H      23D
CONSTANT AREA SIZE = 0000H       0D
VARIABLE AREA SIZE = 0000H       0D
MAXIMUM STACK SIZE = 0006H       6D
30 LINES READ
0 PROGRAM WARNINGS
0 PROGRAM ERRORS

```

DICTIONARY SUMMARY:

```

27KB MEMORY AVAILABLE
3KB MEMORY USED   (11%)
0KB DISK SPACE USED

```

END OF PL/M-86 COMPILATION

Figure 11-8. Sample Program Showing the SET(DEBUG=) Control

The most general form of IF element allows one or more ELSEIF elements to be introduced *before* the ELSE element (if any):

```

$ IF condition 1
text 1
$ ELSEIF condition 2
text 2
$ ELSEIF condition 3
text 3
.
.
.

$ ELSEIF condition n
text n
$ ELSE
text n + 1
$ ENDIF

```

where any of the ELSEIF elements may be omitted, as may the ELSE element.

The conditions are tested in sequence. As soon as one of them yields a value with a 1 as its least significant bit, the associated text is processed normally. All other text in the IF element is skipped. If none of the conditions yields a least significant bit of 1, the text in the ELSE element (if any) is processed normally and all other text in the IF element is skipped.

SET/RESET

These are general controls. The SET control has the general form:

SET (*switch assignment list*)

where

switch assignment list consists of one or more switch assignments separated by commas.

A switch assignment has the form:

switch

or:

switch = *value*

where

switch is a name which is formed according to the PL/M-86 rules for identifiers. Note that a switch name exists only at the compiler control level, and therefore you may have a switch with the same name as an identifier in the program; no conflict is possible. However, note that a PL/M-86 reserved word may *not* be used as a switch name.

value is a whole-number constant in the range 0 to 255. This value is assigned to the switch. If the *value* and the equal sign (=) are omitted from the switch assignment, the default value 0FFH ("true") is assigned to the switch.

The following is an example of a SET control line:

```
$SET(TEST, ITERATION=3)
```

This example sets the switch TEST to “true” (OFFH) and the switch ITERATION to 3. Note that switches do not need to be declared.

Figure 11-8 is an example of a program that was compiled using the SET control.

The RESET control has the form:

```
RESET (switch list)
```

where

switch list consists of one or more switch names that have already occurred in SET controls.

Each switch in the *switch list* is set to “false” (0).

COND/NOCOND

These controls determine whether text within an IF element will appear in the listing if it is skipped. These general controls have the form:

```
COND
```

```
NOCOND
```

Default: COND

The COND control specifies that any text that is skipped is to be listed (without statement or level numbers). Note that a COND control cannot override a NOLIST or NOPRINT control, and that a COND control will not be processed if it is within text which is skipped.

The NOCOND control specifies that text within an IF element which is skipped is not to be listed. However, the controls that delimit the skipped text *will* be listed, providing an indication that something has been skipped. Note that a NOCOND control will not be processed if it is within text that is skipped.

Figure 11-9 is an example of a program that was compiled using the NOCOND control.

```
system-id PL/M-86 V2.1 COMPILATION OF MODULE EXAMPLE
OBJECT MODULE PLACED IN :F1:CEX.OBJ
COMPILER INVOKED BY: :F1:PLM86.86 :F1:CEX.SRC SET(DEBUG=3) NOCOND
```

```
EXAMPLE: DO;

2 1  DECLARE BOOLEAN LITERALLY 'BYTE', TRUE LITERALLY 'OFFH',
    FALSE LITERALLY '0';
3 1  PRINT$DIAGNOSTICS: PROCEDURE (SWITCHES, TABLES) EXTERNAL;
4 2  DECLARE (SWITCHES, TABLES) BOOLEAN;
5 2  END PRINT$DIAGNOSTICS;

6 2  DISPLAY$PROMPT: PROCEDURE EXTERNAL; END DISPLAY$PROMPT;
```

Figure 11-9. Sample Program Showing the NOCOND Control

```

8      2      AWAIT$CR: PROCEDURE EXTERNAL; END AWAIT$CR;

          $IF DEBUG = 1
          $ELSEIF DEBUG = 3
10     1      CALL PRINT$DIAGNOSTICS (TRUE, TRUE);
11     1      CALL PRINT$DIAGNOSTICS (TRUE, TRUE);
          $ SET (TRAP)
          $ENDIF

          $IF TRAP
12     1      CALL DISPLAY$PROMPT;
13     1      CALL AWAIT$CR;
          $ENDIF

14     1      END EXAMPLE;

```

MODULE INFORMATION:

```

CODE AREA SIZE      = 0017H      23D
CONSTANT AREA SIZE = 0000H       0D
VARIABLE AREA SIZE = 0000H       0D
MAXIMUM STACK SIZE = 0006H       6D
30 LINES READ
0 PROGRAM WARNINGS
0 PROGRAM ERRORS

```

DICTIONARY SUMMARY:

```

27KB MEMORY AVAILABLE
3KB MEMORY USED      (11%)
0KB DISK SPACE USED

```

END OF PL/M-86 COMPILATION

Figure 11-9. Sample Program Showing the NOCOND Control (Cont'd.)



In the sample program given in figure 12-1, we first declare a procedure called SORTPROC which implements the following sorting method (assume that the records are all in memory, stored as an array of structures). The key for each record is a member of the structure.

Now we go through the array from the second record (record number 1) upwards. When we reach any given record (the “current” record), we will already have sorted the preceding records. (The first time through, when we look at record number 1, record number 0 is the only preceding record.)

We take the current record, store it temporarily in a buffer, and look backwards through the preceding records until we find one whose key is not greater than that of the current record. Then we put the current record just after this record.

This procedure makes only the following assumptions about the records it is to sort:

- Each record occupies a contiguous set of storage locations. Therefore, by using based variables, each record can be handled as a sequence of bytes, even though the parts of a record are not necessarily BYTE scalars.
- The records themselves are also stored contiguously, so the entire set of records can be regarded as a single sequence of bytes. The location of the first byte of the first record is specified by the POINTER parameter PTR.
- All records are the same size; that is, each occupies the same number of bytes. This size is specified by the WORD parameter RECSIZE and may not exceed 128.
- In each record, the value of one byte is used as the sort key. Within each record, this byte is always in the same relative position, that is, the first byte in the record, or the third, etc. This relative position (or offset) is specified by the WORD parameter KEYINDEX, which resembles an array subscript; that is, it is 0 if the key is the first byte in the record, 1 if the key is the second byte, etc.
- The number of records is specified by the WORD parameter COUNT.

The following is a detailed explanation of the program.

After the PROCEDURE statement and the declaration of the parameters, we declare a based BYTE array called RECORD. This array is based on the parameter PTR, which points to the beginning of the first record to be sorted. Therefore, a reference to a scalar element of RECORD will be a reference to some byte within the set of records to be sorted as long as the subscript used with RECORD is less than the total number of bytes in all the records (i.e., the subscript must be less than COUNT * RECSIZE).

Note that a dimension specifier of 1 is used in declaring RECORD. We need to use a nonzero dimension specifier here in order to use subscripts later in the procedure. However, the value of the dimension specifier is unimportant because RECORD is a based array and does not have any actual storage allocated to it. The value 1 is chosen arbitrarily.

Next, we declare CURRENT, an array of 128 BYTE elements; this array will be used to store the “current” record. Note that the dimension (size) of the array CURRENT is what establishes the maximum size of the records that this procedure can handle. We have chosen 128 here, but, in principle, any dimension could be specified.

SERIES-III PL/M-86 V2.1 COMPILATION OF MODULE SORTMODULE
 OBJECT MODULE PLACED IN :F1:PROG12.OBJ
 COMPILER INVOKED BY: :F1:PLM86.86 :F1:PROG12.SRC

```

SORT$MODULE: DO; /* Beginning of module. */

2  1  SORTPROC: PROCEDURE(PTR, COUNT, RECSIZE, KEYINDEX);
3  2  DECLARE PTR POINTER, (COUNT, RECSIZE, KEYINDEX) WORD;

    /* Parameters:
    PTR is pointer to first record.
    COUNT is number of records to be sorted.
    RECSIZE is number of bytes in each record, max is 128.
    KEYINDEX is byte position within each record of a BYTE scalar
    to be used as sort key. */

4  2  DECLARE RECORD BASED PTR(1) BYTE,
      CURRENT(128) BYTE,
      (I, J) WORD;

5  2  SORT:
      DO J = 1 TO COUNT-1;
6  3      CALL MOVB(@RECORD(J*RECSIZE), @CURRENT, RECSIZE);
7  3      I = J;
8  3      DO WHILE I > 0
          AND (RECORD((I-1)*RECSIZE+KEYINDEX) >
              CURRENT(KEYINDEX));
9  4          CALL MOVB(@RECORD((I-1)*RECSIZE), @RECORD(I*
              RECSIZE), RECSIZE);
10 4          I = I-1;
11 4      END;
12 3      CALL MOVB(@CURRENT, @RECORD(I*RECSIZE), RECSIZE);
13 3  END SORT;

14 2  END SORTPROC;

    /* Program to sort two sets of records, using SORTPROC. */

15 1  DECLARE SET1 (50) STRUCTURE
      (ALPHA WORD,
      BETA (12) BYTE,
      GAMMA INTEGER,
      DELTA REAL,
      EPSILON BYTE);

16 1  DECLARE SET2 (500) STRUCTURE
      (ITEMS (21) INTEGER,
      VOLTS REAL,
      KEY BYTE);

    /* Data is read in to initialize the records. */

17 1  CALL SORTPROC (@SET1, LENGTH(SET1), SIZE(SET1(0)),
      SIZE(SET1(0).ALPHA));
18 1  CALL SORTPROC(@SET2, LENGTH(SET2), SIZE(SET2(0)),
      .SET2(0).KEY - .SET2(0));

    /* Data is written out from the records. */

19 1  END SORT$MODULE; /* End of module. */

```

Figure 12-1. Sample Program: Record Sort

MODULE INFORMATION:

```

CODE AREA SIZE      = 00CEH      206D
CONSTANT AREA SIZE = 0000H        0D
VARIABLE AREA SIZE = 606AH      24682D
MAXIMUM STACK SIZE = 000EH       14D
54 LINES READ
0 PROGRAM WARNINGS
0 PROGRAM ERRORS

```

DICTIONARY SUMMARY:

```

27KB MEMORY AVAILABLE
3KB MEMORY USED      (11%)
0KB DISK SPACE USED

```

END OF PL/M-86 COMPILATION

Figure 12-1. Sample Program: Record Sort (Cont'd.)

The WORD variables I and J are used to control the DO WHILE iterative DO loops. J is the index variable of the DO statement, and is always the subscript of the current record. (When J becomes greater than 127, the sort is done.) I is the index variable used to control the DO WHILE block; I-1 is always the subscript of a previously sorted record. In this program, I and J are also used to calculate subscripts for the based array RECORD.

In the statement following the iterative DO, we used the built-in procedure MOV B to copy a sequence of byte values from one storage location to another. (See Chapter 9 for details.)

In the first activation of MOV B, the parameter @RECORD(J*RECSIZE) is the location of the beginning of the Jth record, and @CURRENT is the location of the beginning of the array CURRENT. Thus, the effect of this CALL statement is to copy the Jth record into the array CURRENT.

To understand the DO WHILE statement, consider that RECORD((I-1)*RECSIZE) would be the first byte of the (I-1)st record, so RECORD((I-1)*RECSIZE + KEYINDEX) is the byte that is to be used as the sort key of the (I-1)st record. Similarly, CURRENT (KEYINDEX) is the sort key of the "current" record into the position of the Ith record.

To illustrate the way this procedure can be used, it is a set in a program that declares two sets of records, SET1 and SET2, and sorts them. Comments are inserted in place of the code that would be used in a working program to read data into the records and write it out after they are sorted.

SET1 is a set of 50 structures, each of which represents one record. Each structure contains a WORD scalar, an array of 12 BYTE scalars, an INTEGER scalar, a REAL scalar, and another BYTE scalar. We want to sort the records using the first element of the 12-byte array as the sort key. Since the key is the second element within the structure, its offset is just the number of bytes occupied by the first element of the structure. Therefore, we will use the built-in function SIZE to calculate the offset of the key and use that as the value of the parameter KEYINDEX.

SET2 is a set of 500 structures, each containing an array of 21 INTEGER scalars, a REAL scalar, and a BYTE scalar that is to be used as the key. This time, the key is deep within the structure. We can calculate the offset of the key manually, but this

is both error prone and inflexible. If the elements within the structure are changed, then we must remember to recalculate the offset of the key. A better method is to let the compiler calculate the offset for us. This can be accomplished by subtracting the location of the key element from the location of the start of the record. The result of this subtraction is the offset of the key, which becomes the value of the parameter KEYINDEX.

In the two CALL statements used to activate SORTPROC, we used the built-in function LENGTH to determine the number of records that are to be sorted, and we used the built-in function SIZE to calculate the number of bytes occupied by each record. These two values are used for the COUNT and RECSIZE parameters for an activation of SORTPROC. See Chapter 9 for complete details on built-in functions.

NOTE

To generate and output reasonable sort data, user-supplied routines must be added to this program in the areas indicated.



13.1 Introduction

Program *segmentation* is a technique used to optimize the object code produced by the compiler. It allows for easier, more efficient memory addressing.

The simplest way to compile PL/M-86 code is to use the program size controls (SMALL, COMPACT, MEDIUM, and LARGE) outlined in Chapter 11. However, PL/M-86 provides extended segmentation facilities to take full advantage of the segmented iAPX 86 architecture and to simplify the development of very large programs. This chapter discusses the segmentation control options available to you.

Each compilation produces an object module composed of several sections (see section 11.4). At link time, the sections from separately compiled modules are combined into segments depending on compiler-generated attributes and your input to the iAPX 86 Linker. Once combined into a segment, all constituent sections can be addressed from the same iAPX 86 segment register. The compiler uses this segment addressability to improve the code produced for data references and procedure calls. The primary purpose of the segmentation controls is to tell the compiler how separately compiled code and data will be located in memory.

Most PL/M-86 programmers need not be concerned about memory addressing techniques on the iAPX 86 because the SMALL, COMPACT, MEDIUM, and LARGE controls transparently handle the mechanics of program segmentation. If needed, a description of iAPX 86 memory concepts is given in the *Introduction to the iAPX 86*.

By making the variety of segmentation schemes open to you virtually unlimited, these controls can reduce the storage required for pointers and the code required to reference external variables and procedures.

Extended Segmentation

You can use segment controls to create a large program where each module is compiled with the same segmentation control, or you can use the extended controls to partition your program into a number of loosely-coupled subsystems.

A *subsystem* is a collection of tightly-coupled, logically-related modules that obey the same model of segmentation. A program is made up of one or more subsystems. (If you only use the simple controls to compile your program modules, then your program consists of one subsystem.) If appropriate, the subsystems within a program can use different segmentation models. Calls and data references within a subsystem are long or short depending on the segmentation model selected for the subsystem. All calls between subsystems are long and most data references require 32-bit pointers.

Optimized code is obtained by breaking LARGE programs (greater than 64K bytes of code, data, or both) into COMPACT subsystems having less than 64K bytes of code and less than 64K bytes of data. References between modules in the same subsystem can then use more efficient 16-bit addresses. (Only references outside a subsystem need to use the full 32-bit address.) Use of these extensions also allows easier access to the 16 megabyte address space.

13.2 Subsystems

By carefully constructing subsystems, you can minimize references to outside resources (code and data) and, in return, receive highly optimized code for all internal resource references. This section describes how to create subsystems using the extended segmentation controls.

Note that all modules in your program should be compiled with the same set of subsystem definitions so the compiler will make consistent assumptions about the location of externals. If inconsistent subsystem definitions are used when compiling modules, the iAPX 86 Linker (LINK86) will generate an error.

Each subsystem in a PL/M-86 program has one segmentation control, which takes one of the following forms:

- 1) $\$model \left(subsystem-id [submodel] HAS module-list \left[\begin{array}{l} ; HAS module-list \\ ; EXPORTS public-list \end{array} \right] \right) \dots$
- 2) $\$model ([submodel];] EXPORTS public-list)$
- 3) $\$model ([submodel[; EXPORTS public-list])$

where

<i>model</i>	specifies the model of segmentation that the subsystem will follow (COMPACT and LARGE are allowed, but LARGE subsystems will not provide optimized code). All modules in the subsystem must be compiled with the same segmentation model.
<i>subsystem-id</i>	specifies a unique name for each subsystem, and can be up to 31 characters long. The <i>subsystem-id</i> is required by all closed subsystems (see below).
<i>submodel</i>	specifies the placement of constants. The <i>submodel</i> can be either - CONST IN CODE - (for burning into ROM; default case for LARGE) - CONST IN DATA - (for efficient access to constants in programs that are loaded into RAM; default case for COMPACT)
HAS <i>module-list</i>	specifies all the modules that make up the subsystem. A HAS list is required by all closed subsystems (see below). Each identifier in the <i>module-list</i> is the name of a module, which can be up to 31 characters long.
EXPORTS <i>public-list</i>	lists the constants, procedures, and variables exported by this subsystem. Any object not named in an EXPORTS list will be local to its subsystem and should not be accessed from outside the subsystem. Each identifier in the <i>public-list</i> is the name of a public object, which can be up to 31 characters long.

In most applications of the subsystem controls, the HAS and EXPORTS lists will have several dozen entries apiece. To accommodate lists of this length, a subsystem control may be continued over more than one control line. (The continuation lines must be contiguous, and each must begin with a dollar sign (\$) in the first column). Also, note that any number of HAS and EXPORTS lists may appear in a control, in any order, allowing you to format your subsystem specification so it can be easily read and maintained.

Consider the following subsystem definition:

```
$COMPACT(Room -CONST IN CODE- HAS Chair, Door, Window)
$COMPACT(Diner HAS Booths, Waitress, Jukebox;
$          EXPORTS Lunches)
$LARGE(Allocate EXPORTS CodeSize, DataSize,
$          MemSize, FreeSpace)
```

This sample program contains three subsystems: Room, Diner, and Allocate. Room and Diner use the COMPACT model of segmentation while Allocate uses the LARGE model. Constants are stored with the code in Room and Allocate. The Room subsystem, which contains the modules Chair, Door, and Window, is apparently the main program because it exports no objects. Allocate supplies four objects: CodeSize, DataSize, MemSize, and FreeSpace.

Open and Closed Subsystems

The subsystems that make up your PL/M-86 program may be either open or closed. The subsystem definition for an *open* subsystem does not have a subsystem name and does not list the modules it contains; in short, it does not contain a HAS list. Modules can be added to this subsystem at any time without changing the subsystem definition. Each program may have only one open subsystem.

The subsystem definition for a *closed* subsystem contains a subsystem name and a HAS list, which specifies all the modules in the subsystem.

In an open subsystem, only the submodel and EXPORTS modifiers are permitted. By omitting the subsystem name, you automatically create an open subsystem that contains all modules not claimed in another subsystem's HAS list. (Note that use of the simple segmentation controls also creates an open subsystem.)

One advantage of using an open subsystem is that it simplifies dealing with a COMPACT program whose code has grown too large. When your program exhausts its 64K code size, take a subset of your modules and put them into a closed subsystem, leaving the rest of the modules in the open subsystem.

For example, assume that the following modules from your original program (ATTACH, OPEN, CLOSE, ERRORS, ALLOCATE, FREE) were compiled with this simple control:

```
$COMPACT
```

If you factor out the modules ALLOCATE and FREE from the original program, creating SUBSYS1, its subsystem definition would be:

```
$COMPACT (SUBSYS1 HAS ALLOCATE, FREE)
```

Now, suppose that the modules remaining in the open subsystem reference entry points AllocBuff and FreeBuff in SUBSYS1. These must be exported from SUBSYS1 as follows:

```
$COMPACT (SUBSYS1 HAS ALLOCATE, FREE;
$          EXPORTS AllocBuff, FreeBuff)
```

or

```
$COMPACT (SUBSYS1 HAS ALLOCATE; EXPORTS AllocBuff;
$          HAS FREE; EXPORTS FreeBuff)
```

The second form illustrates how multiple HAS and EXPORTS lists can be used to document the items exported from each module; it also illustrates the use of a continuation line.

Likewise, if a routine in SUBSYS1 references the procedure FatalError in the module ERRORS, the definition of the open subsystem would then be:

```
$COMPACT (EXPORTS FatalError)
```

No data structures need to be changed, because data reference values can still be 16 bits. All procedures except AllocBuff and FreeBuff will still use the short call and return mechanism.

The EXPORTS List

A symbol included in a subsystem's EXPORTS list must be declared PUBLIC in one of the modules in that subsystem. The symbol, called an exported symbol, may be referenced by modules in other subsystems. A public symbol defined within a subsystem but not listed in its EXPORTS list is called a domestic symbol. It should be referenced only by modules within the same subsystem.

A procedure should be exported only if it must be referenced outside the defining subsystem, because accessing exported procedures will generally require more code and time than is normally required for domestic procedures.

Exported procedures have the following characteristics:

- They use the long form of call and return.
- They save and restore the caller's DS register upon entry and exit.
- They reload DS with their associated data segment upon entry.

However, because short pointers are used in the SMALL (RAM) model of segmentation, care must be used when you call procedures that have pointer parameters and are exported from a subsystem. In these cases, the compiler will always use the value of the current DS register as the selector portion of the long pointer. This means that passing a pointer to any data items declared in the SMALL module will produce the proper result, but the following restrictions must be observed for the special cases:

1. If the actual parameter is the NIL pointer, DS:0 will be passed to the exported procedure. Consequently, the procedure will execute differently if called from a SMALL module than it will if called from a COMPACT, MEDIUM, or LARGE module. For example:

```
$COMPACT (FOO HAS N; EXPORTS FOO)
$SMALL
```

```

M: DO;
  DECLARE PTR POINTER;
  FOO: PROCEDURE (P) EXTERNAL;
    DECLARE P POINTER;
  END FOO;

  CALL FOO (NIL); /* Wrong, will pass DS:0 */

  PTR = NIL;
  CALL FOO (PTR); /* Wrong, will pass DS:0 */

END M;

$COMPACT (FOO HAS N; EXPORTS FOO)
N: DO;
  FOO: PROCEDURE (P) PUBLIC;
    DECLARE P POINTER;
    DECLARE B BYTE;
    B = (P=NIL); /* Will assign FALSE
                  (000H) to B if FOO is
                  called from SMALL;
                  Will assign TRUE
                  (0FFH) to B
                  otherwise */

  END FOO;

  CALL FOO (NIL); /* Right, will pass
                  0:0 */

END N;

```

2. If the actual parameter is a pointer to a procedure, the compiler will extend the short pointer with DS then pass the value of DS:(offset of procedure) to the exported procedure. This situation should be avoided because the result of any reference through such a pointer is undefined. For example:

```

$COMPACT (FOO HAS N; EXPORTS FOO)
$SMALL
M: DO;
  DECLARE PTR POINTER;
  DECLARE TABLE (10) BYTE;
  FOO: PROCEDURE (P) EXTERNAL;
    DECLARE P POINTER;
  END FOO;
  BAZ: PROCEDURE;
  ...
END BAZ;

  CALL FOO (@BAZ); /* Wrong, will pass
                  DS:offset-of-BAZ */

  PTR = @BAZ;
  CALL FOO (PTR); /* Wrong, will pass DS:PTR */

  CALL FOO (@TABLE); /* Right, will pass
                    pointer to TABLE */

```

```
PTR = @TABLE;  
CALL FOO (PTR); /* Right, will pass pointer  
                to TABLE */  
  
END M;
```

Placement of Controls

The segmentation controls have special restrictions associated with their placement. These rules are as follows:

- Only the definition of the open subsystem (with no submodel and no EXPORTS list) may be placed on the invocation line; definitions of all other subsystems must occur inside the source program.
- The definition of the open subsystem (if present) must be the last segmentation control specified.
- Only the open subsystem can be without a HAS list.

The subsystem definitions for your entire program may be included in the compilation of each module using the INCLUDE control. The compiler will extract the information it needs to compile correctly and efficiently each module's intrasubsystem and intersubsystem references.



The compiler may issue five varieties of error messages:

- Source PL/M-86 errors
- Fatal command tail and control errors
- Fatal input/output errors
- Fatal insufficient memory errors
- Fatal compiler failure errors

The source errors are reported in the program listing; the fatal errors are reported on the console device.

14.1 Source PL/M-86 Errors

Nearly all of the source PL/M-86 errors are interspersed in the listing at the point of error and follow the general format:

```
***ERROR mmm IN nnn (LINE ppp), NEAR 'aaa', message
```

or:

```
***WARNING mmm IN nnn (LINE ppp), NEAR 'aaa', message
```

where

<i>mmm</i>	is the error number from the list below.
<i>nnn</i>	is the source statement number where the error occurs.
<i>ppp</i>	is the actual source line number where the error occurs.
<i>aaa</i>	is the source text near where the error is detected.
<i>message</i>	is the error explanation from the list below.

A prefix of W means the message is a warning only, and object code was not suppressed. Errors not so prefixed prevent object code generation.

Source error message list:

1. INVALID CONTROL

See Chapter 11. Example:

```
$NXC0DE; /* probably intended NOC0DE */
```

2. ILLEGAL USE OF PRIMARY CONTROL AFTER NON-CONTROL LINE

Primary controls may appear as control lines in your source program, but they must come first. No other statements may precede them. See Chapter 11.

3. MISSING CONTROL PARAMETER

Certain controls, e.g., INCLUDE, require you to specify a parameter. See Chapter 11.

4. INVALID CONTROL PARAMETER

One example is an illegal pathname for a control like OBJECT. See Chapter 11.

5. INVALID CONTROL FORMAT

See Chapter 11 for correct formatting of control lines. An example that could cause this error is:

```
$LIST (MYPRG.LST);
```

because no pathname is expected on this control. It could also be caused if an INCLUDE was followed by another control on the same line.

(W) 6. ILLEGAL PRINT CONTROL, IGNORED

PRINT (:CI:) would be an example, since you cannot print to the console input device. See Chapter 11.

7. INVALID PATHNAME

See the operating instructions for your specific host system.

(W) 8. ILLEGAL PAGELength, IGNORED

See section 11.6.

(W) 9. ILLEGAL PAGEWIDTH, IGNORED

See section 11.6.

(W) 10. RESPECIFIED PRIMARY CONTROL, IGNORED

See section 11.1.

11. MISPLACED ELSE OR ELSEIF CONTROL

See section 11.8.

12. MISPLACED ENDIF CONTROL

See section 11.8.

13. MISSING ENDIF CONTROL

See section 11.8.

14. SWITCHNAME TOO LONG (31), TRUNCATED

See section 11.8.

15. MISSING OPERATOR

See section 11.8.

(W) 16. INVALID CONSTANT, ZERO ASSUMED

See section 11.8.

17. INVALID OPERAND

See section 11.8.

19. LIMIT EXCEEDED: SAVE NESTING (5)

See section 11.8.

20. LIMIT EXCEEDED: INCLUDE NESTING (5)

For example, if you include a file named A, which includes a file named B, and so on, this error will arise when the limit is exceeded.

21. MISPLACED RESTORE CONTROL

RESTORE can only work if there has been a prior SAVE. See section 11.7.

22. UNEXPECTED END OF CONTROL

A segmentation control was expecting a continuation line or a ')'.
.

23. SYMBOL EXISTS IN MORE THAN ONE HAS LIST

A module name may occur in only one HAS list.

24. SUBSYSTEM ALREADY DEFINED

The subsystem name has already been defined.

25. HAS OR EXPORT LIST RESPECIFIED

You cannot have more than one of each in a sub-system control.

26. ILLEGAL PL/M IDENTIFIER

Identifier does not meet the rules for PL/M identifiers. See section 2.3.

(W) 28. INVALID PLM86 CHARACTER, IGNORED

Look near the text flagged for an invalid character, or one that is inappropriate in context. Delete it or retype the statement.

(W) 29. UNPRINTABLE CHARACTER, IGNORED

Retype the line in question using valid characters.

30. NAME OR STRING TOO LONG, TRUNCATED

Match your intended variable type with the length of the flagged item. For the correct maximum lengths, see Chapters 2 and 3.

31. ILLEGAL CONSTANT TYPE

This might reflect missing operators, e.g., $A=4T$ instead of $4+T$. See Chapter 2 for the list of valid types.

32. INVALID CHARACTER IN CONSTANT

For example, 107B and 0ABCD will cause this error because neither can be valid in any PL/M-86 interpretation; 7 is not a binary numeral, B may not occur in decimal or octal, and neither string ends in H. See Chapter 2.

33. RECURSIVE MACRO EXPANSION

Here is an example causing this error:

```
DECLARE A LITERALLY 'B';
DECLARE B LITERALLY 'A';
.
.
.
B=4; /* error discovered here */
```

No type can be assigned to variables declared circularly, i.e., solely in terms of each other.

34. LIMIT EXCEEDED: MACRO NESTING (5)

This error occurs when too many DECLARE statements refer back through each other to the one that actually supplies a type. For example:

```
DECLARE A LITERALLY 'B';
DECLARE B LITERALLY 'C';
. . . .
. . . .
DECLARE Y LITERALLY 'Z';
DECLARE Z BYTE INTITAL (77);
.
A=7; /* error discovered here */
```

35. LIMIT EXCEEDED: SOURCE LINE LENGTH (128)

A source line longer than 128 characters was read.

36. CONSTANT TOO LARGE FOR TYPE

DECLARE A BYTE INTITAL (259) would be an example because the maximum byte constant is 255.

37. INVALID REAL CONSTANT

Examples: 1.7F or 1.7. See Chapter 3.

(W) 38. REAL CONSTANT UNDERFLOW

An underflow occurred when conversion into floating-point was attempted.

(W) 39. REAL CONSTANT OVERFLOW

An overflow occurred when conversion into floating-point was attempted.

40. NULL STRING NOT ALLOWED

Strings of length zero (e.g., " ") are not supported.

41. DELETED: "<tokens>"

The compiler deleted the following tokens while attempting to recover from a syntax error.

42. INSERTED: "<tokens>"

The compiler inserted the following tokens while attempting to recover from a syntax error.

43. STATEMENTS FOLLOW MODULE END

Information follows logical end-of-module.

44. CONSTANT TOO LARGE

For example, a constant with the value 999,999,999,999.

(W) 45. MISMATCHED BLOCK IDENTIFIER

See section 6.1. If a label is supplied in an END statement, the label must match the first unmatched DO statement above the END. Sometimes the error involves a module name confused with a procedure name. See also Chapters 1 and 7.

46. DUPLICATE PROCEDURE NAME

Procedure names must be unique. See section 8.1.

47. LIMIT EXCEEDED: PROCEDURES (253)

Too many procedures in this module. Break it into smaller modules.

48. DUPLICATE PARAMETER NAME

A parameter must be declared exactly once. This message indicates that the flagged parameter already has a definition at this block level, as in:

```
YAR: PROCEDURE (YAR77, YAR78);  
DECLARE YAR77 BYTE;  
DECLARE YAR77 BYTE;
```

Perhaps a different spelling was intended.

49. NOT AT MODULE LEVEL

The flagged attribute or initialization can only be valid at the module level, not in a procedure. See Chapters 1, 3, 7, and 8.

50. DUPLICATE ATTRIBUTE

Attributes should be specified, at most, once. This message means the compiler has found a declaration like:

```
DECLARE B BYTE EXTERNAL EXTERNAL;
```

51. ILLEGAL INTERRUPT VALUE

Interrupt numbers must be whole-number constants between 0 and 255. Thus -7 or 272 would be invalid. See section 8.5.

52. INTERRUPT WITH PARAMETERS

No parameters are allowed in interrupt procedures. See section 8.5.

53. INTERRUPT WITH TYPED PROCEDURE

Interrupt procedures must be untyped. See section 8.5.

54. INVALID DIMENSION

See section 5.1.

55. STAR DIMENSION WITH STRUCTURE

Star dimensions (*) are for initialization. See section 3.2.

56. STAR DIMENSION WITH STRUCTURE MEMBER

Star dimension(*) must not be used with structures. See section 3.2.

57. CONFLICT WITH PARAMETER

Attributes may not be used with parameters.

58. DUPLICATE DECLARATION

The flagged item already has a definition declared at this block level.

59. ILLEGAL PARAMETER TYPE

Parameters may not be declared of type structure or array. See section 8.1.

60. DUPLICATE LABEL

Each label must be unique within its block or scope. Otherwise, GOTOs and CALLs would have ambiguous targets. See sections 6.3 and 8.2.

61. DUPLICATE MEMBER NAME

For example, in the case:

```
DECLARE AIR STRUCTURE (F4 BYTE, F4 BYTE);
```

subsequent references to AIR.F4 would be ambiguous. See sections 5.2 and 5.3.

62. UNDECLARED PARAMETER

A parameter named in the procedure statement did not get defined in the body of the procedure. See sections 8.1 and 8.2.

63. CONFLICTING ATTRIBUTES

Certain attributes are not allowed when declaring a parameter, e.g., PUBLIC, EXTERNAL, DATA, INITIAL, AT, or BASED.

64. LIMIT EXCEEDED: DO BLOCKS

See Appendix B for correct limit.

65. ILLEGAL PARAMETER ATTRIBUTE

Certain attributes are not allowed when declaring a parameter, e.g., PUBLIC, EXTERNAL, DATA, INITIAL, AT, or BASED.

66. UNDEFINED BASE

A variable was declared BASED using an undeclared identifier.

67. INVALID ATTRIBUTE FOR BASE

A base must be a non-subscripted scalar of type POINTER, WORD, or SELECTOR. It is not permitted to have the attribute BASED. See section 3.5.

68. MISPLACED DECLARATION

You can intersperse declarations and procedures, but not declarations and executable statements. See Chapter 1.

69. INVALID BASE WITH LABEL OR MACRO

BASED may not be used with LABEL or LITERALLY types.

70. INVALID DIMENSION WITH LABEL OR MACRO

LABEL or LITERALLY may not be dimensioned.

71. INITIALIZATION LIST REQUIRED

A list of initial values is required if the INITIAL attribute is used, if the non-external * dimension form is used, or if the non-external DATA attribute is used.

72. BASED CONFLICTS WITH ATTRIBUTES

Examples of attributes conflicting with base include AT, DATA, INITIAL, PUBLIC, and EXTERNAL. See section 3.5.

73. EXECUTABLE STATEMENTS IN EXTERNAL

An EXTERNAL procedure, being defined elsewhere, may not contain executable statements. See Chapter 7 and section 8.5.

74. MISSING RETURN FOR TYPED PROCEDURE

A typed procedure must return a value; thus, its RETURN statement must specify one.

75. INVALID NESTED REENTRANT PROCEDURE

Reentrant procedures may not contain nested procedures. See section 8.5.

76. LIMIT EXCEEDED: FACTORED LIST (64)

Too many variables were named in a factored declaration. Break it into several declarations.

77. LIMIT EXCEEDED: STRUCTURE ELEMENTS (64)

Too many elements were declared in a structure.

78. MISSING PROCEDURE NAME

Every procedure must have a name. See section 8.1.

79. MULTIPLE PROCEDURE LABELS

Procedures must have exactly one name; no more, no less. See section 8.1.

80. DECLARATIONS MAY NOT BE LABELED

Labels may not be used on declaration statements.

81. STAR DIM WITH FACTORED LIST NOT ALLOWED

Star dimensions (*) are for initializations. See section 3.2.

82. SIZE EXCEEDS 64K BYTES

Storage for the declared item exceeds 64K bytes.

(W) 83. PROCEDURE CONTAINS NO EXECUTABLE STATEMENTS

84. WARNING: DATA USED WITH AT; INITIAL ASSUMED

A variable may not be declared using both the DATA attribute and the AT attribute. See section 3.2.

85. WARNING: INITIAL USED WITH ROM OPTION

Variables declared with INITIAL are not initialized until load-time. Thus, if your program is burned into ROM, these initializations will never occur. See section 3.2.

88. LIMIT EXCEEDED: PROGRAM TOO COMPLEX

The program has too many complex expressions, cases, etc. Break it into smaller procedures.

89. COMPILER ERROR: BAD ERROR RECOVERY

An unrecoverable error occurred. Trying a different copy of the compiler on a different drive might reveal that the first copy had been damaged or went bad. Contact Intel.

91. PROGRAM TOO COMPLEX

The program has too many complex expressions, cases, etc. Break it into smaller procedures.

93. LIMIT EXCEEDED: PROGRAM TOO COMPLEX

The program has too many complex expressions, cases, and procedures. Break it into smaller modules.

95. COMPILER ERROR: PARSE BUFFER OVERFLOW

See source error message number 89.

96. LIMIT EXCEEDED: BLOCK NESTING (18)

The program has too many nested DO blocks. Break it into smaller procedures.

97. COMPILER ERROR: STACK UNDERFLOW

See source error message number 89.

98. LIMIT EXCEEDED: STATEMENT TOO COMPLEX

The statement is too large for the compiler. Break it into several smaller statements.

99. COMPILER ERROR: SEMANTIC UNDERFLOW

See source error message number 89.

100. STRING CONSTANT TOO LONG

String constants used as scalars may have a maximum of 4 characters.

101. UNSUBSCRIPTED ARRAY

In the context of the flagged statement, the array reference requires a subscript. See sections 3.4 and 8.2, and Chapter 5.

(W) 102. UNQUALIFIED STRUCTURE

This statement was ambiguous as to which structure or member was intended. See sections 3.4 and 8.2, and Chapter 5.

103. NOT AN ARRAY

Subscripts are permitted only on identifiers declared as arrays. Check spelling consistency. See Chapter 5.

104. MULTIPLE SUBSCRIPTS

For any array TING, references of the form TING(2,4) or TING(3,7,9,6) are invalid because of multiple subscripts. Only references of one subscript are valid, e.g., TING(5). See section 5.1.

105. NOT A STRUCTURE

For example, a reference of the form GNU.F1, where GNU was not declared a structure. See Chapter 5.

106. UNDEFINED IDENTIFIER

Every identifier must be declared. See Chapter 3.

107. UNDEFINED MEMBER

For example, KAPI.HORN, where KAPI is a valid, declared structure but HORN is an undeclared member of the structure. See Chapter 5.

108. ILLEGAL INDEX TYPE

Only expressions of type **BYTE**, **WORD**, and **INTEGER** can be used. See section 5.1.

109. NOT A LABEL

The identifier following **GOTO** must be a label; the flagged item was declared otherwise.

110. MISSING RETURN VALUE

A typed procedure must return a value; thus, its **RETURN** statement must specify one. See section 8.2.

111. INVALID RETURN WITH UNTYPED PROCEDURE

An untyped procedure does not return a value; thus, its **RETURN** statement may not specify one. See section 8.2.

112. INVALID INDIRECT TYPE

Only **WORD** or **POINTER** scalars can be used for indirect calls. This excludes **WORD** or **POINTER** expressions, **BYTE**, **DWORD**, **INTEGER**, or **REAL** scalars, all structures, and all arrays. See section 8.2.

113. INVALID PARAMETER COUNT

The number of actual parameters supplied in a **CALL** must be equal to the number of formal parameters declared in the procedure. See section 8.2. (See also Chapter 9 for the requirements of built-ins.)

114. QUALIFIED PROCEDURE NAME

Procedure names may not be qualified.

115. INVALID FUNCTION REFERENCE

Typed procedures can be invoked only by use in an expression, not by a **CALL**. See sections 8.1 and 8.2.

116. INVALID CASE EXPRESSION TYPE

Case expressions must be of type **BYTE**, **WORD**, or **INTEGER**.

117. LIMIT EXCEEDED: CASES

See Appendix B for correct limit.

118. TYPE CONFLICT

See section 4.6.

119. INVALID BUILT-IN REFERENCE

Built-in reference was qualified with a member name, or OUTPUT/OUTWORD did not appear on the left side of an assignment.

120. INVALID PROCEDURE REFERENCE

Untyped procedures must be invoked by a CALL statement; references to such procedures are not permitted in expressions. See sections 8.1 and 8.2.

121. INVALID LEFT-HAND SIDE OF ASSIGNMENT

An example is PROCEDURE=4 or INWORD(7)=9. See sections 3.2 and 4.7.

122. INVALID REFERENCE

Invalid label reference.

123. Not used**124. PROCEDURE NAME REQUIRED**

Procedure name is required for SET\$INTERRUPT and INTERRUPT\$PTR built-ins.

125. PROCEDURE NAME ONLY

Parameters are not allowed on the procedure name in SET\$INTERRUPT and INTERRUPT\$PTR.

126. BAD INTERRUPT NUMBER

Interrupt numbers must be whole-number constants between 0 and 255. Thus, -7 and 272 would be invalid. See section 8.5.

127. CONSTANT ONLY

In this instance, a constant is required.

128. ARRAY REQUIRED

Some built-ins need an array name. See section 9.1.

129. Not used

130. INVALID RESTRICTED OPERAND

For example, PL/M-86 reserved words and predeclared identifiers would be invalid. See Appendix A and section 9.1.

131. INVALID RESTRICTED OPERATOR

Only + and - may be used in restricted expressions.

132. REAL CONSTANT EXPRESSION

A constant expression with REALs is not allowed.

133. REFERENCE REQUIRED

A variable reference is required for LENGTH, LAST, and SIZE.

134. VARIABLE REQUIRED

The operand to LENGTH, LAST, and SIZE must be a variable.

135. VALUE TOO LARGE

A value is too large for its contextually determined type.

136. ABSOLUTE POINTER WITH SHORT POINTERS

The NIL pointer was used in the SMALL (RAM) case.

137. INVALID RESTRICTED EXPRESSION

Only addresses or constant types are allowed in restricted expressions.

138. PUBLIC AT EXTERNAL

For example:

```
DECLARE DARTH BYTE EXTERNAL;  
DECLARE VADER BYTE PUBLIC AT (.DARTH);
```

See sections 3.2 and 7.2.

139. PUBLIC AT ABSOLUTE

Absolute locations for PUBLICs are supported only under the LARGE option. See Chapter 11.

140. PUBLIC AT MEMORY

PUBLIC AT @ MEMORY is not supported by SMALL.

141. AT BASED VARIABLE

Based variables cannot be used in AT clauses. See section 3.5.

142. ILLEGAL FORWARD REFERENCE

An AT expression cannot have a forward reference.

143. VARIABLE TYPE REQUIRED IN AN AT EXPRESSION

The expression must be AT a variable.

144. LIMIT EXCEEDED: DATA OR STACK SEGMENT TOO LARGE

See Appendix B for correct limit.

145. LIMIT EXCEEDED: CODE OR CONST SEGMENT TOO LARGE

See Appendix B for correct limit.

146. LIMIT EXCEEDED: TOO MANY EXTERNALS

See Appendix B for correct limit.

147. LABEL NOT AT LOCAL OR MODULE LEVEL

See sections 6.3 and 7.3.

148. ABSOLUTE ADDRESS REFERENCE IN SMALL

See section 11.3.

149. ILLEGAL MODULE NAME REFERENCE

Module names cannot be referenced.

(W) 150. USE OF "." WITH FAR PROCEDURE

A subsequent indirect call made through the respective address/pointer generates the wrong type of call. See section 11.4.

(W) 151. USE OF "@" WITH SHORT PROCEDURE

See source error message number 150.

152. INVALID "." OR "@" OPERAND

Must be used with a variable, procedure, or constant list.

153. INVALID RETURN IN MAIN PROGRAM

A main program must have no returns. See section 8.3.

154. STAR DIMENSIONED EXTERNAL WITH LENGTH, LAST,
or SIZE

The LENGTH, LAST, and SIZE built-in functions cannot be used with variables declared with a star (*) and the EXTERNAL attribute.

155. PUBLIC SYMBOL EXPORTED FROM ANOTHER
SUBSYSTEM

A PUBLIC symbol in this module is also exported by another subsystem. See Chapter 13.

156. LONG POINTER REQUIRED FOR THIS CONSTRUCT

A model with long pointers is required.

157. MEMORY MAY NOT BE USED WITH LENGTH, LAST,
or SIZE

The LENGTH, LAST, and SIZE built-in functions cannot be used with variables declared with * and the EXTERNAL attribute.

158. ILLEGAL INITIALIZATION OF AN EXTERNAL

An external variable cannot be initialized.

159. ILLEGAL INTERRUPT PROCEDURE REFERENCE

An interrupt procedure cannot be invoked with the CALL statement.

160. INTERRUPT PROCEDURES MUST BE PUBLIC

An interrupt procedure must also be given the PUBLIC attribute.

161. Not used

162. LIMIT EXCEEDED: PROGRAM COMPLEXITY

Too many complex expressions, cases, etc. Break it into smaller cedures.

163. COMPILER ERROR: SEMANTIC UNDERFLOW

See source error message number 89.

164. COMPILER ERROR: INVALID NODE

See source error message number 89.

165. COMPILER ERROR: INVALID OPERATOR

See source error message number 89.

166. COMPILER ERROR: INVALID TREE

See source error message number 89.

167. COMPILER ERROR: SCOPE STACK UNDERFLOW

See source error message number 89.

168. LIMIT EXCEEDED: PROGRAM COMPLEXITY

Too many complex expressions, cases, etc. Break it into smaller cedures.

169. COMPILER ERROR: INVALID RECORD

See source error message number 89.

170. INVALID DO CASE BLOCK, AT LEAST ONE CASE REQUIRED

See section 6.1.

171. LIMIT EXCEEDED: NUMBER OF ACTIVE CASES

See Appendix B for correct limit.

172. LIMIT EXCEEDED: NESTING OF TYPED PROCEDURE CALLS

See Appendix B for correct limit.

173. LIMIT EXCEEDED: NUMBER OF ACTIVE PROCEDURES
OR DO CASE GROUPS

See Appendix B for correct limit.

174. ILLEGAL NESTING OF BLOCKS, ENDS NOT BALANCED

For every DO, an END is needed. See section 6.1.

175. COMPILER ERROR: INVALID OPERATION

See source error message number 89.

176. LIMIT EXCEEDED: REAL EXPRESSION COMPLEXITY

The REAL stack has eight registers. Heavily nested use of REAL functions with REAL expressions as parameters could get excessively complex. See Appendix F.

177. COMPILER ERROR: REAL STACK UNDERFLOW

See source error message numbers 89 and 176.

178. LIMIT EXCEEDED: BASIC BLOCK COMPLEXITY

You have a very long list of statements without labels, CASEs, IFs, GOTOs, RETURNS, etc. Either break the procedure into several smaller procedures, or add labels to some of your statements.

179. LIMIT EXCEEDED: STATEMENT SIZE

The statement is too large for the compiler. Break it into several smaller statements.

199. LIMIT EXCEEDED: PROCEDURE COMPLEXITY OF
OPTIMIZE(2) (terminal error)

The combined complexity of statements, user labels, and compiler-generated labels is too great. Simplify as much as possible, perhaps breaking the procedure into several smaller procedures.

200. ILLEGAL INITIALIZATION OF MORE SPACE THAN
DECLARED

The number of initialization values exceeds the number of declared elements. See section 3.2.

201. INVALID LABEL: UNDEFINED

No definition for this label was found. See sections 3.3 and 6.3.

202. LIMIT EXCEEDED: NUMBER OF EXTERNAL ITEMS

See Appendix B for correct limit.

203. COMPILER ERROR: BAD LABEL ADDRESS

See source error message number 89.

204. LIMIT EXCEEDED: CODE SEGMENT SIZE

See Appendix B for correct limit.

205. COMPILER ERROR: BAD CODE GENERATED

See source error message number 89.

206. LIMIT EXCEEDED: DATA SEGMENT SIZE

See Appendix B for correct limit.

251. COMPILER ERROR: INVALID OBJECT

See source error message number 89.

252. COMPILER ERROR: SELF NAME LINK

See source error message number 89.

253. COMPILER ERROR: SELF ATTR LINK

See source error message number 89.

254. LIMIT EXCEEDED: PROGRAM COMPLEXITY

The program has too many complex expressions, cases, and procedures.
Break it into smaller modules.

255. LIMIT EXCEEDED: SYMBOLS

See Appendix B for correct limit.

NOTE

If a terminal error is encountered, program text beyond the point of error is not compiled. A terminal error message will appear at the point of error in the program listing.

14.2 Fatal Command Tail and Control Errors

Fatal command tail errors are caused by an improperly specified compiler invocation command or an improper control. The errors that may occur here are as follows:

```
ILLEGAL COMMAND TAIL SYNTAX OR VALUE
UNRECOGNIZED CONTROL IN COMMAND TAIL
INCLUDE FILE IS NOT A DIRECT ACCESS FILE
INVOCATION COMMAND DOES NOT END WITH <cr><LF>
INCORRECT DEVICE SPECIFICATION
SOURCE FILE NOT A DIRECT ACCESS FILE
SOURCE FILE NAME INCORRECT
SOURCE FILE EXTENSION INCORRECT
ILLEGAL COMMAND TAIL SYNTAX
MISPLACED CONTROL: WORKFILES ALREADY OPENED
```

14.3 Fatal Input/Output Errors

Fatal input/output errors occur when the user incorrectly specifies a pathname for compiler input or output. These error messages are of the form:

```
PL/M-86 ERROR -
FILE:
NAME:
ERROR:
COMPILATION TERMINATED
```

14.4 Fatal Insufficient Memory Errors

The fatal insufficient memory errors are caused by a system configuration with not enough RAM memory to support the compiler.

The errors that may occur due to insufficient memory are as follows:

```
NOT ENOUGH MEMORY FOR COMPILATION
DYNAMIC STORAGE OVERFLOW
NOT ENOUGH MEMORY
```

14.5 Fatal Compiler Failure Errors

The fatal compiler failure errors are internal errors that should never occur. If you encounter such an error, please report it to Intel Corporation, 3065 Bowers Avenue, Santa Clara, California 95051, Attn: Software Marketing Department. The errors falling into this class are as follows:

```
89. COMPILER ERROR: BAD ERROR RECOVERY
95. COMPILER ERROR: PARSE BUFFER OVERFLOW
97. COMPILER ERROR: STACK UNDERFLOW
99. COMPILER ERROR: SEMANTIC UNDERFLOW
163. COMPILER ERROR: SEMANTIC UNDERFLOW
164. COMPILER ERROR: INVALID NODE
165. COMPILER ERROR: INVALID OPERATOR
166. COMPILER ERROR: INVALID TREE
167. COMPILER ERROR: SCOPE STACK UNDERFLOW
169. COMPILER ERROR: INVALID RECORD
175. COMPILER ERROR: INVALID OPERATION
```

177. COMPILER ERROR: REAL STACK UNDERFLOW
203. COMPILER ERROR: BAD LABEL ADDRESS
205. COMPILER ERROR: BAD CODE GENERATED
210. COMPILER ERROR: OBJECT MODULE GENERATION ERROR
211. COMPILER ERROR: DEBUG SEGMENT SIZE OVERFLOW
212. COMPILER ERROR: ILLEGAL FIXUP
251. COMPILER ERROR: INVALID OBJECT
252. COMPILER ERROR: SELF NAME LINK
253. COMPILER ERROR: SELF ATTR LINK



APPENDIX A

PL/M-86 RESERVED WORDS AND PREDECLARED IDENTIFIERS

These are the reserved words of PL/M-86. They may not be used as identifiers.

ADDRESS	INITIAL
AND	INTEGER
AT	INTERRUPT
BASED	LABEL
BY	LITERALLY
BYTE	MINUS
CALL	MOD
CASE	NOT
CAUSEINTERRUPT	OR
DATA	PLUS
DECLARE	POINTER
DISABLE	PROCEDURE
DO	PUBLIC
DWORD	REAL
ELSE	REENTRANT
ENABLE	RETURN
END	SELECTOR
EOF	STRUCTURE
EXTERNAL	THEN
GO	TO
GOTO	WHILE
HALT	WORD
IF	XOR

These are the identifiers for the built-in procedures and predeclared variables. If one of these identifiers is declared in a DECLARE statement, the corresponding built-in procedure or predeclared variable becomes unavailable within the scope of the declaration.

ABS	INPUT
ADJUSTRPL	INT
BLOCKINPUT	INTERRUPTPTR
BLOCKINWORD	INWORD
BLOCKOUTPUT	LAST
BLOCKOUTWORD	LOCKSET
BUILDPTR	LENGTH
CARRY	LOW
CMPB	MEMORY
CMPW	MOVB
DEC	MOVE
DOUBLE	MOVRB
FINDB	MOVW
FINDRB	NIL
FINDRW	OFFSETOF
FINDW	OUTPUT
FIX	OUTWORD
FLAGS	PARITY
FLOAT	RESTOREREALSTATUS
GETREALERROR	ROL
HIGH	ROR
IABS	SAL
INITREALMATHUNIT	

SAR
SAVEREALSTATUS
SCL
SCR
SELECTOROF
SETB
SETINTERRUPT
SETW
SHL
SHR
SIGNED

SIZE
SKIPB
SKIPRB
SKIPRW
SKIPW
STACKBASE
STACKPTR
TIME
UNSIGN
XLAT
ZERO



APPENDIX B

PROGRAM CONSTRAINTS

Certain fixed size tables within the compiler constrain various features of a user program to certain maximums. These limits are summarized below:

MAXIMUM:

Nesting of LITERALLY invocations	5
Nesting of INCLUDE controls	5
Number of nested procedures and DO cases	7
Number of labels on a statement	unlimited
Nesting of blocks	18
Number of nested typed procedures	20
Number of elements in a factored list	64
Number of members in a structure	64
Structure size	64K-1
Numbers of characters in a line	128
Length of a string constant	255
Number of DO blocks in a procedure	255
Number of cases in a DO CASE block	255
Number of active cases	255
Number of EXTERNAL items	255
Number of procedures in a module	253
Segment Size	64K-1

NOTE

The PL/M-86 compiler has a symbol capacity of approximately 5000 symbols. Of these, 800 are held in memory when the compiler has a partition size of 96K. Any symbols over this amount will spill onto the workfiles disk, causing performance degradation. (If this happens, the compiler will display a message to the console.) To increase performance, look at the dictionary summary (found in the compilation summary) to determine the amount of additional memory needed to avoid spilling to the disk. If another 64K of memory is added to the compiler's partition (either by adding more memory to the system or by increasing its share of available memory), a total of 2300 symbols will then be held in memory. For large programs or programs with many symbols, providing the compiler with more memory to work in will improve its performance.



This appendix lists the entire syntax of the PL/M-86 language Backus Naur Form (BNF). Since the semantic rules are not included here, this syntax permits certain constructions that are not actually allowed. The terminology used in the BNF syntax has been designed for convenience in constructing concise and rigorous definitions. Its appearance differs substantially from the main body of the manual.

The notations used here are slightly extended from standard BNF notations. An ellipsis (...) indicates that the syntactic element preceding it may be repeated indefinitely. Braces ({ }) indicate that only one of the items stacked vertically within them may be used. Brackets ([]) indicate that whatever appears within them may be omitted. When items are stacked vertically within brackets, only one of the items may be used.

The nonterminals in the syntax that follows are listed in alphabetical order after section C.5.3. Each nonterminal is tagged with the section number (within this appendix) where its primary definition may be found.

C.1 Lexical Elements

C.1.1 Character Sets

```
<character> ::= <apostrophe>  
             | <non-quote character>
```

```
<non-quote character> ::= <letter>  
                        | <decimal digit>  
                        | $  
                        | <special character>  
                        | blank
```

```
<letter> ::= <upper case letter>  
           | <lower case letter>
```

```
<upper case letter> ::= A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
```

```
<lower case letter> ::= a b c d e f g h i j k l m n o p q r s t u v w x y z
```

```
<decimal digit> ::= 0 1 2 3 4 5 6 7 8 9
```

```
<special character> ::= + | - | * | / | < | > | = | ! | ; | . | ( | ) | @ | _
```

```
<apostrophe> ::= '
```

C.1.2 Tokens

```
<token> ::= <delimiter>  
          | <identifier>  
          | <reserved word>  
          | <numeric constant>  
          | <string>
```

C.1.3 Delimiters

<delimiter> ::= <simple delimiter>
 | <compound delimiter>

<simple delimiter> ::= + | - | * | / | < | > | = | ! | ; | . | , | (|) | @

<compound delimiter> ::= <>
 | <=
 | >=
 | :=

C.1.4 Identifiers and Reserved Words

<identifier> ::= <letter> $\left[\begin{array}{c} \text{<letter>} \\ \text{<decimal digit>} \\ \$ \\ _ \end{array} \right] \dots$

<reserved word> see list, Appendix A

C.1.5 Numeric Constants

<numeric constant> ::= <binary number>
 | <octal number>
 | <decimal number>
 | <hexadecimal number>
 | <floating point number>

<binary number> ::= <binary digit> $\left[\begin{array}{c} \text{<binary digit>} \\ \$ \end{array} \right] \dots \text{B}$

<octal number> ::= <octal digit> $\left[\begin{array}{c} \text{<octal digit>} \\ \$ \end{array} \right] \dots \left\{ \begin{array}{l} \text{O} \\ \text{Q} \end{array} \right\}$

<decimal number> ::= <decimal digit> $\left[\begin{array}{c} \text{<decimal digit>} \\ \$ \end{array} \right] \dots [\text{D}]$

<hexadecimal number> ::= <decimal digit> $\left[\begin{array}{c} \text{<hexadecimal digit>} \\ \$ \end{array} \right] \dots \text{H}$

<floating point number> ::= <digit string> <fractional part> [<exponent part>]

<fractional part> ::= . [<digit string>]

<exponent part> ::= E [+ | -] <digit string>

<digit string> ::= <decimal digit> $\left[\begin{array}{c} \text{decimal digit} \\ \$ \end{array} \right] \dots$

<binary digit> ::= 0 | 1

<octal digit> ::= <binary digit> | 2 | 3 | 4 | 5 | 6 | 7

<decimal digit> ::= <octal digit> | 8 | 9

<hexadecimal digit> ::= <decimal digit> | A | B | C | D | E | F

C.1.6 Strings

<string> ::= ' <string body element> ... '

<string body element> ::= <non-quote character>

| "

C.1.7 PL/M Text Structure: Tokens, Blanks, and Comments

<pl/m text> ::= [<token>] ...
 <separator>]

<separator> ::= blank
 | <comment>

<comment> ::= /* [<character>] ... */

C.2 Modules and the Main Program

<compilation> ::= <module> [EOF]
 <module> ::= <module name> : <simple do block>
 <module name> ::= <identifier>

C.3 Declarations

<declaration> ::= <declare statement>
 | <procedure definition>

C.3.1 DECLARE Statement

<declare statement> ::= DECLARE <declare element list>;
 <declare element list> ::= <declare element> [, <declare element>] ...
 <declare element> ::= <factored element>
 | <unfactored element>

<unfactored element> ::= <variable element>
 | <literal element>
 | <label element>

<factored element> ::= <factored variable element>
 | <factored label element>

C.3.2 Variable Elements

<variable element> ::= <variable name specifier>
 [<array specifier>] <variable type>
 [<variable attributes>]

<variable name specifier> ::= <non-based name>
 | <based name> BASED <base specifier>

<non-based name> ::= <variable name>
 <based name> ::= <variable name>
 <variable name> ::= <identifier>
 <base specifier> ::= <identifier> [. <identifier>]

<variable attributes> ::= [PUBLIC] [<locator>] [<initialization>]
 | [EXTERNAL] [<constant attribute>]

<locator> ::= AT (<expression>)

<constant attribute> ::= DATA

<array specifier> ::= <explicit dimension>
 | <implicit dimension>

<explicit dimension> ::= (<numeric constant>)
<implicit dimension> ::= (*)

<variable type> ::= <basic type>
 | <structure type>

<basic type> ::= INTEGER
 | REAL
 | POINTER
 | SELECTOR
 | BYTE
 | WORD
 | DWORD

C.3.3 Label Element

<label element> ::= <identifier> LABEL [PUBLIC
 EXTERNAL]

C.3.4 Literal Elements

<literal element> ::= <identifier> LITERALLY <string>

C.3.5 Factored Variable Element

<factored variable element> ::= (<variable name specifier>
 [, <variable name specifier>] ...)
 [<explicit dimension>] <variable type>
 [<variable attributes>]

C.3.6 Factored Label Elements

<factored label element> ::= (<identifier> [, <identifier>] ...) LABEL [PUBLIC
 EXTERNAL]

C.3.7 The Structure Type

<structure type> ::= STRUCTURE (<member element> [, <member element>] ...)
<member element> ::= <member name> [<explicit dimension>] <basic type>
<member name> ::= <identifier>
 | (<identifier> [, <identifier>] ...)

C.3.8 Procedure Definition

<procedure definition> ::= <procedure statement>
 [<declaration> ...] [<unit> ...] <ending>

<procedure statement> ::= <procedure name> :PROCEDURE
 [<formal parameter list>] [<procedure type>]
 [<procedure attributes>] ;

```

<procedure name>::=<identifier>
<procedure type>::=<basic type>
<basic type>::=INTEGER
                I REAL
                I POINTER
                I SELECTOR
                I BYTE
                I WORD
                I DWORD

<formal parameter list>::=( <formal parameter> [,<formal parameter>]...)
<formal parameter>::= <identifier>

< procedure attributes>::=
    {
        { <interrupt> }
        EXTERNAL
    }
    {
        { <interrupt> }
        PUBLIC
        REENTRANT
    } ...

```

C.3.9 Attributes

C.3.9.1 AT

```
<locator>::=AT ( <expression> )
```

C.3.9.2 INTERRUPT

```
<interrupt>::=INTERRUPT <numeric constant>
```

C.3.9.3 Initialization

```
<initialization>::=
    {
        INITIAL
        DATA
    } ( <initial value> [,<initial value>] ...)
```

```
<initial value>::=<expression>
                I <string>
```

C.4 Units

```

<unit>::=<conditional clause>
        I <do block>
        I <basic statement>
        I <label definition> <unit>

<basic statement>::=<assignment statement>
                I <call statement>
                I <goto statement>
                I <null statement>
                I <return statement>
                I <iAPX 286 dependent statement>

<scoping statement>::=, <simple do statement>
                I <do-case statement>
                I <do-while statement>
                I <iterative do statement>
                I <end statement>
                I <procedure statement>

<label definition>::=<identifier>:

```

C.4.1 Basic Statements

C.4.1.1 Assignment Statement

<assignment statement> ::= <left part> = <expression>;
 <left part> ::= <variable reference> [, <variable reference>] ...

C.4.1.2 CALL Statement

<call statement> ::= CALL <simple variable> [<parameter list>];
 <parameter list> ::= (<expression> [, <expression>] ...)
 <simple variable> ::= <identifier>
 | <identifier> . <identifier>

C.4.1.3 GOTO Statement

<goto statement> ::= $\left\{ \begin{array}{l} \text{GOTO} \\ \text{GO TO} \end{array} \right\}$ <identifier>;

C.4.1.4 Null Statement

<null statement> ::= ;

C.4.1.5 RETURN Statement

<return statement> ::= <typed return>
 | <untyped return>

<typed return> ::= RETURN <expression> ;
 <untyped return> ::= RETURN ;

C.4.1.6 iAPX 86 Dependent Statements

<iAPX 86 dependent statement> ::= <disable statement>
 | <enable statement>
 | <halt statement>
 | <cause interrupt statement>

<disable statement> ::= DISABLE ;
 <enable statement> ::= ENABLE ;
 <halt statement> ::= HALT ;
 <cause interrupt statement> ::= CAUSE\$INTERRUPT (numeric constant);

C.4.2 Scoping Statements

C.4.2.1 Simple DO Statement

<simple do statement> ::= DO ;

C.4.2.2 DO-CASE Statement

<do-case statement> ::= DO CASE <expression> ;

C.4.2.3 DO-WHILE Statement

<do-while statement> ::= DO WHILE <expression> ;

C.4.2.4 Iterative DO Statement

<iterative do statement> ::= DO <index part> <to part> [<by part>] ;

<index part> ::= <index variable> = <start expression>

<to part> ::= TO <bound expression>

<by part> ::= BY <step expression>

<index variable> ::= <simple variable>

<start expression> ::= <expression>

<bound expression> ::= <expression>

<step expression> ::= <expression>

C.4.2.5 END Statement

<end statement> ::= END [<identifier>] ;

C.4.2.6 Procedure Statement

<procedure statement> ::= <procedure name> : PROCEDURE

[<formal parameter list>] [<procedure type>]

[<procedure attributes>] ;

C.4.3 Conditional Clause

<conditional clause> ::= <if condition> <true unit>

! <if condition> <true element> ELSE <>false element>

<if condition> ::= IF <expression> THEN

<true element> ::= [<label definition> ...] <do block>

! [<label definition> ...] <basic statement>

<>false element> ::= <unit>

<true unit> ::= <unit>

C.4.4 DO Blocks

<do block> ::= <simple do block>

! <do-case block>

! <do-while block>

! <iterative do block>

C.4.4.1 Simple DO Blocks

<simple do block> ::= <simple do statement> [<declaration> ...] [<unit> ...] <ending>

<ending> ::= [<label definition> ...] <end statement>

C.4.4.2 DO-CASE Blocks

<do-case block> ::= <do-case statement> { <unit> ... } <ending>

C.4.4.3 DO-WHILE Blocks

<do-while block> ::= <do-while statement> [<unit> ...] <ending>

C.4.4.4 Iterative DO Blocks

<iterative do block> ::= <iterative do statement> [<unit> ...] <ending>

C.5 Expressions

C.5.1 Primaries

<primary> ::= <constant>
 | <variable reference>
 | <location reference>
 | <subexpression>

<subexpression> ::= (<expression>)

C.5.1.1 Constants

<constant> ::= <numeric constant>
 | <string>

C.5.1.2 Variable References

<variable reference> ::= <data reference>
 | <function reference>

<data reference> ::= <name>[<subscript>][<member specifier>]

<subscript> ::= (<expression>)

<member specifier> ::= .<member name>[<subscript>]

<function reference> ::= <name>[<actual parameters>]

<actual parameters> ::= (<expression>[,<expression>]...)

<member name> ::= <identifier>

<name> ::= <identifier>

C.5.1.3 Location References

<location reference> ::= $\left\{ \begin{array}{l} @ \\ . \end{array} \right\}$ <constant list>

 | $\left\{ \begin{array}{l} @ \\ . \end{array} \right\}$ <variable reference>

<constant list> ::= (<constant>[,<constant>]...)

C.5.2 Operators

<operator> ::= <logical operator>
 | <relational operator>
 | <arithmetic operator>

<logical operator> ::= AND

 | OR

 | NOT

 | XOR

<relational operator> ::= <I> | <=I> | <I> | <I> | <I> | <I>

<arithmetic operator> ::= + | - | PLUS | MINUS | * | / | MOD

C.5.3 Structure of Expressions

<expression> ::= <logical expression>
 | <embedded assignment>

<embedded assignment> ::= <variable reference> := <logical expression>
 <logical expression> ::= <logical factor>
 | <logical expression> <or operator> <logical factor>

<or operator> ::= OR
 | XOR

<logical factor> ::= <logical secondary>
 | <logical factor> <and operator> <logical secondary>

<and operator> ::= AND
 <logical secondary> ::= [<not operator>] <logical primary>
 <not operator> ::= NOT
 <logical primary> ::= <arithmetic expression>
 [<relational operator> <arithmetic expression>]

<relational operator> ::= <I> | <I> <= I> | <I> <> | =

<arithmetic expression> ::= <term>
 | <arithmetic expression> <adding operator> <term>

<adding operator> ::= + | - | PLUS | MINUS
 <term> ::= <secondary>
 | <term> <multiplying operator> <secondary>

<multiplying operator> ::= * | / | MOD
 <secondary> ::= [<unary minus>] <primary>
 [<unary plus>]

<unary minus> ::= -
 <unary plus> ::= +

NONTERMINALS

SECTION

<actual parameters>	C.5.1.2
<adding operator>	C.5.3
<and operator>	C.5.3
<apostrophe>	C.1.1
<arithmetic expression>	C.5.3
<arithmetic operator>	C.5.2
<array specifier>	C.3.2
<assignment statement>	C.4.1.1
<base specifier>	C.3.2
<based name>	C.3.2
<basic statement>	C.4
<basic type>	C.3.2
<binary digit>	C.1.5
<binary number>	C.1.5
<bound expression>	C.4.2.4
<by part>	C.4.2.4
<call statement>	C.4.1.2
<character>	C.1.1
<comment>	C.1.7
<compilation>	C.2
<compound delimiter>	C.1.3
<conditional clause>	C.4.3
<constant list>	C.5.1.3
<constant>	C.5.1.1
<data reference>	C.5.1.2

<decimal digit>	C.1.5
<decimal number>	C.1.5
<declaration>	C.3
<declare element list>	C.3.1
<declare element>	C.3.1
<declare statement>	C.3.1
<delimiter>	C.1.3
<digit string>	C.1.5
<disable statement>	C.4.1.6
<do block>	C.4.4
<do-case block>	C.4.4.2
<do-case statement>	C.4.2.2
<do-while block>	C.4.4.3
<do-while statement>	C.4.2.3
<embedded assignment>	C.5.3
<enable statement>	C.4.1.6
<end statement>	C.4.2.5
<ending>	C.4.4.1
<explicit dimension>	C.3.2
<exponent part>	C.1.5
<expression>	C.5.3
<factored element>	C.3.1
<factored label element>	C.3.6
<factored member>	C.3.7
<factored variable element>	C.3.5
<false element>	C.4.3
<floating point number>	C.1.5
<formal parameter list>	C.3.8
<formal parameter>	C.3.8
<fractional part>	C.1.5
<function reference>	C.5.1.2
<goto statement>	C.4.1.3
<halt statement>	C.4.1.6
<hexadecimal digit>	C.1.5
<hexadecimal number>	C.1.5
<identifier>	C.1.4
<if condition>	C.4.3
<implicit dimension>	C.3.2
<index part>	C.4.2.4
<index variable>	C.4.2.4
<initial value>	C.3.9.3
<initialization>	C.3.9.3
<interrupt>	C.3.9.2
<iterative do block>	C.4.4.4
<iterative do statement>	C.4.2.4
<label definition>	C.4
<label element>	C.3.3
<left part>	C.4.1.1
<letter>	C.1.1
<linkage>	C.3.8
<literal element>	C.3.4
<location reference>	C.5.1.3
<locator>	C.3.9.1
<logical expression>	C.5.3
<logical factor>	C.5.3
<logical operator>	C.5.2
<logical primary>	C.5.3
<logical secondary>	C.5.3
<lower case letter>	C.1.1
<member element>	C.3.7

<member name>	C.3.7
<member specifier>	C.5.1.2
<module name>	C.2
<module>	C.2
<multiplying operator>	C.5.3
<name>	C.5.1.2
<non-based name>	C.3.2
<non-quote character>	C.1.1
<not operator>	C.5.3
<null statement>	C.4.1.4
<numeric constant>	C.1.5
<octal digit>	C.1.5
<octal number>	C.1.5
<operator>	C.5.2
<or operator>	C.5.3
<parameter list>	C.4.1.2
<pl/m text>	C.1.7
<primary>	C.5.1
<procedure attributes>	C.3.8
<procedure definition>	C.3.8
<procedure name>	C.3.8
<procedure statement>	C.3.8
<procedure type>	C.3.8
<relational operator>	C.5.2
<reserved word>	C.1.4
<return statement>	C.4.1.5
<scoping statement>	C.4
<secondary>	C.5.3
<separator>	C.1.7
<simple delimiter>	C.1.3
<simple do block>	C.4.4.1
<simple do statement>	C.4.2.1
<simple variable>	C.4.1.2
<special character>	C.1.1
<start expression>	C.4.2.4
<step expression>	C.4.2.4
<string body element>	C.1.6
<string>	C.1.6
<structure type>	C.3.7
<subexpression>	C.5.1
<subscript>	C.5.1.2
<term>	C.5.3
<to part>	C.4.2.4
<token>	C.1.2
<true element>	C.4.3
<true unit>	C.4.3
<typed return>	C.4.1.5
<unary minus>	C.5.3
<unary plus>	C.5.3
<unfactored element>	C.3.1
<unfactored member>	C.3.7
<unit>	C.4
<untyped return>	C.4.1.5
<upper case letter>	C.1.1
<variable attributes>	C.3.2
<variable element>	C.3.2
<variable name specifier>	C.3.2
<variable name>	C.3.2
<variable reference>	C.5.1.2
<variable type>	C.3.2



D.1 General Comparison

PL/M-86 may be regarded as an extension of the PL/M-80 language, described in Intel document 9800268. PL/M-86 differs from PL/M-80 in three principal respects:

- Floating-point arithmetic and signed integer arithmetic are provided. These are supported by two new data types, REAL and INTEGER.
- The extended addressing capability of the iAPX 86 is supported by two new data types, POINTER and SELECTOR, for storage of iAPX 86 locations, and a new location reference operator, @.
- The set of built-in procedures is greatly expanded.

In addition, the PL/M-80 reserved word ADDRESS is replaced by the PL/M-86 reserved word WORD. Thus where PL/M-80 has only the two data types, BYTE and ADDRESS, PL/M-86 has seven: BYTE, WORD, DWORD, INTEGER, REAL, POINTER, and SELECTOR.

The PL/M-86 rules for expression evaluation are more complete than those of PL/M-80, to make proper use of the extended capabilities. There are also various other differences which stem from the ones noted here. In particular, an iterative DO block operates differently if its index variable is an INTEGER variable.

D.2 Compatibility of PL/M-80 Programs and the PL/M-86 Compiler

PL/M-80 programs that operate correctly on an 8080 can be recompiled with the PL/M-86 compiler to produce code that will run on an iAPX 86. The PL/M-80 source code must first be edited as follows:

- All identifiers in the PL/M-80 source code must be examined and changed if they are PL/M-86 reserved words. The PL/M-86 reserved words that might occur as identifiers in a PL/M-80 source program are WORD, DWORD, INTEGER, REAL, POINTER, SELECTOR, and CAUSEINTERRUPT (since these are not reserved words in PL/M-80).
- It is not necessary to change ADDRESS to WORD; ADDRESS is a PL/M-86 reserved word with the same meaning as WORD.

Note that where PL/M-86 programs would normally have POINTER variables and location references formed with the @ operator, PL/M-80 programs have ADDRESS (WORD) variables and location references formed with the "dot" operator. PL/M-80 usage is therefore slightly less restricted than normal PL/M-86 usage, since arithmetic operations are allowed on WORD values. To provide upward compatibility, the PL/M-86 compiler in general supports PL/M-80 usage. However, some restrictions are imposed, affecting the types of expressions allowed in the AT attribute, the INITIAL and DATA initializations, and location references. See also the discussions of size controls and the dot and @ operators in this manual.

(In fact, all of these constructions are formally permitted by the PL/M-86 language, but their use in PL/M-86 programs is not recommended for most purposes, since they will not always produce correct results in a program where POINTER values also appear.)



APPENDIX E ASCII CODES

ASCII CHARACTER	HEX	PL/M-86 CHARACTER?	ASCII CHARACTER	HEX	PL/M-86 CHARACTER?
NUL	00	no	@	40	yes
SOH	01	no	A	41	yes
STX	02	no	B	42	yes
ETX	03	no	C	43	yes
EOT	04	no	D	44	yes
ENQ	05	no	E	45	yes
ACK	06	no	F	46	yes
BEL	07	no	G	47	yes
BS	08	no	H	48	yes
HT	09	no	I	49	yes
LF	0A	no	J	4A	yes
VT	0B	no	K	4B	yes
FF	0C	no	L	4C	yes
CR	0D	no	M	4D	yes
SO	0E	no	N	4E	yes
SI	0F	no	O	4F	yes
DLE	10	no	P	50	yes
DC1	11	no	Q	51	yes
DC2	12	no	R	52	yes
DC3	13	no	S	53	yes
DC4	14	no	T	54	yes
NAK	15	no	U	55	yes
SYN	16	no	V	56	yes
ETB	17	no	W	57	yes
CAN	18	no	X	58	yes
EM	19	no	Y	59	yes
SUB	1A	no	Z	5A	yes
ESC	1B	no	[	5B	no
FS	1C	no	\	5C	no
GS	1D	no	]	5D	no
RS	1E	no	^(↑)	5E	no
US	1F	no	—	5F	yes
space	20	yes	,	60	no
!	21	no	a	61	yes
"	22	no	b	62	yes
#	23	no	c	63	yes
\$	24	yes	d	64	yes
%	25	no	e	65	yes
&	26	no	f	66	yes
'	27	yes	g	67	yes
(	28	yes	h	68	yes
)	29	yes	i	69	yes
*	2A	yes	j	6A	yes
+	2B	yes	k	6B	yes
,	2C	yes	l	6C	yes
—	2D	yes	m	6D	yes
.	2E	yes	n	6E	yes
/	2F	yes	o	6F	yes
0	30	yes	p	70	yes
1	31	yes	q	71	yes
2	32	yes	r	72	yes
3	33	yes	s	73	yes
4	34	yes	t	74	yes
5	35	yes	u	75	yes
6	36	yes	v	76	yes
7	37	yes	w	77	yes
8	38	yes	x	78	yes
9	39	yes	y	79	yes
:	3A	yes	z	7A	yes
;	3B	yes	{	7B	no
<	3C	yes		7C	no
=	3D	yes	}	7D	no
>	3E	yes	~	7E	no
?	3F	no	DEL	7F	no



APPENDIX F

LINKING TO MODULES WRITTEN IN OTHER LANGUAGES

This appendix describes the calling conventions used by iAPX 86 family languages. These calling conventions are standardized so that a module written in PL/M-86 can freely call procedures, subroutines, and subprograms in other modules written in other iAPX 86 family languages.

As a PL/M-86 programmer calling PL/M-86 procedures and functions from PL/M-86, you do not need the information in this appendix. You need to know information about parameters and arguments as described in Chapter 8.

As a PL/M-86 programmer calling ASM86 subroutines, you need to know the calling conventions of stack and register usage described in this appendix. You also need to know the corresponding data types listed at the end of this appendix in order to write a subroutine that can pick up the data your PL/M-86 program passes to it. Refer to the *ASM86 Macro Assembler Operating Instructions* for more information about linking to ASM86 programs and for examples of linking such programs to PL/M-86 programs.

The easiest way to ensure compatibility between assembly-language subroutines that are linked to PL/M-86 programs or procedures is to write a dummy procedure in PL/M-86. This procedure would have the same argument list and the same attributes as the desired assembly language subroutine. The PL/M-86 procedure would then be compiled with the correct size control and with the CODE control specified. This will produce a pseudoassembly listing of the generated iAPX 86 code, which may then be copied as the prologue and epilogue of the assembly language subroutine.

F.1 Introduction

Since PL/M-86 allows you to write and compile separate PL/M-86 modules that can be linked together at a later time, you can solve a big problem with a solution composed of separately-tested modules that are linked together after they are internally bug-free. Not all modules have to be in PL/M-86—you can choose the appropriate languages for each module. Be sure to link the modules properly with the iAPX 86 Linker (LINK86) to satisfy references to externals. Since the iAPX 86 family languages follow the same calling sequence (described in the following section), control will pass to a called module correctly. However, the called module might not be able to deal intelligently with the data passed to it since languages treat some data structures differently.

By specifying arguments in a reference to an external procedure, you pass data to the external procedure. The number of arguments and the order in which you specify them must match the number and order of the corresponding parameters in the external procedures declaration (see Chapter 8).

All arguments for parameters are passed on the iAPX 86 stack or the 8087 register stack in the order in which they were specified. Functions that return values return a simple-type value (except REAL values) in a register, or a REAL value on the top of the 8087 register stack.

F.2 Calling Sequence

The *calling sequence* for each procedure activation places the procedure's actual parameters (if any) on the stack then activates the procedure with a CALL instruction.

Parameters are placed on the iAPX 86 stack or the 8087 register stack in left-to-right order. Since the stack grows from higher locations to lower locations, the first parameter occupies the highest position on the stack, and the last parameter occupies the lowest position. Note that a BYTE parameter occupies two bytes on the stack; the lower byte holds the value, the higher byte is undefined. A POINTER parameter in the SMALL(ROM), COMPACT, MEDIUM, and LARGE cases consists of a selector and an offset. The 16-bit selector is pushed first, followed by the 16-bit offset.

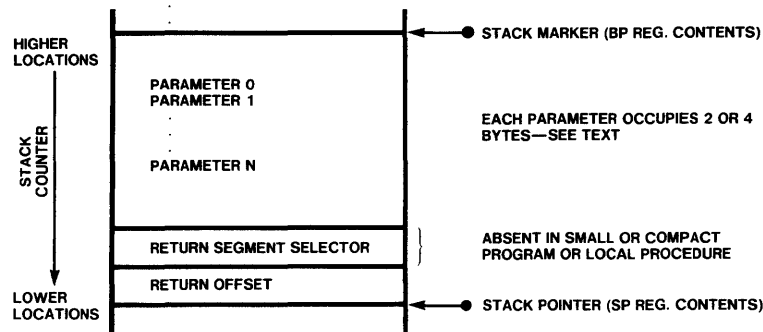
The leftmost seven REAL parameters are passed on the top of the 8087 stack. If more than seven REAL parameters are present, the rest (after the leftmost seven) are passed on the iAPX 86 stack.

After the parameters are passed, the CALL instruction places the return address on the stack. In the SMALL and COMPACT cases with local procedures, this is a 16-bit offset (the contents of the IP register) and occupies two contiguous bytes on the stack.

In the MEDIUM and LARGE cases, and for procedures exported from COMPACT, the type of the return address depends on whether the procedure is local or public. The return address for a local procedure, like any return address for the SMALL case, is a 16-bit offset and occupies two contiguous bytes on the stack. For a public procedure in the MEDIUM or LARGE case, and for procedures exported from a subsystem, the return address is a POINTER value consisting of a selector and an offset; it is passed in the same way a POINTER parameter is passed. The 16-bit segment selector (contents of the CS register) is pushed first, then the 16-bit offset (IP register contents) is pushed.

Control is passed to the code of the procedure by updating the IP register. In MEDIUM and LARGE cases, and for procedures exported from a subsystem, the CS register is also updated.

Figure F-1 shows the stack layout at the point where the procedure gains control.



121945-5

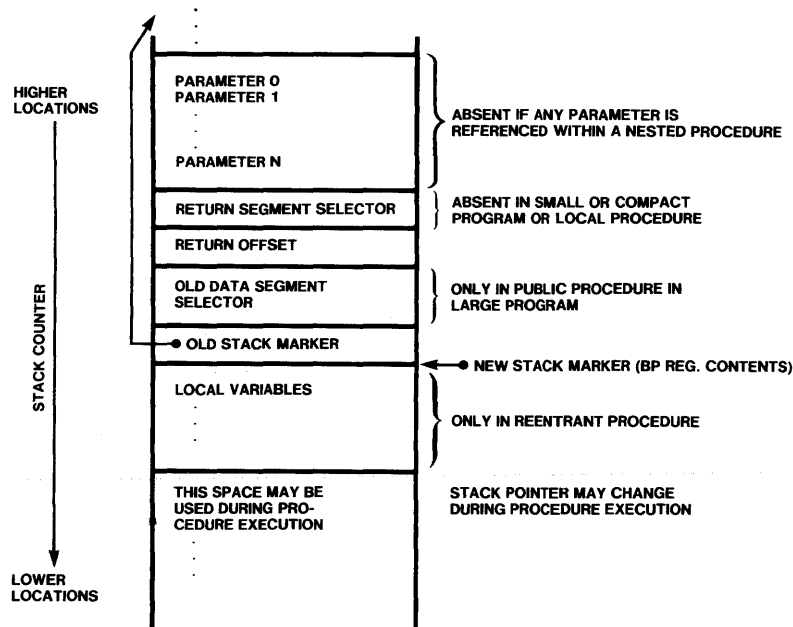
Figure F-1. Stack Layout at Point Where a Non-Interrupt Procedure is Activated

F.3 Procedure Prologue

In compiling the procedure itself, the compiler inserts at the beginning a sequence of code called the *prologue*. This code accomplishes the following:

1. If the procedure has the PUBLIC attribute and the program size is LARGE, or if it is exported from a subsystem, the contents of the DS register are placed on the stack. The DS register is then updated with a value that is found in the current code segment (i.e., the segment containing the procedure). (The DS register contains the segment selector for the current data segment; thus, this step implements the pairing of code and data segments in the LARGE case. It is not needed in the SMALL, COMPACT, and MEDIUM cases because the data segment does not change.)
2. If any parameter of the procedure is referenced by a nested procedure, all parameters are removed from the stack and placed in space reserved for them in the data segment.
3. The stack marker offset (BP register contents) is placed on the stack, and the current stack pointer (SP register contents) is used to update the BP register.
4. If the procedure has the REENTRANT attribute, space is reserved on the stack for any variables declared within the procedure (this does not include based variables, variables with the DATA attribute, or variables with the AT attribute).

Control then passes to the code compiled from the executable statements in the procedure body. Figure F-2 shows the stack layout at this point.



121945-6

Figure F-2. Stack Layout During Execution of a Non-Interrupt Procedure Body

F.4 Procedure Epilogue

To return from the procedure, the compiler inserts a code sequence called the *epilogue*. This accomplishes the following:

1. If the compiler has used stack locations for temporary storage or local variables during procedure execution, the stack pointer (SP register) is loaded with the stack marker (BP register contents). This has the effect of discarding the temporary storage.
2. The old stack marker is restored by popping the stored value from the stack into the BP register.
3. If the procedure has the PUBLIC attribute and the program size is LARGE or it is exported from a subsystem, the old data segment selector is restored by popping the stored value from the stack into the DS register.
4. If the program size is SMALL, the stored return address (a 16-bit offset) is popped into the IP register. Any parameters stored on the stack are discarded.

If the program size is MEDIUM or LARGE and the procedure is local, a return is performed using the same actions described above. If the program size is MEDIUM or LARGE and the procedure is public, the stored return-address offset from the stack is popped into the IP register and the return-address selector is popped into the CS register. Any parameters not stored on the stack are discarded.

F.5 Register Usage

Table F-1 provides a summary of iAPX 86 register usage.

In addition, the 8087 stack contains the first seven REAL arguments passed by the calling program. The 8087 status word is unknown and need not be saved; if it is changed, the 8087 mode word must be saved on entry and restored before exit.

If an assembly language subroutine alters the DS or SS registers, and expects to be called by a PL/M-86 program, it must save the contents of these registers upon entry and restore them before returning to the PL/M-86 program.

PL/M-86 uses the BP register to address the stack. If it uses the BP register, a called assembly language subroutine must save the contents of this register upon entry and restore its contents before returning control to the PL/M-86 program. Before returning, the called subroutine must also adjust the SP register to remove all parameters from the iAPX 86 stack.

The AX, BX, CX, DX, SI, DI, and ES registers do not need to be preserved. A called assembly language subroutine can freely use these registers.

An assembly language program calling a PL/M-86 procedure cannot expect the contents of the general-purpose registers, except BP and SP, to be preserved. If they are needed, they must be saved prior to calling the PL/M-86 procedure.

Table F-2 summarizes the iAPX 86 registers used to hold simple data types that are returned by typed procedures.

F.6 Segment Name Conventions

Table F-3 summarizes the segmentation of a subsystem under the program size controls by giving the name of the segment where each type of program section is stored for each model.

Table F-1. Summary of iAPX 86 Register Usage

Register	Must Preserve	Usage
AX	No	Return BYTE (AL), WORD, DWORD INTEGER, and SELECTOR values
BX	No	Return POINTER offset values
CX	No	—
DX	No	Return DWORD values
SP	Yes*	Stack pointer
BP	Yes	Stack marker
SI	No	—
DI	No	—
FLAGS	No	—
CS	Yes	Called procedure's code segment
DS	Yes	Caller's data segment
SS	Yes	Caller's stack segment
ES	No	Return POINTER segment selector

* SP must be adjusted so that all arguments are removed from the stack upon return.

Table F-2. Registers Used to Hold Simple Data Types

Procedure Type	Register
BYTE	AL
WORD	AX
DWORD	DX:AX
INTEGER	AX
POINTER (SMALL)*	BX
POINTER (COMPACT)	ES:BX
POINTER (MEDIUM)	ES:BX
POINTER (LARGE)	ES:BX
SELECTOR	AX
REAL	Top of RMU stack

* Under the ROM option, the result is returned in ES:BX.

Table F-3. Summary of PL/M-86 Segment Names

Model	SubModel	Code	Data	Const	Stack
SMALL	IN DATA	CODE	DATA	CONST	STACK
	IN CODE	CODE	DATA	CONST	STACK
COMPACT	IN DATA	CODE	DATA	CONST	STACK
	IN CODE	CODE	DATA	CONST	STACK
COMPACT (subsystem)	IN DATA	sCODE	sDATA	CONST	STACK
	IN CODE	sCODE	sDATA	CONST	STACK
MEDIUM	IN DATA	mCODE	DATA	CONST	STACK
	IN CODE	mCODE	DATA	CONST	STACK
LARGE	IN DATA	mCODE	mDATA	mDATA	STACK
	IN CODE	mCODE	mDATA	mCODE	STACK
LARGE (subsystem)	IN DATA	mCODE	mDATA	mDATA	STACK
	IN CODE	mCODE	mDATA	mCODE	STACK

NOTES:

sCODE denotes a segment name composed of the subsystem name and CODE.

sDATA denotes a segment name composed of the subsystem name and DATA.

mCODE denotes a segment name composed of the module name and CODE.

mDATA denotes a segment name composed of the module name and DATA.

F.7 Floating-Point Linkage

This section deals with the issues of choosing the linkage specifications appropriate to your use of the REAL math facility: no use, PL/M-86 use only, or use of routines not written in PL/M-86. What is appropriate also depends on whether execution will use an actual 8087 chip or an emulator.

These linkage specifications make available to your program the libraries of floating-point functions. The circumstances determining which library is appropriate are given in table F-4. The libraries themselves are discussed briefly below the table.

The interface libraries do the following:

- 8087.LIB resolves external references inserted by the translator of an iAPX 86 program so that floating-point instructions will correctly invoke the 8087 chip. 8087.LIB is the library of floating-point functions written for the chip itself rather than for the Emulator.
- E8087.LIB resolves such references to invoke the Emulator software instead of the actual 8087 chip.

Emulation is performed by E8087 or PE8087.

- E8087 is the actual library of emulation routines, which provide every function and feature of an actual 8087 chip except speed. Emulation is invoked automatically as needed, using interrupts 20 through 31. The full Emulator occupies about 16K bytes of code space.
- PE8087 is a subset of E8087. The REAL arithmetic performed by PL/M-86 programs does not require the complete set of routines in the full Emulator. The partial Emulator occupies about 8K bytes of code space.

NOTE

The 8087 Emulator processes exceptions exactly as the 8087 does. However, if your iAPX 86/8087 implementation includes some external interrupt masking device such as an 8259A, the effect of this external device cannot be simulated by the 8087 Emulator. With the Emulator, an Interrupt 16 will occur after the execution of any instruction when the (emulated) interrupt is active and the iAPX 86 interrupt is enabled, even if the 8259A is disabled.

(For examples of how to link interface libraries with your program, see specific host-system appendix.)

To locate the 8087 Emulator at a specified memory location:

- Locate the read-only code by referring to class AQMCODE in the LOC86 invocation.
- Locate the read-write data area by referring to class AQMDATA.

Table F-4. Linkage Choices for REAL-Math Usage

Use of REAL Math Facility	Emulator or Actual Chip Used	Link-List Should Include the Specifications Below (Not Necessarily in the Order Shown) After Object Modules
NONE	NEITHER	(none)
All Floating Point in PL/M-86 ONLY	EMULATOR	E8087.LIB, PE8087
With Some Modules that Use Floating Point NOT in PL/M-86	EMULATOR	E8087.LIB, E8087
ANY	ACTUAL 8087 CHIP	8087.LIB



APPENDIX G

RUN-TIME INTERRUPT PROCESSING

G.1 General Information

An interrupt is initiated when the CPU receives a signal on its “maskable interrupt” pin from some peripheral device or when control is transferred to an interrupt vector by the `CAUSE$INTERRUPT` statement. (If your program runs under an operating system that traps interrupts, the information in this appendix may not be applicable to you.)

Note that the CPU does not respond to this signal unless interrupts are enabled. The compiler does not generate any code to enable or disable interrupts at the start of your main program.

NOTE

If you require your program to begin with interrupts disabled, simply start with the instruction `DISABLE`. Since the iAPX 86 processor does not allow an interrupt to occur until the first machine instruction following the enable instruction has been processed, the resulting code sequence will not allow any maskable interrupt to occur.

If interrupts are enabled and vectored through an interrupt gate, the following actions take place:

1. The CPU completes any instruction currently in execution.
2. The CPU issues an “acknowledge interrupt” signal and waits for the interrupting device to send an interrupt number.
3. The CPU flag registers are placed on the stack (occupying two bytes of stack storage).
4. Interrupts are disabled by clearing the IF flag.
5. Single stepping is disabled by clearing the TF flag.
6. The CPU activates the interrupt procedure corresponding to the interrupt number sent by the interrupting device.
7. When that procedure terminates, the stack is automatically restored to the state it was in when the interrupt was received, and control returns to the point where it was interrupted.

The mechanism for this activation and restoration are described below.

G.2 The Interrupt Vector

If the `NOINTVECTOR` control is not used, an interrupt vector entry is automatically generated by the compiler for each interrupt procedure. Collectively, the interrupt vector entries form the *interrupt vector*. If `NOINTVECTOR` is used, the programmer must supply the interrupt vector as explained in section 6-4.

The interrupt vector is an absolutely located array of `POINTER` values beginning at location 0. Thus the n th entry is at location $4*n$ and contains the location of a procedure declared with the `INTERRUPT n` attribute.

Note that the first and second bytes of each entry contain an offset, while the second two bytes contain a segment address. The entries are always four-byte pointers, and the segment address is always used in transferring to the interrupt procedure, even if the program size is SMALL.

The CPU uses the interrupt vector entry to make a long indirect call to activate the appropriate procedure. At this point, the current code segment address (CS register contents) and instruction offset (IP register contents) are placed on stack.

At the point where the procedure is activated, the stack layout is as shown in figure G-1.

G.3 Interrupt Procedure Prologue and Epilogue

An interrupt procedure begins by declaring its name and nature. For example:

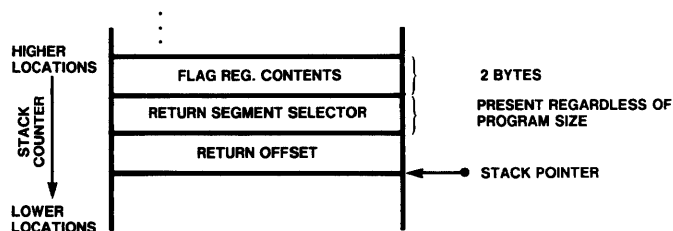
```
HANDLER: PROCEDURE INTERRUPT 4;
```

This alerts the compiler to create a code prologue appropriate to a routine that will, in general, be invoked by interrupts.

At the beginning of each interrupt procedure, the interrupt procedure prologue inserted by the compiler accomplishes the following:

1. Pushes the CPU registers to the stack in the following order—AX, CX, DX, BX, (SP and BP on the iAPX 186 only), SI, and DI.
2. Pushes the ES register contents onto the stack.
3. Pushes the DS register contents onto the stack.
4. Loads the DS register with a new data segment address taken from the current code segment (i.e., the segment containing the interrupt procedure).
5. Pushes the BP register contents onto the stack.
6. Loads the stack marker (BP register) with the address taken from the current stack pointer (SP register).
7. If the procedure is REENTRANT, reserves space on the stack for any local variables declared in the procedure.

However, if the procedure is REENTRANT and contains local variables, steps 5, 6, and 7 will be replaced by the ENTER instruction (see the *iAPX 86 Programmer's Reference Manual*).



121945-5

Figure G-1. Stack Layout at Point Where an Interrupt Procedure Gains Control

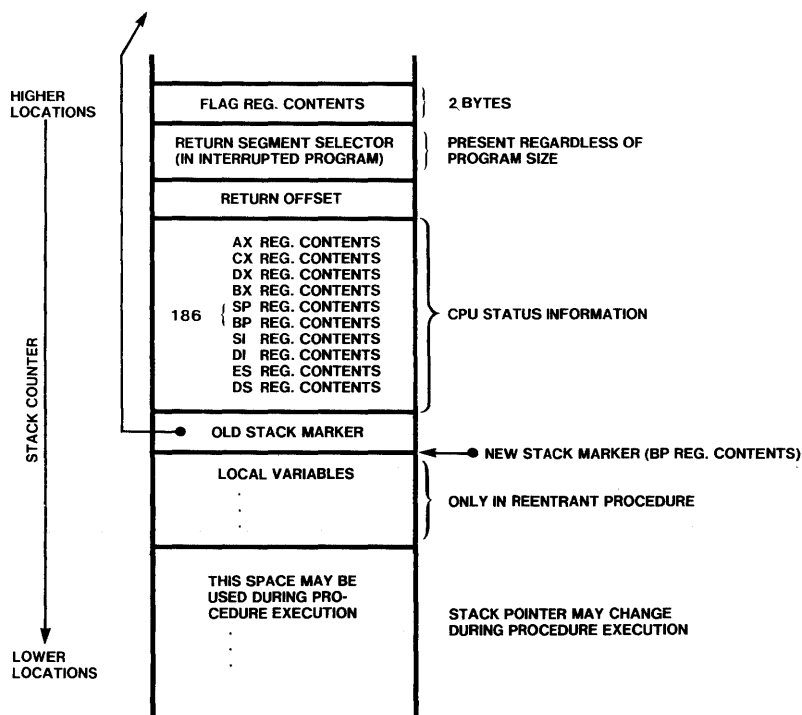
NOTE

The compiler may make temporary use of the DS register in some cases (e.g., string built-ins), but always restores it. Care must be taken when writing your own interrupt procedure in assembly language to note this possibility.

After the interrupt procedure prologue is executed (at the point where the code compiled from the procedure body gains control), the stack layout is as shown in figure G-2.

The return from the procedure body is called the interrupt procedure epilogue; it restores the stack to the state it was in before the interrupt occurred. The interrupt procedure epilogue contains the following steps:

8. If stack locations were used for temporary storage or local variables, loads the stack pointer (SP register) with the current stack marker (BP register). This discards the temporary storage.
9. Pops the stack into the BP register, restoring the old stack marker.
10. Pops the stack into the DS register.
11. Pops the stack into the ES register.
12. Pops the stack into the CPU registers in the following order—DI, SI (BP on the iAPX 186 only), BX, DX, CX, AX. Note that the SP register value is discarded by the POPA instruction on the iAPX 186.
13. Executes an IRET instruction to return from the interrupt procedure. This restores the IP, CS, and the flag register contents from the stack.



121636-4

Figure G-2. Stack Layout During Execution of an Interrupt Procedure Body

If the procedure is REENTRANT and has local variables, steps 8 and 9 will be replaced by the LEAVE instruction (see the *iAPX 186 Programmer's Reference Manual*).

At this point, the stack has been restored to the state it was in before the interrupt occurred, and processing continues normally.

G.4 Writing Interrupt Vectors Separately

In some cases it may be desirable to write the interrupt vector separately (in PL/M-86 or assembly language). This can be done by using NOINTVECTOR to prevent generation of an interrupt vector by the compiler. The separately created interrupt vector can then be linked into the program.

Creation of a separate explicit interrupt vector requires some care. The @ operator in PL/M-86 provides access to a procedure's normal (i.e., called) entry point, not to its interrupt entry point. The interrupt entry point first saves the status of the interrupted program before invoking the interrupt procedure through its normal entry point. The exact length of these operations depends on the compilation options chosen, the attributes of the interrupt procedure, and the version of the compiler being used. The built-in function INTERRUPT\$PTR can be used during execution to return the actual interrupt entry point. Discussion of this function appears in Chapter 9.

The usefulness of a separate created interrupt vector can be seen by considering an example.

Suppose that two modules for a multimodule program are developed separately. Both use interrupt procedures, but at the time when the modules are written the assignment of interrupt numbers to the various interrupt procedures has not been determined.

The two modules are therefore compiled with the NOINTVECTOR control. When this is done, the *n* in an INTERRUPT *n* attribute is ignored—since normally it would only be used to put the procedure's entry in the proper location within the interrupt vector.

Later, when the program is linked together, a separately created interrupt vector can be linked in. Within this interrupt vector, the placement of the entry for a given interrupt procedure determines which interrupt vector, the placement of the entry for a given interrupt procedure determines which interrupt number will activate that procedure.

G.5 Exception Conditions in REAL Arithmetic

As indicated in section 10.4, six exception conditions apply to normal numeric operations:

- Invalid operation
- Denormalized operand

- Zero divide
- Overflow
- Underflow
- Precision

These exceptions are discussed in order below. In each case, only a few of the possible causes are described because most are not likely to occur with PL/M-86 usage. If you want to perform sophisticated numeric processing of extreme precision and flexibility, see the *iAPX 86,88 User's Manual*.

The six exceptions and their default responses are summarized in table G-1.

As the following sections indicate, the masked, default response to most exceptions will provide the least abrupt, most appropriate action for PL/M-86 applications. Many real math exceptions that occur in other processors will not occur with the 8087 because of the extended range of intermediate results it holds. The "soft" recovery of gradual underflow (described under the denormal exception), also extends the range of permissible execution rather than reporting a hard-failure condition.

Programmers who use the recommended setting of the REAL Mode Word (see section 10.4) need to handle only the invalid exception. Study of the other exception conditions is advised, however, to provide a general understanding of their use.

Table G-1. Exception and Response Summary

Exception	Masked Response	Unmasked Response
Invalid Operation	If one operand is NAN, return it; if both are NANs, return NAN with larger absolute value; if neither is NAN, return <i>indefinite</i> NAN.	Request interrupt. (8087 stack unchanged.)
Zerodivide	Return ∞ signed with "exclusive or" of operand signs.	Request interrupt. (8087 stack unchanged.)
Denormalized	Memory operand: proceed as usual. Register operand: convert to valid unnormal, then reevaluate for exceptions.	Request interrupt. (8087 stack unchanged.)
Overflow	Return properly signed ∞ .	Register destination: adjust exponent,* store result, request interrupt. Memory destination: request interrupt.
Underflow	Denormalize result.	Register destination: adjust exponent,* store result, request interrupt. Memory destination: request interrupt.
Precision	Return rounded result.	Return rounded result, request interrupt.

* On overflow, 24,576 decimal is *subtracted* from the true result's exponent; this forces the exponent back into range and permits a user exception handler to ascertain the true result from the adjusted result that is returned. On underflow, the same constant is *added* to the true result's exponent.

Invalid Operation Exception

This exception generally indicates a program error. It could be caused by referencing an uninitialized REAL variable or by referencing a location that does not contain a REAL value (as might occur with an out-of-range subscript for a REAL array). Attempting to take the square root of a negative number or to store a number too large for integer format would also generate this exception.

Another interpretation of this exception is stack error. This may be caused by failing to restore the math unit status before returning from an interrupt routine that had saved the status. Another cause is the generation of more than 8 intermediate results during REAL arithmetic, which can arise if REAL procedure function calls are nested too deeply. The compiler ensures that no single procedure does this, but cannot check what may occur only at run time. This exception can also occur when REAL functions (typed procedures) are used as operands within longer REAL expressions. For example:

```
DELTA$1 = ALPHA * (BETA/GAMMA) + CHI (PSI, RHO, PI)
```

where all these names are typed REAL and CHI is some previously declared REAL function having three parameters.

The following is less likely to cause an exception condition:

```
EPS = CHI (PSI, RHO, PI)
DELTA$1 = ALPHA * (BETA/GAMMA) + EPS
```

because all REAL arithmetic is performed using the 8087 stack, which has eight registers. The first seven REAL parameters supplied in procedure calls are placed on this stack. If the procedure is typed (i.e., invoked as a function), it can be imbedded as one operand within a longer REAL expression.

Since the evaluation of such an expression also involves the use of this stack for prior and subsequent arithmetic operations, stack overflow may occur. This overflow amounts to unpredictable destruction of original parameters or intermediate results. It becomes more likely as you increase the complexity of REAL expressions containing REAL functions. Thus, you are safer using an assignment statement first to store the function's value in a real variable—and using that variable in the larger expression.

If stack error might apply to your program, modify the code for the affected procedures to call the built-in procedures `SAVE$REAL$STATUS` and `RESTORE$REAL$STATUS` as their first and last operations, respectively.

The masked (default) response is to set the result to one of the special bit patterns called Not-A-Number (NaNs), usually the indefinite value, the smallest NaN representable in the specified precision. It also sets Bit 0 of the REAL Error Byte.

If Bit 0 of the REAL Mode Word is 0 (invalid exception unmasked), an interrupt occurs, transferring control to the user-written interrupt handler.

Denormal Operand Exception

This condition arises when numeric operations have resulted in a number whose exponent is literally zero and whose significand is non-zero, or a number whose significand does not begin with a one. Denormals usually arise in response to masked underflow. Gradual underflow is the masked, default response to underflow. A small

denormal added to a large normal REAL number can give an acceptable, in-range answer if the denormal exception is masked. In practice, denormals are very rare since intermediate results are kept in temporary real format (15-bit exponent).

This condition causes Bit 1 of the REAL Error Byte to be set to 1. If Bit 1 of the REAL Mode Word is 1, the response described above occurs; if Bit 1 is 0, an interrupt occurs, transferring control to the user-written interrupt handler.

Zero Divide Exception

This condition arises when in the course of some REAL computation a divisor turns out to be zero. The masked response, when Bit 2 of the REAL Mode Word is 1, is to return infinity appropriately signed. If Bit 1 is 0, an interrupt occurs, giving control to the user-written interrupt handler. In either case, Bit 2 of the REAL Error Byte is set to 1.

Overflow Exception

This error occurs when a real result is too large for the format in use. For REAL assignments, it occurs if the result is greater than about 3.37×10^{38} ; for intermediate REAL computations, if the result is greater than about $10^{49.32}$. The overflow exception can arise during assignment, addition, subtraction, multiplication, division, or conversion to integer.

The masked, default response (Bit 3 of REAL Mode Word = 1) is to return infinity (signed if Affine Mode is set) and set Bit 3 of the REAL Error Byte to 1. Unmasked overflow must go through a user-written interrupt handler.

Underflow Exception

Underflow exception is caused by an exponent too small for the format in use. For REAL assignments, it occurs if the exponent is less than -127 , and for intermediate results if the exponent is less than -16383 . Underflow can be caused by the same type of REAL operations as overflow.

The masked, default response (Bit 4 of REAL Mode Word = 1) is to use the denormal number created by adjusting the very small result. It adjusts the significand, moving significant digits off to the right and raising the exponent until the latter becomes non-zero. For example, a 24-bit significand of .01 with an exponent of zero implies the number 1×2^{-129} because a zero exponent in this format means -127 . If the denormal exception is masked, this would be adjusted into a significand of .001 with an exponent of 1, i.e., 0.001×2^{-126} , prior to operation. This number would then be available for use in subsequent REAL operations that might yield valid results. Zero is returned if it is the rounded result. Bit 4 of the REAL Error Byte is set to 1. Unmasked underflow must go through a user-written interrupt handler.

Precision Exception

This error occurs when the result of an operation is inexact (i.e., rounded) or when an overflow exception occurs. No special action is performed by a masked response (Bit 5 of REAL Mode Word = 1) other than setting Bit 5 of the REAL Error Byte. Unmasked response is as chosen by the user.

G.6 Writing a Procedure to Handle REAL Interrupts

This section partially summarizes the information pertaining to interrupts, floating-point usage, and procedures. (Additional facilities for handling REAL interrupts may be provided by your operating system.)

An interrupt-handling procedure may, for example, begin as follows:

```
HANDLER: PROCEDURE INTERRUPT 16;
```

If HANDLER will do any REAL arithmetic or assignments, its first executable statements should be of the form:

```
ERR$INFO = GET$REAL$ERROR; /* must declare ERR$INFO$ BYTE
                             earlier */
```

or:

```
CALL SAVE$REAL$STATUS (@Local_Save_Area); /* also declare
                                             earlier */
```

Each procedure clears the error byte. The latter procedure also clears out the REAL stack. Thus, after either procedure is used, the REAL Error Byte no longer contains the flagged cause of the exception condition that invoked HANDLER.

Using SAVE\$REAL\$STATUS is a way of avoiding possible stack errors from cumulative usage. This permits errors in HANDLER to be detected independently of the originating exception condition, and allows HANDLER to restore the state of the interrupted procedure despite HANDLER's own use of the REAL facility. SAVE\$REAL\$STATUS also makes available all the information as to the state of the 8087 exceptions, stack, and operations, as shown below.

Thus, the beginning of a typical routine to handle REAL exception conditions could look like this:

```
HANDLER: PROCEDURE INTERRUPT 16;

        DECLARE ERR$INFO BYTE;
        DECLARE LOCAL_SAVE_AREA (94) BYTE;

        ERR$INFO = GET$REAL$ERROR;
```

or, if you want to perform extensive manipulations on the save area, you could declare a structure permitting you access to its component parts by name and/or byte, as follows:

```
HANDLER: PROCEDURE INTERRUPT 16;

    DECLARE ERR$INFO BYTE;
    DECLARE SAVE_AREA STRUCTURE
        ( CONTROL(2) BYTE,
          STATUS(2) BYTE,
          TAG      WORD,
          INSTR_OFF WORD,
          INSTR_SEL SELECTOR,
          OPERAND_OFF WORD,
          OPERAND_SEL SELECTOR,
          STACK_TOP(5) WORD,
          STACK_ONE(5) WORD,
          STACK_TWO(5) WORD,
          STACK_3 (5) WORD,
          STACK_4 (5) WORD,
          STACK_5 (5) WORD,
          STACK_6 (5) WORD,
          STACK_7 (5) WORD);

    CALL SAVE$REAL$STATUS (@SAVE_AREA);

    ERR$INFO = SAVE_AREA.STATUS(0);
```

NOTE

To make use of the words from TAG through OPERAND_SEL, you must employ masks and shifts to access the individual fields shown in figure G-3.

You must call either the SAVE\$REAL\$STATUS procedure or the GET\$REAL\$ERROR function, but not both. If you do not need the extra information gained by the SAVE (i.e., if you need only the exceptions), use the GET\$REAL\$ERROR function. If you call both, the second call will produce incorrect results.

The rest of HANDLER can perform any actions deemed appropriate. This is an application-dependent decision. Among the possibilities:

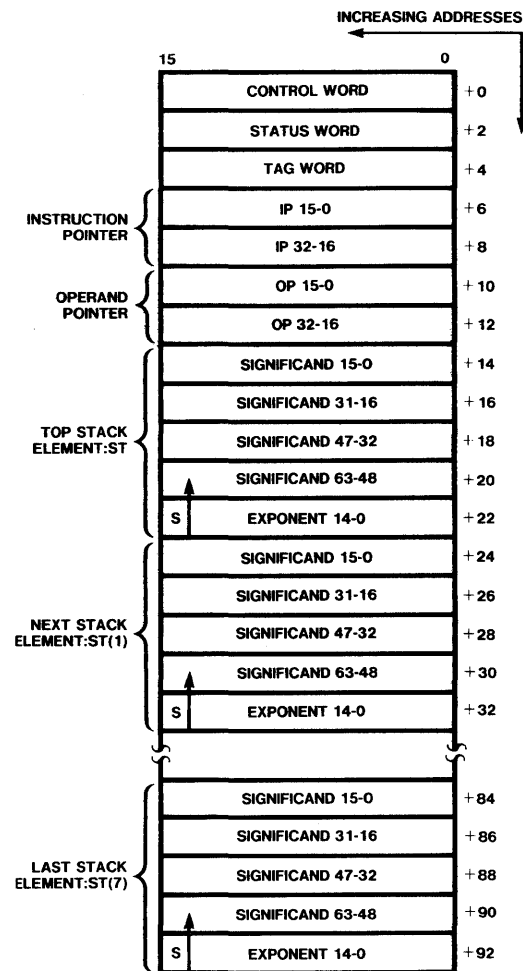
- Incrementing an exception counter for later display
- Printing diagnostic data, e.g., the contents of SAVE_AREA
- Aborting further execution of the calculation causing exception
- Aborting all further execution

The format of the LOCAL_SAVE_AREA as it is filled by the save procedure is shown in figure G-3.

The final action prior to returning (if desired) to the interrupted procedure is to restore the status of the REAL math unit:

```
CALL RESTORE$REAL$STATUS (@LOCAL_SAVE_AREA);
```

However, if you did not use GET\$REAL\$ERROR prior to the SAVE\$REAL\$STATUS call, the local save area will contain the original contents of the error byte. Under these circumstances, you must first clear the lower byte of the saved status word before the above RESTORE so as to avoid retriggering the save exception that invoked HANDLER in the beginning.



NOTES:

S = Sign

BIT 0 OF EACH FIELD IS RIGHTMOST, LEAST SIGNIFICANT BIT OF CORRESPONDING REGISTER FIELD.

BIT 63 OF SIGNIFICAND IS INTEGER BIT (ASSUMED BINARY POINT IS IMMEDIATELY TO THE RIGHT).

Figure G-3. Memory Layout of REAL SAVE AREA

121945-9

To do so, you can use a command of the form:

```
LOCAL_SAVE_AREA (2) = 0; /* should precede
                           restore */
```

or:

```
SAVE_AREA.STATUS (0) = 0;
```



APPENDIX H

COMPILER INVOCATION AND ADDITIONAL INFORMATION FOR SERIES III USERS

This appendix contains information that is specific to the Intellec Series III Microcomputer Development System. It covers the following areas:

- Compiler invocation and file usage
- Examples of system-dependent compiler controls
- Related publications

H.1 Compiler Invocation

The PL/M-86 *compiler* is a program that translates your PL/M instructions into object code modules that can be linked with the iAPX 86 Linker and executed.

This compiler runs under ISIS V4.1 with RUN V1.2, ISIS V4.2 with RUN V1.3, or later versions of both products.

You create a PL/M program by using the CREDIT text editor to type instructions into a file, and then submitting the file to the PL/M-86 compiler. The file you submit is called a source file, and the file containing the compiled program is called an object file. (The content of the object file is also known as object code.) In PL/M-86, you can compile parts of a program. Each separate compilation is known as an object module.

The following discussion assumes that you have a Series III system up and running, and that you have a suitable copy of the PL/M-86 compiler. Details on the operating system environment are provided in the *Intellec Series III Microcomputer Development System Console Operating Instructions*.

The invocation command has the general form:

```
RUN [ :Fd: ] PLM86 [ :Fd: ] source [ controls ] < cr >
```

where

RUN	is the name of the command used to execute the compiler.
:Fd:	specifies on which drive, if not drive 0, the PL/M-86 compiler and/or source program resides. (The source file does not have to be on the same drive as the compiler.)
PLM86	is the name you use for the compiler PLM86.86.
source	is the name of the source file containing the PL/M-86 program.
controls	are optional compiler controls described in Chapter 11. You can have many controls on the invocation line if you use a space between each control, and you can extend the invocation line by using the ampersand (&) as a continuation character.
< cr >	stands for use of the RETURN key on the keyboard.

Examples:

```
-RUN PLM86 :F1:MYPRDG.SRC PRINT(:LP: ) TITLE('TEST  
PROGRAM 4') < cr >
```

In this example, the compiler is on drive 0, but MYPROG.SRC, the source program, is on drive 1. The compiler will direct all printed output to :LP:, and place 'TEST PROGRAM 4' in the header on each page of the listing.

```
- RUN :F1:PLM86 :F1:PROG1.SRC NOPRINT <ctrl>
```

In this example, both the compiler and the source program (PROG1.SRC) are on drive 1. NOPRINT will prohibit all printed output (except error messages) usually generated by the compiler.

H.2 File Usage

Input Files

The compiler reads the PL/M-86 source from the source file specified on the command line (see previous section) and also from any files specified with INCLUDE controls (as described in Chapter 11). These files must be standard ISIS-II disk files. The source input should contain a PL/M-86 source module.

Output Files

Two output files are produced during each compilation unless specific controls are used to suppress them. The output files are the listing and object code files. Each of these may be explicitly directed to some standard Series II pathname (device or file) by using the PRINT and OBJECT controls, respectively. If the user does not control these outputs explicitly, the compiler writes them to disk files on the disk containing the input file. These files then have the same file name as the input file but have the extensions LST for the listing and OBJ for the object code. For example, if the compiler is invoked by:

```
- RUN PLM86 :F1:MYPROG.SRC
```

the listing and all other printed output is written to :F1:MYPROG.LST and the object code to :F1:MYPROG.OBJ. If these files already exist, they are overwritten. If they do not exist, the compiler creates them.

The object code file may be used as input to the ISIS-II linkage facilities. (See the *iAPX 86 Utilities User's Guide*).

Compiler Work Files

The work files used by the compiler during its operation are deleted at the end of compilation. All of these files are on the device :WORK:.

All of the work files have names with the extension TMP. Therefore, you should avoid naming files with the extension TMP on any device used by the compiler for work files—the operation of the compiler may destroy them.

H.3 Linking to Floating-Point Libraries with the Series III

Suppose you write a PL/M-86 program called Easy that, at first, uses no REAL math. As modules are added, you supply a PL/M-86 REAL math routine called ACURAT, and revise EASY to call it. At this point you would need to add an interface library for floating-point math.

If you do not have a 8087 chip installed in your system, the correct linking statement is:

```
LINK86 ACURAT.OBJ, EASY.OBJ, E8087.LIB, PE8087
```

However, if ACURAT were written in another language (e.g., FORTRAN86 or ASM86), use instead:

```
LINK86 ACURAT.OBJ, EASY.OBJ, E8087.LIB, E8087
```

If your system includes a 8087 chip, use:

```
LINK86 ACURAT.OBJ, EASY.OBJ, 8087.LIB
```

More detailed and advanced discussions of the features and functions of the iAPX 86 utilities appear in the *iAPX 86, 88 Family Utilities User's Guide*.

H.4 Series III-Specific Compiler Controls

This appendix includes a fold-out page for the system-specific examples of many of the compiler controls. This page is designed to be opened and used in conjunction with the corresponding text in Chapter 11.

H.5 Related Publications

Below is a list of other Intel publications you might need along with this manual. The manual order number for each publication is given immediately following the title. The paragraph below each title describes the book.

Intellec Series III Product Overview, 121575

A summary description of the set of manuals that describe the Intellec Series III development system and its supporting hardware and software. This short manual includes a description of each manual related to the Series III, plus a glossary of terms used in the manuals.

Intellec Series III Console Operating Instructions, 121609

Intellec Series III Pocket Reference, 121610

Instructions for using the console features of the Series III, including the resident monitor. The *Console Operating Instructions* provides complete instructions, and the *Pocket Reference* gives a summary of this information.

Intellec Series III Programmer Reference Manual, 121618

Instructions for calling system routines from user programs for both microprocessor environments (8080/8085 and 8086) in the Series III.

ISIS-II CREDIT 9CRT-Based Text Editor) User's Guide, 9800902

ISIS-II CREDIT (CRT-Based Text Editor) Pocket Reference, 9800903

Instructions for using CREDIT, the CRT-based text editor supplied with the Series III. The *User's Guide* provides complete operating instructions, and the *Pocket Reference* summarizes this information for quick reference.

iAPX 86,88 Family Utilities User's Guide, 121616

Instructions for using the utility programs LINK86, LIB86, LOC 86, CREF86, and OH86 in iAPX 86-based environments to prepare compiled or or assembled programs for execution.

8086/8087/8088 Macro Assembly Language Reference Manual for 8086-Based Development Systems, 121627

8086/8087/8088 Macro Assembly Operating Instructions for 8086-Based Development Systems, 121628

8086/8087/8088 Macro Assembly Language Pocket Reference, 121674

Instructions for using the 8086/8087/8088 macro assembly language and its assembler in iAPX 86-based environments. The *Language Reference Manual* gives a complete description of the assembly language the *Operating Instructions* gives complete instructions for operating the assembler and the *Pocket Reference* provides summary information for quick reference. You need these publications if you are coding some of your routines in assembly language.

ICE-86 In-Circuit Emulator Operating Instructions for ISIS-II Users, 9800714

ICE-86 Pocket Reference, 9800838

ICE-88 In-Circuit Emulator Operating Instructions for ISIS-II Users, 9800949

ICE-88 Pocket Reference, 9800950

Instructions for using the ICE-86 and ICE-88 In-Circuit Emulators for hardware and software development. The *Operating Instructions* manuals give complete user descriptions of the In-Circuit Emulators, and the *Pocket Reference* guides provide summary information for quick reference. You need the corresponding publications if you are using the ICE-86 or ICE-88 emulator.

COMMENTS
Causes a listing of standard assembly language code to be placed at the end of file PROG1.LST on drive 5.
Causes file TEST6.OBJ (on drive 1) to include information that will later be used to debug the program using a symbolic debugger.
Causes source lines from file SYSLIB.SRC (on drive 1) to be placed in file SYSPRG.SRC (on drive 0).
Sends file PROG1.OBJ to the disk in drive 1 instead of drive 0 (the default location).
Directs the compiler to perform more than the default code optimizations when generating the object code for file SAMPLE.OBJ on drive 4.
Prints file TSTLIB.LST with a pagewidth of 70.
Sends printed output to the line printer.
Causes the subtitle 'Module One' to appear on every page until another subtitle control (if any) appears in INIT1.SRC on drive 1.
Causes a listing of all identifiers in file TSTOBJ.SRC, and their attributes, to appear in file TSTOBJ.LST.
Causes the title 'TEST PROGRAM 5' to appear on every page.
Prevents the type definitions of the variables output in symbols records from appearing in file MOD2.OBJ.
Produces a cross-reference listing of all identifiers in file MYPROG.SRC and appends the listing to default listing file MYPROG.LST.

CONTROL
CODE/NOCODE
DEBUG/NODEBUG
INCLUDE
OBJECT/NOOBJECT
OPTIMIZE
PAGEWIDTH
PRINT/NOPRINT
SUBTITLE
SYMBOLS/NOSYMBOLS
TITLE
TYPE/NOTYPE
XREF/NOXREF

EXAMPLES
-RUN PLM86 :F5:PROG1.SRC CODE <cr>
-RUN PLM86 :F1:TEST6.SRC DB <cr>
-RUN PLM86 SYSPRG.SRC & >>INCLUDE(:F1:SYSLIB.SRC) <cr>
-RUN PLM86 PROG1.SRC & >>OBJECT(:F1:PROG1.OBJ) <cr>
-RUN PLM86 :F4:SAMPLE.SRC OT(2) <cr>
-RUN PLM86 TSTLIB.SRC PW(70) <cr>
-RUN PLM86 EXMPLE.SRC PRINT(:LP:) <cr>
-RUN PLM86 :F1:INIT1.SRC & >>SUBTITLE('MODULE ONE') <cr>
-RUN PLM86 TSTOBJ.SRC SB <cr>
-RUN PLM86 PROG5.SRC & >>TITLE('TEST PROGRAM 5') <cr>
-RUN PLM86 MOD2.SRC NOTYPE <cr>
-RUN PLM86 MYPROG.SRC XREF <cr>



APPENDIX I

COMPILER INVOCATION AND ADDITIONAL INFORMATION FOR iRMX™ 86 USERS

This appendix contains information that is specific to the iRMX 86 Operating System. It covers the following areas:

- System requirements
- Compiler invocation and file usage
- Examples of system-dependent floating-point library linkage
- Examples of system-dependent compiler controls
- Related publications

I.1 System Requirements

To run the compiler on an iRMX 86-based system, you must have the following hardware and software:

- The iRMX 86 Human Interface (and other iRMX 86 layers necessary to support the Human Interface).
- At least one mass storage device. The installation of the compiler always requires a single- or double-density diskette drive, since the product is delivered in diskette form.
- Enough memory to run the compiler (above and beyond that required for the Operating System). This ranges from
minimum – 140K – Amount needed to run compiler assuming connection to only six files
maximum – 512K – Largest amount the compiler can use.

I.2 Compiler Invocation

The compiler is supplied on an iRMX 86-format diskette. It may be desirable to copy the compiler to another disk or to one of the directories that the Operating System automatically searches when commands are entered. The compiler consists of a single file: PLM86.

You can invoke the PL/M-86 Compiler from the system console using the standard command format described in the *iRMX 86 Human Interface Reference Manual*. You can specify continuation lines by using the ampersand (&) as a continuation character. The ampersand can appear any place there is a space or other delimiter.

The invocation command has the general form

```
-[directory]PLM86 sourcepath [controls] <cr>
```

where

directory

is the portion of the pathname that identifies the device and directories that contain the file PLM86. If you omit the directory portion of the pathname, the Operating System automatically searches several directories for the file PLM86. The directories searched and the order of search are iRMX 86 configuration parameters.

- sourcepath** is the pathname of the file containing the PLM86 source module. The beginning portion of this pathname can consist of a logical name enclosed in colons (such as :F1:). This indicates where the Operating System starts its search for the file. If you omit the logical name, the Operating System assumes that the file resides in the default directory (:\$.).
- You can use the slashes (/) and up-arrows or circumflexes (^) as separators between individual components of the pathname (except immediately after the logical name). The slash separator tells the Operating System to search down one level in the directory tree for the next component. The circumflex separator tells the Operating System to search up one level in the directory tree.
- controls** is an optional sequence of assembler controls (see Chapter 11).
- <cr>** indicates a carriage return.

In the interactive sequence shown in figure I-1, comments appearing to the right of semicolons are for clarification, not material entered by the user. This example shows how to compile a complete program that does not require more than 64K bytes of storage for the code or more than 64K bytes for the data.

In the normal usage, the PL/M-86 compiler writes the compilation listing by default to a secondary storage file on the same device as the source file. This file has the same pathname as the source file, but the last component of the pathname has the extension LST. Thus, in the previous example, you can find the listing in :PROG:MYPROG.LST. Similarly, the object code file is on the same device and has the same pathname, but has the extension OBJ. In the example, :PROG:MYPROG.OBJ contains the object code produced by compiling :PROG:MYPROG.SRC.

Examples

1. `PLM86 :F1:PROG1.SOURCE <cr>`

In this example, the compiler resides in a directory that the Operating System automatically searches; thus it does not require a device or directory name. The compiler is directed to compile the source module on :F1:PROG1.SOURCE. This file resides on the device with logical name :F1: and has the name PROG.SOURCE.

2. `WDO:PLM86 <wdo>DIR1/MYPROG.SRC PRINT(ALL) TITLE(1TEST PROGRAM 4)<cr>`

In this example, the compiler resides on device :FDO:. The compiler is directed to compile the source module on :WDO:DIR1/MYPROG.SRC, send the output listing to :LP:, and place 'TEST PROGRAM 4' in the header on each page of the listing.

```
iRMX 86 HI V2.1 : user = WORLD                ; the system identifies itself

-PLM86 :PROG:MYPROG.SRC                        ; the compiler is invoked

iRMX 86 PL/M-86 COMPILER V2.1
PL/M-86 COMPILATION COMPLETE.      0 WARNINGS,      0 ERRORS

                                ; the program may now be linked and relocated
```

Figure I-1. iRMX 86 Interactive Compilation Sequence

I.3 File Usage

Input Files

The compiler reads the PL/M-86 source from the sourcepath file specified on the command line (see previous section) and also from any files specified with INCLUDE controls (as described in Chapter 11). These files must be standard iRMX 86 files. The source input should contain a PL/M-86 source module.

Output Files

Two output files are produced during each compilation unless specific controls are used to suppress them. These are the listing and object code files. Each of these may be explicitly directed to some iRMX 86 file or device by using the PRINT and OBJECT controls respectively. If the user does not control these outputs explicitly, the compiler writes them to secondary storage files on the device containing the input file. These files have the same pathname as the input file, but have the extensions LST (for the listing) and OBJ (for the object code). For example, you invoke the compiler with

```
PLM86 :F1:MYPROG.SRC <cr>
```

The compiler writes the listing and all other printed output to :F1:MYPROG.LST and the object code to :F1:MYPROG.OBJ. If these files already exist, the compiler overwrites them.

The object code file may be used as input to the relocation and linkage facilities (see the *iAPX 86,88 Family Utilities User's Guide*).

Compiler Work Files

The compiler uses work files during its operation that are deleted at the completion of compilation. All of these files are in the directory with logical name :WORK: unless the WORKFILES control is used to specify another directory.

All of the work files have names with the extension TMP. Therefore, if you place those files in the work files directory, you should avoid giving your files names with the extension TMP, because there is a possibility that the compiler will destroy them.

I.4 Linking to Floating-Point Libraries with the iRMX 86 Operating System

Suppose you write a PL/M-86 program called EASY that, at first, uses no REAL math at all. No interface library is needed. As modules are added during the development process, you supply a PL/M-86 REAL math routine call ACURAT, and you revise EASY to call it.

In order to use the REAL math facility on the iRMX 86 Operating System, you must have an 8087 Numeric Processor Extension installed on your iRMX 86 system. In such a case, the correct linking statement is

```
LINK86 ACURAT.OBJ, EASY.OBJ, 8087.LIB, E-87.LIB
```

More detailed and advanced discussions of the features and functions of the iAPX 86 utilities appear in the *iAPX 86,88 Family Utilities User's Guide*.

I.5 iRMX 86 – Specific Compiler Controls

This appendix includes a fold-out page for the system-specific examples of several compiler controls. This page is designed to be opened out and used in conjunction with the corresponding text in Chapter 11.

I.6 Related Publications

This section lists other Intel publications you might need in addition to this one. The manual order number for each publication immediately follow the title. The paragraph following each title describes the manual.

Introduction to the iRMX 86 Operating System, 9803124

A general introduction to the iRMX 86 Operating System. This manual discusses the features of the Operating System and introduces some of the terminology. It also lists and describes each of the manuals in the iRMX 86 manual set.

iRMX 86 Human Interface Reference Manual, 9803202

iRMX 86 Disk Verification Utility Reference Manual, 144133

Instructions for entering commands at an iRMX 86 terminal. The Human Interface manual describes file-naming conventions and provides a complete description of all commands available with the Operating System. The Disk Verification Utility manual describes an interactive utility that examines and restores iRMX 86 volumes.

iRMX 86 Nucleus Reference Manual, 9803122

iRMX 86 Basic I/O System Reference Manual, 9803123

iRMX 86 Extended I/O System Reference Manual, 143308

iRMX 86 Loader Reference Manual, 143318

iRMX 86 System Programmer's Reference Manual, 142721

Run-Time Support Manual for iAPX 86,88 Applications, 121776

Instructions for invoking system calls from user programs.

iRMX 86 Programming Techniques, 142982

Guide to Writing Device Drivers for the iRMX 86 and iRMX 88 I/O Systems, 142926

Additional information about the Operating System that PL/M-86 users might require.

EDIT Reference Manual, 143587

Instructions for using EDIT, the text editor available for use on the iRMX 86 Operating System.

iAPX 86,88 Family Utilities User's Guide, 121616

Instructions for using the utility programs LINK86, LIB86, LOC86, CREF86, and OH86 in iAPX 86-based environments to prepare compiled or assembled programs for execution.

ASM86 Assembly Language Reference Manual, 121627

ASM86 Macro Assembler Operating Instructions for 8086-Based Development Systems, 121628

Instructions for using the 8086/8087/8088 macro assembly language and its assembler in iAPX 86-based environments. The Language Reference Manual gives a complete description of the assembly language. The Operating Instructions Manual gives complete instructions for operating the assembler. You need these publications if you are coding some of your routines in assembly language.

8087 Support Library Reference Manual, 121725

Instructions for calling procedures that implement the REAL math facility of the 8087 Numeric Processor Extension.

COMMENTS	CONTROL	OBJECT	EXAMPLES
Causes object code to be written to file :PROG:TESTS/A.SOURCE			<code>OBJECT(:PROG:TESTS/A.SOURCE)</code>
Causes printed output to be directed to the line printer		PRINT	<code>PRINT(:LP:)</code>
Causes input of source lines from file :WDO:INCLUDE/SYSLIB.SRC		INCLUDE	<code>INCLUDE(:WDO:INCLUDE/SYSLIB.SRC)</code>



- 8087.LIB, Appendix F
- ABS, 9-4, 9-7
- actual parameters, 8-2 thru 8-5
- addition, 3-10, 4-1, 4-3
- addresses
 - physical, 7-5
 - relative, 11-17
 - relocation, 1-5
 - unique, 3-2
- affine, see infinity
- ambiguity
 - in embedded assignments, 4-14
 - in location references, 5-5
 - of address/location, 7-4
- AND, 4-5 thru 4-8, 6-8, 11-32
- apostrophe, 2-2, 2-5
- arithmetic, 3-10 thru 3-12
 - floating-point, 3-11, 3-12, Chapter 10, Appendix G
 - modulo, 4-4, 6-6
 - operators, 4-3, 4-4, 4-6
 - signed, 3-11
 - summary of rules, 4-10 thru 4-12
 - unsigned, 3-10
- array, 3-2, 5-1 thru 5-5
 - contiguity, 5-1
 - declaration, 3-2, 5-1, 5-3
 - elements, 5-1, 9-1, 9-2
 - initialization, 3-4 thru 3-6
 - number of bytes, 9-1, 9-2
 - number of elements, 5-1, 9-1, 9-2
 - reference, 3-13
 - storage of, 5-1
 - structures, 5-3, 5-4, Chapter 12
 - subscripts, 5-1, 5-2
- ascending
 - order in string moves, 9-9 thru 9-13
- ASCII, 2-1, 2-5, 3-5, 4-2, 6-10, 11-24, Appendix E
- assembly language linkage, Appendix F
- assignment
 - embedded, 3-1, 4-9, 4-14, 4-15
 - floating-point, 3-11, 3-12, Appendix G
 - statement, 1-4, 3-4, 4-12 thru 4-15, 5-2, 6-7
- asterisk, 2-1, 2-5, 3-6, 4-3
- @ operator, 2-1, 3-5, 3-13 thru 3-15, 8-3, 9-14
- AT, 3-1, 3-4, 3-7, 3-13, 3-17 thru 3-19, 7-5
- attributes, 3-1, 3-4, 5-2
 - affecting section, 11-30
 - initialization, 3-1, 3-4, 3-5
 - linkage, 3-1, 3-4 thru 3-7, 8-7, 8-8
 - location, 3-1
- BASED, 3-15 thru 3-17, 8-2, 8-7
- based, 3-4, 3-13, 3-15 thru 3-17, 5-2, 7-5, 8-2, 8-3, 8-7, 11-8
- binary
 - number, see constant
 - point, 3-12
 - scientific notation, 3-12
- blank, 2-1, 2-3, 6-3
- block, 1-2, 1-3, 3-4
 - kinds, 1-2
 - levels, 1-4, 1-5
 - names, 1-4
 - nesting, 7-1 thru 7-5
 - structure, 1-3, Chapter 7
- BLOCKINPUT, 10-5
- BLOCKINWORD, 10-5
- BLOCKOUTPUT, 10-5
- BLOCKOUTWORD, 10-5
- BNF, Appendix C
- BOOLEAN expressions
 - see logical, short-circuit
- BUILD\$PTR, 9-16
- built-in
 - arrays, 10-5
 - functions, 9-1 thru 9-11, 9-14 thru 9-16, 10-4
 - REAL, 10-9 thru 10-11
 - procedures, 1-4, 9-9 thru 9-13, 10-5
 - REAL, 10-8 thru 10-10
 - variables, 1-4, 10-3, 10-4, 10-6, 10-8, 10-9
- By, see DO, iterative
- BYTE, 3-1, 3-10, 4-1 thru 4-14
- CALL, 4-12, 6-2, 6-12, 8-1, 8-4 thru 8-6, 8-9
- calling sequence, Appendix F
- carriage return, 2-1, 2-5
- CARRY, 10-2, 10-3
- CASE, see DO
- CAUSE\$INTERRUPT, 8-10, 10-1
- character
 - set, 2-1
 - strings, see string
- closed subsystems, 13-3
- CMPB, 9-10
- CMPW, 9-10
- CODE control, 11-24, 11-25, Appendix F
- code
 - section, 11-6, 11-8 thru 11-22
 - space efficiency, 11-9
- colon, 2-1, 2-2
- comma, 2-1, 3-3, 3-4
- comment, 2-1, 2-5
- communicating values, 1-5
 - see also parameters
- COMPACT control, 11-20, 11-21
 - restrictions, 11-20, 11-21
- compilation
 - constant, 3-1, 3-7, 3-8
 - steps, 11-1
 - summary, 11-31
- compiler controls, Chapter 11
- compiler files, Appendix H, Appendix I
 - input, Appendix H, Appendix I
 - output, Appendix H, Appendix I
 - work, Appendix H, Appendix I
- compiler invocation, Appendix H, Appendix I

- complement
 - ones, 4-5
 - twos, 3-11
- compound
 - delimiter, 2-2
 - operands, 4-2, 4-3, 4-7
- COND control, 11-36, 11-37
- conditional
 - execution 6-7 thru 6-11
 - compilation, 11-33 thru 11-37
- constant, 2-3 thru 2-5, 3-4 thru 3-8
 - character, 2-5
 - compilation, 3-1
 - execution, 3-1, 3-6
 - expression, 3-4, 4-13, 4-14, 6-3
 - list, 3-14
 - numeric, 4-1
 - binary, 2-3, 2-4
 - decimal, 2-3, 2-4
 - floating-point, 2-4
 - hexadecimal, 2-3, 2-4
 - octal, 2-3, 2-4
 - real, 2-4
 - string, 2-5
 - whole-number, 2-3, 2-4
 - type, 2-4
- constraints, Appendix B
- context, signed or unsigned, 4-1, 4-10 thru 4-14
- contiguity, 3-2, 5-1, 8-2
 - of storage, 3-3, 5-1, 5-2
- control line, 11-1
- controls, Chapter 11
 - list, 11-2
- conversion
 - explicit, 9-3 thru 9-7
 - implicit, 4-13, 8-5
- count, 9-6 thru 9-11
- counter, 6-1
- cross-reference listing, 11-28, 11-30
- CS register, 11-18 thru 11-22

- DATA, 3-1, 3-4 thru 3-7, 7-5
- data section, 11-6, 11-18 thru 11-22
- DEBUG control, 11-17
- DEC, 10-3
- decimal adjust, 10-3
- decimal point, 2-1, 2-4, 4-1
- DECLARE, 1-3, 3-1
 - combining, 3-3
 - elements, 3-8, 3-9
- declarations, 1-3, 3-1 thru 3-9
 - elements, 3-8, 3-9
 - factored, 3-2
 - local, 7-3
 - multiple, 1-3, 3-1, 7-3
 - procedure, 3-9
 - variable, 3-1
 - results, 3-3
- default
 - control word setting, Appendix G
 - recovery, Appendix G
- defining
 - declaration, 7-5, 8-7, 8-8
 - delay, 9-13
 - delimiters, 2-2, 2-5
 - denormal, Appendix G
 - descending
 - order in string moves, 9-9 thru 9-13
 - destination
 - location in string moves, 9-10 thru 9-13
 - dimension specifier, 5-1, 8-2, 8-7
 - implicit, 3-6
 - DISABLE, 10-1
 - division, 3-10, 4-1 thru 4-4
 - DO
 - as unit, 6-2, 6-4, 6-8
 - block, 1-2, 1-3, 6-4, 6-8
 - CASE, 6-1, 6-3, 6-4
 - iterative, 6-1, 6-5 thru 6-7
 - simple, 6-1 thru 6-3
 - WHILE, 6-1, 6-4, 6-5
 - exit, 6-2
 - labeled, 6-3
 - loop, 6-1
 - nested, 6-3, 7-1 thru 7-4
 - statement, 1-3
 - dollar sign, 2-2, 11-1
 - dot operator, 2-1, 3-5, 3-15, 3-17
 - DOUBLE, 9-4
 - DS register, 11-18 thru 11-22
 - DWORD, 3-10, 4-1, 4-3, 4-10 thru 4-14

 - E8087, Appendix F
 - E8087.LIB, Appendix F
 - EJECT control, 11-27
 - element
 - in declarations, 3-1, 3-3, 8-2
 - in initializations, 3-4
 - in string moves, 9-9
 - of expression, 1-4
 - elimination of
 - common subexpressions, 11-5
 - superfluous branches, 11-5
 - ELSE part, 6-9
 - see IF statement or control
 - embedded assignment, 4-9, 4-14, 4-15
 - possible ambiguity, 4-15
 - ENABLE, 10-1
 - END, 1-3, 3-9, 6-1, 8-1, 8-5
 - entry point, 3-13, 8-5, 8-9, Appendix F, Appendix G
 - equal sign, 2-1, 2-2, 11-28
 - errors, 2-1, 3-4, 5-4, 7-5, 11-30, Chapter 14
 - REAL, handling, 10-9, Appendix G
 - evaluation order, 4-6, 4-9
 - exception conditions for REALs, 10-8, 10-9, Appendix G
 - exception handling procedure, Appendix G
 - executable statement, 1-4, 2-1, 2-8, 6-2, 8-1
 - execution
 - constant names, 3-1
 - faster, 11-9
 - suspending, 8-10
 - exclusive extent, 7-1 thru 7-3
 - explicit
 - label declaration, 3-8
 - type conversion, 9-3 thru 9-7
 - exponent of REAL number, 3-11, 3-12
 - EXPORTS list, 13-4

- expression, 1-4, 3-2, 4-1 thru 4-15, 8-4
 - analysis, 4-6 thru 4-9
 - constant, 4-10 thru 4-14
 - evaluation order, 4-9
 - floating-point, 3-11, 3-12
 - REAL, 3-11, 3-12
 - restricted, 3-4
 - subscript, 5-2
 - summary of rules, 4-9 thru 4-14
- extended scope, 7-4, 7-5, 8-7, 8-8
- extended segmentation controls, Chapter 13
- extent, inclusive or exclusive, 7-1 thru 7-3
- EXTERNAL, 3-1, 3-6 thru 3-9, 6-11, 7-4 thru 7-7, 8-7, 8-8, 11-23
- factored declaration, 3-3
- false, 3-11, 4-5, 4-6, 6-4, 6-7
- far references to procedures, Appendix F
- fatal errors, 14-20
- fields of a REAL number, 3-11
- file usage, Appendix H, Appendix I
- FINDB, 9-11
- FINDRB, 9-11
- FINDRW, 9-11
- FINDW, 9-11
- FIX, 9-3, 9-5
- FLAGS, 10-3
- FLOAT, 9-3, 9-5
- floating-point linkage, Appendix F
- floating-point number, 3-11, 3-12
 - format in memory, 3-11
- flow control statements, 6-1 thru 6-12
- folding of constant expressions, 11-5
- formal parameters, 8-2 thru 8-7
- fraction of REAL number, 3-11
- function, 8-4
 - REAL, 10-15
 - references, 4-2, 4-9, 4-12, 8-1 thru 8-5
 - see also built-in
- general controls, 11-1
- GET\$REAL\$ERROR, 10-8, 10-9, Appendix G
- GOTO, 3-9, 6-2, 6-11, 7-6 thru 7-9, 8-5, 8-9
- gradual underflow, Appendix G
- grammar of PL/M-86, Appendix C
- greater than, see relational
- HALT, 10-1
- hardware
 - features, Chapter 10
 - flags, 10-1 thru 10-3
 - registers, 10-3, 10-4
- HIGH, 4-3, 9-4
- high-level languages, 1-1, 1-2
- HLT, 10-1
- IABS, 9-4, 9-7
- identifier, 1-4, 3-2, 5-1, 5-2
 - definition, 2-3
 - examples, 2-3
 - listing, 11-28
 - predeclared, Appendix A
- IF control, 11-33
- IF statement, 4-5, 4-11, 6-3, 6-7, 6-8
 - enclosing DO blocks, 6-8 thru 6-10
 - nested, 6-8 thru 6-10
- implicit
 - dimension specifier, 3-6
 - label declaration, 3-8
 - type conversion, 4-13, 8-5
- INCLUDE control, 11-2, 11-32, 13-6, Appendix H, Appendix I
- inclusive extent, 7-1 thru 7-3
- index
 - in string moves, 9-9
 - variable, 5-2, 6-6
 - see also DO, iterative
- infinity control, Appendix G
- INIT\$REAL\$MATH\$UNIT, 10-8
- INITIAL, 3-1, 3-4 thru 3-7, 3-13, 7-5, 11-23
- initializations, 3-4 thru 3-7
 - multiple, 3-4
 - of REAL math facility, 10-6
 - string constant, 2-5
- INPUT, 1-5, 10-4, 10-5
- inner level, 6-11
 - see outer
- input/output, 1-5, 10-4, 10-5
 - ports, 1-5
- insertion sort example, Chapter 12
- INT, 9-3, 9-5
- INTEGER, 3-11, 4-1 thru 4-4
 - least significant bits, 3-11
- interface list, 13-4
- intermediate results, 10-6, Appendix G
- internal REAL format, 3-11
- INTERRUPT attribute, 8-7 thru 8-9
- interrupt
 - attribute, 8-7
 - enabled or disabled, 8-8, 8-9, Appendix G
 - emulator usage, Appendix F
 - entry point, Appendix G
 - epilogue, Appendix G
 - gates, Appendix G
 - masking, Appendix F, Appendix G
 - mechanism, 8-9, Appendix G
 - prologue, Appendix G
 - procedures, 8-8 thru 8-10, Appendix G
 - REAL, 10-6, Appendix G
 - signal, 8-8, 8-9
 - software, 9-13
 - vectors, 8-8, 11-4, Appendix G
- interrupt request, 10-6
- INTVECTOR control, 11-4
- invalid operation, Appendix G
- invocation
 - line, Appendix H, Appendix I
 - of procedure, 1-3
- INWORD, 1-5, 10-4, 10-5
- iRMX, Appendix I
- LABEL, 3-1, 3-8
- label, 3-8, 6-2
 - as target of GOTO, 6-11
 - declaration, 3-8, 3-9, 7-1
 - results, 3-9
 - definition, 3-1, 3-8, 3-9, 8-2

- generated, 11-28
 - scope, 7-6 thru 7-9
- LARGE, 11-22, 11-23
 - restrictions, 11-23
- LAST, 5-5, 9-2
- LEFTMARGIN, 11-2
- left-to-right, 4-6 thru 4-9
- LENGTH, 5-5, 9-1
- length
 - in string moves, 9-9
- less than, see relational
- letters
 - upper and lower case, 2-1
- level
 - block, 7-1 thru 7-4
 - module, 3-7
 - outer, 1-3
- libraries of floating-point functions, Appendix F
- line-feed, 2-1, 2-5
- LINK86, 3-10, 11-16
- linkage, 3-1
 - assembly language, Appendix F
 - floating-point, Appendix F
- list
 - as array, 3-2, 5-1
 - as structure, 3-2, 5-2 thru 5-5
 - of controls on line, 11-1
 - of initialization values, 3-4 thru 3-7
 - restriction, 3-6, 3-7
 - of parameters, Chapter 8
 - of scalars, 3-2, 5-1
- LIST control, 11-24
- listing format controls, 11-26 thru 11-31
- listing, sample, 11-28
- listing selection and content controls, 11-24, 11-25
- LITERALLY, 3-1, 3-7, 3-8
- loading, 3-4
- LOC86, 1-1, 11-16
- local meaning, 9-1, 11-17
- local_save_area, Appendix G
- location
 - address, 3-3
 - attribute, 3-1
 - contents, 1-3
 - references, 2-5, 3-5, 3-13 thru 3-19, 5-5, 9-1
 - in string moves, 9-9
- lock, 9-14
- LOCKSET, 9-14
- logical
 - operation, 4-1
 - operator, 4-1, 4-5 thru 4-9, 11-32
- loop optimization, 11-7
- LOW, 9-3, 9-4

- machine code optimization, 11-4, 11-5
- main
 - module, 7-6, 7-9
 - program, 1-4, 11-6
- manual organization, Preface
- masked error, 10-6, Appendix G
- matrix as structure, 5-4
- MEDIUM, 11-21, 11-22
 - restrictions, 11-22

- member
 - reference, 5-4, 5-5, 8-2
 - structure, 5-2 thru 5-5
- MEMORY, 1-1, 9-14
- memory
 - concepts, 13-1
 - free, 9-14
 - shared, 9-14
- MINUS, 10-2
- minus sign, 4-3, 4-4, 4-6
- MOD, operator, 1-4, 4-3, 4-4, 4-6
- MOD86/MOD186
 - control, 11-2, 11-3
- models of segmentation, 11-18 thru 11-22, Chapter 13
- modular programming, 6-1
 - advantages, 7-1, 8-1
 - use of procedures, 8-1
- module, 7-1 thru 7-9
 - as block, 1-3
 - level, 3-7
 - main, 1-4
 - object, 11-16
- modulo arithmetic in DOs, 6-6
- MOVB, 9-10
- MOVE, 9-13
- MOVRB, 9-10
- MOVW, 9-10
- multiple
 - assignment, 4-14
 - declarations, 1-3, 3-1, 7-3
 - initializations, 3-4
- multiprocessor
 - lock, 9-14
- multi-tasking, Appendix G

- name, 1-3, 3-2
 - on a DO, 1-3
 - of procedure, 1-3, 3-9, 8-1, 8-2, 8-5
 - recognized in blocks, 7-1 thru 7-9
- NAN, Appendix G
- negative values, 3-10
- nesting
 - of blocks, 1-3, 6-3, 7-1 thru 7-4, 11-28
 - of procedures, 8-6, 8-10
- NIL pointers, 9-16
- NOCODE control, 11-25
- NOCOND control, 11-37
- NODEBUG control, 11-17
- NOINTVECTOR control, 11-4
- NOLIST control, 11-24
- NOOBJECT control, 11-16
- NOOVERFLOW control, 11-4
- NOPAGING control, 11-26
- NOPRINT control, 11-24
- NOSYMBOLS control, 11-25
- NOT, 4-5, 11-32
- notation, Preface
- NOTYPE control, 11-18
- NOXREF control, 11-25

- object
 - code, 11-16
 - file controls, Chapter 11
 - module, 11-16

- OBJECT control, 11-16, Appendix H, Appendix I
- object module sections, 11-16
- offset
 - of REAL exponent, 3-11
- OFFSET\$OF, 9-16
- ones complement, 4-5
- open subsystems, 13-3
- operand, 1-4, 4-1 thru 4-15
- operator, 1-4, 4-1, 4-6
 - precedence, 4-6
- optimization
 - and hardware flags, 10-1
- OPTIMIZE controls, 11-4 thru 11-15
- optimizing
 - indeterminate storage operations, 11-8
 - machine code, 11-6
 - pointer comparisons, 11-9
- OR, 4-5, 11-32
- order
 - of multiple assignments, 4-14
 - of operand evaluation, 4-6, 4-9, 4-14
 - of parameter evaluation, 8-3
- outer level, 1-3, 3-4, 3-13, 6-11, 7-1 thru 7-9
- out-of-range
 - case number, 6-3
- OUTPUT, 1-5, 10-4, 10-5
- OUTWORD, 1-5, 10-4, 10-5
- OVERFLOW control, 11-4
- overflow, Appendix G

- PAGELength control, 11-26
- PAGEWIDTH control, 11-26
- PAGING control, 11-26
- parameters,
 - control, 11-1
- parentheses, 2-2, 3-2 thru 3-7, 4-2, 4-6, 4-14, 4-15, 8-2, 8-4
- PARITY, 10-2
- pathname, 11-16, 11-26
- PE8087, Appendix F
- PL/M-80, Appendix D
- PL/M-86
 - compiler, Appendix H, Appendix I
 - PL/M-80, Appendix D
 - sample program, 1-6, Chapter 12
 - statements, 1-4
- PLM86.LIB, 4-1, Chapter 14
- PLUS, 10-2
- plus sign, 2-2, 2-5, 4-3 thru 4-6
- POINTER, 3-2, 3-4, 3-13, 4-13
- pointer, 3-4, 3-13
- POINTER and SELECTOR related functions, 9-16, 9-17
- precedence, 4-6 thru 4-9
- precision, 10-8, Appendix G
- predeclared identifiers, Appendix A
- primary
 - controls, 11-1
 - operands, 4-3
- PRINT control, 11-24
- PROCEDURE, 3-9, 8-1, 8-2
- procedure, 1-3, 1-6
 - activation, 1-3, 6-11, 6-12
 - block, 7-1 thru 7-7
 - body, 8-1, 8-6, 8-7
 - declaration, 1-3, 3-1, 3-9
 - definition block, 1-3, 3-9
 - epilogue, Appendix F
 - handling REAL interrupts, Appendix G
 - invoking, 1-3, 1-6, 6-12
 - in location references, 3-13, 3-14
 - name, 3-9
 - parameters, 3-9, 4-2
 - prologue, Appendix F
 - reentrant, 7-1, 8-10
 - termination, 1-3, 8-5
 - typed, 4-2, 8-3, Appendix F
 - untyped, 6-12, 8-3
 - value, 1-3, 4-2
- program, 1-2, 1-3, 7-1
 - control, 8-1
 - documentation, 2-5
 - example, 1-6, Chapter 12
 - main, 1-2
 - size controls, 11-18 thru 11-22, Chapter 13
- program constraints, Appendix B
- program development process, 1-5
- projective, see infinity
- prologue, 8-8, Appendix F
- PUBLIC, 3-1, 3-6 thru 3-9, 6-11, 7-4 thru 7-7, 8-7, 8-8

- qualified references, 4-1, 4-2
- quote, see apostrophe

- RAM control, 11-28
- REAL, 2-4, 3-11, 3-12, 4-1, 4-3, 4-4, 4-7 thru 4-14, 10-6 thru 10-10, Appendix G
- REAL error
 - byte, 10-6, 10-7
 - categories, Appendix G
- REAL exceptions, Appendix G
- REAL math facility, 10-6 thru 10-10
- REAL mode word, 10-6, 10-7
 - initial value, 10-6
 - suggested value, Appendix G
- REAL-parameter passing, Appendix F
- recursion, 8-10
- REENTRANT, 3-9, 8-10
- reentrant procedure, 7-1, 8-1, 8-10
- references
 - to arrays and structures, 5-4, 5-5, 8-3, 9-1, 9-2
 - external, see scope
 - location, 2-5, 3-13 thru 3-15, 5-5, 9-1
 - qualified
 - fully, 4-1, 4-2, 4-12, 5-4, 9-1
 - partially, 5-4, 5-5, 9-1 thru 9-3
 - unqualified, 5-4, 5-5
- related publications, Preface, Appendix H
- relational
 - operation, 3-10, 4-8, 4-9
 - operator, 3-10, 4-4 thru 4-9, 11-32
- removal of unreachable code, 11-6 thru 11-8
- representation of REAL values, 3-11, 3-12
- reserved words, 2-3, Appendix A
- RESET control, 11-35
- RESTORE control, 11-32
- RESTORE\$REAL\$STATUS, 10-10, Appendix G
- restoring REAL status, 10-8
- restricted expression, 3-4, 3-5
- RETURN, 4-12, 6-12, 8-5, 8-6
- return address, Appendix F

- reuse of duplicate code, 11-6, 11-7
 - restriction, 11-6
- reversal of branch condition, 11-6
- ROL, 9-7
- ROM control, 11-23
- ROR, 9-7
- rotation functions, 9-7, 9-8, 10-3
- rounding, 10-8, Appendix G
- RUN, Appendix H
- run-time, 4-1, 6-3, 8-3, Appendix F, Appendix H
- SAL, 9-8
- sample
 - listing, 11-28
 - programs, 1-6, Chapter 12
- SAR, 9-8
- SAVE control, 11-32
- saving REAL status, 10-9
- SAVE\$REAL\$STATUS, 10-10, Appendix G
- scalar, 3-2 thru 3-5, 4-2, 4-12, 5-4
- SCL, 10-3
- scope, 7-1 thru 7-9, 9-1
 - extended, 7-4 thru 7-7, 8-7, 8-8
 - of labels, 3-9
 - of procedure name, 8-1, 8-2
 - of variables, 1-4
- SCR, 10-3
- segment
 - address, 13-1
 - overlap, 11-8
- segmentation
 - controls, 11-18 thru 11-22, Chapter 13
- select_expression, see DO CASE
- SELECTOR, 3-15, 4-3, 4-10, 4-13
- SELECTOR\$OF, 3-15, 9-16
- semicolon, 2-2
- separators, 2-2, 2-3
- SETB, 9-12
- SET control, 11-35
- SET\$INTERRUPT, 9-15
- SET\$REAL\$MODE, 10-8
- SETW, 9-12
- shared memory, 9-14
- shift functions, 9-7 thru 9-9
- SHL, 9-8
- short-circuit Boolean evaluation, 11-4
- SHR, 9-8
- side effects, 4-9, 8-3, 8-4
 - see also order
- SIGN, 10-2
- SIGNED, 9-6
- sign of REAL number, 3-11
- SIZE, 5-5, 9-1, 9-2
- size controls, 11-18 thru 11-22, Chapter 13
- SKIPB, 9-11
- SKIPRB, 9-11
- SKIPRW, 9-11
- SKIPW, 9-11
- slash, 3-1, 4-3, 4-4
- SMALL, 11-18 thru 11-20
 - restrictions, 11-19
- soft recovery from REAL underflow, Appendix G
 - see underflow, denormal

- source
 - code, 1-6, Chapter 12
 - location in string moves, 9-9
- source inclusion controls, 11-32
- source PL/M-86 errors, 14-1 thru 14-19
- space, see blank
- special characters, 2-1 thru 2-3
- SS, 11-18 thru 11-22
- STACKBASE, 10-4
- STACKPTR, 10-4
- stack, Appendix F
 - market offset, Appendix F
 - maximum, 11-30
 - overflow, Appendix F
 - pointer, 10-4, Appendix F
 - REAL, Appendix F
 - segment
 - base address, 10-4, Appendix F
 - usage, 8-10, Appendix F, Appendix G
- stack section, 11-17 thru 11-22
- start_expr, 6-1
- statement number, 11-29
- step_expr, see DO, iterative
- strength reduction, 11-5
- string
 - in compilation constant, 3-7, 3-8
 - comparison, 9-9
 - constant, 2-1, 2-3, 2-5, 3-1, 3-6, 3-15, 4-2
 - copying, 9-9
 - definition, 9-9
 - index, 9-9
 - manipulation, 9-9 thru 9-13
 - order of copy, 9-9
 - target, 9-11
 - translation, 9-12
 - type, 2-5
 - value, 2-5, 3-5, 4-2
 - value assignment, 9-12
- STRUCTURE, 3-1, 3-3, 3-5, 5-2
- structure, 3-2, 3-5, 3-6, 5-2 thru 5-5
 - arrays, 5-2 thru 5-5
 - as matrix, 5-4
 - declaration, 5-2
 - example, 3-3, 3-5, 3-18
 - references, 5-5
 - storage of, 5-2
 - type, 5-2
- subexpression, 4-2, 4-6 thru 4-9, 11-5, 11-33
- subroutine, see procedure
- subscript, 3-2, 3-3, 3-9, 3-15 thru 3-17, 4-11, 5-1 thru 5-5, 8-2
- subsystems
 - modular, Chapter 13
 - unnamed, 13-3
- SUBTITLE control, 11-27
- subtraction, 3-10, 4-3
- suffix, 2-4
- superfluous
 - branches, 11-5
 - operations, 11-5
- support library, 3-10
- symbolic and cross-reference listing, 11-28, 11-30
- symbolic
 - debugging, 11-17
 - names, see variable, SYMBOLS

SYMBOLS control, 11-25, 11-28

syntax

BNF description, Appendix C

tab, 2-1, 2-5, 6-3

target

label in GOTO, 6-11, 7-6 thru 7-9

in string moves, 9-11

THEN part

see IF statement or control

TIME, 9-13

TITLE control, 11-27

token, 2-2

true, 4-5, 4-6, 6-4, 6-7

twos complement, 3-11

type, 3-1

of arithmetic, 1-4, 3-10, 4-3, 4-4

conflict, 7-5

conversion

explicit, 9-3 thru 9-7

implicit, 4-8, 4-13

of counter in iterative DO, 6-6

data, 3-10 thru 3-15

mixing, 4-8

procedure, 8-3, 8-4, 8-7, Appendix F

TYPE control, 11-18

unary operators, 4-6

underflow, Appendix G

underscore, 2-3

unmasked error, 10-8, Appendix G

UNSIGN, 9-3, 9-6

untyped procedure, 6-12, 8-3, 8-4, 8-7

usage

declaration, 7-5, 8-7, 8-8

variable, 1-4, 3-1 thru 3-3, 3-10

area size, 11-30

assignment, 4-12 thru 4-15

based, 3-15 thru 3-17

declaration, 3-1

results, 3-3

definition, 3-1

initialization, 3-1, 3-4 thru 3-7

names, 1-4, 3-1

negative, 3-10

REAL, 3-11, 3-12

references, 1-4, 4-2

reinitialization, 3-4

subscripted, 5-1 thru 5-4

types, 3-10 thru 3-15, 4-3 thru 4-5, 4-8, 4-10 thru 4-15

WAIT state, 10-10

warnings, 11-31, Chapter 14

WHILE, see DO

whole-numbers,

context, 4-1

WORD, 3-1, 3-10, 4-1 thru 4-14

least significant bits, 3-10

XLAT, 9-12

XOR, 4-5, 11-32

XREF control, 11-25, 11-28

ZERO, 10-2

zero divide, Appendix G



REQUEST FOR READER'S COMMENTS

Intel's Technical Publications Departments attempt to provide publications that meet the needs of all Intel product users. This form lets you participate directly in the publication process. Your comments will help us correct and improve our publications. Please take a few minutes to respond.

Please restrict your comments to the usability, accuracy, readability, organization, and completeness of this publication. If you have any comments on the product that this publication describes, please contact your Intel representative. If you wish to order publications, contact the Intel Literature Department (see page ii of this manual).

1. Please describe any errors you found in this publication (include page number).

2. Does the publication cover the information you expected or required? Please make suggestions for improvement.

3. Is this the right type of publication for your needs? Is it at the right level? What other types of publications are needed?

4. Did you have any difficulty understanding descriptions or wording? Where?

5. Please rate this publication on a scale of 1 to 5 (5 being the best rating)._____

NAME _____ DATE _____

TITLE _____

COMPANY NAME/DEPARTMENT _____

ADDRESS _____

CITY _____ STATE _____ ZIP CODE _____
(COUNTRY)

Please check here if you require a written reply. ☐

WE'D LIKE YOUR COMMENTS ...

This document is one of a series describing Intel products. Your comments on the back of this form will help us produce better manuals. Each reply will be carefully reviewed by the responsible person. All comments and suggestions become the property of Intel Corporation.



**NO POSTAGE
NECESSARY
IF MAILED
IN U.S.A.**



BUSINESS REPLY MAIL

FIRST CLASS PERMIT NO. 1040 SANTA CLARA, CA

POSTAGE WILL BE PAID BY ADDRESSEE

**Intel Corporation
Attn: Technical Publications M/S 6-2000
3065 Bowers Avenue
Santa Clara, CA 95051**



INTEL CORPORATION, 3065 Bowers Avenue, Santa Clara, California 95051 (408) **987-8080**

Printed in U.S.A.